



## **Modular QoS Configuration Guide for Cisco 8000 Series Routers, Cisco IOS XR Releases**

**First Published:** 2025-10-31

**Last Modified:** 2025-12-17

### **Americas Headquarters**

Cisco Systems, Inc.  
170 West Tasman Drive  
San Jose, CA 95134-1706  
USA  
<http://www.cisco.com>  
Tel: 408 526-4000  
800 553-NETS (6387)  
Fax: 408 527-0883

THE SPECIFICATIONS AND INFORMATION REGARDING THE PRODUCTS IN THIS MANUAL ARE SUBJECT TO CHANGE WITHOUT NOTICE. ALL STATEMENTS, INFORMATION, AND RECOMMENDATIONS IN THIS MANUAL ARE BELIEVED TO BE ACCURATE BUT ARE PRESENTED WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED. USERS MUST TAKE FULL RESPONSIBILITY FOR THEIR APPLICATION OF ANY PRODUCTS.

THE SOFTWARE LICENSE AND LIMITED WARRANTY FOR THE ACCOMPANYING PRODUCT ARE SET FORTH IN THE INFORMATION PACKET THAT SHIPPED WITH THE PRODUCT AND ARE INCORPORATED HEREIN BY THIS REFERENCE. IF YOU ARE UNABLE TO LOCATE THE SOFTWARE LICENSE OR LIMITED WARRANTY, CONTACT YOUR CISCO REPRESENTATIVE FOR A COPY.

The Cisco implementation of TCP header compression is an adaptation of a program developed by the University of California, Berkeley (UCB) as part of UCB's public domain version of the UNIX operating system. All rights reserved. Copyright © 1981, Regents of the University of California.

NOTWITHSTANDING ANY OTHER WARRANTY HEREIN, ALL DOCUMENT FILES AND SOFTWARE OF THESE SUPPLIERS ARE PROVIDED "AS IS" WITH ALL FAULTS. CISCO AND THE ABOVE-NAMED SUPPLIERS DISCLAIM ALL WARRANTIES, EXPRESSED OR IMPLIED, INCLUDING, WITHOUT LIMITATION, THOSE OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OR ARISING FROM A COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

IN NO EVENT SHALL CISCO OR ITS SUPPLIERS BE LIABLE FOR ANY INDIRECT, SPECIAL, CONSEQUENTIAL, OR INCIDENTAL DAMAGES, INCLUDING, WITHOUT LIMITATION, LOST PROFITS OR LOSS OR DAMAGE TO DATA ARISING OUT OF THE USE OR INABILITY TO USE THIS MANUAL, EVEN IF CISCO OR ITS SUPPLIERS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Any Internet Protocol (IP) addresses and phone numbers used in this document are not intended to be actual addresses and phone numbers. Any examples, command display output, network topology diagrams, and other figures included in the document are shown for illustrative purposes only. Any use of actual IP addresses or phone numbers in illustrative content is unintentional and coincidental.

All printed copies and duplicate soft copies of this document are considered uncontrolled. See the current online version for the latest version.

Cisco has more than 200 offices worldwide. Addresses and phone numbers are listed on the Cisco website at [www.cisco.com/go/offices](http://www.cisco.com/go/offices).

Cisco and the Cisco logo are trademarks or registered trademarks of Cisco and/or its affiliates in the U.S. and other countries. To view a list of Cisco trademarks, go to this URL: <https://www.cisco.com/c/en/us/about/legal/trademarks.html>. Third-party trademarks mentioned are the property of their respective owners. The use of the word partner does not imply a partnership relationship between Cisco and any other company. (1721R)

© 2025 Cisco Systems, Inc. All rights reserved.



## CONTENTS

|                  |                                                         |
|------------------|---------------------------------------------------------|
| <b>PREFACE</b>   | <b>Preface</b> xi                                       |
| <b>CHAPTER 1</b> | <b>YANG Data Models for QoS Features</b> 1              |
|                  | Using YANG Data Models 1                                |
| <b>CHAPTER 2</b> | <b>Modular QoS Command-line Interface</b> 3             |
|                  | Modular QoS command-line interface 3                    |
| <b>CHAPTER 3</b> | <b>Traffic Management with VOQs</b> 5                   |
|                  | Traffic management with VOQs 5                          |
|                  | Limitations of VOQ model 6                              |
|                  | How traffic management with VOQs work 7                 |
| <b>CHAPTER 4</b> | <b>Classify Packets to Identify Specific Traffic</b> 11 |
|                  | Packet classification 11                                |
|                  | IP Precedence to prioritize traffic 15                  |
|                  | Advantages of IP Precedence 15                          |
|                  | How IP Precedence Works 15                              |
|                  | Packet classification systems on your router 16         |
|                  | DiffServ tunneling modes 17                             |
|                  | Short-pipe mode 18                                      |
|                  | How Short-Pipe mode for QoS classification works 19     |
|                  | Best practices for configuring short-pipe mode 20       |
|                  | Enable short-pipe mode for MPLS to IP traffic 21        |
|                  | Traffic classes in QoS 24                               |
|                  | Traffic class configuration criteria 25                 |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Elements and supported actions of a traffic policy                    | 27 |
| Configure packet classification and specify QoS actions               | 28 |
| Configure QoS actions for the default class                           | 30 |
| Additional Guidelines for Packet Classification and QoS Actions       | 31 |
| Enhanced Running Configuration Display for Policy Maps and Class Maps | 31 |
| Multicast traffic scheduling on egress queues                         | 34 |
| QoS policy inheritance                                                | 35 |
| Limitations of the QoS policy inheritance model                       | 36 |

---

**CHAPTER 5**
**Packet Marking for Priority Differentiation 37**

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Packet marking                                                          | 37 |
| Unconditional QoS packet marking                                        | 39 |
| QoS marking modes for MPLS traffic                                      | 39 |
| QoS marking for IP-to-MPLS traffic                                      | 41 |
| Benefits of QoS marking for IP-to-MPLS traffic                          | 42 |
| Limitations for configuring QoS marking for IP-to-MPLS traffic          | 43 |
| Configure QoS marking for IP-to-MPLS encapsulated traffic               | 43 |
| QoS marking in tunneling and encapsulation scenarios                    | 45 |
| QoS marking on GRE tunnels                                              | 45 |
| Guidelines for configuring QoS marking on GRE tunnels                   | 46 |
| Explicit QoS marking for SRv6 encapsulation                             | 47 |
| Limitations for configuring explicit QoS marking for SRv6 encapsulation | 47 |
| Default QoS marking for VXLAN encapsulation                             | 48 |
| QoS marking on Layer 2 subinterfaces                                    | 49 |
| Configure QoS marking on Layer 2 subinterfaces                          | 49 |

---

**CHAPTER 6**
**QoS Policies for Link Bundles 53**

|                                                   |    |
|---------------------------------------------------|----|
| QoS policies for link bundles                     | 53 |
| Usage guidelines for QoS policies on link bundles | 53 |
| Restrictions for QoS policies on link bundles     | 55 |
| Configure QoS policies on link bundles            | 55 |

---

**CHAPTER 7**
**Congestion Management 61**

|                               |    |
|-------------------------------|----|
| Traffic congestion management | 61 |
|-------------------------------|----|

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Low-latency queuing with strict priority queuing                               | 62 |
| Guidelines for configuring low-latency queuing with strict priority queuing    | 66 |
| Configure low-latency queuing with strict priority queuing                     | 67 |
| Traffic shaping                                                                | 68 |
| Guidelines for configuring traffic shaping                                     | 68 |
| Configure traffic shaping                                                      | 69 |
| Egress class-level traffic shaping on subinterfaces                            | 70 |
| Guidelines for configuring egress class-level traffic shaping on subinterfaces | 72 |
| Traffic policing                                                               | 72 |
| Single-rate two-color policers                                                 | 73 |
| Guidelines for configuring single-rate two-color policers                      | 73 |
| How the single-rate two-color policer works                                    | 73 |
| Configure single-rate two-color policers                                       | 74 |
| Two-rate three-color policers                                                  | 76 |
| Guidelines for configuring two-rate three-color policers                       | 76 |
| How the two-rate three-color policer works                                     | 76 |
| Configure two-rate three-color policers                                        | 79 |

---

## CHAPTER 8

### Congestion Avoidance 81

|                                                     |    |
|-----------------------------------------------------|----|
| Congestion Avoidance                                | 81 |
| Queuing modes                                       | 83 |
| Switch the queuing mode on the router               | 84 |
| Queuing policy for main interfaces                  | 85 |
| Queuing policy behavior on subinterfaces            | 86 |
| Congestion detection and handling                   | 88 |
| Tail drop and the FIFO queue                        | 89 |
| Key attributes of tail drop queue management        | 89 |
| How tail drop works                                 | 90 |
| Guidelines for configuring tail drop thresholds     | 91 |
| Configure a tail drop threshold for a traffic class | 91 |
| Congestion avoidance in VOQs                        | 93 |
| Dual queue limit                                    | 93 |
| Guidelines for configuring dual queue limits        | 94 |
| Configure dual queue limits                         | 94 |

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| Equitable traffic flow using fair VOQ                              | 96  |
| Configuration options for fair VOQ modes and sharing of counters   | 99  |
| Guidelines and limitations for fair VOQ                            | 99  |
| Maximum supported interfaces by fair VOQ and counter sharing modes | 100 |
| Configure fair VOQ                                                 | 101 |
| Random Early Detection and TCP                                     | 103 |
| Guidelines for configuring Random Early Detection                  | 103 |
| How Random Early Detection works                                   | 104 |
| Configure Random Early Detection                                   | 104 |
| Explicit Congestion Notification                                   | 106 |
| How ECN marking works                                              | 107 |
| ECN bit combinations and their meanings                            | 107 |
| ECN policy restrictions                                            | 108 |
| Configure ECN in a policy map                                      | 108 |
| Virtual Output Queue Watchdog                                      | 109 |
| Guidelines and restrictions for VOQ watchdog                       | 111 |
| How VOQ watchdog works on line cards and fabric cards              | 112 |
| VOQ watchdog behavior on line and fabric cards                     | 112 |
| Enable VOQ watchdog                                                | 114 |

---

**CHAPTER 9**
**Congestion Monitoring and Optimization 115**

|                                                                                      |     |
|--------------------------------------------------------------------------------------|-----|
| Congestion monitoring and optimization                                               | 115 |
| Sharing of VOQ statistics counters                                                   | 115 |
| Guidelines for configuring VOQ counter sharing                                       | 116 |
| Shared VOQ counter modes                                                             | 116 |
| Configure shared VOQ counters                                                        | 116 |
| Traffic class queue high water marks monitoring                                      | 117 |
| High water mark data types                                                           | 119 |
| Recommendations and requirements for monitoring traffic class queue high water marks | 120 |
| Configure traffic class queue high water marks monitoring                            | 120 |
| Descriptions of show command output fields for queue high water marks                | 123 |
| VOQ evictions to High Bandwidth Memory                                               | 125 |
| How VOQ eviction to HBM works                                                        | 127 |
| Best practices for viewing VOQ eviction to HBM                                       | 128 |

View evicted VOQs to HBM 128

## CHAPTER 10

### Priority Flow Control 131

Stages of Priority Flow Control configuration 131

Priority Flow Control 132

Benefits of Priority Flow Control 136

How Priority Flow Control works 136

Priority Flow Control modes 137

Hardware support for Priority Flow Control modes 137

Best practices for Priority Flow Control 138

buffer-internal mode 139

Best practices for configuring buffer-internal mode 139

Configure buffer-internal thresholds 140

Configurable ECN threshold and marking probability values 142

How Explicit Congestion Notification works within Priority Flow Control buffer-internal mode 142

PFC buffer internal mode configuration options and behaviors 144

Best practices for configuring ECN threshold and marking probability 145

Configure ECN threshold and maximum marking probability values 146

buffer-extended mode 148

Best practices for configuring buffer-extended mode 148

Configure buffer-extended thresholds 149

Configure PFC on interfaces 152

High Bandwidth Memory congestion detection 155

How HBM congestion detection works 156

Best practices for HBM congestion detection 157

Configure High Bandwidth Memory congestion detection 157

Check available Shared Memory System and High Bandwidth Memory buffers 161

Available buffer in Shared Memory System and High Bandwidth Memory 162

Global pause frames for High Bandwidth Memory congestion 163

Recommendations for configuring global pause frames for High Bandwidth Memory congestion 164

Configure global pause frames for High Bandwidth Memory congestion 164

buffer-extended hybrid mode 166

Best practices for configuring buffer-internal hybrid mode 167

Configure buffer-extended hybrid mode with default settings 167

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| Configure buffer-extended hybrid mode with user-specified settings | 168 |
| Configure buffer-extended hybrid mode                              | 170 |

---

**CHAPTER 11**
**Priority Flow Control Watchdog and Monitoring Metrics 171**

|                                                                                                       |     |
|-------------------------------------------------------------------------------------------------------|-----|
| Stages of Priority Flow Control Watchdog and monitoring                                               | 171 |
| Priority Flow Control watchdog                                                                        | 172 |
| How the Priority Flow Control watchdog works                                                          | 172 |
| Best practices for Priority Flow Control watchdog configuration                                       | 173 |
| Configure a Priority Flow Control watchdog interval                                                   | 173 |
| Global statistics counters for Priority Flow Control and Priority Flow Control watchdog               | 176 |
| View global statistics for Priority Flow Control and Priority Flow Control watchdog                   | 177 |
| Supported commands for global statistics for Priority Flow Control and Priority Flow Control watchdog | 178 |
| Traffic class queue pause duration                                                                    | 179 |
| How traffic class queue pause duration works                                                          | 181 |
| Best practices for using traffic class queue pause duration                                           | 182 |
| View traffic class queue pause duration                                                               | 183 |

---

**CHAPTER 12**
**QoS Deployment across Network Scenarios 187**

|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| Information flow                                                      | 187 |
| QoS support for VPWS traffic                                          | 188 |
| Configure ingress QoS classification for VPWS                         | 190 |
| QoS Policy Propagation via BGP                                        | 193 |
| Benefits of QoS Policy Propagation via BGP                            | 195 |
| How QoS Policy Propagation via BGP works                              | 195 |
| Best practices for configuring QoS Policy Propagation via BGP         | 196 |
| Configure QoS Policy Propagation via BGP                              | 197 |
| QoS on Pseudowire Headend                                             | 200 |
| How QoS on Pseudowire Headend works                                   | 202 |
| Functional details and behaviors of QoS on PWHE                       | 203 |
| General features and support information for QoS on PWHE              | 204 |
| Traffic mapping for Pseudowire Headend interfaces to pin-down members | 205 |
| Port shaping effect on pin-down members                               | 206 |
| Bandwidth distribution for QoS on Pseudowire Headend                  | 206 |



|                                                                    |     |
|--------------------------------------------------------------------|-----|
| QoS accounting for Pseudowire Headend traffic                      | 206 |
| Marking for QoS on Pseudowire Headend                              | 207 |
| Policing and queuing for QoS on Pseudowire Headend                 | 207 |
| Pseudowire Headend aggregate shaper                                | 208 |
| Pseudowire Ethernet subinterface policy                            | 209 |
| Scale information for QoS on Pseudowire Headend interfaces         | 209 |
| Pseudowire Headend without QoS policy                              | 210 |
| Limitations for QoS on Pseudowire Headend                          | 210 |
| Configuration options for QoS on Pseudowire Headend                | 210 |
| Configure Flat QoS on Pseudowire Headend                           | 211 |
| Configure Hierarchical QoS on Pseudowire Headend                   | 215 |
| Configure QoS with aggregate shaper on Pseudowire Headend          | 221 |
| Bridged Virtual Interface two-pass QoS                             | 235 |
| Benefits of BVI two-pass QoS                                       | 236 |
| How BVI two-pass QoS works                                         | 236 |
| Best practices for BVI two-pass QoS                                | 237 |
| Restrictions while configuring BVI two-pass QoS for Release 25.4.1 | 238 |
| Configure BVI two-pass QoS for ingress traffic                     | 238 |





## Preface

This cumulative guide provides a single, continuously updated version that includes all the latest IOS XR features and release updates. It simplifies your experience by letting you bookmark one link and access the complete guide, instead of navigating through multiple release-specific versions.

Specific changes or updates tied to individual releases are clearly called out within the relevant sections. For a list of features introduced in a specific release, refer to the [Release Notes](#) or the [IOS XR Feature Finder](#).

The table lists the release numbers for which this document has been updated since its initial publication.

**Table 1: Changes to this document**

| Date          | Summary                            |
|---------------|------------------------------------|
| December 2025 | Republished for Release 25.4.1     |
| October 2025  | First published for Release 25.3.1 |

### Related resources

- [Modular QoS Command Reference for Cisco 8000 Series Routers](#)
- [Release Notes](#)
- [IOS XR Feature Finder](#)
- [MIBs dashboard](#)
- [Error Messages dashboard](#)
- [YANG dashboard](#)





## CHAPTER 1

# YANG Data Models for QoS Features

---

This chapter provides information about the YANG data models for QoS features.

- [Using YANG Data Models, on page 1](#)

## Using YANG Data Models

Cisco IOS XR supports a programmatic way of configuring and collecting operational data of a network device using YANG data models. Although configurations using CLIs are easier and human-readable, automating the configuration using model-driven programmability results in scalability.

The data models are available in the release image, and are also published in the [Github](#) repository. Navigate to the release folder of interest to view the list of supported data models and their definitions. Each data model defines a complete and cohesive model, or augments an existing data model with additional XPaths. To view a comprehensive list of the data models supported in a release, navigate to the **Available-Content.md** file in the repository.

You can also view the data model definitions using the [YANG Data Models Navigator](#) tool. This GUI-based and easy-to-use tool helps you explore the nuances of the data model and view the dependencies between various containers in the model. You can view the list of models supported across Cisco IOS XR releases and platforms, locate a specific model, view the containers and their respective lists, leaves, and leaf lists presented visually in a tree structure. This visual tree form helps you get insights into nodes that can help you automate your network.

To get started with using the data models, see the *Programmability Configuration Guide*.





## CHAPTER 2

# Modular QoS Command-line Interface

---

- [Modular QoS command-line interface, on page 3](#)

## Modular QoS command-line interface

Modular QoS command-line interface (MQC) is a Quality of Service (QoS) CLI configuration framework that

- defines traffic classification rules using class-map objects
- associates classification with treatment actions for traffic packets in policy-map objects, and
- attaches traffic policies to interfaces (ingress or egress), thereby enabling consistent QoS control across platforms.

MQC enables users to create QoS policies by linking defined traffic classes with specific traffic treatment actions (marking, policing, shaping, queuing, or dropping) and then applying these policies to interfaces. The system enforces classification and subsequent treatment on packets flowing through the interfaces and enforces the QoS behavior described in the associated policy map.

MQC comprises of three fundamental logical components.

- Class-map: defines a traffic class used later by a policy-map as a match criteria to apply QoS treatment.
- Policy-map: associates one or more traffic classes with QoS actions such as marking, queueing, scheduling, shaping, WRED, or policing.
- Service-policy: applies a policy-map to an interface (ingress or egress), activating the configured QoS treatments for matching traffic.

### MQC attributes

MQC exhibits several structural characteristics that define how it organizes and applies QoS policies.

- Implements a three-stage object model: classification (class-map), policy creation (policy-map), and policy attachment to interfaces (service-policy).
- Enables reuse of named objects (classes and policies) to reduce duplicated configuration.
- Distinguishes classification logic from traffic treatment logic so that traffic definition and QoS action remain decoupled.

- Includes default handling: traffic that doesn't match any class is bound to a default class with baseline behavior.
- Supports nested class-maps using match class-map to refine matching hierarchy.

**Table 2: Differences between MQC and traditional QoS CLI frameworks**

| Attribute       | MQC framework                                                         | Traditional QoS CLI framework                                            |
|-----------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------|
| Type            | Configuration framework consists of modular command hierarchy         | Interface-centric command model consists of flat or monolithic CLI       |
| Structure       | Uses modular objects: class-map, policy-map, and service-policy       | Defines QoS directly under each interface with feature-specific commands |
| Reusability     | High—classes and policies are reusable across interfaces and services | Low—each interface must be configured independently                      |
| Consistency     | Ensures identical QoS behavior across multiple interfaces             | Risk of inconsistency due to repetitive manual configuration             |
| Maintainability | Simplifies updates—change one policy and re-apply                     | Requires per-interface configuration changes for every modification      |

### Benefits of MQC

MQC offers multiple operational advantages that enhance the consistency and scalability of QoS configuration.

- Promotes consistency in QoS configurations by using the same architecture on multiple platforms.
- Improves readability and maintenance by decoupling classification from service treatment.
- Enables policy reuse and modular design, reducing operational effort in large deployments.
- Allows flexible traffic behavior management, supporting multiple QoS features within a unified framework.
- Simplifies troubleshooting and verification by separating counters and policies per class and interface.





## CHAPTER 3

# Traffic Management with VOQs

- [Traffic management with VOQs, on page 5](#)
- [Limitations of VOQ model, on page 6](#)
- [How traffic management with VOQs work, on page 7](#)

## Traffic management with VOQs

A traffic management model with Virtual Output Queue (VOQ) is a QoS queuing architecture that

- enables per-egress interface queuing at ingress using VOQs
- prevents head-of-line (HOL) blocking by isolating traffic destined for different egress interfaces, and
- provides granular control over network traffic congestion and bandwidth allocation for data packets.

**VOQs**—VOQs are buffers at the ingress that holds traffic for a specific egress port, ensuring traffic is queued only when the destination is ready to receive. Your routers support up to eight output queues per main interface or physical port. For every egress output queue, the VOQ model earmarks buffer space on every ingress pipeline. This buffer space is in the form of dedicated VOQs. These queues are called virtual because the queues physically exist on the ingress interface only when the line card actually has packets enqueued to it.

**Head-of-line blocking**—In traditional ingress queuing systems, traffic destined for multiple egress ports may be held in a single shared queue. If the egress port for the first packet is congested, all subsequent packets—even those destined for other non-congested ports—are blocked from forwarding. This effect is referred to as head-of-line blocking and results in reduced traffic throughput and increased latency.

### Key features of VOQ-based traffic management

These are the key features of the VOQ-based traffic management model:

- **Per-egress Queuing at Ingress:** Virtual Output Queues are created per traffic class and per egress interface, allowing granular traffic isolation and precise scheduling.
- **Eight VOQs per Egress Port:** For each egress interface, eight VOQs—corresponding to eight traffic classes—are reserved across all ingress interfaces, ensuring efficient prioritization of data packets.

For more information on the various types of traffic classes and packet classification, see *Classify Packets to Identify Specific Traffic*.

- **Head-of-line Blocking Prevention:** By dedicating VOQs for each destination, the model eliminates congestion caused by blocked packets for other egress ports.

- **Dynamic VOQ Instantiation:** VOQs are created only when packets are present, optimizing buffer usage across ingress pipelines.
- **Connector Mesh:** A logical mesh of connectors enables integrated handling of credit requests, grant responses, and data transmission between ingress and egress.
- **Credit-Based Scheduling:** Egress ports issue credits to ingress VOQs based on availability, ensuring bandwidth is distributed according to QoS priorities.
- **Throughput Optimization and Loss Reduction:** The architecture minimizes packet drops during congestion and maximizes throughput by forwarding only when the egress is ready.

### Benefits of VOQ model

The VOQ-based traffic management model improves traffic handling efficiency and network performance by introducing a queuing architecture that

- reduces head-of-line blocking by separating queues per egress destination,
- minimizes packet drops by queuing only when egress resources are available, and
- supports lossless packet forwarding for high-priority traffic under congestion scenarios.

**Table 3: Differences between non-VOQ-based traffic management and VOQ-based traffic management**

| Non-VOQ-based traffic management                                                                               | VOQ-based traffic management                                                                                  |
|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Ingress queues traffic without visibility into egress port availability.                                       | Ingress buffers traffic per egress port using dedicated VOQs.                                                 |
| Prone to congestion due to head-of-line blocking.                                                              | Prevents head-of-line blocking by isolating queues per destination.                                           |
| Inefficient use of bandwidth when packets are dropped mid-pipeline at ingress due to congestion at the egress. | Minimizes wastage of router resources by dropping traffic packets only at ingress when egress is unavailable. |

## Limitations of VOQ model

Before you install the QoS features on your router, consider these limitations that could impact scalability, system memory usage, and system behavior under high-load conditions:

- **Replication overhead:** Each egress queue must be replicated as an ingress VOQ on every slice of every NPU or ASIC. This increases memory usage significantly.
- **Reduced scalability:** The total egress queue scale is lower due to the replication requirement, limiting system-wide queueing flexibility.
- **Resource consumption:** As the number of egress ports or linecards increases, the system must replicate VOQs across all ingress NPUs, consuming significant buffer and memory resources.

For example, adding a new NPU with 20 interfaces may require 160 additional VOQs on every existing NPU (20 interfaces  $\times$  8 traffic classes per interface).

- On-chip buffer usage: In high-scale scenarios with 1000+ active VoQs, on-chip buffer (OCB) may be exhausted, causing unexpected traffic drops even when traffic rates are within shaping thresholds.

## How traffic management with VOQs work

### Summary

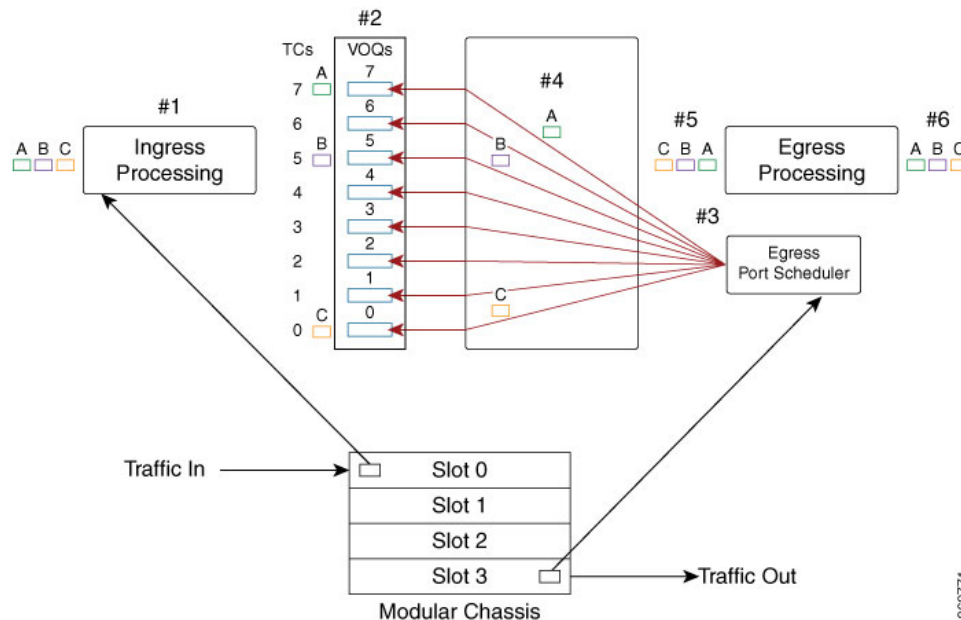
The key components involved in VOQ-based traffic management are:

- Ingress Interface: Classifies, marks, and polices incoming data packets. The ingress interface is always mapped to a physical port.
- Ingress VOQ: Buffers packets based on egress port and traffic class; instantiated dynamically when traffic is present. Every ingress interface port maintains eight VOQs—one for each traffic class—dedicated to a specific egress port.
- Connectors: Logical links between ingress NPUs and egress ports that handle credit requests, grant responses, and data transmission across the fabric cards.
- Egress Scheduler: Allocates credits based on available bandwidth and priority policies.
- Egress Interface: Applies final markings and transmits packets to the next hop. The egress interface is always mapped to a physical port.

Traffic management with VOQs involves a coordinated sequence of queuing and scheduling actions between ingress and egress components to ensure efficient and lossless packet forwarding. The key components—ingress interface, Virtual Output Queues (VOQs), connector mesh, egress scheduler, and egress interface—work together to classify incoming traffic, buffer it per egress destination, allocate transmission credits based on availability, and transmit packets across the switch fabric with minimal latency and packet loss.

## Workflow

Figure 1: Traffic flow from ingress port on slot 0 to egress port on slot 3



These stages describe how traffic management with VOQs work.

1. The ingress interface classifies and marks incoming packets.

The router receives packets—A (green), B (pink), and C (brown)—on the ingress interface. This is where packet marking, classification, and policing are applied based on QoS policies.



**Note** This diagram illustrates a modular chassis with multiple slots (Slot 0 to Slot 3).

2. The ingress interface maps packets to dedicated VOQs.

Per traffic-class, each packet is enqueued into a VOQ corresponding to its egress port. For example, in the figure:

- Packet A (green) is mapped to TC7,
- Packet B (pink) is mapped to TC5, and
- Packet C (brown) is mapped to TC0.

This one-to-one VOQ mapping ensures proper congestion management and accurate scheduling for each flow.

3. The egress scheduler allocates transmission credits.

Based on the availability of egress bandwidth, the egress port scheduler issues credits that determine the sequence and rate of packet transmission towards the egress interface.

4. The fabric switch forwards eligible packets to the egress port.

Once credits are received, the eligible packets are transmitted across the fabric card toward the appropriate egress port.

5. The egress interface marks and prepares packets for forwarding.

At the egress interface, any final packet markings or required shaping is enforced before forwarding to the next hop.

6. The router transmits packets to their outbound destination.

At this stage, congestion is managed in such a way that no packets are dropped, and all packets are transmitted to the next hop.





## CHAPTER 4

# Classify Packets to Identify Specific Traffic

- [Packet classification, on page 11](#)
- [Packet classification systems on your router, on page 16](#)
- [DiffServ tunneling modes, on page 17](#)
- [Short-pipe mode, on page 18](#)
- [Traffic classes in QoS, on page 24](#)
- [Elements and supported actions of a traffic policy, on page 27](#)
- [Configure packet classification and specify QoS actions, on page 28](#)
- [Enhanced Running Configuration Display for Policy Maps and Class Maps, on page 31](#)
- [Multicast traffic scheduling on egress queues, on page 34](#)
- [QoS policy inheritance, on page 35](#)

## Packet classification

Packet classification is a process that

- sorts network traffic into specific categories or classes based on predefined criteria
- categorizes a packet within a specific class and assigns it a traffic descriptor to implement Quality of Service (QoS) policies where different traffic types (such as voice, video, or data) are treated according to their priority, and
- ensures traffic flows meet the required network performance metrics, such as minimal delay, high throughput, or low packet loss.

A traffic descriptor is a set of specific attributes attached to a packet that

- indicates the forwarding treatment (QoS) that the packet should receive, such as priority, bandwidth allocation, and queuing treatment
- is used by traffic policers and traffic shapers to ensure that the packet adheres to a specified traffic profile or contract (such as rate limits or traffic priority), and
- facilitates QoS handling throughout the network, allowing routers, switches, and other devices to apply the appropriate traffic management policies based on the packet's markings (such as DSCP and IP Precedence).

Table 4: Feature History Table

| Feature Name                             | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingress and Egress Packet Classification | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br>*This feature is supported on Cisco 8011-4G24Y4H-I routers.                                                                                                                                                                                                                                                                                                                                                                                                              |
| Ingress and Egress Packet Classification | Release 24.4.1      | Introduced in this release on: Fixed Systems (8700 [ASIC:K100])(select variants only*)<br>*This feature is supported on Cisco 8712-MOD-M routers.                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Ingress and Egress Packet Classification | Release 24.3.1      | Introduced in this release on:<br>Modular Systems (8800 [LC ASIC: P100]) (select variants only*), Fixed Systems (8200) (select variants only*), Fixed Systems (8700 (P100, K100)) (select variants only*)<br><br>You can categorize packets into specific groups or classes and assign them traffic descriptors for QoS to classify and manage network traffic effectively.<br><br>*This feature is supported on:<br><ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |
| Ingress and Egress Packet Classification | Release 24.2.1      | Introduced in this release on:<br>Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>Categorizing packets into specific groups or classes and assigning them traffic descriptors for QoS helps classify and manage network traffic effectively. At the ingress, the QoS map and TCAM are used for classification. The QoS map is used for classification at the egress when policy matches only on DSCP, while TCAM is used for other criteria such as MPLS.<br><br>*This feature is supported on 88-LC1-36EH.                                    |

Your router uses packet classification in a staged approach to identify, group, and assign QoS actions. The table summarizes these key stages and the corresponding sections for more detail.



**Table 5: Understanding the Stages of Packet Classification**

| Stage                           | Description                                                                                                                                                                                                                                | See section                                                                    |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Packet classification technique | Determines how traffic is identified using methods such as IP Precedence and Differentiated Services Code Point.                                                                                                                           | <a href="#">Types of packet classification</a>                                 |
| Packet handling mode            | Determines which packet header fields are visible to classification based on the configured DiffServ tunneling mode. Packet handling models influence how QoS markings are interpreted during packet traversal but do not perform marking. | <a href="#">DiffServ tunneling modes, on page 17</a>                           |
| Traffic class definition        | Defines match criteria and logic for grouping similar packets.                                                                                                                                                                             | <a href="#">Traffic classes in QoS, on page 24</a>                             |
| Traffic policy application      | Applies QoS actions (mark, shape, police, queue) based on traffic classes.                                                                                                                                                                 | <a href="#">Elements and supported actions of a traffic policy, on page 27</a> |

### Types of packet classification

Your router offers several advanced packet classification techniques to manage network traffic effectively. These methods help prioritize and control traffic flows based on network parameters. They ensure efficient resource use and QoS adherence. Also see [Packet classification systems on your router, on page 16](#).

Classification identifies traffic, while marking writes QoS values. For marking actions, see [Packet marking, on page 37](#).

**Table 6: Packet Classification Types**

| Classification technique | What it is                                                                                                                                                    | Used for                                                                                              |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| IP Precedence            | A 3-bit field in the IPv4 header, allowing traffic to be categorized into one of 8 priority levels, ranging from 0 (lowest priority) to 7 (highest priority). | Prioritizing traffic at the network edge when basic prioritization (like voice or video) is required. |

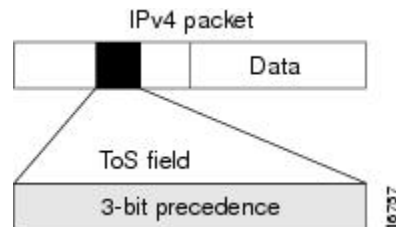
| Classification technique                  | What it is                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Used for                                                                                                                                                                   |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Differentiated Services Code Point (DSCP) | <p>A 6-bit field in the IP header used to classify traffic more granularly than IP precedence.</p> <p>Common DSCP Values are:</p> <ul style="list-style-type: none"> <li>• Expedited Forwarding (EF): For real-time traffic such as voice and video (low latency)</li> <li>• Assured Forwarding (AF): For assured delivery of traffic, with different drop priorities.</li> <li>• Best Effort: For traffic that can be dropped during congestion if needed.</li> </ul> | Fine-grained control over traffic, especially in large-scale networks where different types of traffic require varied treatment, such as real-time voice versus bulk data. |
| Experimental (EXP) bits in MPLS           | A part of the MPLS label which uses 3 bits, allowing for 8 levels of priority.                                                                                                                                                                                                                                                                                                                                                                                         | MPLS VPNs and traffic engineering where precise control over traffic routing and prioritization is required.                                                               |
| Priority Code Point (PCP)                 | A 3-bit field in the Class of Service (CoS) portion of an Ethernet frame. It is used to prioritize Layer 2 (Ethernet) traffic. PCP values range from 0 to 7, with 0 being the lowest and 7 being the highest priority.                                                                                                                                                                                                                                                 | Ethernet networks and VLANs where Layer 2 traffic needs to be prioritized, such as voice traffic in a VLAN.                                                                |
| Drop Eligibility Indicator (DEI)          | A part of the CoS field in the Ethernet header which indicates whether a packet is eligible to be dropped during congestion.                                                                                                                                                                                                                                                                                                                                           | Ethernet networks where low-priority traffic (such as best-effort data) should be dropped first during periods of congestion.                                              |
| QoS group                                 | A technique to classify traffic based on QoS group numbers.                                                                                                                                                                                                                                                                                                                                                                                                            | Grouping traffic flows for specialized handling or service-level agreements                                                                                                |
| Access Control Lists (ACLs)               | A technique to classify traffic by matching specific attributes such as IP address, protocol type, and port numbers.                                                                                                                                                                                                                                                                                                                                                   | Custom traffic filtering or when you need to define very specific conditions for traffic classification.                                                                   |

The sections that follow focus on IP Precedence as an illustration of how typical packet classification techniques work.

## IP Precedence to prioritize traffic

IP Precedence is a method for prioritizing network traffic by assigning a priority level to IP packets. It uses the first 3 bits of the Type of Service (ToS) field in the IPv4 header, allowing for 8 different priority levels (0-7). Each precedence level has a corresponding name defined in RFC 791.

*Figure 2: Type of Service field in IPv4 header*



## Advantages of IP Precedence

- **Differentiated Services (DiffServ):** You can assign higher priority to critical traffic (for example, VoIP) and lower priority to less time-sensitive traffic. You can do this by setting precedence levels and combining them with QoS queuing features to ensure service differentiation.
- **End-to-end QoS:** When you set IP Precedence at the edge of your network, core network devices can enforce QoS policies based on those markings, ensuring consistent prioritization across the network.
- **Integration with QoS features:** You can use IP Precedence with features like Low Latency Queuing (LLQ) and traffic shaping to manage congestion and bandwidth allocation.
- **Layer 2 mapping:** You can map IP Precedence (Layer 3) values to Class of Service (CoS) values at Layer 2 (in the 802.1Q VLAN tag), extending QoS policies across different network segments.

### Considerations for deploying IP Precedence in packet classification

- **Edge deployment:** IP Precedence is usually deployed as close to the edge of the network or administrative domain as possible. This allows core network devices to implement QoS based on the precedence already set.
- **Reserved values:** IP Precedence bit settings 6 and 7 are reserved for network control traffic, such as for routing updates. These values must not be used for user traffic.
- **Class-based marking and LLQ:** Class-based unconditional packet marking and Low Latency Queuing (LLQ) features can use IP Precedence to classify and prioritize traffic.

## How IP Precedence Works

### Summary

The key components involved in applying IP Precedence are:

- **Edge router:** Classifies and marks incoming traffic by setting the IP Precedence value in the packet header.

- Policy map: Defines QoS policies that specify how traffic is handled based on its precedence level.
- Network devices (such as core routers and switches): Reference the IP Precedence value to enforce traffic handling policies such as prioritization, bandwidth allocation, and congestion management.

The IP Precedence process involves marking packets at the edge of the network and applying QoS policies throughout the network path. The edge router assigns a precedence value to each packet based on policy configurations. This value is used by downstream devices to prioritize traffic, allocate bandwidth appropriately, and manage congestion.

Higher-precedence traffic is given preferential treatment in queuing and transmission, ensuring that critical applications—such as voice and video—are delivered with lower latency and better performance.

The process occurs in real time as packets are received and forwarded through the network. This ensures consistent traffic prioritization from edge to core.

### Workflow

These stages describe the IP Precedence classification process:

1. Edge router sets the IP Precedence value in the packet's IPv4 header based on configured classification policies.
2. Policy map defines how traffic should be treated based on the assigned IP Precedence value. This includes prioritization, bandwidth allocation, and congestion behavior.
3. Network devices (such as core routers and switches) read the IP Precedence value and determine the appropriate handling of the packet:
  - a. Prioritize higher-precedence packets in the queue.
  - b. Allocate bandwidth as defined by the policy map.
  - c. Drop lower-precedence packets first during congestion.
4. All network devices continue to forward packets through the network, consistently applying QoS actions based on the IP Precedence value set at the edge.

## Packet classification systems on your router

Your router uses two packet classification systems to determine traffic marking and policy application.

| Traffic direction | Classification system                                                                        |
|-------------------|----------------------------------------------------------------------------------------------|
| Ingress           | QoS map and Ternary Content Addressable Memory (TCAM) are used to classify incoming traffic. |
| Egress            | Only QoS map is used to classify outgoing traffic.                                           |

### Selection of classification system

Your router determines the classification system based on the match conditions you configure in the QoS policy.

- The router uses the QoS map when the policy matches on any of these fields:
  - DSCP
  - EXP
  - PCP
  - DEI
  - QoS group
- If the match conditions do not include these fields, the router uses TCAM for classification.

Ternary Content Addressable Memory (TCAM) extends the concept of the Content Addressable Memory (CAM) table.

The table compares CAM and TCAM by match capabilities, use cases, and advantages in traffic classification.

**Table 7: Comparison of CAM and TCAM**

| Attribute  | CAM                                               | TCAM                                                 |
|------------|---------------------------------------------------|------------------------------------------------------|
| Match type | Exact match (0 or 1)                              | Ternary match (0, 1, or a wild card or “don’t care”) |
| Use case   | MAC address lookup, switch port mapping           | QoS policy classification, ACLs                      |
| Advantage  | Enables fast lookups for simple key-value queries | Supports complex and flexible matching conditions    |

## DiffServ tunneling modes

A DiffServ tunneling mode is a QoS packet-handling model that

- defines how inner and outer traffic markings are interpreted as packets traverse a network core
- specifies which header drives classification and queuing decisions, and
- influences how end-to-end service differentiation is maintained or isolated across domains.

The supported DiffServ tunneling modes are: Uniform mode, Pipe mode, and Short-Pipe mode. Tunneling modes influence which values become available for marking but do not perform marking. Explicit marking requires policy actions.

### Uniform mode

Uniform mode enables a consistent QoS model across interconnected domains. When a packet enters a tunnel, the marking from the inner header is copied to the outer header. When the packet exits the tunnel, the marking is restored to the inner header. This ensures that classification uses the same QoS value end-to-end, regardless of whether the packet is encapsulated.

### Pipe mode

Pipe mode provides QoS isolation between domains. Within the tunnel, only the outer header marking is used for classification and queuing. The inner marking is preserved for use at the tunnel edge but is not used inside the core unless a policy explicitly copies or rewrites it.

### Short-pipe mode

Inside the tunnel, forwarding uses the outer header marking. When the packet exits the tunnel, classification and queuing decisions at the egress device are based on the inner header marking, not the outer one. This mode is useful when customer or application markings must be honored at the network edge.

Short-pipe mode is not supported in all releases. See [Short-pipe mode, on page 18](#) for support details.

DiffServ tunneling modes may preserve or expose QoS marking fields, but they are not marking mechanisms. Marking is performed only when explicit policy-map commands, such as **set dscp**, **set cos**, or **set mpls experimental**, are used. To learn about marking commands and how they are applied, see [Packet marking, on page 37](#).

## Short-pipe mode

Short-pipe mode is a QoS classification mechanism that

- uses the Differentiated Services Code Point (DSCP) value from the inner IP header to determine packet treatment at the egress router
- applies per-hop behavior (PHB) based on the original IP packet markings after MPLS decapsulation, and
- preserves end-to-end service differentiation for user and application traffic as it exits the MPLS core.

Short-pipe mode is especially relevant for networks that use VPNs or tunnels, where traffic needs to be classified and prioritized based on original application or user markings instead of MPLS label markings imposed in the provider core.

**Table 8: Feature History Table**

| Feature Name    | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Short-pipe mode | Release 25.4.1      | <p>Introduced on Fixed Systems (8200 [ASIC:Q200], Centralized Systems (8600 [ASIC:Q200]), Modular Systems (8800 [LC ASIC: Q200])</p> <p>Short-pipe mode ensures that your device applies QoS policies only to customer traffic, excluding network overhead such as tunnel headers or the MPLS encapsulation header. This helps you achieve fairer bandwidth allocation and better prioritization of user data, especially in service provider or large-scale enterprise networks.</p> |

## How Short-Pipe mode for QoS classification works

In short-pipe mode, QoS classification and policy decisions on the egress Provider Edge (PE) router are based on the original IP packet's DSCP value, rather than the MPLS EXP bits. This approach allows end-to-end QoS to be preserved for user traffic as it traverses the MPLS core. The workflow diagram illustrates a typical MPLS VPN setup using short-pipe mode, with traffic flowing from CE1 (Customer Edge) through PE and P routers across an Internet Service Provider (ISP) core to CE2.

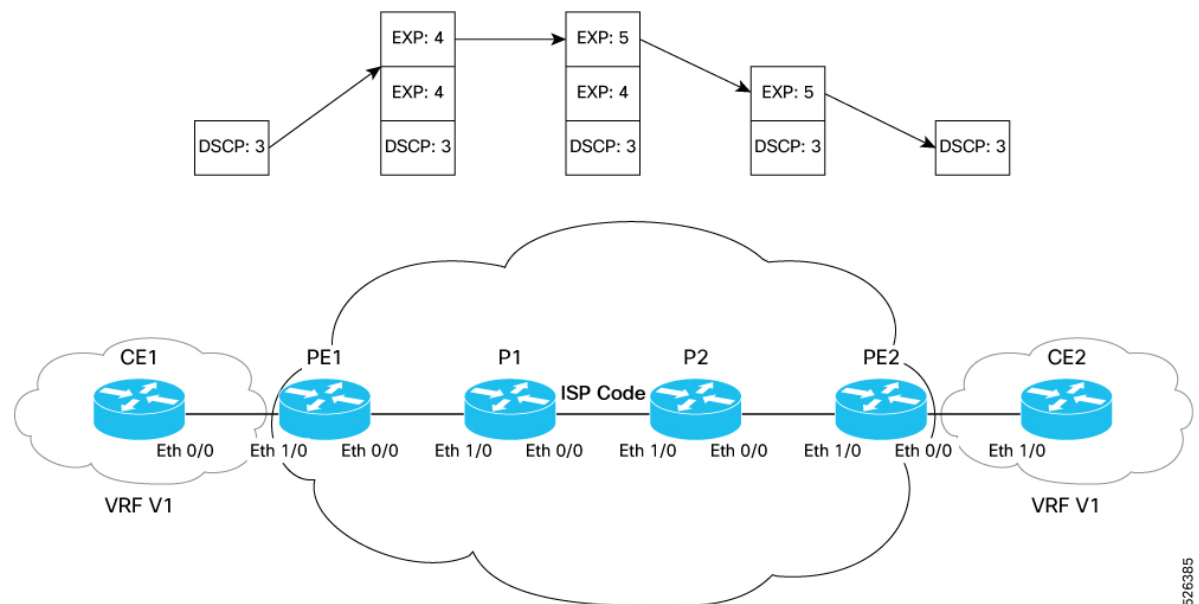
### Summary

Key components involved in short-pipe mode QoS classification are:

- Customer Edge (CE) Routers (CE1 and CE2 in the figure): Devices connecting customer networks to the MPLS VPN.
- Provider Edge (PE) Routers (PE1 and PE2): Routers at the boundary of the MPLS provider core, responsible for label imposition (ingress) and disposition (egress).
- Provider (P) Routers: Core routers that forward labeled packets across the MPLS network.
- Packets: Each packet may have an original DSCP value in its IP header and MPLS labels added by PE routers.

### Workflow

**Figure 3: Traffic flow in short-pipe mode showing inner DSCP preservation across the MPLS core**



These stages describe how short-pipe mode for QoS classification works. Refer to the diagram for visual reference.

1. The CE1 router sends a packet into the network: The packet is marked with a DSCP value (such as DSCP:3) corresponding to its QoS requirements.

2. The ingress PE1 router receives the packet: PE1 imposes an MPLS label stack onto the packet. The top label carries the EXP value (such as EXP:4) mapped from the DSCP value.
3. The packet traverses the MPLS core (P1 and P2): P routers may remark the EXP value as needed (such as EXP:5) to reflect new QoS policies. The EXP value is used to schedule and prioritize the packet at each hop.
4. At the penultimate router (P2), the top MPLS label (with EXP:5) is removed (penultimate hop popping or PHP): This action exposes the next label in the stack (with EXP:4). The packet is then forwarded to the egress PE2 router.
5. At the edge (PE2), the remaining MPLS label is removed (label disposition): The original DSCP value (or QoS marking) is restored or mapped as needed.
6. The CE2 router receives the packet with the intended QoS treatment intact.

## Best practices for configuring short-pipe mode

### Account for system-wide behavior when enabling short-pipe mode

Be aware of these points while enabling short-pipe mode:

- Ensure that enabling short-pipe mode applies the configuration system-wide. All interfaces will adopt short-pipe behavior.
- Pipe mode is the default tunneling behavior for Layer 3 VPN. The router uses pipe mode unless short-pipe mode is explicitly enabled.
- Disabling short-pipe mode reverts the router to pipe mode.
- No system reload is required after enabling or disabling short-pipe mode.

### Apply only DSCP-based classification for decapsulated packets

Take note of these important points for classification and matching actions in short-pipe mode.

- Use only DSCP-based classification for MPLS decapsulation packets. Matching on the MPLS EXP (outer header) value is not supported.
- Do not use complex QoS matching criteria, such as ACL-based matching, with short-pipe mode enabled.

### Avoid ingress remarking when using short-pipe mode

Short-pipe mode does not rewrite DSCP or EXP values. DSCP and other QoS values cannot be remarked or modified on ingress.

### Apply short-pipe mode only to MPLS-to-IP flows

Use short-pipe mode only for MPLS-to-IP flows. The feature does not apply to MPLS flows with underlay headers other than IP.

### Ensure an ingress QoS policy is applied on all core-facing interfaces

Short-pipe mode requires an ingress QoS policy for correct classification behavior.

### Verify hardware support before enabling short-pipe mode

The feature is supported on Cisco 8000 Series Routers with Cisco Silicon One Q200 ASICs.



## Use short-pipe mode only for L3VPN deaggregation flows

Short-pipe classification activates when the PE router removes the VPN label. The decapsulated IP packet is mapped to the appropriate Virtual Routing and Forwarding (VRF) instance for routing and DSCP-based short-pipe classification. This deaggregation step is required for the router to use the inner DSCP value for per-hop behavior (PHB).

## Configure per-VRF label allocation to support short-pipe operation

Short-pipe mode works on a per-VRF basis and requires MPLS label allocation in per-VRF mode. Ensure that the MPLS configuration includes **label mode per-vrf** so that each VRF receives its own VPN label for correct short-pipe classification at egress.

# Enable short-pipe mode for MPLS to IP traffic

Enable short-pipe mode so that QoS classification for MPLS-to-IP flows is based on the DSCP value in the IP header at the egress router.

Use this feature when the egress interface of your device should classify and queue IP packets according to their DSCP value, rather than the MPLS EXP value, after the MPLS label is removed. This approach enables precise service differentiation for tunneled or VPN traffic.

### Before you begin

- Short-pipe mode addresses QoS classification at the decapsulation edge. Complete all standard tunnel and VPN setup steps before you enable short-pipe mode. For comprehensive instructions on configuring MPLS tunnels, L3VPNs, or other foundational network elements, refer to the [Cisco IOS XR MPLS and L3VPN Configuration Guides](#) for your router. These guides provide details on:
  - setting up MPLS tunnels
  - configuring VRFs and PE-CE connections, and
  - applying interface and routing policies.
- If your QoS set up requires rewriting DSCP or EXP values, see [Packet marking, on page 37](#).

Follow these steps to enable short-pipe mode for MPLS-to-IP traffic:

### Procedure

**Step 1** Enter configuration mode on your router.

**Example:**

```
Router#config
```

**Step 2** Enable short-pipe mode system-wide.

**Example:**

```
Router#hw-module profile qos mode l3vpn-short-pipe
Router#commit
```

The router does not require a reload after enabling this mode.

**Step 3** Confirm the **l3vpn-short-pipe** mode is enabled.

**Example:**

```
Router#show hw-module profile qos mode
```

| Knob                      | Status     | Applied | Action |
|---------------------------|------------|---------|--------|
| QOS L3VPN Short Pipe Mode | Configured | Yes     | None   |

The **Status: Configured** and **Applied: Yes** fields indicate that the **l3vpn-short-pipe** mode is enabled and active on the hardware. Enablement of this mode is required for proper operation of QoS classification.

**Step 4** Examine the QoS policy map configuration.

**Example:**

```
Router#show policy-map pmap-name policymap_1_0_0 detail
class-map match-any classmap_1_1_0_0
  match precedence 1
end-class-map
!
class-map match-any classmap_1_2_0_0
  match dscp 16
end-class-map
!
class-map match-all classmap_1_3_0_0
  match dscp 24
end-class-map
!
class-map match-any classmap_1_4_0_0
  match precedence 4
end-class-map
!
class-map match-all classmap_1_5_0_0
  match precedence 5
end-class-map
!
class-map match-all classmap_1_6_0_0
  match dscp 48
end-class-map
!
class-map match-any classmap_1_7_0_0
  match dscp 56
end-class-map
!
class-map match-any class-default
end-class-map
!
policy-map policymap_1_0_0
  class classmap_1_1_0_0
    set traffic-class 1
    set qos-group 1
  !
  class classmap_1_2_0_0
    set traffic-class 2
    set qos-group 2
  !
  class classmap_1_3_0_0
    set traffic-class 3
    set qos-group 3
  !
  class classmap_1_4_0_0
    set traffic-class 4
    set qos-group 4
```

```

!
class classmap_1_5_0_0
  set traffic-class 5
  set qos-group 5
!
class classmap_1_6_0_0
  set traffic-class 6
  set qos-group 6
!
class classmap_1_7_0_0
  set traffic-class 7
  set qos-group 7
!
class class-default
  set traffic-class 0
  set qos-group 0
!
end-policy-map
!

```

This example confirms that the policy map **polycymap\_1\_0\_0** contains class maps that match packets based on DSCP or IP precedence values. For each matching class, the policy sets the appropriate traffic-class and qos-group, which determine how the router applies internal QoS handling, queuing, and marking decisions. The **class-default** entry ensures that all unmatched traffic is assigned a default internal forwarding class.

**Step 5** Verify active traffic classification on the ingress interface.

**Example:**

```
router#show policy-map type qos interface HundredGigE0/0/0/0 input pmap-name polycymap_1_0_0
```

```
HundredGigE0/0/0/0 input: polycymap_1_0_0
```

```

Class classmap_1_1_0_0
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched                    :      459380561/178674499066      9921297
    Transmitted                 :      459380561/178674499066      9921297
    Total Dropped               :                0/0                0
Class classmap_1_2_0_0
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched                    :      459380732/178674790252      9921296
    Transmitted                 :      459380732/178674790252      9921296
    Total Dropped               :                0/0                0
Class classmap_1_3_0_0
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched                    :      459380862/178674965756      9921295
    Transmitted                 :      459380862/178674965756      9921295
    Total Dropped               :                0/0                0
Class classmap_1_4_0_0
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched                    :      459380988/178674990652      9921314
    Transmitted                 :      459380988/178674990652      9921314
    Total Dropped               :                0/0                0
Class classmap_1_5_0_0
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched                    :      459381140/178675150032      9921295
    Transmitted                 :      459381140/178675150032      9921295
    Total Dropped               :                0/0                0
Class classmap_1_6_0_0
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched                    :      459381249/178675130694      9921287
    Transmitted                 :      459381249/178675130694      9921287
    Total Dropped               :                0/0                0
Class classmap_1_7_0_0

```

```

Classification statistics      (packets/bytes)      (rate - kbps)
Matched                      :      459381373/178675175370      9921284
Transmitted                  :      459381373/178675175370      9921284
Total Dropped                :                0/0                0
Class class-default
Classification statistics      (packets/bytes)      (rate - kbps)
Matched                      :      459381499/178675117678      9921007
Transmitted                  :      459381499/178675117678      9921007
Total Dropped                :                0/0                0
Policy Bag Stats time: 1761564022245 [Local Time: 10/27/25 11:20:22.245]

```

This example confirms that the ingress PE classifies L3VPN traffic using the DSCP or precedence values of inner IP packets. Non-zero **Matched** counters for class maps with **match dscp** or **match precedence** show that classification is occurring after the MPLS VPN label is removed and the packet is delivered into the VRF instance. Because the router is not using the MPLS EXP field for classification, this behavior indicates that **l3vpn-short-pipe** mode is enabled and that the PE router is applying QoS decisions based on the restored or mapped DSCP value following decapsulation.

## Traffic classes in QoS

The purpose of traffic classes is to define the criteria for identifying a specific type of traffic. Each class includes a name, match conditions, and logic to evaluate those match conditions. Traffic that matches a defined class is assigned to that class and treated according to the associated QoS policy.

### Components of a traffic class

| Component              | Description                                                            |
|------------------------|------------------------------------------------------------------------|
| Name                   | The identifier assigned to the traffic class.                          |
| Match commands         | The criteria used to classify packets                                  |
| Match evaluation logic | Specifies whether the packet must match any or all defined conditions. |

### Packet classification behavior

When a packet arrives at the router, it is compared against the match commands defined in the traffic class. This table describes how the router classifies packets.

| Classification case                | Behavior                                                                                                                                                                                      |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Packet matches class criteria      | <ul style="list-style-type: none"> <li>The packet is considered a member of that class, and</li> <li>it is handled according to the QoS settings in the associated traffic policy.</li> </ul> |
| Packet does not match any criteria | It is assigned to the default traffic class, which applies default forwarding behavior.                                                                                                       |

## Traffic class configuration criteria

### Match conditions

- You can specify multiple values for the same match type in a single line of configuration. If the first value doesn't match, the next value in the match statement is considered for classification.
- You can use the **not** keyword with a match command to classify traffic that does not match specific values.
- Although all match commands are optional, you must configure at least one match criterion to define a valid traffic class.

### Supported match types

| Match Type Supported               | Min, Max       | Max Entries          | Support for Match NOT | Support for Ranges | Direction Supported on Interfaces |
|------------------------------------|----------------|----------------------|-----------------------|--------------------|-----------------------------------|
| IPv4 DSCP<br>IPv6 DSCP             | (0,63)         | 64                   | Yes                   | Yes                | Ingress                           |
| DSCP                               |                |                      |                       |                    | Egress                            |
| IPv4 Precedence<br>IPv6 Precedence | (0,7)          | 8                    | Yes                   | No                 | Ingress                           |
| Precedence                         |                |                      |                       |                    | Egress                            |
| MPLS<br>Experimental<br>Topmost    | (0,7)          | 8                    | Yes                   | No                 | Ingress<br>Egress                 |
| Access-group                       | Not applicable | 8                    | No                    | Not applicable     | Ingress                           |
| Match<br>qos-group                 | (1-31)         | 7 +<br>class-default | No                    | No                 | Egress                            |
| Protocol                           | (0, 255)       | 1                    | Yes                   | Not applicable     | Ingress                           |
| CoS                                | (0,7)          | 8                    | Yes                   | No                 | Ingress and Egress                |
| DEI                                | (0,1)          | 2                    | Yes                   | No                 | Ingress and Egress                |

### Match logic

- The default behavior is **match-any**, where the packet must match at least one of the specified match statements for classification.
- When you configure **match-all**, the packet must match all specified criteria to be considered part of the traffic class.

- Although all match commands are considered optional, you must configure at least one match criterion to define a valid traffic class.

### ACL considerations

- The **match access-group** command does not support matching on packet length or time-to-live (TTL) fields.
- When used in a class map, the ACL **deny** action is ignored. Traffic is classified solely on the basis of a match.

### Egress classification rules

- These match types are valid only in the egress direction:
  - **match qos-group**
  - **match traffic-class**
  - **match dscp**
  - **match precedence**
  - **match mpls experimental (EXP)**
- The **class-default** implicitly matches qos-group 0 traffic.
- You can classify traffic into up to seven distinct groups using **match qos-group**, with supported values ranging from **qos-group 1** to **qos-group 31**.
- You cannot configure match **qos-group 0**. In egress policies, **qos-group 0** traffic is automatically matched by **class-default**.

### Multicast and egress QoS handling

- On your routers, multicast and unicast traffic follow different paths but converge at the egress in a fixed 20:80 multicast-to-unicast ratio per interface.
- Egress QoS for multicast traffic treats traffic classes 0–5 as low-priority and traffic classes 6–7 as high priority. This prioritization behavior is not user-configurable.
- Egress shaping does not apply to multicast traffic in high-priority traffic classes. Shaping is only effective for unicast traffic in these classes.

### Ingress and egress traffic class considerations

- If you assign a traffic class at ingress and do not define a matching class at egress, then the traffic will not be counted in the class-default at the egress policy map.
- Only traffic class 0 maps to the egress class-default. Any non-zero traffic class assigned at ingress, but not mapped to an egress class or queue, will not be included in either the default class or any other class at egress.

### Default traffic class behavior

- Packets not matching any explicitly user-defined class are assigned to the default class (**class-default**.)
- The default class always exists, even if not explicitly configured.
- If no QoS actions are configured for the default class, packets receive no treatment—no marking, policing, shaping, or prioritization.
- To apply QoS to unclassified traffic, configure actions under the default class in a policy map.

### Layer 2 classification using Layer 3 headers

From Cisco IOS XR Release 7.2.12, classification on Layer 2 transport interfaces using Layer 3 header fields is supported. This capability is limited to physical and bundle main interfaces and does not apply to sub-interfaces.

## Elements and supported actions of a traffic policy

Traffic policies define the QoS actions for traffic that matches one or more classes. These policies enforce how classified traffic is marked, shaped, policed, or prioritized on a router.

### Elements of a traffic policy

A traffic policy contains these elements:

- Name: Identifies the traffic policy. The name is referenced when you apply the policy to an interface.
- Traffic class association: Specifies one or more traffic classes (defined using class-map) to include in the policy. The Modular QoS CLI (MQC) allows multiple traffic classes within a single policy.
- QoS actions: Defines the actions to be applied to each traffic class. Supported actions include marking, policing, shaping, prioritizing, and queuing traffic.

### Order of traffic class entries in a policy map

The order of class entries in a policy map determines how hardware evaluates the match rules.

- Class match rules are programmed into the TCAM in the order specified in the policy map.
- If a packet matches multiple classes, only the first matching class is used and only its configured actions are applied.

### Number of classes per policy map

You can configure:

- up to 8 traffic classes per ingress policy map, and
- up to 8 traffic classes per egress policy map.

### Supported QoS actions in a traffic policy map

In the MQC, you apply specific QoS actions in a policy-map after classifying traffic via class-map.

The table

- outlines the supported actions
- specifies the direction (ingress or egress) in which each action can be applied and,
- helps prevent configuration errors by guiding you to apply only valid combinations.

**Table 9: QoS actions supported in policy maps**

| QoS action                 | Direction          | Notes                                                                                                                                                                                                                                                                                                    |
|----------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>bandwidth remaining</b> | Egress             | Configures remaining bandwidth allocation for a class.                                                                                                                                                                                                                                                   |
| <b>mark</b>                | Ingress and egress | See <i>Packet Marking</i> for marking options.                                                                                                                                                                                                                                                           |
| <b>police</b>              | Ingress            | Applies a traffic policing action.                                                                                                                                                                                                                                                                       |
| <b>priority</b>            | Egress             | Supports priority levels 1 to 7.                                                                                                                                                                                                                                                                         |
| <b>queue-limit</b>         | Egress             | Sets a threshold for queue depth.                                                                                                                                                                                                                                                                        |
| <b>shape</b>               | Egress             | Configures traffic shaping per class.                                                                                                                                                                                                                                                                    |
| <b>red</b>                 | Egress             | Enables Random Early Detection (RED). Supports discard-class values: <ul style="list-style-type: none"> <li>• 0—typically associated with default or lower-priority discard behavior.</li> <li>• 1—often used for slightly higher-priority traffic, still eligible for drop under congestion.</li> </ul> |

## Configure packet classification and specify QoS actions

Use this task to classify traffic into user-defined classes and apply QoS actions such as traffic shaping. This helps manage bandwidth and prioritize traffic on an interface.

To classify and shape outgoing traffic, you apply a traffic policy to an interface. Classification is based on match criteria, such as QoS groups. Shaping controls the transmission rate of classified traffic.

### Before you begin

Determine the match criteria for classifying traffic and the QoS actions based on the interface bandwidth.



## Procedure

**Step 1** Create a class map and specify the match criteria.

**Example:**

```
Router#configure
Router(config)#class-map match-any qos-1
```

This example creates a class map named **qos-1** and specifies that traffic is matched if any of the defined conditions are met.

**Step 2** Match traffic with QoS group 1 to the **qos-1** class.

**Example:**

```
Router(config-cmap)#match qos-group 1
Router(config-cmap)#end-class-map
Router(config-cmap)#commit
```

QoS groups are internal markers that may be set based on ACLs, routing protocols, or other criteria.

**Step 3** Create a policy map named **test-shape-1**.

**Example:**

```
Router(config)#policy-map test-shape-1
```

**Step 4** Bind or associate the **qos-1** class to the policy map.

**Example:**

```
Router(config)#policy-map test-shape-1
```

**Step 5** Specify QoS actions for the traffic class.

**Example:**

```
Router(config-pmap-c)#shape average percent 40
Router(config-pmap-c)#exit
Router(config-pmap)#end-policy-map
```

This example configures the policy to shape traffic in the **qos-1** class to 40% of the interface bandwidth.

**Step 6** Attach the traffic policy to an interface.

**Example:**

```
Router(config)#interface fourHundredGigE 0/0/0/2
Router(config-int)#service-policy output test-shape-1
Router(config-int)#commit
```

This example applies the policy map to the egress-direction of interface **FourHundredGigE 0/0/0/2**.

**Step 7** Verify policy configuration and traffic statistics.

**Example:**

```
Router# show qos interface fourHundredGigE 0/0/0/2 output
```

NOTE:- Configured values are displayed within parentheses

```
Interface FourHundredGigE0/0/0/2
  ifh 0x80000080 -- output policy
```

## Configure QoS actions for the default class

```

NPU Id:                0
Total number of classes: 2
Interface Bandwidth:    400000000 kbps
Policy Name:            test-shape-1
Accounting Type:        Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class
=
    Level2 Class        qos-1

Level1 Class
=
    class-default

Node 0/0/CPU0, Interface FourHundredGigE0/0/0/2 Ifh 0x80000080 (FourHundredGigE) -- output policy
NPU Id:                0
Total number of classes: 2
Interface Bandwidth:    400000000 kbps
Policy Name:            test-shape-1
Accounting Type:        Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class
=
    qos-1
    Shape Average Rate(kbps)    160000000 (40%)
    Queue ID                    50331648 (LP queue)
    Tail Drop Packets           0
    Tail Drop Bytes             0

Level1 Class
=
    class-default
    Queue ID                    50331647 (LP queue)
    Tail Drop Packets           0
    Tail Drop Bytes             0

```

The **Shape Average Rate (kbps) 160000000 (40%)** line shows the configured shaping rate (160 Gbps), which accounts for 40% of the 400 Gbps interface bandwidth.

## Configure QoS actions for the default class

Use this task to apply a QoS marking action to unclassified traffic (the class-default class) in a traffic policy.

Traffic that does not match any user-defined class is automatically treated as part of the **class-default** class. Applying actions to this class ensures that even unclassified traffic receives appropriate QoS treatment, such as DSCP marking.

Use these steps to configure QoS actions for the default class.

### Before you begin

Ensure that no other policy is applied to the same interface that would conflict with this default class policy.

### Procedure

**Step 1** Define a traffic policy and assign the default class.

**Example:**

```
Router# configure
Router(config)# policy-map egress_policy1
Router(config-pmap)# class class-default
```

This step creates a policy map named **egress\_policy1** and selects the **class-default** class, which handles all unmatched traffic.

**Step 2** Set the QoS action—in this example, the DSCP value—for the default class.

**Example:**

```
Router(config-pmap-c)# set dscp default
Router(config-pmap-c)# exit
Router(config)# commit
```

This step marks all unclassified traffic with the DSCP value default (equivalent to **DSCP 0**), which typically indicates best-effort service.

---

The policy map **egress\_policy1** is created, and unclassified traffic is marked with **DSCP 0** when the policy is applied to an interface.

## Additional Guidelines for Packet Classification and QoS Actions

- Hierarchical policy limitation: You cannot nest one policy map inside another or reference a policy map within another policy. Hierarchical policies are not supported.
- Traffic rate accuracy: Traffic rate estimates in QoS statistics are approximations based on an exponential decay filter and may not reflect precise values.
- BGP Flowspec impact: When a Flowspec police rule and a QoS policy apply to the same ingress traffic, the Flowspec action takes precedence. This may cause QoS counters for the policy map to display zero or incorrect values.

## Enhanced Running Configuration Display for Policy Maps and Class Maps

Enhanced running configuration displays are CLI output improvements that

- display each class map or policy map configuration instance on a separate line
- improve readability of QoS configurations, and
- aid in easier verification and troubleshooting.

Table 10: Feature History Table

| Feature Name                                                          | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enhanced Running Configuration Display for Policy Maps and Class Maps | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is now supported on Cisco 8011-4G24Y4H-I routers.                                                                                                                                                                                                                                                    |
| Enhanced Running Configuration Display for Policy Maps and Class Maps | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 8712-MOD-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |
| Enhanced Running Configuration Display for Policy Maps and Class Maps | Release 24.2.11     | You can view each class map or policy map running configuration instance on a separate line.<br><br>The feature modifies the output display of this command:<br><br>CLI: <b>show run formal</b>                                                                                                                                                                                                                   |

Before this enhancement, the running configuration outputs for policy maps and class maps were compressed and difficult to read. With this enhancement, the **show run formal** command outputs each QoS component—such as class maps and their matches, or policy maps and their associated classes—on dedicated lines.

### Running Configuration Example

The configuration example associates policy map **p1** with the class maps **DSCP** and **MPLS**.

After the configuration, its **show run formal** running configuration is displayed.

```
/* class map DSCP */

Router# config
Router(config)# class-map match-any DSCP
Router(config-cmap)# match dscp 1
Router(config-cmap)# end-class-map

/* class map MPLS */

Router(config)# class-map match-any MPLS
```

```

Router(config-cmap)# match mpls experimental topmost 2
Router(config-cmap)# end-class-map

/* DSCP-to-pl association */

Router(config)# policy-map p1
Router(config-pmap)# class DSCP
Router(config-pmap-c)# bandwidth remaining ratio 80
Router(config-pmap-c)# root

/* MPLS-to-pl association */

Router(config)# policy-map p1
Router(config-pmap)# class MPLS
Router(config-pmap-c)# bandwidth remaining ratio 60
Router(config-pmap-c)# exit
Router(config-pmap)# end-policy-map
Router(config)# commit

```

### Verification

The **show run formal** running configuration displays each class map and policy map running configuration instance on a separate line.

```

Router# show run formal
..
class-map match-any DSCP match dscp 1
class-map match-any MPLS match mpls experimental topmost 2
policy-map p1 class DSCP bandwidth remaining ratio 80
policy-map p1 class MPLS bandwidth remaining ratio 60
..

```

**show run** and **show run formal** running configuration comparison:

| <b>show run</b> displays indented configuration                                                                                                                                                                                                                                                                  | <b>show run formal</b> displays a running configuration instance on a separate line                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class-map match-any DSCP   match dscp 1   end-class-map ! class-map match-any MPLS   match mpls experimental topmost 2   end-class-map ! policy-map p1   class DSCP     bandwidth remaining ratio 80   !   class MPLS     bandwidth remaining ratio 60   !   class class-default   ! end-policy-map </pre> | <pre> class-map match-any DSCP match dscp 1 class-map match-any MPLS match mpls experimental topmost 2 policy-map p1 class DSCP bandwidth remaining ratio 80 policy-map p1 class MPLS bandwidth remaining ratio 60 </pre> |

**Table 11: Configuration output before and after enhancement**

| Display aspect   | Before enhancement (show run command) | After enhancement (show run formal) |
|------------------|---------------------------------------|-------------------------------------|
| Class map output | Compressed or inline                  | Shown on a separate line            |

| Display aspect               | Before enhancement (show run command) | After enhancement (show run formal)      |
|------------------------------|---------------------------------------|------------------------------------------|
| Policy map class association | Merged into single policy block       | Shown per class, each on a separate line |
| Troubleshooting readability  | Moderate-to-low                       | High                                     |



**Note** Use the **show run formal** command instead of the **show run** command to view the enhanced QoS configuration display with clearly separated class map and policy map lines.

## Multicast traffic scheduling on egress queues

Multicast traffic scheduling on egress queues is a QoS traffic queuing feature that

- schedules multicast traffic in the second pass of egress processing
- applies egress queuing policy parameters such as shaping, priority, and queuing on multicast traffic, and
- enables granular control of multicast traffic per egress queue.

**Table 12: Feature History Table**

| Feature Name                                  | Release Information | Feature Description                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multicast Traffic Scheduling on Egress Queues | Release 25.1.1      | Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)<br><br>This feature is supported on: <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                          |
| Multicast Traffic Scheduling on Egress Queues | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is now supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> </ul> |

| Feature Name                                  | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multicast Traffic Scheduling on Egress Queues | Release 24.3.1      | <p>Introduced in this release on: Modular Systems (8800 [LC ASIC: ASIC: P100])(select variants only*)</p> <p>We have introduced multicast traffic scheduling on egress queues. Now you can have more granular control over multicast traffic by applying egress queuing policy map parameters such as traffic shaping, priority, and queuing, to each egress queue. This allows for specific management of multicast traffic for different receivers, ensuring efficient and prioritized handling of multicast data.</p> <p>*This feature is enabled by default and supported on:</p> <ul style="list-style-type: none"> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

## QoS policy inheritance

QoS policy inheritance is a QoS classification feature that

- applies a single QoS policy on a main interface and automatically enforces it on all attached subinterfaces
- supports all QoS operations including classification, marking, policing, and shaping, and
- supports cumulative policy-map statistics visibility for the main interface that includes its subinterfaces.

Running the [show policymap](#) interface command displays the cumulative statistics for an interface, and these numbers include the subinterfaces as well.

**Table 13: Feature History Table**

| Feature Name           | Release Information | Feature Description                                                                                                                                                                                                                              |
|------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS Policy Inheritance | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul> |

| Feature Name           | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS Policy Inheritance | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-52Y8H-EM</li> <li>• 88-LC1-12TH24FH-E</li> </ul>                                                                                                                                 |
| QoS Policy Inheritance | Release 7.3.15      | <p>To create QoS policies for subinterfaces, you had to apply the policy on each subinterface manually. From this release, all you do is create and apply a single QoS policy on the main interface and the subinterfaces automatically inherit the policy.</p> <p>The inheritance model provides an easily maintainable method for applying policies, enabling you to create targeted policies for a group of interfaces and their subinterfaces. This model saves your time and resources while creating QoS policies.</p> |

## Limitations of the QoS policy inheritance model

- ECN marking and egress marking policies cannot be used simultaneously on a main interface and its subinterfaces.

Avoid configuring ECN-enabled policies and egress marking policies across the same main interface–subinterface hierarchy to prevent ECN marking failures.

- The inheritance model is the default option; policy inheritance cannot be selectively overridden on individual subinterfaces.

To prevent policy inheritance on specific subinterfaces, remove the policy from the main interface and explicitly configure policies on the required subinterfaces.





## CHAPTER 5

# Packet Marking for Priority Differentiation

This chapter describes how Quality of Service (QoS) packet marking enables differentiated traffic treatment by assigning priority values—such as IP precedence, DSCP, Class of Service (CoS), and MPLS Experimental (EXP) bits—to packets, ensuring consistent handling of traffic classes across interfaces and network domains.

- [Packet marking, on page 37](#)
- [Unconditional QoS packet marking, on page 39](#)
- [QoS marking modes for MPLS traffic, on page 39](#)
- [QoS marking for IP-to-MPLS traffic, on page 41](#)
- [QoS marking in tunneling and encapsulation scenarios, on page 45](#)
- [QoS marking on Layer 2 subinterfaces, on page 49](#)

## Packet marking

Packet marking is a QoS traffic differentiation model that

- assigns a priority value to each packet to indicate its relative importance in the network
- embeds this priority value into header fields such as IP precedence, Differentiated Services Code Point (DSCP), Class of Service (CoS), or MPLS Experimental (EXP) bits, and
- enables consistent forwarding treatment and queuing decisions across interfaces and network domains.

You can use packet marking in input policy maps to set or modify the attributes for traffic belonging to a specific class. For example, you can change the CoS value in a class or set DSCP or IP precedence values for a specific type of traffic. These new values are then used to determine how the traffic should be treated.

### Types of packet marking

QoS uses packet and frame markings at different layers of the network stack to identify traffic classes, assign priorities, and influence queuing or drop behavior.

The primary marking categories are organized by the protocol layer at which they operate.

### Layer 3 QoS markings

- Differentiated Services Code Point (DSCP): a six-bit field that defines per-hop behavior for traffic classes.
- IP Precedence: a three-bit field in the IP header that represents eight levels of priority.

- MPLS Experimental (EXP) bits: a three-bit field in the MPLS header that conveys packet priority across MPLS networks.

### Layer 2 QoS markings

- Class of Service (CoS): a three-bit field in the 802.1Q VLAN tag that specifies a priority level (0-7) for a Layer 2 Ethernet frame.
- Priority Code Point (PCP): a three-bit field in the 802.1Q VLAN tag that indicates frame priority at Layer 2.
- Drop Eligibility Indicator (DEI): a one-bit field in the 802.1Q VLAN tag that indicates frame drop precedence.

For more information on the different types of packet marking, refer to the *Types of packet classification* section in the *Classify Packets to Identify Specific Traffic* chapter.

### ECN—a system-controlled marking mechanism

Explicit Congestion Notification (ECN) is not to be confused with QoS packet marking mechanisms such as DSCP, CoS, or MPLS EXP, because ECN does not classify or prioritize traffic. Instead of defining how packets are queued or forwarded, ECN signals network congestion between devices by using two bits in the IP header. Routers and switches set or clear these bits dynamically when congestion is detected, allowing end hosts to reduce transmission rates without dropping packets.

Unlike DSCP or CoS markings, which administrators configure through Modular QoS CLI (MQC) policies to define per-hop behavior and resource allocation, ECN is system-controlled and not user-configurable.

### Packet marking commands

The table lists the packet marking commands available in Modular QoS CLI (MQC) and shows their applicability for ingress and egress traffic directions, supported value ranges, and conditional or unconditional usage.

**Table 14: Packet marking commands for egress and ingress interfaces**

| QoS marking command           | Range | Unconditional marking—applicable direction | Conditional marking—availability |
|-------------------------------|-------|--------------------------------------------|----------------------------------|
| set ip encapsulation          | 0-63  | ingress                                    | No                               |
| set discard-class             | 0-1   | ingress                                    | No                               |
| set dscp                      | 0-63  | ingress, egress                            | No                               |
| set mpls experimental topmost | 0-7   | ingress, egress                            | No                               |
| set precedence                | 0-7   | ingress, egress                            | No                               |
| set qos-group                 | 0-31  | ingress                                    | No                               |
| set cos                       | 0-7   | ingress, egress                            | No                               |
| set dei                       | 0-1   | ingress, egress                            | No                               |

### Default QoS marking behavior for VLAN and MPLS encapsulations

When a packet enters or leaves a router, additional encapsulations such as VLAN tags or MPLS labels may be added to the frame. To maintain consistent QoS behavior, the router applies default marking profiles that specify which CoS or EXP values are written into those encapsulations. These profiles define how unclassified or unmarked traffic is treated, allowing the device to maintain a predictable baseline for scheduling, shaping, and congestion management operations.

During initialization, the system automatically creates one ingress default QoS mapping profile and one egress default QoS mapping profile for each device. These mappings determine the default CoS and EXP field values for all tagged or labeled traffic when no explicit class-based policy overrides them.

## Unconditional QoS packet marking

Unconditional QoS packet marking is a QoS traffic differentiation feature that

- overwrites the packet's existing marking with the value configured in the policy map
- operates independently of policer conformance state or the packet's current header markings, and
- applies through MQC set-actions (for example, setting DSCP, CoS, MPLS EXP, or qos-group) at the configured interface and direction.

Conditional QoS packet marking is a QoS traffic differentiation feature that

- modifies the packet's marking based on specific traffic conditions such as policer conformance results (for example, conform, exceed, or violate actions), and
- allows differentiated treatment of packets depending on whether they meet defined rate or threshold criteria.

## QoS marking modes for MPLS traffic

QoS marking modes for MPLS traffic is a QoS traffic differentiation feature for MPLS traffic that

- determines how the network preserves or rewrites packet priority markings as traffic moves between network domains
- applies to MPLS networks that carry IPv4 or IPv6 packets, defining how DSCP or Class of Service (CoS) values are mapped to or restored from MPLS Experimental (EXP) bits, and
- uses uniform mode to preserve ingress markings across domains and pipe mode to apply domain-specific QoS policies.

**Table 15: Feature History Table**

| Feature Name                             | Release Information | Feature Description                                                                                                                                        |
|------------------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Packet Marking for IPv4 and IPv6 Traffic | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is supported on Cisco 8011-4G24Y4H-I routers. |

| Feature Name                             | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Packet Marking for IPv4 and IPv6 Traffic | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8700) (select variants only*)</p> <p>You can use uniform or pipe modes to implement QoS for IPv4 and IPv6 traffic.</p> <p>*This functionality is now extended to the Cisco 8712-MOD-M routers.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Packet Marking for IPv4 and IPv6 Traffic | Release 24.3.1      | <p>Introduced in this release on: Modular Systems (8800 [LC ASIC: P100]) (select variants only*), Fixed Systems (8200) (select variants only*), Fixed Systems (8700 (P100, K100)) (select variants only*)</p> <p>You can use uniform or pipe modes to implement QoS for IPv4 and IPv6 traffic.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> </ul>                                                                                                                                                                                                                                                                                                        |
| Packet Marking for IPv4 and IPv6 Traffic | Release 24.2.11     | <p>Introduced in this release on: Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>Using uniform or pipe modes, you can implement QoS for IPv4 and IPv6 traffic. Uniform mode maintains DSCP marking for packets across different network segments and is useful where the network policy requires consistent treatment of packets across multiple domains. Pipe mode separates QoS policies between network segments. In this mode, ingress DSCP markings classify and queue the packets, but the packets can remarked before they egress. Such treatment is beneficial when you need to apply one set of QoS policies within your network and a different set when packets exit the network towards your customer.</p> <p>*This feature is supported on 88-LC1-36EH.</p> |

### MPLS QoS marking modes

You can apply QoS marking on IPv4 and IPv6 packets that traverse an MPLS network using uniform or pipe modes.

**Uniform mode:** It's a DiffServ-aware MPLS marking mode that preserves the incoming IP header's DSCP or CoS values throughout the MPLS domain. When packets are label-imposed and later disposed, the same DSCP values are copied into and out of the MPLS EXP field. This approach ensures consistent, end-to-end QoS treatment across provider and customer networks, making it suitable where network domains share a common QoS policy.

**Pipe mode:** It's a DiffServ-aware MPLS marking mode that isolates QoS policies between domains. In this mode, the MPLS domain applies its own EXP-based treatment that may differ from the IP DSCP values received at ingress. The original IP markings are restored (or re-applied) when packets egress, enabling independent QoS control within the MPLS provider network.

This table shows the details for uniform and pipe modes.

**Table 16: Differences between uniform mode and pipe mode**

| Uniform mode                                                                                    | Pipe mode                                                                                                                          |
|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Preserves ingress DSCP or CoS markings across the MPLS domain.                                  | Applies new MPLS EXP markings based on provider QoS policy.                                                                        |
| Provides consistent end-to-end QoS treatment across interconnected network domains.             | Enables independent QoS enforcement within the MPLS provider network.                                                              |
| DSCP values are copied into and out of MPLS EXP fields during label imposition and disposition. | DSCP values are used only at ingress for classification; packets may be remarked at egress.                                        |
| Suitable when customer and provider networks share a common QoS model.                          | Suitable when customer and provider networks maintain distinct QoS models to enable policy separation and administrative autonomy. |

## QoS marking for IP-to-MPLS traffic

QoS marking for IP-to-MPLS traffic is a QoS traffic differentiation feature that

- transfers the DSCP value into the MPLS experimental (EXP) field when IP packets are encapsulated into MPLS
- maintains end-to-end service differentiation across the IP and MPLS domains by aligning per-hop behaviors between IP and label-switched segments, and
- enables consistent QoS treatment for customer traffic traversing MPLS networks through DSCP-to-EXP mapping configured in QoS policies.

This feature ensures that traffic classified at the IP layer retains its intended QoS behavior as it transits an MPLS-enabled core, preserving prioritization and congestion-handling characteristics across heterogeneous domains.

Table 17: Feature History Table

| Feature Name                                            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mark IP DSCP and MPLS EXP Values for IP to MPLS Traffic | Release 7.5.4       | <p>We've provided you with more granular control for IP to MPLS traffic, so you can collect billing information for your customers upstream at the MPLS core and correlate them with subsequent next-hop IP nodes. This helps you accurately bill your customers as they pay for specific services based on their network requirements.</p> <p>Such control is possible because we've enabled the remarking of IP DSCP values in the inner IP header—that remain even after the POP tag removes the outer header—along with the existing ability to remark MPLS EXP values on the outer MPLS header.</p> <p>There's no action required for you to enable this functionality, and there are no changes to command options.</p> |

### QoS behavior for DSCP and EXP marking in IP-to-MPLS encapsulation

In earlier software releases, remarking the DSCP value in an ingress QoS policy for IP-to-MPLS traffic resulted in incomplete propagation of the marked value.

- The inner IP header retained its original DSCP value, preventing the intended QoS action from being applied across the network.
- The outer MPLS header, however, received the remarked value through the EXP bits that reflected the QoS policy action.

Starting from Cisco IOS XR Release 7.5.4, the QoS marking behavior for IP packets transiting an MPLS core has been enhanced.

- During MPLS imposition, the inner IP header now receives the remarked DSCP value from the ingress QoS policy, ensuring that the IP packet carries consistent QoS intent across the network.
- The top MPLS label continues to receive the remarked EXP value, derived from the three most significant bits of the DSCP field.

## Benefits of QoS marking for IP-to-MPLS traffic

The ability to remark IP DSCP and MPLS EXP values at the ingress provides:

- Granular QoS control across multiple hops, ensuring consistent per-hop behavior and prioritization across both IP and MPLS domains.
- Improved service visibility, enabling correlation between traffic treatment in the IP edge and forwarding performance in the MPLS core.
- Enhanced operational efficiency, allowing service providers to collect and align customer-specific billing or usage data with the QoS policies applied at different network layers.

## Limitations for configuring QoS marking for IP-to-MPLS traffic

- This feature is supported only on Cisco Silicon One Q200–based ASICs, available in both fixed and distributed systems.
- In distributed systems, you can use a Cisco Silicon One Q100–based ASIC line card for ingress classification; however, the egress line card must be equipped with a Q200 ASIC.

## Configure QoS marking for IP-to-MPLS encapsulated traffic

### Before you begin

Complete these prerequisites before you begin:

- Ensure MPLS forwarding and label imposition are enabled on the PE router interfaces.
- Verify that class maps and policy maps required for ingress DSCP remarking and egress classification are defined in your configuration plan.
- Confirm that the platform uses Cisco Silicon One Q200–based ASICs for egress functionality.

### Procedure

- Step 1** Create an ingress class map and policy map to remark DSCP values for IP packets before MPLS encapsulation. This configuration sets the DSCP value to AF41 for traffic originally marked as AF31 to define a higher assured forwarding PHB.

#### Example:

```
Router(config)#class-map AF31
Router(config-cmap)#match dscp AF31
Router(config-cmap)#end-class-map
Router(config)#
Router(config)#policy-map INGRESS_REMARK
Router(config-pmap)#class AF31
Router(config-pmap-c)#set dscp AF41
Router(config-pmap-c)#end-policy-map
Router(config)#
Router(config)#interface FourHundredGigE0/0/0/0
Router(config-if)#service-policy input INGRESS_REMARK
Router(config-if)#commit
```

- Step 2** Create an egress class map and policy map to classify packets based on remarked DSCP values.

This configuration classifies incoming traffic as AF31 or AF41 and maps it to the appropriate traffic-class values for forwarding decisions.

**Example:**

```
Router(config)#class-map AF31
Router(config-cmap)#match dscp AF31
Router(config-cmap)#end-class-map
Router(config)#
Router(config)#class-map AF41
Router(config-cmap)#match dscp AF41
Router#end-class-map
Router(config)#
Router(config)#policy-map INGRESS_CLASSIFY
Router(config-pmap)#class AF31
Router(config-pmap-c)#set traffic-class 3
Router(config-pmap-c)#exit
Router(config-pmap)#
Router(config-pmap)#class AF41
Router(config-pmap-c)#set traffic-class 4
Router(config-pmap-c)#exit
Router(config-pmap)#end-policy-map
Router(config)#
Router(config)#int FourHundredGigE0/0/0/1
Router(config-if)#service-policy input INGRESS_CLASSIFY
Router(config-if)#commit
```

**Step 3** Verify DSCP remarking and classification at the egress PE router.

Use the `show policy-map interface` command to verify that traffic marked as AF41 at ingress is now being classified and counted correctly at egress.

**Example:**

```
Router#show policy-map interface FourHundredGigE0/0/0/1 input
```

```
FourHundredGigE0/0/0/1 input: INGRESS_CLASSIFY
```

```
Class AF31
  Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                      :          0/0              0
  Transmitted                  :          0/0              0
  Total Dropped                :          0/0              0
Class AF41
  Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                      : 9813350/13738690000      936847
  Transmitted                  : 9813350/13738690000      936847
  Total Dropped                :          0/0              0
Class class-default
  Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                      :          0/0              0
  Transmitted                  :          0/0              0
  Total Dropped                :          0/0              0
```

The nonzero counters for Class AF41 confirm that the ingress DSCP remarking has been applied successfully and is recognized at the egress PE router.



# QoS marking in tunneling and encapsulation scenarios

This topic introduces how QoS markings are preserved, mapped, or redefined when packets undergo encapsulation across GRE, MPLS, or Segment Routing tunnels.

## QoS marking on GRE tunnels

QoS marking on Generic Routing Encapsulation (GRE) tunnels is a QoS traffic differentiation feature that

- preserves or maps DSCP and Type of Service (ToS) values between inner and outer IP headers when packets are encapsulated or decapsulated
- ensures consistent QoS treatment for tunneled traffic by maintaining class-of-service information across GRE boundaries, and
- coordinates with other encapsulation types, such as MPLS, to copy or derive appropriate QoS field values when MPLS labels coexist with GRE headers.

### Generic Routing Encapsulation (GRE)

GRE is an IP tunneling protocol that encapsulates a wide variety of network layer packets inside IP tunnels. It allows one network to transport packets across another network by adding an outer IP header. GRE is commonly used to create logical point-to-point links between routers, enabling the transport of traffic types that are not natively routable across intermediate networks.

For more information on GRE tunnels, refer to the *Interfaces Configuration Guide for Cisco 8000 Series Routers*.

### Type of Service (ToS)

ToS is an eight-bit field in the IPv4 header that specifies how an IP packet should be handled during transmission. The ToS field includes bits for DSCP and Explicit Congestion Notification (ECN), which together determine packet priority and congestion handling.

**Table 18: Feature History Table**

| Feature Name                                                                  | Release Information | Feature Description                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS Behavior for Generic Routing Encapsulation (GRE) Tunnels: Default Marking | Release 25.1.1      | Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)<br><br>*This feature is supported on: <ul style="list-style-type: none"><li>• 8712-MOD-M</li><li>• 8011-4G24Y4H-I</li></ul> |

| Feature Name                                                                  | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS Behavior for Generic Routing Encapsulation (GRE) Tunnels: Default Marking | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is now supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |
| QoS Behavior for Generic Routing Encapsulation (GRE) Tunnels: Default Marking | Release 7.3.1       | <p>With the support for GRE encapsulation and decapsulation tunnel interfaces, there are some important updates to QoS behavior for GRE tunnels. These updates are applicable for default packet marking and involve Type of Service (ToS) and MPLS experimental bits.</p>                                                                                                                       |

## Guidelines for configuring QoS marking on GRE tunnels

### GRE Encapsulation

During encapsulation of IPv4/IPv6 payload inside the GRE header, QoS behavior is as follows:

- Ingress: QoS supports classification on the payload Layer 3 fields or EXP and remarking payload IP header DSCP.
- Egress: QoS supports setting outer GRE IP header DSCP. It doesn't overwrite the Tunnel Type of Service (ToS) configuration and doesn't remark GRE IP header DSCP.

### GRE Decapsulation

During decapsulation of the outer GRE header (in which the inner IPv4/IPv6/MPLS payload is forwarded to the next-hop router), QoS behavior is as follows:

- Ingress: QoS supports classification on Layer 3 fields of outer GRE using the **set qos-group** command. Setting DSCP on the ingress interface sets DSCP for the inner headers.
- Egress: QoS supports classification using **qos-group** to set DSCP or EXP for egress packets.

## Explicit QoS marking for SRv6 encapsulation

Explicit QoS marking for SRv6 encapsulation is a QoS traffic differentiation feature that

- sets DSCP or Traffic Class values in the outer IPv6 header during SRv6 encapsulation, and
- ensures consistent QoS treatment across SRv6 transport paths by aligning encapsulated markings with class-based policy actions.

Explicit QoS marking feature is also known as packet marking.

**Table 19: Feature History Table**

| Feature Name                          | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Set IP Marking for SRv6 Encapsulation | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is supported on Cisco 8011-4G24Y4H-I routers.                                                                                                                                                                                                                                                            |
| Set IP Marking for SRv6 Encapsulation | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is now supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 8712-MOD-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |
| Set IP Marking for SRv6 Encapsulation | Release 24.2.11     | With this feature support for IP marking for SRv6 packets that are encapsulated, there are some important updates to the QoS behavior.<br><br>This is an explicit packet marking feature that applies only to ingress QoS policies.<br><br><b>CLI:</b> This feature introduces the <a href="#">set ip encapsulation</a> command.                                                                                      |

## Limitations for configuring explicit QoS marking for SRv6 encapsulation

### Configuration constraints

- The explicit packet marking feature is supported only on ingress QoS policies.
- The **set ip encapsulation class-of-service** command is supported only in ingress QoS policies.

- The **set qos-group** configuration cannot be used together with the **set ip encapsulation class-of-service** command.
- The **set ip encapsulation class-of-service** and **set mpls experimental** commands are mutually exclusive and cannot coexist in the same policy configuration.

## QoS marking mode conditions

- The encapsulation marking behavior applies exclusively in pipe mode.
- In uniform mode or propagate mode, the CoS value from the inner header is automatically copied to the outer header, regardless of the QoS policy encapsulation settings.

## Default QoS marking for VXLAN encapsulation

Default QoS marking for VXLAN encapsulation is a QoS traffic differentiation feature that

- assigns a DSCP value of 0 (Class Selector 0) to the outer IP header when a Provider Edge (PE) device encapsulates IP traffic into a VXLAN tunnel,
- ensures VXLAN transport packets are treated with best-effort service across the underlay network.

VXLAN (Virtual eXtensible LAN) tunnels encapsulate Layer 2 Ethernet frames in UDP/IP packets. When the PE performs this encapsulation, the system must decide what value to place in the outer IP header's DSCP field, which determines per-hop QoS treatment for the VXLAN packet itself.

The default behavior is to clear or reset that field to DSCP 0 (Class Selector 0, best effort).

**Table 20: Feature History Table**

| Feature Name                              | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Set VXLAN Outer IP Header DSCP Value to 0 | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is now supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 8712-MOD-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |
| Set VXLAN Outer IP Header DSCP Value to 0 | Release 24.2.11     | When a PE device transports IP traffic over a VXLAN tunnel that originates on the device, it automatically sets the DSCP value in the VXLAN outer IP header to 0 (CS0).                                                                                                                                                                                                                                               |

## QoS marking on Layer 2 subinterfaces

QoS marking on Layer 2 subinterfaces is a QoS traffic differentiation feature that

- enables per-subinterface traffic classification and marking on Layer 2 transport interfaces
- applies CoS-based marking directly on Layer 2 frames
- supports marking of Layer 2 Ethernet packets in the ingress and egress directions, and
- ensures independent QoS policy enforcement for each subinterface within a physical port or bundle.

**Table 21: Feature History Table**

| Feature Name                              | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Classification and Marking for L2 Traffic | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is supported on Cisco 8011-4G24Y4H-I routers.                                                                                                                                                                                                                                                            |
| Classification and Marking for L2 Traffic | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is now supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 8712-MOD-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |
| Classification and Marking for L2 Traffic | Release 7.7.1       | You can now perform packet remarking on Layer 2 subinterfaces. Traffic marking allows you to fine tune the traffic attributes on your network. The increased granularity helps single out traffic that requires special handling, and achieves optimal application performance.                                                                                                                                       |

## Configure QoS marking on Layer 2 subinterfaces

### Before you begin

Complete these prerequisites before you begin:

- Enable Layer 2 service on the device or subinterface as needed. For configuration details, refer to the *L2VPN Configuration Guide*.
- Prepare the class maps and policy maps used for ingress classification and egress remarking. For more information, refer to the *Create a Traffic Class* section in the *Classify Packets to Identify Specific Traffic* chapter.

Follow these steps to configure QoS marking on Layer 2 subinterfaces.

## Procedure

**Step 1** Create an ingress policy to classify traffic and set internal qos-group values.

This policy assigns traffic classes and associates each class with a traffic-class and qos-group value for later egress remarking.

### Example:

```
Router# configure terminal
Router(config)# policy-map INGRESS_L2_AC
Router(config-pmap)# class CONTROL_PLANE
Router(config-pmap-c)# set traffic-class 7
Router(config-pmap-c)# set qos-group 27
Router(config-pmap-c)# exit

Router(config-pmap)# class VOIP
Router(config-pmap-c)# set traffic-class 6
Router(config-pmap-c)# set qos-group 26
Router(config-pmap-c)# exit

Router(config-pmap)# class VIDEO_STREAM
Router(config-pmap-c)# set traffic-class 5
Router(config-pmap-c)# set qos-group 25
Router(config-pmap-c)# exit

Router(config-pmap)# class TRANSACTIONAL_DATA
Router(config-pmap-c)# set traffic-class 4
Router(config-pmap-c)# set qos-group 24
Router(config-pmap-c)# exit

Router(config-pmap)# class DB_SYNC
Router(config-pmap-c)# set traffic-class 3
Router(config-pmap-c)# set qos-group 23
Router(config-pmap-c)# exit

Router(config-pmap)# class BULK_DATA
Router(config-pmap-c)# set traffic-class 2
Router(config-pmap-c)# set qos-group 22
Router(config-pmap-c)# exit

Router(config-pmap)# class SCAVENGER
Router(config-pmap-c)# set traffic-class 1
Router(config-pmap-c)# set qos-group 21
Router(config-pmap-c)# exit

Router(config-pmap)# class class-default
Router(config-pmap-c)# end-policy-map
Router(config)# commit
```

- Step 2** Create an egress policy to remark frames based on `qos-group` classification.  
Map each `qos-group` to the desired Layer 2 CoS and DEI marking for egress.

**Example:**

```
Router# configure terminal
Router(config)# policy-map REMARK_EGRESS_QG_L2_AC
Router(config-pmap)# class QoS_GROUP_27
Router(config-pmap-c)# set cos 7
Router(config-pmap-c)# exit

Router(config-pmap)# class QoS_GROUP_26
Router(config-pmap-c)# set cos 7
Router(config-pmap-c)# exit

Router(config-pmap)# class QoS_GROUP_25
Router(config-pmap-c)# set cos 5
Router(config-pmap-c)# exit

Router(config-pmap)# class QoS_GROUP_24
Router(config-pmap-c)# set cos 5
Router(config-pmap-c)# exit

Router(config-pmap)# class QoS_GROUP_23
Router(config-pmap-c)# set cos 3
Router(config-pmap-c)# set dei 0
Router(config-pmap-c)# exit

Router(config-pmap)# class QoS_GROUP_22
Router(config-pmap-c)# set cos 3
Router(config-pmap-c)# set dei 1
Router(config-pmap-c)# exit

Router(config-pmap)# class QoS_GROUP_21
Router(config-pmap-c)# set cos 1
Router(config-pmap-c)# set dei 1
Router(config-pmap-c)# exit

Router(config-pmap)# class class-default
Router(config-pmap-c)# set dei 1
Router(config-pmap-c)# exit
Router(config)# commit
```

- Step 3** Attach the egress policy to the Layer 2 subinterface.

**Example:**

```
Router(config)# interface bundle-Ether 1.104
Router(config-subif)# service-policy output REMARK_EGRESS_QG_L2_AC
Router(config-subif)# commit
```

The egress remarking policy is now active on the specified Layer 2 subinterface.

- Step 4** Verify egress remarking on the Layer 2 subinterface.  
Confirm class counters and transmitted byte rates for each `qos-group` class on the subinterface.

**Example:**

```
Router# show policy-map interface Bundle-Ether 1.104 output

Bundle-Ether1.104 output: REMARK_EGRESS_QG_L2_AC

Class QoS_GROUP_7
```

## Configure QoS marking on Layer 2 subinterfaces

```

Classification statistics          (packets/bytes)      (rate - kbps)
Matched           :          21319554/29847375600      985424
Transmitted       :          21319554/29847375600      985424
Total Dropped       :              0/0                  0
Class QoS_GROUP_6
Classification statistics          (packets/bytes)      (rate - kbps)
Matched           :          21581741/30214437400      997540
Transmitted       :          21581741/30214437400      997540
Total Dropped       :              0/0                  0
Class QoS_GROUP_5
Classification statistics          (packets/bytes)      (packets/bytes)      (rate - kbps)
Matched           :          106597824/149236953600      4927114
Transmitted       :          106597824/149236953600      4927114
Total Dropped       :              0/0                  0
Class QoS_GROUP_4
Classification statistics          (packets/bytes)      (rate - kbps)
Matched           :          490350115/686490161000      22664721
Transmitted       :          490350115/686490161000      22664721
Total Dropped       :              0/0                  0
Class QoS_GROUP_3
Classification statistics          (packets/bytes)      (rate - kbps)
Matched           :              0/0                  0
Transmitted       :              0/0                  0
Total Dropped       :              0/0                  0
Class QoS_GROUP_2
Classification statistics          (packets/bytes)      (rate - kbps)
Matched           :              0/0                  0
Transmitted       :              0/0                  0
Total Dropped       :              0/0                  0
Class QoS_GROUP_1
Classification statistics          (packets/bytes)      (rate - kbps)
Matched           :              0/0                  0
Transmitted       :              0/0                  0
Total Dropped       :              0/0                  0
Class class-default
Classification statistics          (packets/bytes)      (rate - kbps)
Matched           :          1087299229/1522218920600      50256530
Transmitted       :          1087299229/1522218920600      50256530
Total Dropped       :              0/0                  0

Policy Bag Stats time: 1657531125148 [Local Time: 07/11/22 09:18:45.148]

```





## CHAPTER 6

# QoS Policies for Link Bundles

- [QoS policies for link bundles, on page 53](#)
- [Usage guidelines for QoS policies on link bundles, on page 53](#)
- [Restrictions for QoS policies on link bundles, on page 55](#)
- [Configure QoS policies on link bundles, on page 55](#)

## QoS policies for link bundles

A link bundle QoS policy is a Modular QoS feature that

- applies unified QoS policies across a group of aggregated Ethernet or VLAN interfaces
- ensures consistent bandwidth management, load balancing, and scheduling for all member links, and
- simplifies configuration by allowing a single QoS policy to govern multiple physical links treated as one logical interface.

### Load balancing on link bundles

Load balancing on link bundles distributes traffic across member links by using Layer 3 routing information. This mechanism includes per-destination load balancing. It ensures that all packets from the same source–destination pair use the same link, preserving packet order and preventing re-sequencing delays when multiple links are available. Per-destination load balancing is only supported on the router, which can distribute packets over any of the links in the bundle.

Layer 3 load balancing for link bundles operates on Ethernet Flow Points (EFPs) associated with the bundle interface. It is based on the IPv4 source and destination addresses in the packet. When IPv4 information is unavailable, the system automatically reverts to the default load balancing that uses the source and destination MAC (SA/DA) fields.

## Usage guidelines for QoS policies on link bundles

### Applying and inheriting QoS policies

- QoS policies are configured on link bundles in the same manner as they are on individual interfaces.

- When you apply a QoS policy on a bundle (in either ingress or egress direction), the queuing policy is automatically applied to each member interface. The reference bandwidth used for calculating shaper or bandwidth values is derived from the physical member interface, not the aggregate bundle bandwidth.
- The QoS policy applied to the bundle is inherited by all member links. The reference bandwidth used for shaping and scheduling is based on the bandwidth of each physical member interface, not the aggregate bandwidth of the bundle.
- If a QoS policy is not applied to a bundle interface, both the ingress and egress traffic use the default queue of the per link member port.
- The shape rate specified in a bundle policy map is not an aggregate rate for all bundle members. The rate applied to the bundle depends on traffic distribution.

For example, if a 10 Mbps shaping policy is applied to a bundle with two member links and all traffic flows through one member, the total shape rate remains 10 Mbps. However, if traffic is balanced evenly across both members, the combined shaping rate becomes 20 Mbps.

- Add bundle members before you apply the QoS policy to the bundle. This action ensures that the QoS policy applies across the bundle and on the new member.

If you add a new member after the policy is applied, and the new member's configuration is incompatible, the policy may fail to propagate and cause configuration errors.

## Troubleshooting queue-limit errors

- When you encounter a queue-limit error, remove the existing QoS policy from the bundle interface. The removal process may take up to ten minutes to take effect, depending on the time elapsed since the error occurred. To monitor progress, check the **Retries performed** information in the syslog message.
- If a queue-limit error occurs, the syslog lists only one interface, even if multiple interfaces are affected. Review **operations** in the syslog to determine the number of affected interfaces.

```
Router#Aug 17 09:56:37.417 PDT: BM-DISTRIB[1156]:
%L2-BM-4-ERR_OP_RETRY_THRESHOLD :
Exceeded threshold for the number of retries for 4 operations on members.
First (and error) FortyGigE0/0/0/34 ('DPA_QOSEA' detected the 'warning'
condition 'Invalid Queue Limit.
Value should be in range 614400-390070272 bytes or equivalent value in usec or
msec based on the service rate of the queue'),
retries performed for 10 hrs 16 mins
```

```
Router#Aug 17 10:06:40.388 PDT: BM-DISTRIB[1156]:
%L2-BM-4-ERR_OP_RETRY_THRESHOLD :
Exceeded threshold for the number of retries for 4 operations on members.
First (and error) FortyGigE0/0/0/34 ('DPA_QOSEA' detected the 'warning'
condition 'Invalid Queue Limit.
Value should be in range 614400-390070272 bytes or equivalent value in
usec or msec based on the service rate of the queue'),
retries performed for 10 hrs 26 mins
```

## Managing statistics and bundle changes

When you remove a member interface from a bundle, the total bundle statistics change because the counters associated with the detached link are lost. Always capture statistics before removing member links to ensure accurate reporting.

- Classification resources (TCAM or MAP) and counters are allocated per bundle interface.

- Queuing resources (VOQs) and counters are allocated per bundle member interface.

## Configuring policing and queuing parameters

- You can apply a policing policy on either the bundle's main interface or a bundle subinterface. You cannot apply it to both at the same time.
- Configure Committed Information Rate (CIR) and Peak Information Rate (PIR) values using percentage units only. You cannot use absolute values on bundle interfaces.
- Policers and their counters are allocated to each bundle interface. They are not allocated to individual member links.
- You can apply the queuing policy on the bundle's main interface or on its subinterface. You cannot apply it to both interfaces simultaneously.
- You can apply classification and marking policies on the bundle main interface or on its subinterface. You cannot apply these policies to both interfaces at the same time.

# Restrictions for QoS policies on link bundles

## Interface and link bundling

- Link bundling is supported only on Ethernet interfaces. Other interface types cannot participate in a bundle.
- Each bundle must contain only physical links. You cannot add subinterfaces or virtual interfaces to a link bundle.
- All links within a bundle must run the same bundling protocol—either IEEE 802.3ad (LACP) or EtherChannel (non-LACP). Mixing protocols within a single bundle is not supported.

## QoS shaping and accounting

- QoS shapers on bundle interfaces support only relative, percentage-based configurations. Absolute shaper values are not supported.
- Ethernet link bundles do not support MAC-level accounting. To collect traffic statistics, use higher-layer QoS counters instead.

## Scalability limits

- Each bundle can include a maximum of 64 Ethernet links.
- The maximum number of link bundles supported is 128.

# Configure QoS policies on link bundles

## Before you begin

When you attach a QoS policy to a bundle interface, the policy is enforced on each member interface. Complete these prerequisites before you begin this procedure.

- Prepare your class and policy definitions.

For more information on how to configure class maps and policy maps, see *Classify Packets to Identify Specific Traffic* in the *Modular QoS Configuration Guide for Cisco 8000 Series Router*.

- Create the link bundle.

## Procedure

**Step 1** Configure an Ethernet link bundle and attach the traffic policy to it.

### Example:

```
Router(config)#interface Bundle-Ether 12000
Router(config-if)#mtu 9100
Router(config-if)#service-policy input ingress
Router(config-if)#service-policy output egress
Router(config-if)#ipv4 address 100.12.0.0 255.255.255.254
Router(config-if)#bundle maximum-active links 64
Router(config-if)#commit
```

This running configuration example shows how a queuing policy is applied on an Ethernet link bundle. The policy is applied to all interfaces that are members of the Ethernet link bundle.

```
/* Policy-map */
policy-map ingress
class inet4-classifier-af1
set qos-group 1
!
class inet4-classifier-af2
set qos-group 2
!
class inet4-classifier-af3
set qos-group 3
!
class inet4-classifier-af4
set qos-group 4
!
class inet4-classifier-bel
set qos-group 5
!
class inet4-classifier-nc1
set qos-group 6
!
class class-default
!
end-policy-map
!
/* Ether Bundle */
interface Bundle-Ether12000
mtu 9100
service-policy input ingress
service-policy output egress
ipv4 address 100.12.0.0 255.255.255.254
load-interval 30
!
```

**Step 2** Verify that the bundle status is up.

**Example:**

```

router#show bundle bundle-ether 1200
Wed Dec 16 19:55:49.974 PST
Bundle-Ether12000
Status: Up
Local links <active/standby/configured>: 35 / 0 / 35
Local bandwidth <effective/available>: 3500000000 (3500000000) kbps
MAC address (source): ea3b.745f.c4b0 (Chassis pool)
Inter-chassis link: No
Minimum active links / bandwidth: 1 / 1 kbps
Maximum active links: 64
Wait while timer: 2000 ms
Load balancing: Default
LACP: Operational
Flap suppression timer: Off
Cisco extensions: Disabled
Non-revertive: Disabled
mLACP: Not configured
IPv4 BFD: Not configured
Port Device State Port ID B/W, kbps
-----
Hu0/4/0/0 Local Active 0x8000, 0x0009 100000000
Link is Active
Hu0/4/0/1 Local Active 0x8000, 0x000a 100000000
Link is Active
- - -
Hu0/4/0/35 Local Active 0x8000, 0x002b 100000000
Link is Active

```

**Step 3**

Verify the bundle statistics.

**Example:**

```

router#show policy-map interface bundle-ether 12000
Bundle-Ether12000 input: ingress
Class inet4-classifier-af1
Classification statistics (packets/bytes) (rate - kbps)
Matched : 4647401962/21236124455654 26403040
Transmitted : 4647401962/21236124455654 26403040
Total Dropped : 0/0 0
Class inet4-classifier-af2
Classification statistics (packets/bytes) (rate - kbps)
Matched : 4502980177/20576584333939 25571493
Transmitted : 4502980177/20576584333939 25571493
Total Dropped : 0/0 0
Class inet4-classifier-af3
Classification statistics (packets/bytes) (rate - kbps)
Matched : 4647404125/21236213667880 26389086
Transmitted : 4647404125/21236213667880 26389086
Total Dropped : 0/0 0
Class inet4-classifier-af4
Classification statistics (packets/bytes) (rate - kbps)
Matched : 9291188840/42456120548683 52771168
Transmitted : 9291188840/42456120548683 52771168
Total Dropped : 0/0 0
Class inet4-classifier-bel
Classification statistics (packets/bytes) (rate - kbps)
Matched : 4647413429/21235847852686 26393414
Transmitted : 4647413429/21235847852686 26393414
Total Dropped : 0/0 0
Class inet4-classifier-nc1
Classification statistics (packets/bytes) (rate - kbps)
Matched : 9294887621/42473100149807 52778258

```

```

Transmitted : 9294887621/42473100149807 52778258
Total Dropped : 0/0 0
Class class-default
Classification statistics (packets/bytes) (rate - kbps)
Matched : 0/0 0
Transmitted : 0/0 0
Total Dropped : 0/0 0
Bundle-Ether12000 output: egress
Class c1
Classification statistics (packets/bytes) (rate - kbps)
Matched : 16665494532/75878118942463 8760591
Transmitted : 16655834643/75834136022017 8760591
Total Dropped : 9659889/43982920446 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 9659889/43982920446
Class c2
Classification statistics (packets/bytes) (rate - kbps)
Matched : 16665421959/75877849543188 8718687
Transmitted : 16665421959/75877849543188 8718687
Total Dropped : 0/0 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 0/0
Class c3
Classification statistics (packets/bytes) (rate - kbps)
Matched : 16665247833/75877509455458 8703470
Transmitted : 16665187414/75877234624197 8703470
Total Dropped : 60419/274831261 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 60419/274831261
Class c4
Classification statistics (packets/bytes) (rate - kbps)
Matched : 33330896131/151755393012945 17470745
Transmitted : 33330745421/151754709368565 17470745
Total Dropped : 150710/683644380 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 150710/683644380
Class c5
Classification statistics (packets/bytes) (rate - kbps)
Matched : 16878910340/76849791869834 8833394
Transmitted : 16878849464/76849514633309 8833394
Total Dropped : 60876/277236525 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 60876/277236525
Class c6
Classification statistics (packets/bytes) (rate - kbps)
Matched : 33330898844/151756094112925 17456785
Transmitted : 33330752668/151755427708382 17456785
Total Dropped : 146176/666404543 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 146176/666404543
Class c7
Classification statistics (packets/bytes) (rate - kbps)
Matched : 244106/79922040 74
Transmitted : 244106/79922040 74
Total Dropped : 0/0 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 0/0

```

```
Class class-default
Classification statistics (packets/bytes) (rate - kbps)
Matched : 267075066180/1215993441123215 139917482
Transmitted : 267075066180/1215993441123215 139917482
Total Dropped : 0/0 0
Queueing statistics
Queue ID : None (Bundle)
Taildropped(packets/bytes) : 0/0
```

---







## CHAPTER 7

# Congestion Management

- [Traffic congestion management, on page 61](#)
- [Low-latency queuing with strict priority queuing, on page 62](#)
- [Traffic shaping, on page 68](#)
- [Traffic policing, on page 72](#)

## Traffic congestion management

Congestion management is a QoS traffic-management model that

- defines how traffic is organized into queues when congestion occurs on an interface
- defines how classified packets are mapped to those queues, and
- defines how queues are scheduled for transmission based on priority or policy rules.

### Key features for congestion management

- [Low Latency Queueing with Strict Priority Queueing](#): Ensures time-sensitive traffic (such as voice or real-time video) is placed in a strict-priority queue so it is serviced first, minimizing latency and jitter while other traffic is still managed fairly.
- [Traffic Shaping](#): Smooths traffic bursts by buffering and pacing packets so that the egress rate matches downstream link speeds or contracted rates, helping prevent congestion caused by data-rate mismatches.
- [Traffic Policing](#): Enforces a maximum traffic rate on an interface using token-bucket algorithms and drops (or marks) packets that exceed the configured rate, helping protect bandwidth and maintain service quality across classes of traffic.

The table lists the key differences among Low Latency Queueing (with Strict Priority), Traffic Shaping, and Traffic Policing to help you understand how each feature manages traffic under congestion.

Table 22: Key differences among low latency queuing (with strict priority), traffic shaping, and traffic policing

| Aspect                              | LLQ with strict priority queuing                                                                  | Traffic shaping                                                                       | Traffic policing                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Primary goal</b>                 | Prioritize time-sensitive traffic to minimize latency and jitter.                                 | Match traffic output rate to downstream speed or contracted profile.                  | Enforce a maximum allowed traffic rate using token-bucket metering.                        |
| <b>Behavior with excess traffic</b> | Strict priority can cause lower-priority queues to not get serviced if priority traffic is heavy. | Buffers excess packets and sends them later. It <i>smooths</i> bursts.                | Does not buffer. Packets that exceed rate may be marked or dropped.                        |
| <b>Traffic direction</b>            | Egress                                                                                            | Egress only (explicit).                                                               | Ingress (egress policing is not supported).                                                |
| <b>Impact on other traffic</b>      | May starve lower-priority queues unless an optional shaper is used.                               | Prevents downstream bottlenecks by smoothing bursts; does not drop traffic by design. | Can drop packets based on exceed/violate actions; partitions traffic into service classes. |
| <b>Handling of traffic bursts</b>   | Priority traffic bypasses others; does not inherently smooth bursts or buffer.                    | Buffers bursts and releases later to match rate profile.                              | Bursts may trigger exceed/violate events; uses burst parameters.                           |

## Low-latency queuing with strict priority queuing

Low-latency queuing with strict priority queuing is a QoS congestion management feature that

- integrates strict priority queuing into the Class-Based Weighted Fair Queuing (CBWFQ) model
- ensures delay-sensitive traffic is always dequeued ahead of other traffic classes, and
- minimizes latency and jitter for critical applications such as voice and real-time video.

CBWFQ is a QoS scheduling model that assigns bandwidth to traffic classes based on configured weights and ensures each class receives its allocated portion of service during congestion.

Priority Queuing is a scheduling method that dequeues packets from the highest-priority queue before any other queue, allowing delay-sensitive traffic to pre-empt lower-priority classes; in strict priority mode, lower-priority queues may not be serviced if high-priority traffic is continuously present.

Table 23: Feature History Table

| Feature Name                                       | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Priority Propagation on Egress Queues              | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)</p> <p>This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                                                                                                                                       |
| Low Latency Queueing with Strict Priority Queueing | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)</p> <p>*This feature is supported on Cisco 8011-4G24Y4H-I routers.</p>                                                                                                                                                                                                                                                                                     |
| Low Latency Queueing with Strict Priority Queueing | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8700) (select variants only*)</p> <p>This feature ensures that time-sensitive traffic receives dedicated priority, ensuring minimal latency and jitter while congestion for other types of traffic is managed fairly, guaranteeing high performance for critical applications and efficient overall traffic management.</p> <p>*This functionality is now supported on Cisco 8712-MOD-M routers.</p> |

| Feature Name                          | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Priority Propagation on Egress Queues | Release 24.3.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>You can now ensure that high-priority traffic is consistently prioritized across multiple sub-interfaces on the same network port, enhancing the performance of critical applications.</p> <p>This is achieved by enabling support for egress queuing policy maps that allow high-priority P1 traffic (on class TC7) to take precedence over non-P1 traffic across sub-interfaces. This feature ensures that critical traffic, such as voice or real-time video, is always given the highest priority, regardless of the sub-interface it is associated with, thereby maintaining optimal network performance and reducing latency for essential services.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name                                       | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Low Latency Queueing with Strict Priority Queueing | Release 24.3.1      | <p>Introduced in this release on:<br/>Modular Systems (8800 [LC ASIC: P100]) (select variants only*),<br/>Fixed Systems (8200) (select variants only*), Fixed Systems (8700 (P100, K100)) (select variants only*)</p> <p>This feature ensures that time-sensitive traffic receives dedicated priority, ensuring minimal latency and jitter while congestion for other types of traffic is managed fairly, guaranteeing high performance for critical applications and efficient overall traffic management.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"><li>• 88-LC1-12TH24FH-E</li><li>• 88-LC1-52Y8H-EM</li><li>• 8212-48FH-M</li><li>• 8711-32FH-M</li></ul> |

| Feature Name                                       | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Low Latency Queueing with Strict Priority Queueing | Release 24.2.11     | <p>Introduced in this release on: Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>With this feature, time-sensitive traffic receives dedicated priority, ensuring minimal latency and jitter while congestion for other types of traffic is managed fairly, guaranteeing high performance for critical applications and efficient overall traffic management.</p> <p>You can also configure low latency queueing with strict priority queueing at the subinterface level, ensuring that the strict priority policy applies only to traffic on that specific subinterface, not the entire physical interface. This configuration is useful in scenarios where multiple services share the same physical link but have different QoS requirements.</p> <p>*This feature is supported on 88-LC1-36EH.</p> |

## Guidelines for configuring low-latency queuing with strict priority queuing

### Priority and scheduling behavior

- Only priority levels 1 to 7 are supported, where 1 is the highest and 7 is the lowest priority; the default CoSQ 0 has the lowest priority among all classes.
- Egress policing is not supported, and in strict priority mode, heavy priority traffic can prevent lower-priority queues from being serviced.
- The platform enforces a strict class hierarchy, where higher traffic classes take precedence over lower ones. This hierarchy can be redefined through an egress queuing policy map when configuring LLQ with strict priority.
- When operating in strict-priority mode, egress scheduling follows the default class order TC7 > TC6 > TC5 > TC4 > TC3 > TC2 > TC1 > TC0, unless a queuing policy map overrides this behavior.

### Shaping and rate control

- You can configure shape average and queue-limit commands along with the priority command.
- An optional shaper can be configured to cap the maximum traffic rate and prevent starvation of lower-priority traffic classes.

## Interface-level configuration

LLQ with strict priority queuing can also be applied at the subinterface level, allowing strict priority behavior to affect only the specific subinterface rather than the entire physical interface.

# Configure low-latency queuing with strict priority queuing

### Before you begin

Ensure that you have an existing QoS class map that classifies the intended traffic.

Follow these steps to configure LLQ with strict priority on an egress interface.

### Procedure

- Step 1** Create or modify a policy-map that applies LLQ with strict priority.  
This policy-map defines the class of traffic that receives strict priority handling.

#### Example:

```
Router# configure
Router(config)# policy-map test-priority-1
Router(config-pmap)# class qos1
Router(config-pmap-c)# priority level 7
Router(config-pmap-c)# exit
Router(config-pmap)# exit
```

The policy-map now includes a strict priority class with the specified priority level.

- Step 2** (Optional) Shape the strict priority class to a specific bit rate.  
Shaping prevents lower-priority queues from starving during high volumes of strict priority traffic.

#### Example:

```
Router(config-pmap-c)# shape average percent 40
```

- Step 3** Apply the policy-map to the egress interface.  
Attaching the policy-map makes strict priority queuing active for egress traffic on the specified interface.

#### Example:

```
Router(config)# interface HundredGigE 0/6/0/18
Router(config-if)# service-policy output test-priority-1
Router(config-if)# no shutdown
Router(config-if)# commit
```

The LLQ strict priority policy is now active on the selected output interface.

- Step 4** Verify the strict priority behavior on the interface.  
Confirm that the strict priority class (for example, HP7) is visible in the queue statistics.

#### Example:

```
Router# show qos interface HundredGigE 0/6/0/18 output
```

```

NOTE:- Configured values are displayed within parentheses
Interface HundredGigE0/6/0/18 ifh 0x3000220 -- output policy
NPU Id:                                     3
Total number of classes:                   3
Interface Bandwidth:                       100000000 kbps
VOQ Base:                                 11176
VOQ Stats Handle:                         0x88550ea0
Accounting Type:                          Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class (HP7)                         = qos1
Egressq Queue ID                          = 11177 (HP7 queue)
TailDrop Threshold                        = 125304832 bytes / 10 ms (default)
WRED not configured for this class

Level1 Class (HP6)                         = qos-2
Egressq Queue ID                          = 11178 (HP6 queue)
TailDrop Threshold                        = 125304832 bytes / 10 ms (default)
WRED not configured for this class

Level1 Class (class-default)
Egressq Queue ID                          = 11176 (Default LP queue)
Queue Max. BW.                           = 101803495 kbps (default)
Queue Min. BW.                           = 0 kbps (default)
Inverse Weight / Weight                   = 1 (BWR not configured)
TailDrop Threshold                        = 1253376 bytes / 10 ms (default)
WRED not configured for this class

```

The interface shows the strict priority class and confirms that LLQ with strict priority is functioning as expected.

## Traffic shaping

Traffic shaping is a QoS congestion management feature that

- regulates the flow of outbound traffic to match the speed or bandwidth profile of the downstream interface
- buffers excess packets during congestion instead of dropping them, and
- ensures traffic conforms to contracted or expected rate profiles to prevent bottlenecks in topologies with data-rate mismatches.

### Key functions of traffic shaping

- **Buffering:** is a key function of traffic shaping in which excess packets are temporarily stored instead of being dropped, allowing the system to release them later at a controlled rate when bandwidth becomes available.
- **Rate profiling:** is a traffic-control concept that defines the maximum transmission rate a flow must conform to so that it matches the expected bandwidth or contract of the downstream interface.

## Guidelines for configuring traffic shaping

### Supported traffic direction



Only egress traffic shaping is supported.

## Class and policy requirements

- You must configure all eight qos-group classes—including class-default—in the egress QoS policy.
- You can configure the **shape average** command together with the **priority** command when shaping traffic.
- We recommended that you configure all eight traffic-class classes—including class-default—in the egress policies.
- Only a limited set of traffic-class class combinations is supported.

## Traffic-type considerations

Egress interfaces that carry both unicast and multicast traffic may handle traffic classes inconsistently when shaping is enabled. Therefore, configure shaping primarily on ports that carry only unicast traffic.

# Configure traffic shaping

This procedure applies only to configuring traffic shaping on main interfaces.

### Before you begin

Ensure that you have an existing QoS class map for the traffic class you intend to shape.

Follow these steps to configure traffic shaping on a main interface.

### Procedure

- Step 1** Create or modify a policy-map to configure traffic shaping for a specific class.  
This policy-map defines the shaping rate applied to the outbound traffic class.

#### Example:

```
Router# configure
Router(config)# policy-map egress_policy1
Router(config-pmap)# class c5
Router(config-pmap-c)# shape average percent 40
Router(config-pmap-c)# exit
Router(config-pmap)# exit
```

The policy-map now includes a shaping configuration for the selected traffic class.

- Step 2** Attach the shaping policy-map to the egress interface.  
Applying the service policy activates traffic shaping for outbound traffic on that interface.

#### Example:

```
Router(config)# interface HundredGigE 0/1/0/0
Router(config-if)# service-policy output egress_policy1
Router(config-if)# commit
```

The traffic shaping configuration is now active on the selected output interface.

### Running Configuration

```

policy-map egress_policy1
  class c5
    shape average percent 40
  !
  class class-default
  !
end-policy-map
!

interface HundredGigE0/6/0/18
  service-policy input 100g-s1-1
  service-policy output egress_policy1
!
```

- Step 3** Verify the shaping configuration on the interface.  
Check that the class displays the expected shaping rate and queue behavior.

#### Example:

Router# **show qos interface HundredGigE 0/6/0/18 output**

```

NOTE:- Configured values are displayed within parentheses
Interface HundredGigE0/6/0/18 ifh 0x3000220 -- output policy
NPU Id:                               3
Total number of classes:              2
Interface Bandwidth:                  100000000 kbps
VOQ Base:                             11176
VOQ Stats Handle:                     0x88550ea0
Accounting Type:                      Layer1 (Include Layer 1 encapsulation and above)
-----
```

```

Level1 Class                               = c5
Egressq Queue ID                        = 11177 (LP queue)
Queue Max. BW.                             = 40329846 kbps (40 %)
Queue Min. BW.                          = 0 kbps (default)
Inverse Weight / Weight                  = 1 (BWR not configured)
Guaranteed service rate                  = 40000000 kbps
TailDrop Threshold                       = 50069504 bytes / 10 ms (default)
WRED not configured for this class
```

```

Level1 Class                               = class-default
Egressq Queue ID                        = 11176 (Default LP queue)
Queue Max. BW.                          = 101803495 kbps (default)
Queue Min. BW.                          = 0 kbps (default)
Inverse Weight / Weight                  = 1 (BWR not configured)
Guaranteed service rate                  = 50000000 kbps
TailDrop Threshold                       = 62652416 bytes / 10 ms (default)
WRED not configured for this class
```

The interface output shows that shaping is applied with the configured rate and class behavior.

## Egress class-level traffic shaping on subinterfaces

Egress class-level traffic shaping on subinterfaces is a QoS traffic shaping feature that

- provides class-level rate limiters or shaping on each traffic class in an egress subinterface queuing policy map

- provides granular control of outbound traffic on individual classes, and
- enables efficient bandwidth allocation when multiple services share the same physical link or LAG.

**Table 24: Feature History Table**

| Feature Name                                         | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Egress class-level traffic shaping for subinterfaces | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)</p> <p>Egress class-level traffic shaping for subinterfaces is now supported on the fixed systems using K100 and A100 Silicon One ASICs.</p> <p>This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                    |
| Egress class-level traffic shaping for subinterfaces | Release 24.3.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>You can now granularly control the traffic flow on each class in the egress queuing policy map ensuring optimal bandwidth allocation and improved network performance.</p> <p>This is achieved by introducing support for enabling class-level rate limiting traffic shapers for each class in the egress subinterface-level (main and LAG) queuing policy maps.</p> <p>*This feature is enabled by default and supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

# Guidelines for configuring egress class-level traffic shaping on subinterfaces

## Traffic-type considerations

Shaping of multicast traffic is not supported on egress subinterfaces configured in the non-ETM mode.

## Traffic policing

Traffic policing is a QoS congestion management feature that

- limits the maximum rate of traffic entering or leaving an interface by enforcing compliance with a configured bandwidth profile
- uses a token bucket algorithm to determine whether each packet conforms to or exceeds a configured rate profile and applies the corresponding transmit or drop action, and
- applies immediate actions—such as transmitting, marking, or dropping packets—without buffering excess traffic.

### Key concepts of traffic policing

- **Token bucket:** is a rate-control mechanism that accumulates tokens at a configured rate and uses them to determine whether packets conform to or exceed a traffic policer's allowed bandwidth profile.
- **Conform, exceed, and violate actions:** are policing outcomes that classify each packet based on whether it fits within the allowed rate: conforming packets are transmitted, exceeding packets may be marked or dropped, and violating packets are typically dropped according to the configured policing mode.
- **Committed Information Rate (CIR):** is the sustained bandwidth level that a policer guarantees for a traffic flow, defining the average rate at which tokens are added to the token bucket.
- **Peak Information Rate (PIR):** is the upper bandwidth limit configured for two-rate policers, defining the maximum rate at which traffic is allowed to be sent when both the committed and peak token buckets contain sufficient tokens.
- **Committed Burst (Bc):** or Committed Burst Size (CBS) is the maximum amount of data that can be sent at once while still conforming to the CIR, representing the number of bytes or time interval that the committed token bucket can hold at full capacity.

This value represents the capacity of the primary token bucket. The larger the value, the more tolerant the policer is of momentary traffic bursts above the average rate.

- **Excess Burst (Be):** or Excess Burst Size (EBS) is the additional amount of data that may be permitted beyond the committed burst when excess tokens are available, allowing short-term transmission above the CIR before packets are treated as exceeding or violating the rate.

Excess Burst (Be) is an optional parameter used in traffic policing, specifically within a two-bucket, three-color policing mode, to allow for a secondary, higher level of burst tolerance beyond the Committed Burst (Bc)

### Types of traffic policing modes

This section outlines the traffic-policing modes supported on your router.

- **Single-Rate policer**

- **Single-rate two-color (1R2C) policer:** is a rate-control mode that uses a single token bucket and classifies each packet as either *conform* or *exceed* based on the CIR, applying corresponding actions such as transmit or drop.

- **Two-rate policer**

- **Two-rate three-color (2R3C) policer:** is a metering mode that uses two token buckets—one for the CIR and one for the peak information rate (PIR)—to classify packets into *conform*, *exceed*, or *violate* actions, enforcing both a sustained rate limit and a peak rate limit.

## Single-rate two-color policers

The single-rate two-color (1R2C) policer is a traffic-policing feature that

- uses a single token bucket to meter packets against a configured committed information rate (CIR)
- classifies each packet as either conforming or exceeding the allowed rate based solely on the available committed burst tokens, and
- applies fixed policing actions—transmit for conforming packets and drop for exceeding packets—without buffering excess traffic.

## Guidelines for configuring single-rate two-color policers

### Default policer actions

- The policer always transmits packets that conform to the CIR.
- The policer always drops packets that exceed the CIR.
- The default actions cannot be modified, regardless of class configuration or policy-map settings.

## How the single-rate two-color policer works

### Summary

The key components involved in the single-rate two-color policer are:

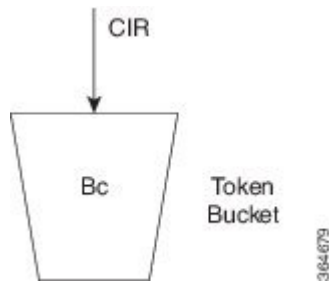
- **Traffic policer:** A metering function on the interface that evaluates each packet against a configured committed information rate (CIR) and burst parameters, then applies a conform or exceed action based on that evaluation.
- **Token bucket (Tc):** A logical bucket that accumulates tokens at the CIR up to a configured committed burst value (Bc). The number of tokens represents how much traffic can be sent at any given moment.
- **Committed information rate (CIR):** The configured average rate (in bits per second) at which the policer replenishes tokens and against which packet sizes are evaluated.

The single-rate two-color policer meters traffic by updating a single token bucket at the committed information rate and comparing the size of each packet against the available tokens. Based on this comparison, the policer

classifies each packet as conforming or exceeding the CIR and applies the corresponding default action. This process limits the maximum traffic rate on the interface without buffering excess packets.

### Workflow

**Figure 4: Workflow for single-rate two-color policers**



These stages describe how the single-rate two-color policer works.

1. The router receives packets on an interface with a single-rate two-color policer configured.  
The traffic policer on the interface inspects each incoming packet and prepares to meter it against the configured committed information rate (CIR) and burst parameters.
2. The policer updates the token bucket based on the committed information rate.  
At every refresh interval, the policer adds tokens to the token bucket (Tc) at a rate equal to the CIR, up to the committed burst value (Bc). Tokens represent the amount of traffic that can be sent at the committed rate.
3. The policer compares the packet size to the available tokens.  
For each packet of size B, the policer checks whether the current token count in the bucket is sufficient to accommodate the packet. If the bucket holds enough tokens, the packet can conform to the CIR; otherwise, the packet exceeds the CIR.
4. The policer classifies conforming packets and applies the conform action.  
If the packet size B is less than or equal to the number of tokens in the bucket, the packet conforms. The policer decrements the token bucket by B and applies the conform action, which transmits the packet.
5. The policer classifies exceeding packets and applies the exceed action.  
If the packet size B is greater than the number of available tokens in the bucket, the packet exceeds the CIR. The policer applies the exceed action, which drops the packet for the single-rate two-color policer. Excess packets are not buffered for later transmission.
6. The policer continues metering traffic to enforce the configured rate.

### Result

As traffic continues to arrive, the policer repeats the cycle of replenishing tokens, classifying packets as conform or exceed, and applying the corresponding actions. This ongoing process enforces the configured committed information rate on the interface without buffering excess packets.

## Configure single-rate two-color policers

### Before you begin

Prepare or identify a class map that matches the traffic you want to police.

Follow these steps to configure traffic policing (1R2C) on an ingress interface.

## Procedure

**Step 1** Create or modify a policy-map to apply the single-rate two-color policer to a traffic class.

This step creates a policy-map that applies a 1R2C policer to a class of traffic. The policer limits the rate for that class on the ingress interface.

### Example:

```
Router# configure
Router(config)# policy-map test-police-1R2C
Router(config-pmap)# class dscp1
Router(config-pmap-c)# police rate 10 gbps
Router(config-pmap-c-police)# exit
Router(config-pmap-c)# exit
Router(config-pmap)# exit
```

The policy-map now includes a single-rate two-color policer that enforces a 10-Gbps committed rate for the `dscp1` class.

**Step 2** Attach the policing policy-map to the ingress interface.

Attaching the policy-map enables traffic policing on the selected interface in the ingress direction.

### Example:

```
Router(config)# interface HundredGigE 0/0/0/18
Router(config-if)# service-policy input test-police-1R2C
Router(config-if)# commit
```

The single-rate two-color policer is now active on the specified input interface.

### Running Configuration:

```
class-map match-any dscp1
  match dscp ipv4 1
end-class-map
!
!
policy-map test-police-1R2C
  class dscp1
    police rate 10 gbps
  !
  !
  class class-default
  !
  !
end-policy-map
!
!
interface HundredGigE0/0/0/8
  service-policy input test-police-1R2C
!
```

**Step 3** Verify the traffic policing configuration and policer statistics.

### Example:

```

Router# show qos interface HundredGigE 0/0/0/8 input
NOTE:- Configured values are displayed within parentheses
Interface HundredGigE0/0/0/8 ifh 0xf0001e8 -- input policy
NPU Id:                                0
Total number of classes:                2
Interface Bandwidth:                    100000000 kbps
Policy Name:                            test-police-1R2C
Accounting Type:                        Layer1 (Include Layer 1 encapsulation and above)
-----
Levell Class                            = dscpl
Policer committed rate                  = 10000000 kbps (10 gbits/sec)
Policer conform burst                   = 1024000 bytes (default)
Policer conform action                  = Just TX
Policer exceed action                   = DROP PKT

Levell Class                            = class-default
Policer not configured for this class

```

The output confirms that the input interface is using the `test-police-1R2C` policy and that the `dscpl` class is policed at the configured rate with the expected conform and exceed actions.

## Two-rate three-color policers

The two-rate three-color (2R3C) policer is a traffic-policing feature that

- uses two token buckets, a committed bucket and a peak bucket, to meter traffic at both the committed information rate (CIR) and the peak information rate (PIR)
- classifies each packet into one of three conformance levels—conform, exceed, or violate—based on the configured rates and burst parameters, and
- applies predefined per-color actions in which conform and exceed traffic are transmitted and violate traffic is dropped.

## Guidelines for configuring two-rate three-color policers

### Default policer actions

- The policer always transmits packets that conform to the committed or peak information rates.
- The policer always drops packets that violate the configured rate thresholds.
- You cannot modify these default actions, regardless of class configuration or policy-map settings.

## How the two-rate three-color policer works

### Summary

The key components involved in two-rate three-color policing are:

- **Committed information rate (CIR):** The configured average rate (in bits per second) at which the policer replenishes tokens and against which packet sizes are evaluated.

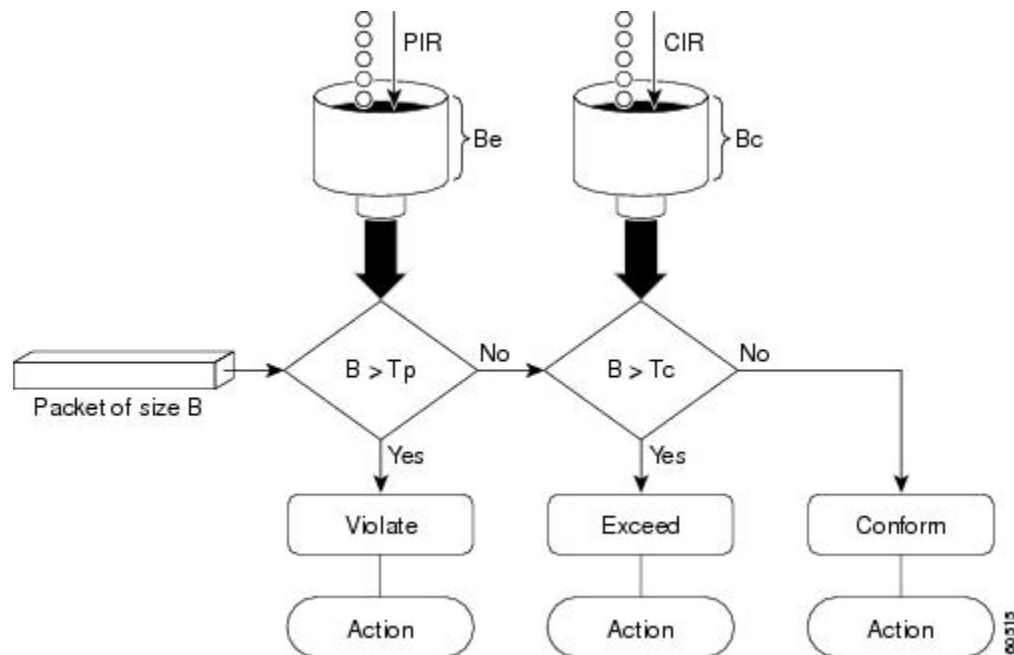


- **Peak information rate (PIR):** The higher metering rate that allows short-term bursts above the CIR, up to a configured peak limit.
- **Committed token bucket (Tc):** A token bucket associated with the CIR that determines whether packets conform to the committed rate.
- **Peak token bucket (Tp):** A token bucket associated with the PIR that determines whether packets exceed the committed rate but still remain within the peak rate.

The two-rate three-color policer uses the committed and peak token buckets together to meter packets against CIR and PIR and to assign a color-based action that enforces the configured traffic policy at the ingress interface.

### Workflow

*Figure 5: Workflow for two-rate three-color policers*



These stages describe how the two-rate three-color policer works.

1. The policer initializes the committed and peak token buckets.

The committed bucket is sized by the committed burst (Bc), and the peak bucket is sized by the peak burst (Be). Both buckets start accumulating tokens as soon as the policer is active.

Based on the configured committed information rate (CIR), peak information rate (PIR), and burst parameters, the router initializes the committed and peak token buckets used to meter incoming traffic.

2. The policer refreshes tokens based on the CIR and PIR.

When a packet arrives at the interface, then

- the committed bucket is refilled at the CIR rate, up to Bc, and
- the peak bucket is refilled at the PIR rate, up to Be.

This dual-rate update occurs each time traffic arrives, ensuring both long-term and peak-rate metering.

3. The policer evaluates the packet size against both token buckets.

The policer compares the packet's size (B) with the committed and then the peak bucket.

**Table 25: Packet evaluation against token buckets**

| When...                                                                                          | Then...                                                                                                                                          |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| the packet size <b>B</b> is less than or equal to the committed token bucket (Tc)                | the policer determines that the committed bucket has enough tokens and classifies the packet as <b>conform</b> .                                 |
| the packet size <b>B</b> is greater than Tc but less than or equal to the peak token bucket (Tp) | the policer determines that the committed bucket overflows, but the peak bucket can accommodate the packet, and classifies it as <b>exceed</b> . |
| the packet size <b>B</b> is greater than Tp                                                      | the policer determines that neither bucket has enough tokens and classifies the packet as <b>violate</b> .                                       |

4. The policer assigns a color to the packet based on CIR and PIR.

**Table 26: Packet color assignment**

| When...                                                                                          | Then...                                                                                                                                                |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| the packet size <b>B</b> is less than or equal to the committed token bucket (Tc)                | the policer assigns the packet the <b>conform (green)</b> color.                                                                                       |
| the packet size <b>B</b> is greater than Tc but less than or equal to the peak token bucket (Tp) | the policer assigns the packet the <b>exceed (yellow)</b> color because it does not fit in the committed bucket but still fits within the peak bucket. |
| the packet size <b>B</b> is greater than Tp                                                      | the policer assigns the packet the <b>violate (red)</b> color because it does not fit in either token bucket and exceeds the PIR.                      |

5. The policer applies the default transmit or drop actions for each color.

**Table 27: Color-based policer actions**

| When...                                          | Then...                                                                                                                                 |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| a packet is classified as <b>conform (green)</b> | the packet is <b>transmitted</b> , and the policer decrements both the committed and peak token buckets by the packet size B.           |
| a packet is classified as <b>exceed (yellow)</b> | the packet is <b>transmitted</b> , the committed bucket is decremented by B, and the peak bucket is decremented by the overflow amount. |
| a packet is classified as <b>violate (red)</b>   | the packet is <b>dropped</b> , and the peak token bucket is not decremented.                                                            |

## Result

As traffic continues to arrive, the policer repeatedly refreshes tokens in both the committed and peak buckets, evaluates packet sizes against the available tokens, assigns each packet a conform, exceed, or violate color,

and applies the corresponding predefined actions. This continuous metering process enforces both the committed and peak information rates on the interface and ensures that exceeding and violating traffic is handled according to the policer's color-based rules.

## Configure two-rate three-color policers

### Before you begin

Prepare or identify a class map that matches the traffic you want to police.

Follow these steps to configure two-rate three-color traffic policing (2R3C) on an ingress interface.

### Procedure

#### Step 1

Create or modify a policy-map to apply the two-rate three-color policer to a traffic class.

This step creates a policy-map that applies a 2R3C policer to a class of traffic. The policer meters traffic using a committed information rate (CIR) and a peak information rate (PIR).

#### Example:

```
Router# configure
Router(config)# policy-map test-police-2R3C
Router(config-pmap)# class dscp1
Router(config-pmap-c)# police rate 10 gbps peak-rate 20 gbps
Router(config-pmap-c-police)# exit
Router(config-pmap-c)# exit
Router(config-pmap)# exit
```

The policy-map now includes a two-rate three-color policer with a 10 Gbps committed rate and a 20 Gbps peak rate for the dscp1 class.

#### Step 2

Attach the policing policy-map to the ingress interface.

Attaching the policy-map enables two-rate three-color traffic policing on the selected interface in the ingress direction.

#### Example:

```
Router(config)# interface HundredGigE 0/0/0/8
Router(config-if)# service-policy input test-police-2R3C
Router(config-if)# commit
```

The two-rate three-color policer is now active on the specified input interface.

#### Running Configuration:

```
class-map match-any dscp1
match dscp ipv4 1
end-class-map
!
!
policy-map test-police-2R3C
class dscp1
police rate 10 gbps peak-rate 20 gbps
!
!
class class-default
!
!
end-policy-map
!
!
```

```
interface HundredGigE0/0/0/8
service-policy input test-police-2R3C
!
```

**Step 3** Verify the two-rate three-color traffic policing configuration and policer statistics.

a) Verify the policer configuration applied on the ingress interface.

**Example:**

```
Router# show qos interface HundredGigE 0/0/0/8 input
NOTE:- Configured values are displayed within parentheses
|Interface HundredGigE0/0/0/8 ifh 0xf0001e8 -- input policy
|NPU Id:||0
|Total number of classes:          2
|Interface Bandwidth:              100000000 kbps
|Policy Name:|                    test-police-2R3C
|Accounting Type:|                Layer1 (Include Layer 1 encapsulation and above)
|-----|
|Levell Class||                  = dscp1
|Policer committed rate|          = 10000000 kbps (10 gbits/sec)
|Policer peak rate||=            20000000 kbps (20 gbits/sec)
|Policer conform burst|           = 1024000 bytes (default)
|Policer exceed burst|            = 2048000 bytes (default)
|Policer conform action|           = Just TX
|Policer exceed action|            = DROP PKT
|Policer violate action|           = DROP PKT
|
|Levell Class||                  = class-default
|Policer not configured for this class
```

b) Review detailed policer statistics on the ingress interface.

**Example:**

```
Router# policy-map interface HundredGigE 0/0/0/8 input
|HundredGigE0/0/0/8 input: test-police-2R3C
|
|Class dscp1
|Classification statistics          (packets/bytes)      (rate - kbps)
|Matched| :                289228439/289228439000      27734775
|Transmitted :|56422213/56422213000      5410359
|Total Dropped :                232806226/232806226000      22324416
|Policing statistics|          (packets/bytes)      (rate - kbps)
|Policed(conform) :|56422213/56422213000      5410359
|Policed(exceed) :|56422215/56422215000      5410358
|Policed(violate) :                176384011/176384011000      16914058
|Policed and dropped :                232806226/232806226000
|Class class-default
|Classification statistics          (packets/bytes)      (rate - kbps)
|Matched| :|61136620/61136620000      0
|Transmitted :|61136620/61136620000      0
|Total Dropped :|0/0|      0
|Policy Bag Stats time: 1570155764000 [Local Time: 10/04/19 02:22:44.000]
```

The output confirms that the input interface is using the test-police-2R3C policy and that the dscp1 class is metered at the configured committed and peak rates with the expected conform, exceed, and violate actions.



## CHAPTER 8

# Congestion Avoidance

---

- [Congestion Avoidance, on page 81](#)
- [Queuing modes, on page 83](#)
- [Queuing policy for main interfaces, on page 85](#)
- [Queuing policy behavior on subinterfaces, on page 86](#)
- [Congestion detection and handling, on page 88](#)

## Congestion Avoidance

Congestion avoidance is a traffic management technique that

- helps prevent buffer overflows in network queues by managing packet drops before queues reach full capacity
- monitors queue thresholds, and
- triggers early drops of lower-priority packets to maintain the availability of buffer memory.

### Queueing for congestion avoidance

Queueing for congestion avoidance manages queue buffers to prevent memory overflow in the ASIC or NPU. When queues fill beyond set thresholds, low-priority packets are dropped early to preserve memory for critical traffic and maintain overall system performance.

Shaping and scheduling context

- Shaping buffers packets within a queue to smooth bursty traffic and control the transmission rate.
- Scheduling moves packets out of queues based on priority and available bandwidth, ensuring traffic flow continuity.

Table 28: Feature History Table

| Feature Name                      | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Queueing for Congestion Avoidance | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)</p> <p>*This feature is now supported on Cisco 8011-4G24Y4H-I routers.</p>                                                                                                                                                                                                                                                                                                                                                                                                       |
| Queueing for Congestion Avoidance | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8700) (select variants only*)</p> <p>You can shape traffic to control the traffic flow from queues and also configure queues to ensure certain traffic classes get a guaranteed amount of bandwidth.</p> <p>*This functionality is now supported on Cisco 8712-MOD-M routers.</p>                                                                                                                                                                                                                                          |
| Queueing for Congestion Avoidance | Release 24.3.1      | <p>Introduced in this release on: Modular Systems (8800 [LC ASIC: P100]) (select variants only*), Fixed Systems (8200) (select variants only*), Fixed Systems (8700 (P100, K100)) (select variants only*) ,</p> <p>You can shape traffic to control the traffic flow from queues and also configure queues to ensure certain traffic classes get a guaranteed amount of bandwidth.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> </ul> |

| Feature Name                      | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Queueing for Congestion Avoidance | Release 24.2.11     | <p>Introduced in this release on: Modular Systems (8800 [LC ASIC: P100]) (select variants only*)</p> <p>By placing packets in different queues based on priority, queueing helps prevent traffic congestion and ensures that high-priority traffic is transmitted with minimal delay. You can shape traffic to control the traffic flow from queues and also configure queues to ensure certain traffic classes get a guaranteed amount of bandwidth. Queueing provides buffers to temporarily store packets during bursts of traffic and also supports strategies that enable dropping lower-priority packets when congestion builds up.</p> <p>*This feature is supported on 88-LC1-36EH.</p> |

## Queuing modes

Your router line cards support two fixed queuing modes—8 xVOQ and 4xVOQ—that determine how many VOQs and related memory resources the system allocates to every interface. Selecting the appropriate mode lets you balance traffic-class isolation against logical-interface scale and is a mandatory first step before you apply any interface-level QoS policy.



**Remember** From Release 7.2.12 onward, all Layer 3 queuing capabilities also apply to Layer 2 physical and bundle interfaces, but not to their sub-interfaces.

**Table 29: Comparison of 8xVOQ and 4xVOQ queuing modes**

| Attribute                          | 8xVOQ                                      | 4xVOQ                                                                                           |
|------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------|
| VOQs per interface                 | 8                                          | 4                                                                                               |
| Supported internal traffic classes | 8 (TC0–TC7), each mapped to a separate VOQ | 8 mapped to 4; traffic classes must be remapped in the QoS policy                               |
| Logical-interface scale            | Standard                                   | Approximately double (since fewer VOQs per interface, the system supports twice the interfaces) |

| Attribute               | 8xVOQ                                             | 4xVOQ                                                                                                               |
|-------------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Default queue hierarchy | P1 + P2 + 6 PN hierarchy (main interface default) | Same hierarchy, but applies to only four VOQs                                                                       |
| Typical use case        | Maximum traffic-class isolation                   | High interface fan-out (number of active interfaces per line card or NPU) where fewer dedicated VOQs are acceptable |



**Note** P1 + P2 + 6 PN refers to the default queuing and scheduling hierarchy where:

- P1—one hardware queue reserved for the single most critical class (TC 7).
- P2—one queue for the next-most-critical class (TC 6).
- 6 PN—six normal-priority queues, one each for TC 5, 4, 3, 2, 1, 0

That adds up to the eight VOQs that the 8xVOQ mode allocates per interface. If you move the router to 4xVOQ mode, TC7 still maps to P1 and TC 6 to P2, but the eight internal traffic classes must be remapped so that the four VOQs can carry multiple classes.

In both modes, queues are allocated per interface regardless of policy applied.

## Switch the queuing mode on the router

Use this task to change the system-wide Virtual Output Queue (VOQ) mode—8xVOQ for maximum traffic-class isolation or 4xVOQ for greater interface scale.

The VOQ mode change applies to the entire chassis. The change takes effect only after every line card reloads, which interrupts traffic.

### Before you begin

Make a list of interfaces (main, sub-interfaces, and bundles) with egress queuing service policies attached.

Follow these steps to switch the queuing mode on the router.

### Procedure

**Step 1** Verify the current VOQ mode.

#### Example:

```
Router#show hw-module profile qos voq-mode
Current system VOQ mode: 8
```

This command confirms the chassis is running in 8xVOQ mode before you begin.

**Step 2** Detach every egress queuing service policy from all interfaces.

#### Example:



```

Router#configure
Router(config)#interface TenGigE0/0/0/0
Router(config-if)#no service-policy output Q-OUT
Router(config-if)#exit
Router(config)#interface Bundle-Ether10
Router(config-if)#no service-policy output Q-OUT
Router(config-if)#exit
! ...repeat "no service-policy output ..." for every main interface, sub-interface, and bundle
Router(config)#commit
Router(config)#exit

```

Remove only the egress queuing (**service-policy output ...**) bindings. Retain ingress classification or marking policies.

**Step 3** Configure the new VOQ mode and commit.

**Example:**

```

Router#configure
Router(config)#hw-module profile qos voq-mode 4
Router(config)#commit
Router(config)#exit

```

The **voq-mode 4** setting applies to the entire chassis and persists across future reboots.

**Step 4** Reload all nodes to program the new mode into the ASICs.

**Example:**

```

Router#admin
Router(admin)#reload location all
Proceed with reload the following node(s): 0/0/CPU0 0/RP0/CPU0 [confirm] y

```

All line cards reload. Traffic is interrupted until the router returns to the IOS XR prompt.

**Step 5** Verify that the router is now running in 4xVOQ mode.

**Example:**

```

Router#show hw-module profile qos voq-mode
Current system VOQ mode: 4

```

Successful output confirms the chassis is operating in the new VOQ mode.

---

### What to do next

After the change, review your QoS policies to be sure each hardware queue now aggregates the intended traffic classes, especially when you reduce the queue count from 8 to 4.

## Queuing policy for main interfaces

After you set the chassis to 8xVOQ or 4xVOQ (see [Switch the Queuing Mode on the Router](#)), each main interface automatically receives the queue hierarchy described here. Use this information to verify how each internal traffic class is handle and to decide whether you need a custom egress-queuing policy on the main interface.

The 8xVOQ configuration allocates: P1 to TC 7, P2 to TC 6, and PN0–PN5 to TC 5 through 0. (See [Queuing modes, on page 83](#))

Next steps:

- If the default priority and bandwidth distribution is acceptable, no further action is required.
- To change scheduling weights, shaping rates, or WRED profiles, create an egress queuing policy and attach it to the main interface (see *Attach a Queuing Policy to an Interface*).

## Queuing policy behavior on subinterfaces

*Table 30: Feature History Table*

| Feature Name                 | Release Information | Feature Description                                                                                                                                                                                                                                                   |
|------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Subinterface Queueing Policy | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is supported on Cisco 8011-4G24Y4H-I routers.                                                                                                            |
| Subinterface Queueing Policy | Release 24.4.1      | Introduced in this release on: Fixed Systems (8700) (select variants only*)<br><br>You can manage traffic flows with fine granularity by configuring QoS policies at the subinterface level.<br><br>*This functionality is now supported on Cisco 8712-MOD-M routers. |

| Feature Name                 | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Subinterface Queueing Policy | Release 24.3.1      | <p>Introduced in this release on:<br/> Modular Systems (8800 [LC ASIC: P100]) (select variants only*),<br/> Fixed Systems (8200) (select variants only*), Fixed Systems (8700 (P100, K100)) (select variants only*)<br/> Modular Systems (8800 [LC ASIC: P100]) , Fixed Systems (8200) , Fixed Systems (8700 (P100, K100)) (select variants only*)</p> <p>You can manage traffic flows with fine granularity by configuring QoS policies at the subinterface level.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> </ul> |
| Subinterface Queueing Policy | Release 24.2.11     | <p>Introduced in this release on:<br/> Modular Systems (8800 [LC ASIC: P100]) (select variants only*)</p> <p>To manage traffic flows with fine granularity, you can configure QoS policies at the subinterface level. Also, with QoS support on mixed Layer 2 and Layer 3 subinterfaces under the same main interface, you can ensure that multiple traffic types with varying QoS requirements coexist on a single main physical interface.</p> <p>*This feature is supported on 88-LC1-36EH.</p>                                                                                                                                                           |

A subinterface uses the main interface VOQs by default. When you apply an egress queuing policy, the router allocates a dedicated set of VOQs—4 or 8, depending on the chassis-wide VOQ mode you selected earlier. Each subinterface can host up to three QoS policies (ingress, egress marking, and egress queuing). If you remove the egress-queuing policy later, its VOQs are released and the subinterface traffic reverts to the main interface queues.

Use this table to understand the limits and defaults that apply after a policy is attached.

| Attribute              | 8xVOQ          | 4xVOQ          |
|------------------------|----------------|----------------|
| VOQs per sub-interface | 8              | 4              |
| Default hierarchy      | P1 + P2 + 6 PN | P1 + P2 + 2 PN |

**Note**

- P1—one hardware queue reserved for the single most critical class (typically TC 7).
- P2—one queue for the next-most-critical class (typically TC 6).
- PN (normal priority queues):
  - In 8xVOQ mode—one queue each for TC5, TC4, TC3, TC2, TC1, and TC0 (six PN queues).
  - In 4xVOQ mode—one queue each for TC1 and TC0 (two PN queues)

A line card has a fixed pool of VOQs and scheduler state blocks, so there is a limit to the number of policies you can deploy before resources are exhausted. To monitor the remaining VOQ and scheduler resources by viewing packet and byte counts per traffic class, as well as drops, which can indicate resource constraints, run the **show controllers npu stats voq** command.

Next steps:

1. Decide which traffic classes need their own scheduler or shaper.
2. Configure an egress queuing policy that defines those actions.
3. Apply the policy with **service-policy output <policy-name>** under the subinterface.
4. Verify that the policy is active with **show qos interface**.

## Congestion detection and handling

Your router assigns a virtual output queue (VOQ) to each traffic class, which uses both on-chip packet buffers and high-bandwidth memory (HBM). If the VOQ becomes congested, the router places excess packets in HBM and applies congestion-control mechanisms such as dynamic drop thresholds or packet marking.

The table lists congestion detection and handling techniques. Each technique includes a brief description and an analogy to help you relate it to a real-world scenario.

**Table 31: Congestion handling techniques and their analogies**

| Technique                                    | Short Description                                                                                                                       | Highway Traffic Analogy                                                                                         |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <a href="#">Tail drop and the FIFO queue</a> | Drops incoming packets once the queue reaches its configured limit; processes packets in arrival order with no traffic differentiation. | Like a toll booth that shuts its gate when the line of cars reaches the maximum allowed length (no exceptions). |

| Technique                                                         | Short Description                                                                                                                                   | Highway Traffic Analogy                                                                                              |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <a href="#">Congestion avoidance in VOQs, on page 93</a>          | Moves a VOQ from on-chip Shared Memory System (SMS) to HBM when congestion thresholds are exceeded.                                                 | Like diverting cars from the main highway lane to an overflow lane when the main lane gets jammed.                   |
| <a href="#">Dual queue limit, on page 93</a>                      | Configures two queue thresholds—one for high-priority traffic and one for low-priority traffic—to ensure critical flows continue during congestion. | Like having an express lane that closes early to non-emergency vehicles so ambulances can always get through.        |
| <a href="#">Equitable traffic flow using fair VOQ, on page 96</a> | Allocates dedicated Virtual Output Queues (VOQs) per source to destination port pair to ensure fair bandwidth distribution across ports.            | Like giving every on-ramp its own dedicated lane to the highway so no one ramp hogs the traffic flow.                |
| <a href="#">Random Early Detection and TCP, on page 103</a>       | Randomly drops packets before queues fill to signal senders to slow down, preventing sudden congestion.                                             | Like a traffic light that occasionally turns red early to prevent gridlock at an intersection.                       |
| <a href="#">Explicit Congestion Notification, on page 106</a>     | Marks packets instead of dropping them to signal congestion, preserving flow without packet loss.                                                   | Like flashing a warning sign telling drivers to slow down before a traffic jam, instead of stopping them completely. |
| <a href="#">Virtual Output Queue Watchdog, on page 109</a>        | Detects and clears VOQs that are stuck and not transmitting, even when packets are queued.                                                          | Like a traffic sensor detecting a blocked lane and triggering a tow truck to clear it.                               |

## Tail drop and the FIFO queue

A tail-drop queue management method is a congestion avoidance mechanism that:

- discards new packets once the output queue is full
- treats every flow identically and,
- operates on a first-in-first-out (FIFO) buffer that drains at the line-rate of the interface.

### Key attributes of tail drop queue management

This table helps you understand the main features of tail drop queuing, including its discipline, drop policy, fairness, and how it is typically used to manage congestion.

| Attribute        | Description                                                           |
|------------------|-----------------------------------------------------------------------|
| Queue discipline | Packets exit in the order in which they arrive (First In, First Out). |

| Attribute   | Description                                                                                                                                                                                                                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Drop policy | When the queue reaches its limit, each subsequent packet is dropped until the backlog decreases below the limit.                                                                                                                                                                                |
| Fairness    | Does not differentiate between classes of traffic. It works best when simple best-effort service is acceptable.                                                                                                                                                                                 |
| Typical use | Basic congestion protection, especially on devices or on network links with limited buffer memory. It is commonly used as a default-queue management mechanism. You can also use tail drop as a foundation for more advanced QoS policies, such as priority queuing with explicit queue limits. |

## How tail drop works

In packet-switched networks, congestion occurs when traffic arriving at an interface exceeds the available buffer capacity.

Tail drop is a common approach. When the queue reaches its configured limit, all subsequent packets are discarded until space becomes available. This mechanism protects buffer memory and provides a straightforward form of congestion control.

### Summary

The tail drop process consists of these key components:

- Queuing subsystem: Enqueues incoming packets until the queue limit is reached.
- Scheduler: Dequeues and transmits packets in FIFO order from the output queue.
- Packet sender: Retransmits packets that are dropped.

Packets are enqueued in FIFO order until the queue reaches its configured limit. At that point, any new packets are tail-dropped (discarded). The sender must retransmit those packets. Once the scheduler transmits enough packets and the queue depth falls below the limit, enqueueing resumes.

Enqueueing resumes after the scheduler transmits enough packets and the queue depth falls below the limit.

### Workflow

These stages describe how the tail-drop process protects buffer memory on an egress queue.

1. When the queuing subsystem enqueues a packet, it is accepted if the queue depth is below the queue limit.
2. When a new packet arrives and queue depth equals the queue limit, the packet is tail-dropped and the sender must retransmit.
3. The scheduler transmits packets in FIFO order from the output queue. If the queue depth falls below the limit, packet enqueueing resumes.

## Guidelines for configuring tail drop thresholds

Always configure queue-limit with priority, shape average, bandwidth, or bandwidth remaining. The only exception is for the default class. If you do not follow this approach, the router rejects the policy commit.

### Efficient use of rate class profiles with tail drop thresholds

Fine-tune tail drop threshold values so that multiple traffic classes can share the same rate class profile.

This principle applies when configuring tail drop thresholds on routers where each ASIC supports a maximum of 64 rate class profiles. Rate class profiles define performance characteristics such as bandwidth limits, priority levels, queuing policies, and drop thresholds.

Because each ASIC can support only 64 rate class profiles, creating a unique profile for every traffic class quickly consumes available hardware resources. By allowing multiple traffic classes with the same tail drop threshold to share a profile, you conserve hardware resources and design a more scalable system.

If you configure each traffic class with a unique tail drop threshold, the device may exhaust the limited pool of available rate class profiles. This can lead to inefficient hardware utilization and restrict further QoS configuration flexibility.

Review your traffic classes and adjust tail drop thresholds so that classes with similar performance needs use the same profile. This approach preserves rate class profiles for essential use cases. It also ensures that critical services receive the required bandwidth and latency treatment.

## Configure a tail drop threshold for a traffic class

Use this task to define the maximum queue length for a traffic class and set service attributes, so that packets beyond this limit are dropped.

You configure tail drop for a class in a policy map and then attach the policy map to an interface. You must specify queue limits and set either priority, average shaping, or bandwidth for non-default classes.

### Before you begin

- Determine your traffic class and required tail drop threshold.
- Identify the interface for policy attachment.

Follow these steps to configure a tail drop threshold for a traffic class.

### Procedure

**Step 1** Define a class map and match the traffic class.

#### Example:

```
Router(config)#class-map qos-1
Router(config-cmap)#match traffic-class 1
Router(config-cmap)#commit
```

This step specifies the match criterion for a class map. In this example, packets belonging to **class 1** are matched. Only traffic identified as **class 1** will be selected for the policy actions associated with this class map.

**Step 2** Create or modify a policy map and assign the class.

#### Example:

```
Router(config)#policy-map test-qlimit-1
Router(config-pmap)#class qos-1
```

This example specifies a class named **qos-1** within the policy map for further configuration.

**Step 3** Set the queue limit for the selected class.

**Example:**

```
Router(config-pmap-c)#queue-limit 100 us
```

This example sets the maximum queue length for the class to the buffer depth that corresponds to 100 microseconds of traffic at the interface rate. Packets arriving after this limit is reached are dropped (tail drop).

**Step 4** Assign a priority level to the class.

**Example:**

```
Router(config-pmap-c)#priority level 7
```

**Step 5** Enter interface configuration mode for the specified interface.

**Example:**

```
Router(config)#interface HundredGigE 0/6/0/18
```

**Step 6** Apply the QoS policy map to the interface for outbound traffic.

**Example:**

```
Router(config)#interface HundredGigE 0/6/0/18
Router(config-if)#service-policy output test-qlimit-1
Router(config-if)#commit
```

This example attaches the policy map named **test-qlimit-1** to the interface as an output (egress) service policy. All traffic leaving the interface is subject to the QoS rules defined in **test-qlimit-1**.

**Step 7** Verify the tail drop and queue configuration on an interface.

**Example:**

```
Router#show qos int hundredGigE 0/6/0/18 output
Interface HundredGigE0/6/0/18 ifh 0x3000220 -- output policy

NPU Id:                               3
Total number of classes:               2
Interface Bandwidth:                   100000000 kbps
VOQ Base:                             11176
VOQ Stats Handle:                      0x88550ea0
Accounting Type:                       Layer1 (Include Layer 1 encapsulation and above)

-----

Level1 Class (HP7)                     = qos-1
Egressq Queue ID                       = 11177 (HP7 queue)
TailDrop Threshold                     = 1253376 bytes / 100 us (100 us)
WRED not configured for this class

Level1 Class                           = class-default
Egressq Queue ID                       = 11176 (Default LP queue)
Queue Max. BW.                         = 101803495 kbps (default)
Queue Min. BW.                         = 0 kbps (default)
Inverse Weight / Weight                 = 1 (BWR not configured)
TailDrop Threshold                     = 1253376 bytes / 10 ms (default)
```

- **qos-1** is shown as a high-priority class (**HP7**) with a queue-limit of 100 us.



- The **class-default** queue uses default values and does not have explicit priority or queue limit overrides.
- Both classes show **WRED not configured for this class** because Weighted Random Early Detection (WRED) was not set.

---

The specified class will drop packets at the configured queue threshold, optimizing queue behavior according to the class's service requirements.

## Congestion avoidance in VOQs

Congestion avoidance in a VOQ protects the on-chip Shared Memory System (SMS) from overruns. When a VOQ's SMS occupancy exceeds its eviction threshold, subsequent packet buffers for that VOQ are allocated from High-Bandwidth Memory (HBM). While the VOQ uses HBM, congestion controls from its policy map, such as Random Early Detection (RED) or Explicit Congestion Notification (ECN), apply. If no congestion controls are configured, the queue relies on tail drop. When the occupancy falls below the return threshold, buffer allocation shifts back to SMS.

**Shared Memory System (SMS):** SMS is the on-chip packet-buffer subsystem that provides primary buffer space for each virtual output queue (VOQ) under normal traffic conditions. It offers low latency and serves as the first stage of packet storage before any spillover to external memory.

**High-Bandwidth Memory (HBM):** HBM is an external, high-capacity packet-buffer attached to the NPU. When a VOQ's SMS occupancy exceeds its eviction threshold, subsequent packet buffers for that VOQ are allocated from HBM. Once the queue drains below the return threshold, buffer allocation shifts back to SMS.



---

**Remember** RED applies only to VOQs that are currently in HBM. Weighted Random Early Detection (WRED) is not supported.

---

## Dual queue limit

A dual queue limit is a pair of length thresholds that

- gives you flexibility to set two queue limits on a single policy map
- an egress VOQ enforces based on the discard class—0 for low-priority or 1 for high-priority traffic—that you mark on the packets in an ingress policy, and
- keeps critical traffic flowing even when buffer space is scarce.

### Queue threshold limits for discard classes

When the queue grows:

- packets marked **discard-class 1** stop being enqueued once the queue length reaches the lower limit.
- packets marked **discard-class 0** can continue to fill the queue until it reaches the higher limit; only then are those packets dropped.

## Guidelines for configuring dual queue limits

This reference lists the guidelines you must follow when configuring dual queue limits. Dual limits use two thresholds (for **discard-class 0** and **discard-class 1**) to control how different priority packets are dropped as the queue grows. Observing these guidelines ensures predictable drop behavior, prevents invalid configurations, and accounts for how NPUs enforce thresholds.

**Table 32: Guidelines and rationale while configuring dual queue limits**

| Guideline                                                                                                                      | Rationale                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use the same unit—bytes, packets, or time—for both limits.                                                                     | Mixed units are rejected.                                                                                                                                                                                                                                                                     |
| Set the <b>discard-class 0</b> limit so that it is higher than the <b>discard-class 1</b> limit.                               | Enforces a logical and deterministic drop behavior, ensuring that higher priority traffic is less likely to be dropped compared to lower priority traffic. Reversed values are rejected to maintain proper queue management and traffic prioritization.                                       |
| Configure both limits to ensure a valid configuration. Defining only one discard class makes the queue limit invalid           | By configuring both <b>discard-class 0</b> and <b>discard-class 1</b> limits, the router avoids ambiguous or undefined drop actions. This means that when the queue reaches these thresholds, packets are dropped in a controlled and expected manner according to their priority and limits. |
| Configure the thresholds per your requirement. The ASIC rounds each threshold value to the nearest hardware buffer block size. | Each NPU enforces limits only in fixed-size buffer blocks. Any value you enter is converted to the closest multiple, so the number shown in <b>show qos interface</b> output may differ slightly from what you configured, but the enforced limit matches that rounded value.                 |



### Remember

Dual limits protect against loss, not necessarily latency. This is because priority traffic may experience delays behind large low-priority bursts.

## Configure dual queue limits

### Before you begin

Ensure that packets are already classified and marked with **discard-class 0** or **discard-class 1** on ingress.

Follow these steps to configure dual queue limits on an egress policy map.

### Procedure

**Step 1** Create the egress policy map.

#### Example:

```
router(config)#policy-map EGRESS_DQL
```

**Step 2** Enter the class that requires the dual limit.

**Example:**

```
router(config-pmap)#class TC7
```

**Step 3** Set the high-priority threshold.

**Example:**

```
router(config-pmap-c)#queue-limit discard-class 0 100 mbytes
```

This example specifies that packet drops (or, RED or ECN marking, if configured) start when the queue's total occupancy for all packets reaches about 100 MB. The NPU rounds this value to the nearest hardware buffer-block size before enforcing it.

**Step 4** Set the low-priority threshold.

**Example:**

```
router(config-pmap-c)#queue-limit discard-class 50 mbytes
```

This example specifies that the queue start dropping (or marking) packets as soon as the total queue occupancy reaches about 50 MB.

**Step 5** (Optional) Make the class strict-priority.

**Example:**

```
router(config-pmap-c)#priority level 1
router(config-pmap-c)#exit
```

This example instructs the scheduler to treat this class as strict-priority (low-latency) traffic. The class still follows the discard-class thresholds you set earlier (discard-class 1 drops at ~50 MB; discard-class 0 drops at ~100 MB).

**Step 6** (Optional) Specify a bandwidth remaining weight for the default class.

**Example:**

```
Router(config-pmap)#class class-default
router(config-pmap-c)#bandwidth remaining ratio 1
router(config-pmap-c)#exit
```

This is an optional step. If you omit this command, the bandwidth remaining ratio defaults to 1.

**Step 7** Verify that the dual queue limits are active on the egress interface.

**Example:**

```
Router#show qos interface hundredGigE 0/0/0/30 output
NOTE:- Configured values are displayed within parentheses Interface HundredGigE0/0/0/30 ifh 0xf000210
-- output policy NPU Id:      0
Total number of classes:      2
Interface Bandwidth:          100000000 kbps
Policy Name:                  egress_pol_dql
VOQ Base:                     464
Accounting Type:              Layer1 (Include Layer 1 encapsulation and above) VOQ Mode:      8
Shared Counter Mode:          1
  Level1 Class (HP1)          = tc7
Egressq Queue ID              = 471 (HP1 queue)
Queue Max. BW.                = no max (default)
Discard Class 1 Threshold      = 25165824 bytes / 2 ms (50 mbytes)
Discard Class 0 Threshold      = 75497472 bytes / 5 ms (100 mbytes)
```

```

WRED not configured for this class

Level1 Class          =      class-default
Egressq Queue ID      =      464 (Default LP queue)
Queue Max. BW.        =      no max (default)
Inverse Weight / Weight =      1 / (1)
TailDrop Threshold    =      749568 bytes / 6 ms (default)

WRED not configured for this class

```

What to look for:

- **Discard Class 1 Threshold** shows the lower limit (~ 50 MB, rounded to the nearest hardware block).
- **Discard Class 0 Threshold** shows the upper limit (~ 100 MB, also rounded).
- Both values confirm enforcement of the dual-queue-limit pair on **egress queue ID 471** for **class tc7**.
- **class-default** appears separately with its own tail-drop threshold and bandwidth-remaining weight. This is not affected by the dual limits.

## Equitable traffic flow using fair VOQ

A Fair Virtual Output Queue (VOQ) is a queuing-allocation mechanism that

- allocates a dedicated VOQ for every source-port and destination-port pair
- distributes the bandwidth of each destination port evenly across all requesting source ports, and
- supports 4-queue (fair-4) or 8-queue (fair-8) modes that you enable with **hw-module profile qos voq-mode**.

Per default behavior, an NPU slice shares four or eight VOQs per destination port; multiple source ports can therefore compete for the same queue and starve each other. Fair VOQ removes that contention by giving every port its own VOQ.

**Table 33: Feature History Table**

| Feature Name                          | Release Information | Feature Description                                                                                                                                            |
|---------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Equitable Traffic Flow Using Fair VOQ | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is now supported on Cisco 8011-4G24Y4H-I routers. |

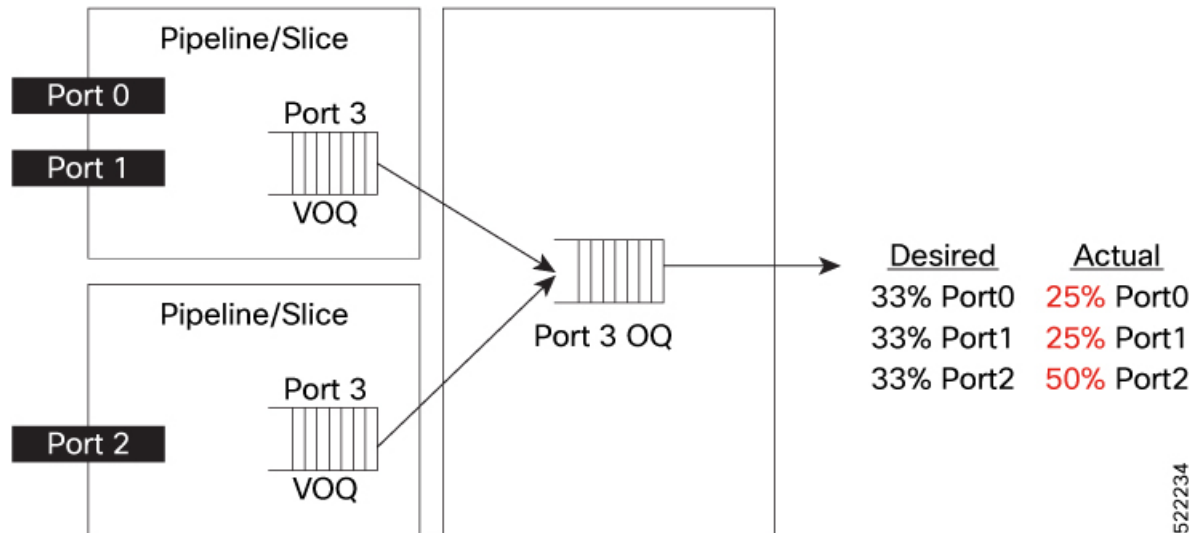
| Feature Name                                                     | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Equitable Traffic Flow Using Fair VOQ                            | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 8712-MOD-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul>                                                                                                                                                                                                                                                          |
| Equitable Traffic Flow Using Fair VOQ on Cisco 8201-32FH Routers | Release 7.5.2       | You can now ensure that the bandwidth available at the destination port for a given traffic class is distributed equally to all source ports requesting bandwidth on Cisco 8201-32FH routers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Equitable Traffic Flow Using Fair VOQ                            | Release 7.3.3       | <p>Configuring this feature ensures that ingress traffic from various source ports on every network slice of an NPU is assigned a unique virtual output queue (VOQ) for every source port and destination port pair. This action ensures that the bandwidth available at the destination port for a given traffic class is distributed equally to all source ports requesting bandwidth.</p> <p>In earlier releases, the traffic wasn't distributed equitably because each slice wasn't given its fair share of the output queue bandwidth.</p> <p>This feature introduces the <b>fair-4</b> and <b>fair-8</b> keywords in the <b>hw-module profile qos voq-mode</b> command.</p> |

### Balancing traffic shares between slices

Consider two 100 G ports (port 0 and port 1) on slice 0 and one 100 G port (port 3) on slice 1, all sending traffic to the same destination port.

Without Fair VOQ, port 0 and port 1 share a single ingress VOQ and each receives ~25 % of the egress buffer traffic. The ingress VOQ in slice-1 is available exclusively for port-3, so port 3 receives ~50 % of the buffer traffic.

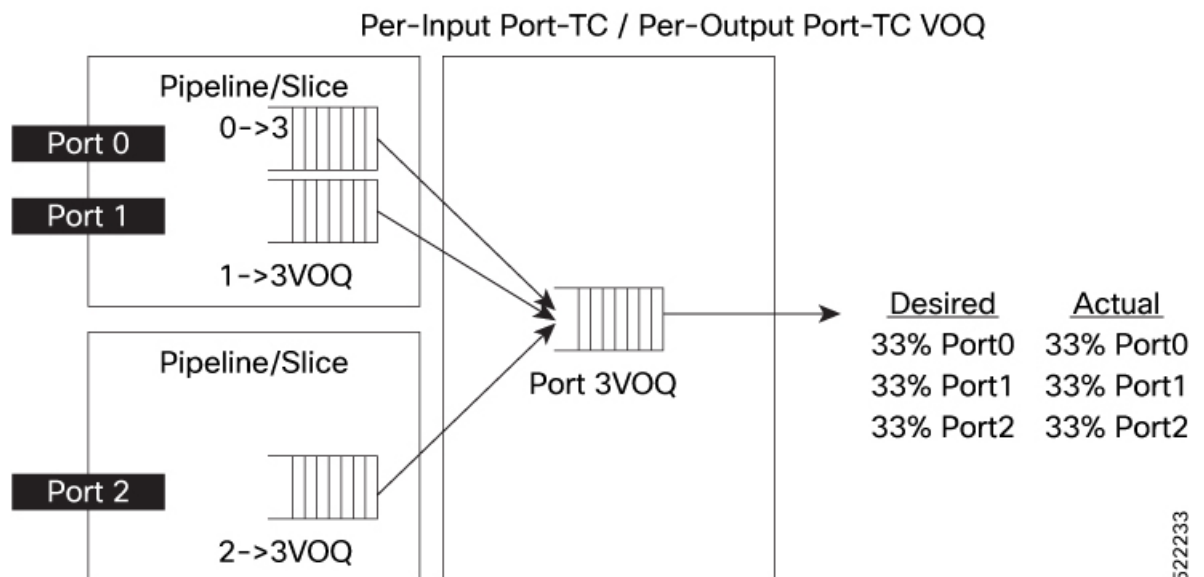
**Figure 6: Existing behavior : Source ports on slice share one VOQ per destination port**



522234

With Fair VOQ enabled, each ingress port has its own VOQ, so all three ports receive an equal 33 % share.

**Figure 7: Fair VOQ behavior: each source port on slice has one dedicated VOQ per destination port**



522233

## Configuration options for fair VOQ modes and sharing of counters

You can configure fair VOQ for 8xVOQ mode (**fair-8**) and 4xVOQ mode (**fair-4**) using these options in the **hw-module profile qos voq-mode** command:

- **hw-module profile qos voq-mode fair-8**
- **hw-module profile qos voq-mode fair-4**

You can also share VOQ statistics counters in both fair VOQ modes. The table presents the supported counter options and other relevant information. (For details on why sharing counters is essential and how to configure counters sharing, see [Sharing of VOQ statistics counters, on page 115](#).)

**Table 34: Fair VOQ modes and sharing counters**

| Fair VOQ mode | Dedicated counter (1 per VOQ) | Two VOQs share a counter | Four VOQs share a counter | Notes                                                                                                                  |
|---------------|-------------------------------|--------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------|
| fair-8        | Not supported                 | Supported                | Supported                 | Dedicated counters not supported in fair-8 mode. Use 2-per-counter for non-breakout and 4-per-counter for 4x breakout. |
| fair-4        | Supported                     | Supported                | Supported                 | Recommended: 2-per-counter for non-breakout, 4-per-counter for 4x breakout.                                            |

## Guidelines and limitations for fair VOQ

### Router support

Fair VOQ is available only on:

- Cisco 8202 routers—12 × QSFP56-DD 400 G ports and 60 × QSFP28 100 G ports.
- Cisco 8201-32FH routers—32 × QSFP-DD 400 G ports.

### Router reload required

After you configure **hw-module profile qos voq-mode** in the **fair-4** or **fair-8** mode (and the matching counter-sharing value), you must reload the router for the new VOQ map to take effect.

### Layer 2 traffic not supported

You cannot use fair VOQ for bridged (Layer 2) frames in either **fair-4** or **fair-8** mode because these frames are forwarded by slice-level queues.

### Subinterface queuing unsupported

You cannot attach egress queuing service-policies to subinterfaces or bundle subinterfaces when fair VOQ is enabled, because those policies require dedicated VOQs. However, egress marking policies on subinterfaces are still supported.

## ERSPAN mirroring traffic unsupported

If you rely on Encapsulated Remote Switched Port Analyzer (ERSPAN), disable fair VOQ because ERSPAN session traffic bypasses the fair VOQ mechanism.

## Counter-sharing restrictions

**hw-module profile stats voqs-sharing-counters 1** is not supported with **fair-8**. Configure one of these valid pairs before you reload:

- **voq-mode fair-8** with voqs-sharing-counters 2 or 4
- **voq-mode fair-4** with voqs-sharing-counters 1, 2, or 4

If your applications use many hardware counters, do not use counter-share 2 because it may exhaust the available pool.

In these environments, use counter-share 4 or dedicated counters in **fair-4** mode.

## Breakout limited to 400 G ports

In fair VOQ mode (**fair-4** or **fair-8**) on Cisco 8202 routers, physical-layer breakout is supported only on 400 G ports. 100 G breakout ports inherit their parent's VOQ allocation

## Additional CLI visibility

When you enable fair VOQ, the **show controllers npu stats voq** command adds the **src-interface** (logical ingress interface) and **src-slice** (NPU slice ID) fields. These fields are absent in slice-based VOQ mode.

## Maximum supported interfaces by fair VOQ and counter sharing modes

The table shows how many interfaces are supported with only basic IPv4 settings—no QoS policies, ACLs, or sub-interfaces configured—under each combination of VOQ mode and counter-sharing option.

*Table 35: Maximum interfaces based on fair VOQ Mode and counter sharing modes*

| VOQ Mode      | Sharing Counter Mode | Maximum Interfaces for Cisco 8202            | Maximum Interfaces for Cisco 8201-32FH       |
|---------------|----------------------|----------------------------------------------|----------------------------------------------|
| <b>fair-8</b> | 1                    | The router doesn't support this combination. | The router doesn't support this combination. |



| VOQ Mode      | Sharing Counter Mode | Maximum Interfaces for Cisco 8202                                                                                 | Maximum Interfaces for Cisco 8201-32FH                        |
|---------------|----------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>fair-8</b> | 2                    | 96 = 60 (100G) + 8x4 + 4 (400G) ==> you can configure only eight 400G interfaces in 4x10G or 4x25G breakout mode. | 128 = 32x4 (4x10G or 4x25G - breakout on all 32 ports - 400G) |
| <b>fair-8</b> | 4                    | 108 = 60 + 12 x 4 (breakout on all 12 ports - 400G)                                                               |                                                               |
| <b>fair-4</b> | 1                    | 96 = 60(100G) + 8x4 + 4 (400G) ==> you can configure only eight 400 G interfaces in 4x10G or 4x25G breakout mode. |                                                               |
| <b>fair-4</b> | 2                    | 108 = 60 + 12 x4 (breakout on all 12 ports - 400G)                                                                |                                                               |
| <b>fair-4</b> | 4                    | 108 = 60 + 12 x4 (breakout on all 12 ports - 400G)                                                                |                                                               |

## Configure fair VOQ

Allocate a dedicated VOQ for every source-port and destination-port pair so that all ingress ports share egress buffers and scheduler time evenly.

Run this task on routers that support this feature when you need to remove slice-level VOQ contention. This configuration is typically used for 'many-to-one' traffic patterns, such as when many 100 G access links feed a few 400 G uplinks.

### Before you begin

- Choose your VOQ mode:
  - fair-8**: eight queues per port pair (finest statistics granularity)
  - fair-4**: four queues per port pair (largest interface scale)

Choose the counter sharing mode:

- 4**: halves VOQ counter usage; best for routers that already track many ACL, QoS, or NetFlow statistics.
- 2**: finer VOQ stats but doubles counter consumption; use only if you have free counter capacity.

Perform these steps on the active route processor.

## Procedure

**Step 1** Set the counter-sharing value.

**Example:**

```
Router(config)#hw-module profile stats voqs-sharing-counters 4
```

Replace **4** with **2** if finer VOQ accounting is needed and spare counters are available.

**Step 2** Enable fair VOQ.

**Example:**

```
Router(config)#hw-module profile qos voq-mode fair-8
```

For higher port scales, choose **fair-4**.

**Note**

If you configure **fair-8** mode without counter sharing, configuration failures or unexpected behaviors may occur

**Step 3** Commit the profile changes, then reload the entire chassis.

**Example:**

```
Router(config)#commit
Router#reload location all
```

**Step 4** Verify that fair VOQ is active.

**Example:**

```
Router#show controllers npu stats voq ingress interface hundredGigE 0/0/0/20 instance 0 location
0/RP0/CPU0
```

```
Interface Name      = Hu0/0/0/20
Interface Handle    = f000118
Location            = 0/RP0/CPU0
Asic Instance       = 0
Port Speed(kbps)    = 100000000
Local Port          = local
Src Interface Name   = ALL
VOQ Mode            = Fair-8
Shared Counter Mode = 4

ReceivedPkts    ReceivedBytes    DroppedPkts    DroppedBytes
-----
TC_{0,1} = 11110    1422080        0              0
TC_{2,3} = 0         0              0              0
TC_{4,5} = 0         0              0              0
TC_{6,7} = 0         0              0              0
```

Check for:

- VOQ Mode : Fair-8
- Shared Counter Mode : 4

The router runs in fair VOQ mode. Every ingress port owns eight dedicated queues per destination port, ensuring equitable buffer usage and scheduler time.

### What to do next

If you need sub-interface queuing later, remove the profile (**no hw-module profile qos voq-mode**), commit, and reload to return to slice-based VOQs.

## Random Early Detection and TCP

Random early detection (RED) is a congestion avoidance mechanism that:

- proactively manages queue lengths by randomly dropping packets before a queue becomes full
- interacts with congestion control protocols such as Transmission Control Protocol (TCP) by signaling sources to reduce their transmission rate and,
- helps maintain network stability and performance.

### Understanding Random Early Detection Queue Size and TCP Congestion Control

- RED operates on the average queue size—not the instantaneous queue size.
- When RED is enabled, it starts dropping packets at a configurable rate as congestion occurs, prompting TCP senders to slow down.

### RED packet drops for TCP congestion notification

If a network interface begins to experience congestion, RED randomly drops packets to notify TCP sources to reduce their sending rate before the queue overflows.

### Random Early Detection as a traffic light for congestion control

RED is like a traffic light that occasionally turns red early to prevent cars from piling up at an intersection and smooth the overall flow.

## Guidelines for configuring Random Early Detection

### Recommendation for setting Random Early Detection thresholds

Configure Random Early Detection (RED) using the **random-detect <min threshold> <max threshold>** command with a maximum threshold higher than the system-supported minimum threshold value. This approach prevents configuration errors."

### Requirement for configuring Random Early Detection with class policies

"When you configure random-detect on any class, including class-default, configure shape average or bandwidth remaining as well.

### Caution when configuring queue limits

If you configure a queue-limit lower than the system minimum, the value changes to the supported minimum.

### Tip for using Random Early Detection to smooth traffic flow

RED distributes packet drops over time, helping TCP adapt transmission rates gradually, which prevents sudden congestion and maintains low queue depths.

## How Random Early Detection works

Random Early Detection (RED) is implemented on network devices to manage congestion by signaling Transmission Control Protocol (TCP) sources to adjust their sending rates before queue overflow occurs.

### Summary

RED monitors the average queue size on an interface. When the average exceeds a minimum threshold, RED starts randomly dropping packets. If the average reaches a maximum threshold, all incoming packets may be dropped until congestion subsides. This proactive approach allows TCP to reduce its transmission rate, smoothing out traffic bursts and maintaining network performance.

### Workflow

1. The device calculates the average queue size.
2. If the average queue size exceeds the minimum threshold, RED randomly drops incoming packets at a configured probability.
3. TCP sources detect dropped packets, interpret this as congestion, and decrease their sending rates.
4. As congestion clears and the average queue size decreases, the probability of drops is reduced, allowing normal transmission rates to resume.

*Table 36: Random Early Detection behavior based on average queue size*

| When the average queue size                   | Then                                                      |
|-----------------------------------------------|-----------------------------------------------------------|
| is below the minimum threshold                | all packets are accepted.                                 |
| is between the minimum and maximum thresholds | packets are randomly dropped with increasing probability. |
| exceeds the maximum threshold                 | all packets may be dropped.                               |

### Result

RED ensures that packet loss signals are spread over time, preventing global synchronization of TCP flows and keeping queue lengths manageable.

## Configure Random Early Detection

Enable Random Early Detection (RED) on a network interface to proactively manage congestion and signal TCP sources to adjust their sending rates.

### Before you begin

- Ensure you know the system-supported minimum and maximum threshold values for RED.
- Confirm that **shape average** or **bandwidth remaining** is configured if using RED.

### Procedure

- 
- Step 1** Create or modify a policy map.

**Example:**

```
Router(config)#policy-map red-abs-policy
```

This example creates a new policy map named red-abs-policy or edits it if it already exists. Policy maps are used to define QoS policies.

**Step 2** Specify the traffic class within the policy.

**Example:**

```
Router(config-pmap)#class qos-1
```

This example selects the traffic class named qos-1. QoS classes group traffic with similar requirements.

**Step 3** Enable RED with minimum and maximum thresholds.

**Example:**

```
Router(config-pmap-c)#random-detect 10000000 20000000
```

This example enables RED for this class with a minimum threshold of 10,000,000 bytes and a maximum threshold of 20,000,000 bytes. When the average queue size exceeds 10 MB, packets are randomly dropped with increasing probability. If the queue size exceeds 20 MB, all packets may be dropped.

**Step 4** Shape the traffic for the class.

**Example:**

```
Router(config-pmap-c)#shape average percent 10
```

This example configures traffic shaping for this class so the average bandwidth does not exceed 10% of the interface's bandwidth

**Step 5** Select the output interface.

**Example:**

```
Router(config)#interface HundredGigE0/0/0/12
```

**Step 6** Apply the policy map to the interface.

**Example:**

```
Router(config-if)#service-policy output red-abs-policy
Router(config-if)#commit
```

This example applies the red-abs-policy as an output service policy on the interface. This activates the RED configuration on outbound traffic.

**Step 7** Verify QoS policy application and RED status.

**Example:**

```
show policy-map interface HundredGigE0/0/0/12 output
```

```
Interface HundredGigE0/0/0/12 output: pm-out-queue
Service-policy output: red-abs-policy
```

```
Class-map: qos-1 (match-all)
  Queueing
    queue limit 75 Mbytes
    shape average 10000000 (bps)
    random-detect 10000000 20000000 (bytes)
  Packets matched: 125000
  Bytes matched: 300000000
  ...
```

In this example:

- The policy map, **red-abs-policy**, is attached to the interface for outbound (output) traffic
- Class-map [**qos-1 (match-all)**]: Indicates the traffic class to which RED is applied.
- queue limit (**75 Mbytes**): The maximum queue length for this class.
- shape average [**10000000 (bps)**]: Traffic for this class is shaped to an average of 10 Mbps.
- random-detect [**10000000 20000000 (bytes)**]: RED is enabled with a minimum threshold of 10 MB and a maximum threshold of 20 MB.
- **Packets matched** and **Bytes matched**: Shows how many packets or bytes have been processed by this class, confirming traffic is being handled by the policy.

**Step 8** Verify RED and Explicit Congestion Notification (ECN) statistics.

**Example:**

```
show qos int HundredGigE0/0/0/12 output
```

```
Interface HundredGigE0/0/0/12 ifh 0x3000220 -- output policy
NPU Id: 3
```

```
Level1 Class = qos-1
Egressq Queue ID = 11177 (LP queue)
Queue Max. BW. = 10082461 kbps (10 %)
TailDrop Threshold = 12517376 bytes / 10 ms (default)
Default RED profile
  RED Min. Threshold = 10000000 bytes
  RED Max. Threshold = 20000000 bytes
RED random drops(packets/bytes): 150/2250000
RED ecn marked & transmitted(packets/bytes): 300/4500000
```

In this example:

- Egressq Queue ID and Queue Max. BW: Show the queue and its maximum bandwidth allocation.
- TailDrop Threshold: The maximum amount of data the queue will hold before dropping packets (if not using RED).
- Default RED profile: Confirms RED is active with the configured min/max thresholds.
- RED random drops (packets/bytes) = **150/2250000**: Number of packets and bytes that RED has randomly dropped to signal congestion.
- RED ecn marked & transmitted(packets/bytes) = **300/4500000**: Number of packets that were marked with ECN and transmitted, if ECN is enabled. This allows TCP endpoints to recognize congestion without packet loss.

## Explicit Congestion Notification

Explicit congestion notification (ECN) is a congestion avoidance mechanism that

- is an extension to the Random Early Detection (RED) mechanism
- enables routers and end hosts to detect network congestion by marking packets rather than dropping them and,

- improves overall network performance without unnecessary packet loss.

RED is a queue management algorithm that preemptively drops packets to signal congestion, using thresholds and weights for different traffic classes. ECN builds on RED by introducing packet marking, which informs endpoints of congestion without incurring packet loss.

### ECN standardization in IP networks

RFC 3168, "The Addition of Explicit Congestion Notification (ECN) to IP," specifies the use of ECN in IP networks, allowing routers to actively manage congestion signals.

## How ECN marking works

In Random Early Detection (RED), routers signal congestion by dropping packets. With Explicit Congestion Notification (ECN), routers mark packets to signal congestion to endpoints, allowing your network devices to react without data loss.

### Summary

The ECN process uses two bits in the IP header: ECT (ECN-Capable Transport) and CE (Congestion Experienced) to encode four possible states, which inform routers and endpoints of congestion status.

### Workflow

These stages describe how ECN marking and congestion signaling work in a network:

1. When packets enter the network, edge routers assign IP precedences.
2. Core routers use RED and ECN to decide packet treatment according to queue lengths and IP precedences.
3. When the average queue length is between the minimum threshold and the maximum threshold:
  - If a packet is ECN-capable (ECT=1 and CE=0, or ECT=0 and CE=1) and the RED algorithm determines that the packet would be dropped based on drop probability, the router marks it as "congestion experienced" (ECT=1, CE=1) and transmits it.
  - If a packet is not ECN-capable (ECT=0, CE=0), the router transmits it. If the queue length exceeds the maximum threshold, the router drops the packet.
  - If a packet is already marked as "congestion experienced" (ECT=1, CE=1), it is transmitted without further marking.
4. If the queue length exceeds the maximum threshold, all packets are dropped, regardless of ECN marking.

### Result

ECN reduces packet loss during congestion, improves network responsiveness, and allows endpoints to dynamically adjust sending rates in response to congestion signals.

## ECN bit combinations and their meanings

ECN bit combinations in an IP header are shown with their meaning for congestion signaling and transport behavior.

| ECT bit | CE bit | Combination | Meaning         |
|---------|--------|-------------|-----------------|
| 0       | 0      | 00          | Not ECN-capable |

| ECT bit | CE bit | Combination | Meaning                       |
|---------|--------|-------------|-------------------------------|
| 0       | 1      | 01          | ECN Capable Transport: ECT(1) |
| 1       | 0      | 10          | ECN Capable Transport: ECT(0) |
| 1       | 1      | 11          | Congestion Experienced (CE)   |

## ECN policy restrictions

Do not use ECN when configuring **qos-group** or MPLS experimental values with a traffic class in the ingress policy.

This restriction applies to ingress policies where multiple marking or classification features could conflict with ECN behavior.

## Configure ECN in a policy map

Use this procedure to enable ECN marking in a class within a policy map for congestion management.

### Before you begin

Ensure you have defined your policy map and class.

Follow these steps to configure ECN in a policy map.

### Procedure

**Step 1** Define a policy map and class, and allocate bandwidth.

#### Example:

```
Router#configure
Router(config)#policy-map policy1
Router(config-pmap)#class class1
Router(config-pmap-c)#bandwidth percent 50
```

**Step 2** Set Random Early Detection (RED) thresholds for the class.

#### Example:

```
Router(config-pmap-c)#random-detect 1000 packets 2000 packets
```

**Step 3** Enable ECN marking.

#### Example:

```
Router(config-pmap-c)#random-detect ecn
Router(config-pmap-c)#exit
Router(config-pmap)#exit
Router(config)#commit
```

**Step 4** Check the policy map statistics on the relevant interface to verify the feature.

#### Example:



```
Router#show policy-map interface HundredGigE0/0/0/35 output
```

```
HundredGigE0/0/0/35 output: egress_qosgrp_ecn
```

```
Class class1
```

```
Classification statistics          (packets/bytes)    (rate - kbps)
Matched      :    2000000/1800000000                0
Transmitted  :    1950000/1750000000                0
Total Dropped:      50000/50000000                  0
```

```
Queueing statistics
```

```
Queue ID      : 10001
Taildropped(packets/bytes) : 30000/30000000
WRED profile for
  RED Transmitted (packets/bytes)      : N/A
  RED random drops(packets/bytes)      : N/A
  RED maxthreshold drops(packets/bytes) : N/A
  RED ecn marked & transmitted(packets/bytes): 47000/45000000
```

How to read the verification output:

- **Class class1** shows statistics for the class you configured.
- **Matched:** Number of packets and bytes that match your class.
- **Transmitted:** Number of packets and bytes successfully sent.
- **Total Dropped:** Number of packets and bytes dropped for this class.
- **Taildropped:** Packets dropped because the queue length exceeded the max threshold (2000 packets).
- **RED ecn marked & transmitted:** Number of packets that were marked with ECN and transmitted (instead of being dropped).

For example, 47,000 packets (45,000,000 bytes) were ECN marked and sent, as shown by **47000/45000000**.

- **RED random or maxthreshold drops** are not applicable (N/A) in this ECN-only configuration
- The **RED ecn marked & transmitted** value shows how effective ECN marking is. If this number increases, ECN works as intended
- Taildropped packets should be minimal if ECN is reducing congestion effectively.

## Virtual Output Queue Watchdog

A Virtual Output Queue (VOQ) watchdog is a congestion monitoring mechanism that

- detects and handles Virtual Output Queues (VOQs) that remain idle despite containing packets and,
- ensures QoS compliance by recovering from hardware or traffic-related stalls in queue transmission.

A VOQ is considered stuck if it contains packets but does not transmit for over one minute.

### VOQ Watchdog in Q100 and Q200 ASICs for traffic disruption prevention

On Cisco 8000 routers using Silicon One Q100 or Q200 ASICs, the VOQ watchdog helps prevent traffic flow disruption caused by:

- Priority Flow Control (PFC) pauses
- Starvation of low-priority traffic classes

When a port's high-priority traffic consumes all available bandwidth, lower-priority queues can become starved and stuck.

**Table 37: Feature History Table**

| Feature Name                  | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Virtual Output Queue Watchdog | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8010 [ASIC: A100], 8700 [ASIC: K100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8011-4G24Y4H-I</li> <li>• 8712-MOD-M</li> </ul>                                                                                                                                                       |
| Virtual Output Queue Watchdog | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is now supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name                  | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Virtual Output Queue Watchdog | Release 24.2.11     | <p>We ensure the continuous movement of traffic queues, which is crucial for enforcing QoS policies, even when hardware issues disrupt the Virtual Output Queue (VOQ) and impede the flow of traffic. With this feature, if the router detects a stuck queue on a line card, it shuts down the line card, and if it detects a stuck queue on a fabric card, the router triggers a hard reset on the NPU. A queue is considered stuck only when there is no transmission for one minute.</p> <p>The feature is disabled by default and can be enabled using the command <b>hw-module voq-watchdog feature enable</b>.</p> <p>The feature is supported only on Cisco 8000 Series Routers (Modular) with Cisco Silicon One Q100 or Q200 ASICs.</p> <p>The feature introduces these changes:</p> <p>CLI:</p> <ul style="list-style-type: none"> <li>• <b>hw-module voq-watchdog feature enable</b></li> <li>• <b>hw-module voq-watchdog cardshut disable</b></li> </ul> |

## Guidelines and restrictions for VOQ watchdog

Follow these guidelines and restrictions when enabling or disabling the VOQ watchdog feature.

- Enablement scope: You can enable or disable the feature on both line and fabric cards. You cannot enable or disable the feature on only one type of card.
- Line card shut down action: Disabling the shutdown action for line cards has no effect on fabric cards.
- Fabric card restriction: You cannot disable the shutdown action on fabric cards.
- Support: The VOQ watchdog feature is supported only on Cisco 8000 Series Routers with Cisco Silicon One Q200 ASICs.

These restrictions ensure consistent behavior across all cards and guarantee recovery on fabric cards, where the shutdown action cannot be disabled. This design prevents stuck VOQs in the fabric from persisting and disrupting overall system stability.

## How VOQ watchdog works on line cards and fabric cards

To use the VOQ watchdog, enable it on both line cards and fabric cards. It is disabled by default.

### Summary

Key components of the process are:

- Router: Monitors and manages VOQs.
- Line cards: Can be shut down if stuck VOQs are detected.
- Fabric cards: Can be reset if stuck VOQs are detected.

### Workflow

The VOQ watchdog process includes these stages:

1. The router monitors VOQs for stuck conditions (no traffic movement for about one minute).
2. If a stuck VOQ is detected:
  - The router raises a notification.
  - On line cards: By default, the affected line card is shut down.
  - On fabric cards:  
The affected fabric element (FE) is hard reset.  
After five hard resets, the fabric card undergoes a graceful reload.
  - Syslog messages are generated.
  - If shutdown action is disabled (**hw-module voq-watchdog cardshut disable**), only a notification is sent.

| When VOQ watchdog is                                 | Then                                                               |
|------------------------------------------------------|--------------------------------------------------------------------|
| Disabled                                             | no stuck VOQs are detected, and no notifications or actions occur. |
| Enabled and a stuck VOQ is detected on a line card   | the line card is shut down unless shutdown is disabled.            |
| Enabled and a stuck VOQ is detected on a fabric card | the fabric card is hard reset. Shutdown cannot be disabled.        |

### Result

The VOQ watchdog enables effective detection and remediation of stuck VOQs and maintains QoS and network stability.

## VOQ watchdog behavior on line and fabric cards

By default, the VOQ watchdog feature is disabled on both line cards and fabric cards. In this state, stuck VOQs are not detected, and no syslog notifications are generated.

After you enable the feature using the **hw-module voq-watchdog feature enable** command, the router regularly checks for packets stuck in VOQs. When a stuck VOQ is detected, the router generates syslog notifications and takes recovery actions depending on the card type and configuration.

### VOQ watchdog behavior on line cards

When enabled on line cards, the router monitors for stuck VOQs.

Default action: The affected line card is shut down, and syslog notifications are generated.

Optional behavior: You can prevent line card shutdowns by using the **hw-module voq-watchdog cardshut disable** command. The router still raises notifications, helping you identify the root cause of the problem.

#### Syslog Notification - Stuck VOQ; Action: Line card Shutdown

```
LC/0/0/CPU0:Feb 22 09:16:56.090 UTC: npu_drvr[203]: %FABRIC-NPU_DRV-3-VOQ_HARDWARE_WATCHDOG
:
[7127] : npu[1]: hardware_watchdog.voq_error: VOQ slc:2 voqnum:19955 isinhbm:1 smscntxtnum:3
hbmcntxtnum:14
isstuck:1 nochangesec:64 rdptr:1728 wrptr:1735 avblcrdts:-16668 is_fabric:0
```

#### Syslog Notification - Stuck VOQ; Action: Line card Shutdown

```
LC/0/0/CPU0:Feb 22 09:16:56.090 UTC: npu_drvr[203]: %FABRIC-NPU_DRV-3-VOQ_HARDWARE_WATCHDOG
:
[7127] : npu[1]: hardware_watchdog.voq_error: VOQ slc:2 voqnum:19955 isinhbm:1 smscntxtnum:3
hbmcntxtnum:14
isstuck:1 nochangesec:64 rdptr:1728 wrptr:1735 avblcrdts:-16668 is_fabric:0
```

```
LC/0/0/CPU0:Jan 30 15:10:57.299 UTC: npu_drvr[241]:
%PKT_INFRA-FM-2-FAULT_CRITICAL : ALARM_CRITICAL :
VOQ WATCHDOG Alarm : DECLARE :: Shutdown card due to voq watchdog error on ASIC 1
```

After disabling the shutdown action on the line card, the router displays these messages when it detects a stuck VOQ.

#### Syslog Notification - Stuck VOQ; Action: None

```
LC/0/0/CPU0:Feb 22 09:16:56.090 UTC: npu_drvr[203]:
%FABRIC-NPU_DRV-3-VOQ_HARDWARE_WATCHDOG : [7127] :
npu[1]: hardware_watchdog.voq_error: VOQ slc:2 voqnum:19955
isinhbm:1 smscntxtnum:3 hbmcntxtnum:14 isstuck:1 nochangesec:64
rdptr:1728 wrptr:1735 avblcrdts:-16668 is_fabric:0
Stuck VOQ; Action: None
```

```
LC/0/0/CPU0:Feb 22 09:16:56.090 UTC: npu_drvr[203]:
%FABRIC-NPU_DRV-3-VOQ_HARDWARE_WATCHDOG : [7127] :
VOQ Watchdog Action Handling Skipped Due to User Configuration
```

### VOQ watchdog behavior on fabric cards

When enabled on fabric cards, the router monitors for stuck VOQs.

Default action: The affected fabric element device is hard reset.

Additional safeguard: After five consecutive hard resets, the fabric card is reloaded gracefully.

#### Syslog Notification - Stuck VOQ; Action: None

```
RP/0/RP0/CPU0:Feb 22 09:16:47.721 UTC: npu_drvr[335]:
%FABRIC-NPU_DRV-3-ASIC_ERROR_ACTION : [7912] :
npu[6]: HARD_RESET needed for hardware_watchdog.voq_error
```

## Enable VOQ watchdog

Enable the VOQ watchdog to automatically detect and resolve stuck VOQs.

### Before you begin

The device must be a Cisco 8000 Series Router with Silicon One Q100 or Q200 ASICs.

Follow these steps to enable the VOQ watchdog feature.

### Procedure

**Step 1** Enter global configuration mode and enable VOQ watchdog.

#### Example:

```
Router#config
Router(config)#hw-module voq-watchdog feature enable
Router(config)#commit
```

**Step 2** (Optional) Prevent automatic shutdown of line cards.

#### Example:

```
Router(config)#hw-module voq-watchdog cardshut disable
Router(config)#commit
```

The router monitors for stuck VOQs and takes appropriate action according to your configuration.

**Step 3** Display the running configuration.

#### Example:

```
Router#show running-config | include voq-watchdog
hw-module voq-watchdog feature enable
hw-module voq-watchdog cardshut disable    (if shutdown is disabled)
```

**Step 4** Check the current watchdog status.

#### Example:

```
Router#show platform hardware voq-watchdog status
VOQ Watchdog Feature: Enabled
Card Shutdown: Disabled
```

**Step 5** Review syslog messages for VOQ watchdog actions.

#### Example:

```
Router#show logging | include VOQ
%FABRIC-NPU_DRV-3-VOQ_HARDWARE_WATCHDOG : VOQ slc:2 voqnum:19955 ... Stuck VOQ; Action: Line card Shutdown
%FABRIC-NPU_DRV-3-VOQ_HARDWARE_WATCHDOG : VOQ Watchdog Action Handling Skipped Due to User Configuration
%FABRIC-NPU_DRV-3-ASIC_ERROR_ACTION : npu[6]: HARD_RESET needed for hardware_watchdog.voq_error
```

The VOQ watchdog feature is enabled and operating based on your configuration



## CHAPTER 9

# Congestion Monitoring and Optimization

- [Congestion monitoring and optimization, on page 115](#)

## Congestion monitoring and optimization

The table summarizes monitoring and optimization techniques, with brief descriptions and highway traffic analogies to illustrate how each method works in practice.

| Technique                                                                    | Short Description                                                                                                      | Highway Traffic Analogy                                                                                            |
|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <a href="#">Sharing of VOQ statistics counters, on page 115</a>              | Allows multiple VOQs to share a set of counters to conserve hardware resources while still tracking traffic and drops. | Like using a shared traffic camera to monitor several nearby intersections instead of one camera per intersection. |
| <a href="#">Traffic class queue high water marks monitoring, on page 117</a> | Records peak queue occupancy and delay for each traffic class to help identify congestion patterns.                    | Like noting the worst traffic buildup times at each highway exit to plan better traffic control.                   |
| <a href="#">VOQ evictions to High Bandwidth Memory , on page 125</a>         | Shows which VOQs have been moved from on-chip SMS to high-bandwidth memory due to congestion.                          | Like tracking which highway lanes have been diverted onto an overflow road during heavy traffic.                   |

## Sharing of VOQ statistics counters

Sharing of Virtual Output Queue (VOQ) counters is a congestion optimization technique that

- reduces the counter consumption required to monitor enqueued and dropped packets across VOQs and,
- allows multiple VOQs to share a set of counters, thus conserving hardware counter resources.

### Key attributes of sharing VOQ statistics counters

- Each VOQ group that shares counters reports both enqueued and dropped packets (in packets and bytes).
- Sharing options include 1 (no sharing), 2, 4, or 8 VOQs per counter set.

- Sharing VOQ counters enables scalability in large-scale deployments by mitigating device counter capacity limits

## Guidelines for configuring VOQ counter sharing

### Configure VOQ counter sharing before applying egress policies

To conserve VOQ counter resources and prevent counter exhaustion, configure VOQ counter sharing (using **hw-module profile stats voqs-sharing-counters**) before applying any egress queuing policy.

If you do not configure VOQ counter sharing first, the egress queuing policies may use default or previously set counter-sharing configurations, which could exceed hardware limits or misrepresent queue statistics.

Delete all existing egress queuing policies and reload all nodes (using **router#reload location all**) to apply the new shared counter configuration successfully.

## Shared VOQ counter modes

Use the **hw-module profile stats voqs-sharing-counters** command to configure VOQ counter-sharing. The available VOQ counter-sharing modes and their intended use cases are described in this table.

| VOQ counter mode | Description                     | Use case                                                                  |
|------------------|---------------------------------|---------------------------------------------------------------------------|
| 1                | One counter per VOQ             | Highest granularity. Use when counters are abundant.                      |
| 2                | Two VOQs share one counter set  | This mode provides a good balance between granularity and resource usage. |
| 4                | Four VOQs share one counter set | This mode is recommended when conserving counter space.                   |

## Configure shared VOQ counters

Use this task to specify how many VOQs share a set of counters, optimizing counter resource usage.

This configuration is hardware-level and affects all VOQs across the device. It must be applied before deploying queuing policies.

### Before you begin

- Ensure no egress queuing policy is applied on any interface.
- Prepare to reload all nodes

### Procedure

**Step 1** Choose one of the available options (1, 2, or 4) based on your needs for granularity versus counter resource conservation.

#### Example:

```
Router(config)#hw-module profile stats voqs-sharing-counters 2
Router(config)#commit
```



**Step 2** Reload all nodes on your router. This step is required for the configuration to take effect.

**Example:**

```
Router#reload location all
```

This example sets the mode where 2 VOQs share counters.

**Step 3** Verify the VOQ statistics.

**Example:**

```
Router#show controllers npu stats voq ingress interface hundredGigE 0/0/0/16 instance all location 0/RP0/CPU0
```

```
Interface Name Interface Handle Location Asic Instance
= = = =
Hu0/0/0/16 f0001b0 0/RP0/CPU0 0
VOQ Base = 10288 Port Speed(kbps) Local Port VOQ Mode
= = =
100000000 local 8 Shared Counter Mode = 2
ReceivedPkts ReceivedBytes DroppedPkts DroppedBytes
TC_{0,1} = 114023724 39908275541 113945980 39881093000
TC_{2,3} = 194969733 68239406550 196612981 68814543350
TC_{4,5} = 139949276 69388697075 139811376 67907466750
TC_{6,7} = 194988538 68242491778 196612926 68814524100
```

- The **HundredGigE 0/0/0/16** interface is actively collecting VOQ statistics.
- The VOQ counter sharing mode is set to 2, meaning two VOQs are sharing a single counter set, as configured.
- The counters are actively capturing received and dropped packets and bytes across various traffic classes, showing traffic flow and congestion events

## Traffic class queue high water marks monitoring

Traffic class queue high water marks are metrics that

- indicate the peak congestion levels experienced by traffic class queues on an egress interface and,
- reflect the maximum queue occupancy and maximum queue delay, helping identify potential bottlenecks.
- Queue occupancy is the amount of buffer space utilized by a traffic class at a given moment.
- Queue delay is the duration that packets wait in the queue before transmission.

For example, by using high water mark data, you can determine whether certain traffic classes are consistently congested and adjust QoS policies.

Table 38: Feature History Table

| Feature Name                                    | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Traffic Class Queue High Water Marks Monitoring | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8010 [ASIC: A100], 8700 [ASIC:K100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8011-4G24Y4H-I</li> <li>• 8712-MOD-M</li> </ul>                                                                                                                                                        |
| Traffic Class Queue High Water Marks Monitoring | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is now supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name                                    | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Traffic Class Queue High Water Marks Monitoring | Release 24.2.11     | <p>Introduced in this release on Cisco 8000 Series Routers with Cisco Silicon One Q200 network processors. The Cisco 8608 router is not currently supported.</p> <p>This feature monitors egress interface traffic class queues and records the queue occupancy and queue delay high water marks information for each traffic class. This information includes the virtual output queue that experienced the high water mark and a timestamp indicating when the high water mark was recorded.</p> <p>You can use this data to identify network bottlenecks and prevent traffic congestion.</p> <p>This feature introduces these changes:</p> <p>CLI:</p> <ul style="list-style-type: none"> <li>• <a href="#">hw-module profile qos high-water-marks</a></li> <li>• <a href="#">show controllers npu qos high-water-marks</a></li> <li>• <a href="#">clear controller npu qos high-water-marks</a></li> </ul> <p>YANG Data Models:</p> <ul style="list-style-type: none"> <li>• cisco-IOS-XR-ofa-npu-qos-oper.yang</li> <li>• cisco-IOS-XR-ofa-npu-qos-act.yang</li> <li>• cisco-IOS-XR-um-8000-hw-module-profile-cfg.yang</li> <li>• cisco-IOS-XR-npu-hw-profile-cfg.yang</li> </ul> |

## High water mark data types

High water mark data types are key metrics that reflect the maximum observed values for queue occupancy and queue delay on an interface's traffic class queue.

The table lists and describes high water mark data types.

| Data Type                        | Description                                                                                                                               |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Queue occupancy high water marks | Maximum queue occupancy observed in kilobytes and percentage of queue size; includes timestamp and Virtual Output Queue (VOQ) identifier. |

| Data Type                    | Description                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Queue delay high water marks | Maximum queue delay observed in nanoseconds (ns); includes timestamp, queue occupancy at the time, and VOQ identifier. |

## Recommendations and requirements for monitoring traffic class queue high water marks

### Priority Flow Control compatibility

This feature can co-exist with Priority Flow Control (PFC). When using PFC in **buffer-internal** mode, configure **ecn** and **max-threshold** values.

### Bundle interface monitoring

High water marks are monitored per bundle member interface, not the bundle interface itself.

### Subinterface contributions

Packets egressing from subinterfaces and bundle subinterfaces contribute to the high water marks on the corresponding parent interface.

### Queue occupancy monitoring support

Queue occupancy monitoring is supported only for packets sourced from Q200-based network processors (NPUs).

### Queue delay monitoring support

Packets from non-Q200 NPUs are also supported for queue delay monitoring.

### Reload requirement

Ensure that you manually reload the chassis or all line cards after enabling or disabling this feature.

### VOQ mapping factors

The number of VOQs per traffic class depends on the VOQ mode of the router and the number of slices.

### Occupancy percentage basis

Queue occupancy percentages are calculated using the maximum queue-limit, not custom limits.

## Configure traffic class queue high water marks monitoring

Use this task to enable monitoring of peak congestion levels for egress traffic class queues.

This task helps identify congestion patterns on supported Cisco 8000 Series Routers by using high water mark data.

### Before you begin

- Verify that your router supports Q200-based line cards.
- Plan for a reload of the chassis or all line cards to activate or deactivate the feature.

Use these steps to configure the traffic class queue high water marks monitoring feature.

## Procedure

**Step 1** Enable the monitoring feature for egress traffic class queues.

**Example:**

```
Router(config)#hw-module profile qos high-water-marks
Router(config)#commit
```

**Step 2** Reload the chassis or all line cards to apply the changes.

**Example:**

```
Router(admin)#reload location all
```

**Step 3** Verify that the feature is enabled

**Example:**

```
Router#show controllers npu qos high-water-marks interface all
Mon Jun  3 06:02:50.138 UTC
Not supported or not enabled on location 0/0/CPU0
RP/0/RP0/CPU0:ios#
```

The output indicates that the feature is not enabled on location **0/0/CPU0**.

**Step 4** (Optional) View the monotonic high water marks for all traffic classes.

**Example:**

```
Router#show controllers npu qos high-water-marks interface fourHundredGigE 0/0/0/11
```

```
Interface Name      = FH0/0/0/11
Interface Handle    = 0x1F8
System Port Gid     = 96
Asic Instance       = 0
```

| Queue Occupancy High Water Marks |                              |                   |                                  |                   | Queue Delay High Water Marks |                          |     |
|----------------------------------|------------------------------|-------------------|----------------------------------|-------------------|------------------------------|--------------------------|-----|
| Sys Port                         | Max Occupancy<br>% kilobytes | Queue<br>Delay ns | Src Sys Port<br>Slot/NPU/Slc/Gid | Timestamp         | Max Queue<br>Delay ns        | Occupancy<br>% kilobytes | Src |
| TC_0 = 0/0/1/44                  | 6.00 30965                   | 73728             | 0/0/2/40                         | 04/08/23 08:39:35 | 102400                       | 3.00 15482               |     |
| TC_1 = 0/0/0/0                   | 0.00 0                       | 0                 | 0/0/0/0                          | -                 | 0                            | 0.00 0                   |     |
| TC_2 = 0/0/0/48                  | 25.00 129024                 | 1114112           | 0/0/0/48                         | 04/07/23 01:10:23 | 1179648                      | 15.00 77414              |     |
| TC_3 = 0/1/1/58                  | 70.00 361267                 | 8912896           | 0/1/1/56                         | 04/02/23 08:41:44 | 8912896                      | 70.00 361267             |     |
| TC_4 = 3/0/2/5                   | 40.00 206438                 | 2228224           | 3/0/2/4                          | 04/09/23 06:38:35 | 2359296                      | 25.00 129024             |     |
| TC_5 = 0/0/0/0                   | 0.00 0                       | 0                 | 0/0/0/0                          | -                 | 0                            | 0.00 0                   |     |
| TC_6 = 7/0/2/76                  | 78.00 599                    | 6437184           | 3/1/0/24                         | 04/10/23 16:35:00 | 8628192                      | 64.00 492                |     |
| TC_7 = 0/2/2/66                  | 25.00 129024                 | 139264            | 3/0/0/14                         | 04/06/23 08:39:41 | 155648                       | 15.00 77414              |     |

## Configure traffic class queue high water marks monitoring

```
[ ----- Occupancy High Water Marks ----- ] [ ----- Delay High Water Marks ----- ]
```

The output displays monotonic high water marks data for all traffic classes on interface **fourHundredGigE 0/0/0/11**, recorded since bootup or after the most recent clear operation.

**Tip**

Monotonic high water marks are displayed if neither the **monotonic** or **periodic** keyword is used.

**Step 5** (Optional) View the monotonic high water marks for a single traffic class.

**Example:**

```
Router#show controllers npu qos high-water-marks monotonic interface fourHundredGigE 0/0/0/2
traffic-class 5
```

```
Interface Name      = FH0/0/0/2
Interface Handle    = 0xF000120
System Port Gid     = 6
Asic Instance      = 0
```

| Queue Occupancy High Water Marks                                                  |                   |              |          |                   | Queue Delay High Water Marks |           |        |           |
|-----------------------------------------------------------------------------------|-------------------|--------------|----------|-------------------|------------------------------|-----------|--------|-----------|
| Max Occupancy                                                                     | Queue             | Src Sys Port |          |                   | Max Queue                    | Occupancy | Src    |           |
| Sys Port                                                                          | %                 | kilobytes    | Delay ns | Slot/NPU/Slc/Gid  | Timestamp                    | Delay ns  | %      | kilobytes |
| Slot/NPU/Slc/Gid                                                                  | Timestamp         |              |          |                   |                              |           |        |           |
| TC_5 = 40.00                                                                      | 206438            | 6815744      | 3/0/0/15 | 11/11/23 17:43:30 | 1811939328                   | 25.00     | 129024 |           |
| 7/1/2/89                                                                          | 11/27/23 11:21:26 |              |          |                   |                              |           |        |           |
| [ ----- Occupancy High Water Marks ----- ] [ ----- Delay High Water Marks ----- ] |                   |              |          |                   |                              |           |        |           |

The output displays monotonically increasing high water marks data for traffic class 5 on interface **fourHundredGigE 0/0/0/2**, recorded since bootup or after the most recent clear operation. .

**Step 6** (Optional) View the periodic high water marks for a single traffic class.

**Example:**

```
Router#show controllers npu qos high-water-marks periodic last 3 interface fourHundredGigE 0/0/0/5
traffic-class 7
```

```
Interface Name      = FH0/0/0/5
Interface Handle    = 0xF000138
System Port Gid     = 9
Asic Instance      = 0
```

| High Water Marks |                   |                   |       |        | Queue Occupancy High Water Marks |           |              | Queue Delay      |          |
|------------------|-------------------|-------------------|-------|--------|----------------------------------|-----------|--------------|------------------|----------|
| Occupancy        | Src Sys Port      |                   |       |        | Max Occupancy                    | Queue     | Src Sys Port | Max Queue        |          |
| Interval         | Start             | End               |       |        | %                                | kilobytes | Delay ns     | Slot/NPU/Slc/Gid | Delay ns |
| %                | kilobytes         | Slot/NPU/Slc/Gid  |       |        |                                  |           |              |                  |          |
| TC_7 = 1         | 12/01/23 17:46:30 | 12/01/23 17:46:59 | 50.00 | 258048 | 34680274                         | 7/1/2/91  |              | 34680274         |          |
| 50.00            | 258048            | 7/1/2/91          |       |        |                                  |           |              |                  |          |
| 2                | 12/01/23 17:45:58 | 12/01/23 17:46:30 | 60.00 | 309657 | 52296260                         | 0/2/1/68  |              | 61348106         |          |
| 50.00            | 258048            | 7/1/2/91          |       |        |                                  |           |              |                  |          |
| 3                | 12/01/23 17:45:30 | 12/01/23 17:45:58 | 40.00 | 206438 | 15290430                         | 0/2/1/68  |              | 15290430         |          |
| 40.00            | 206438            | 0/2/1/68          |       |        |                                  |           |              |                  |          |

```

Delay High Water Marks ----- ] [----- Occupancy High Water Marks ----- ] [ -----

```

The output displays high water marks data for traffic class 7 on interface **fourHundredGigE 0/0/0/5** for the last three periodic collection intervals.

## Descriptions of show command output fields for queue high water marks

This section describes the fields in the show command outputs that help you monitor queue high water marks for different traffic classes and scenarios.

The table presents the common fields displayed in the show command output for queue high water marks.

**Table 39: Common Fields**

| Field                                  | Description                                                                                                                                                |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interval Start and End (periodic only) | Displays the periodic collection interval number, and the start and end time of that interval.                                                             |
| TC_Number =<br>(Number range is 0-7)   | Identifies the traffic class associated with the high water mark data. For periodic output, the field appears only for the traffic class's first interval. |

The table describes queue occupancy fields in the show command output, including maximum occupancy values, delay, source port details, and timestamps.

These fields report the maximum queue occupancy values observed for a traffic class. Use them to determine how full the queue became and the conditions under which that peak occurred.

**Table 40: Queue Occupancy Fields**

| Field                   | Description                                                                                                                                                                                                                                                                                       |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Max Occupancy %         | Displays the maximum queue occupancy for this traffic class as a percentage of the total queue size. Because of limited queue quantization thresholds provided by the NPU, this value is an estimate and may differ from the actual maximum occupancy.                                            |
| Max Occupancy kilobytes | Displays the maximum queue occupancy for this traffic class in kilobytes. The value is calculated assuming that all buffers are fully packed (for example, 384 bytes per Shared Memory System buffer). As a result, the displayed value may be higher than the actual number of kilobytes queued. |
| Queue Delay ns          | Displays the delay, in nanoseconds, when the maximum queue occupancy high water mark occurred.                                                                                                                                                                                                    |

| Field                            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Src Sys Port<br>Slot/NPU/Slc/Gid | <p>Identifies the slot, NPU, slice, and global identifier (GID) of the virtual output queue where the queue occupancy high water mark occurred. The GID is the global identifier of the source system port whose packet was dequeued when the maximum occupancy was reached. In most cases, all ports on a slice share a virtual output queue. The identified source port may not be the only contributor; other ports sharing the same queue could also have contributed to the burst of packets.</p> <p><b>Note</b><br/>In fair-4 or fair-8 VOQ mode, each source port has its own virtual output queue. In Priority Flow Control (PFC) <b>buffer-internal</b> mode, a port may share a virtual output queue with other ports in the same interface group (IFG). Each slice has two IFGs.</p> <p>Use the <b>show controllers npu voq-usage</b> command to verify which other ports share a virtual output queue with the identified source port.</p> |
| Timestamp (monotonic only)       | <p>Displays the timestamp when the maximum queue occupancy high water mark was recorded. The timestamp reflects when the NPU reported the data, not when the event occurred. Because the NPU is queried every 30 seconds, the timestamp shows the end of the 30-second window during which the high water mark occurred.</p> <p>For example, a timestamp of 16:56:44 indicates that the event occurred sometime between 16:56:14 and 16:56:44.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

The table describes the queue delay fields in the show command output, including maximum delay, occupancy details, source port identifiers, and timestamps.

These fields report the maximum queue delay experienced by a traffic class. Use them to understand the delay impact and the corresponding queue occupancy at the time of the peak delay.

**Table 41: Queue Occupancy Fields**

| Field              | Description                                                                   |
|--------------------|-------------------------------------------------------------------------------|
| Max Queue Delay ns | Displays the maximum delay experienced by this traffic class, in nanoseconds. |



| Field                            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Queue Occupancy %                | Displays the queue occupancy as a percentage of the total queue size when the maximum queue delay high water mark occurred. Because of limited queue quantization thresholds provided by the NPU, this value is an estimate and may differ from the actual maximum occupancy.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Queue Occupancy kilobytes        | Displays the queue occupancy in kilobytes at the time the maximum queue delay high water mark occurred. The value is calculated assuming all buffers are fully packed (for example, 384 bytes per Shared Memory System buffer). As a result, the displayed value may be higher than the actual number of kilobytes queued.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Src Sys Port<br>Slot/NPU/Slc/Gid | <p>Identifies the slot, NPU, slice, and global identifier (GID) of the virtual output queue where the maximum queue delay high water mark occurred. The GID identifies the source system port whose packet was dequeued when the maximum delay was reached. In most cases, all ports on a slice share a virtual output queue. The identified source port may not be the only contributor; other ports sharing the same queue could also have contributed to the burst of packets.</p> <p><b>Note</b><br/>In fair-4 or fair-8 VOQ mode, each source port has its own virtual output queue. In Priority Flow Control <b>buffer-internal</b> mode, a port may share a virtual output queue with other ports in the same interface group (IFG). Each slice has two IFGs.</p> <p>Use the <b>show controllers npu voq-usage</b> command to verify which other ports share a virtual output queue with the identified source port.</p> |
| Timestamp (monotonic only)       | <p>Displays the timestamp when the maximum delay high water mark was recorded. The timestamp reflects when the NPU reported the data, not when the event occurred. Because the NPU is queried every 30 seconds, the timestamp shows the end of the 30-second window during which the event occurred.</p> <p>For example, a timestamp of 16:56:44 indicates that the high water mark was observed sometime between 16:56:14 and 16:56:44.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

## VOQ evictions to High Bandwidth Memory

VOQ evictions to High Bandwidth Memory (HBM) are buffer-management optimizations that

- offload virtual output queues (VOQs) from the on-chip Shared Memory System (SMS) to external High Bandwidth Memory (HBM),
- are triggered when local occupancy thresholds are exceeded, and
- help preserve packet buffering capacity during periods of high traffic.

Table 42: Feature History Table

| Feature Name            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ViewVOQs Evicted to HBM | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8010 [ASIC: A100], 8700 [ASIC:K100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8011-4G24Y4H-I</li> <li>• 8712-MOD-M</li> </ul>                                                                                                                                                        |
| ViewVOQs Evicted to HBM | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is now supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ViewVOQs Evicted to HBM | Release 24.2.11     | <p>The newly introduced command displays the virtual output queues (VOQs) that are evicted to the High Bandwidth Memory (HBM) and the VOQs' HBM buffer usage details. You can use this information when monitoring and debugging congestion scenarios.</p> <p>This feature introduces the <a href="#">show controllers npu voq in-extended-memory instance</a> command.</p> <p>This feature modifies the <a href="#">Cisco IOS XR 8000 platform qos hbm pag</a> data model.<br/>(see <a href="#">GitHub</a>, <a href="#">YANG Data Models Navigator</a>)</p> |

## How VOQ eviction to HBM works

When a VOQ in the Shared Memory System (SMS) exceeds its congestion threshold, it may be evicted to High Bandwidth Memory (HBM) to avoid packet drops.

### Summary

This process dynamically moves VOQs between SMS and HBM based on buffer pressure and traffic conditions. The HBM acts as overflow memory for congestion scenarios.

### Workflow

These stages describe how VOQ eviction to HBM works.

1. Normal operation: The NPU places packets into the SMS buffer for every VOQ.
2. Congestion detection: When SMS usage for a VOQ exceeds the congestion threshold, the VOQ is marked for eviction.
3. Eviction to HBM: The VOQ is offloaded to HBM, which serves as extended buffer memory.
4. HBM buffer monitoring: As traffic drains from the HBM-based VOQ, the system tracks usage and adapts the VOQ size dynamically.
5. Reversion to SMS: When HBM pressure reduces and VOQ activity stabilizes, the VOQ is returned to the SMS buffer.

### Result

This system-level process helps avoid packet loss by using adaptive buffering and reducing memory contention among active VOQs.

## Best practices for viewing VOQ eviction to HBM

### Requirement for automation scripts

Do not use the **show controllers npu voq in-extended-memory instance** command in an automation script.

### Guidelines for viewing VOQ eviction to HBM

Use these guidelines to correctly interpret Virtual Output Queue (VOQ) eviction behavior under different Priority Flow Control (PFC) modes.

- Priority handling with PFC **buffer-extended** mode: In PFC **buffer-extended** mode, associated VOQs are evicted to HBM on priority, while the remaining VOQs stay in SMS.
- Retention behavior with PFC **buffer-internal mode** mode: In PFC **buffer-internal mode**, associated VOQs are retained in Shared Memory System (SMS), while the remaining VOQs are evicted to High Bandwidth Memory (HBM).
- Eviction policy when PFC is disabled: If PFC isn't enabled on a device, VOQs are evicted to the HBM based on the VOQs' age and buffer usage.

## View evicted VOQs to HBM

Use this task to identify congestion events and diagnose buffer pressure by viewing Virtual Output Queues (VOQs) evicted to High Bandwidth Memory (HBM) on your Cisco 8000 router.

This task is useful when analyzing congestion symptoms or buffer-related behavior during traffic surges.

### Before you begin

- You must have access to the router in privileged EXEC mode.
- Ensure that the router is running Cisco IOS XR Release 24.2.11 or later.

### Procedure

View the VOQs evicted to the HBM for all device instances.

#### Example:

```
Router#show controllers npu voq in-extended-memory instance all location 0/6/CPU0
```

\* Use this CLI with caution.

\* This should not be integrated with any automation scripts.

```
Total Entries  0  | Slot 6  | Device  0  | Percent in Evict Voq Buff(s) 0.000000
-----
Egress Interface | VOQ_Base | TC  | Slice | Buff_Usage | In_Bytes |
-----
Total Entries  2  | Slot 6  | Device  1  | Percent in Evict Voq Buff(s) 0.004883
-----
Egress Interface | VOQ_Base | TC  | Slice | Buff_Usage | In_Bytes |
-----
          FH0/6/0/0          28672      2    0      14722      87837459
          FH0/6/0/3          28728      2    0      1049       6335369
Total Entries  0  | Slot 6  | Device  2  | Percent in Evict Voq Buff(s) 0.000000
```

```
-----  
Egress Interface | VOQ_Base | TC | Slice | Buff_Usage | In_Bytes |
```

The output displays VOQs that are evicted to the HBM for node **0/6/cpu0** and all instances. In this case, VOQs from device instance **Device 1** are evicted to the HBM.

Output fields for evicted VOQs:

- **Egress Interface:** The egress interface of the virtual output queue.
- **VOQ\_Base:** Base VOQ ID.
- **TC:** Traffic Class number.
- **Slice:** Source slice number.
- **Buff\_Usage and In\_Bytes:** Buffer usage in blocks and in bytes, respectively

---

You can view VOQs evicted to HBM and correlate congestion events with traffic patterns.





## CHAPTER 10

# Priority Flow Control

- [Stages of Priority Flow Control configuration, on page 131](#)
- [Priority Flow Control, on page 132](#)
- [buffer-internal mode, on page 139](#)
- [Configurable ECN threshold and marking probability values, on page 142](#)
- [buffer-extended mode, on page 148](#)
- [Configure PFC on interfaces, on page 152](#)
- [High Bandwidth Memory congestion detection, on page 155](#)
- [Global pause frames for High Bandwidth Memory congestion, on page 163](#)
- [buffer-extended hybrid mode, on page 166](#)

## Stages of Priority Flow Control configuration

Use this information to understand the key stages involved in configuring Priority Flow Control (PFC) on your router.

Each stage provides a high-level summary of the configuration flow, helping you locate the section that aligns with your operational goal, which could be selecting a PFC buffer mode, tuning Explicit Congestion Notification (ECN) thresholds, or enabling congestion protection for High Bandwidth Memory (HBM).

**Table 43: Stages of Priority Flow Control Configuration**

| Stage                       | Description                                                                                                                                                            | See Section                                              |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Understand PFC fundamentals | Learn how PFC uses pause frames to prevent packet loss on congested queues.                                                                                            | <a href="#">Priority Flow Control, on page 132</a>       |
| Select a PFC mode           | Compare buffer-internal, buffer-extended, and buffer-extended hybrid modes and identify which one best suits your traffic profile, link distance, and latency targets. | <a href="#">Priority Flow Control modes, on page 137</a> |

| Stage                                                  | Description                                                                                                                                                                                  | See Section                                                                                                   |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Configure buffer-internal mode                         | Configure pause-threshold, headroom, and ECN values; adjust ECN thresholds and marking probability.                                                                                          | <a href="#">buffer-internal mode, on page 139</a>                                                             |
| Configure ECN threshold and marking probability values | Optimize congestion handling by defining ECN minimum and maximum thresholds and marking probability for buffer-internal PFC.                                                                 | <a href="#">Configurable ECN threshold and marking probability values, on page 142</a>                        |
| Configure buffer-extended and hybrid modes             | Apply interface-level ECN and queue limit settings, define hybrid buffer partitions, and follow tuning guidelines for supported hardware.                                                    | <a href="#">buffer-extended mode, on page 148</a><br><a href="#">buffer-extended hybrid mode, on page 166</a> |
| Configure PFC on an interface                          | Enable PFC on specific interfaces so the router can send and respond to 802.1Qbb pause frames, generate pause-duration telemetry, and operate with <a href="#">PFC watchdog monitoring</a> . | <a href="#">Configure PFC on interfaces, on page 152</a>                                                      |
| Detect and manage HBM congestion                       | Enable High Bandwidth Memory (HBM) congestion detection in the buffer-extended mode to monitor when VOQs spill into external memory and trigger pause protection if required.                | <a href="#">High Bandwidth Memory congestion detection, on page 155</a>                                       |
| Enable global pause protection                         | Configure global X-off frames to prevent packet drops when HBM congestion occurs on buffer-extended devices.                                                                                 | <a href="#">Global pause frames for High Bandwidth Memory congestion, on page 163</a>                         |

## Priority Flow Control

A priority flow control (PFC) is a link layer congestion management mechanism that

- prevents frame loss due to congestion
- pauses only the affected classes of service (CoS) instead of the entire link, and
- uses pause frames with timers per CoS to temporarily stop traffic until congestion clears.

### Additional information about PFC

- PFC is standardized as IEEE 802.1Qbb.
- This mechanism is also called Priority-Based Flow Control (PFC), Class-Based Flow Control (CBFC), or Per-Priority Pause (PPP).



- Unlike IEEE 802.3x Flow Control (pause frames), which stops all traffic, PFC provides granularity at the CoS level.
- PFC pause frame includes a two-octet timer per CoS, expressed in quanta (transmission time of 512 bits).
- The timer range is 0 to 65,535 quanta. The maximum pause time depends on the interface speed.
- Pause frames are not forwarded by the peer; they operate on a hop-by-hop basis only.

**Table 44: Feature History Table**

| Feature Name                    | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Shortlink Priority Flow Control | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)</p> <p>This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                                                                                  |
| Shortlink Priority Flow Control | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is now supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name          | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Priority Flow Control | Release 25.3.1      | <p>Introduced in this release on: Fixed Systems (8100 [ASIC: Q200], 8200 [ASIC: Q200]) (select variants only*)</p> <p>You can now enable Priority Flow Control (PFC) to pause specific classes of traffic without impacting others during network congestion. This feature allows the device to apply flow control on a per-priority basis, preventing packet loss for critical traffic while maintaining overall network performance.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8101-32FH</li> <li>• 8201-32FH</li> </ul> |
| Priority Flow Control | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)</p> <p>This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                                                                                                                                                                                                                                                     |
| Priority Flow Control | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul>                                                                                                                                                                        |

| Feature Name                                                                                | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Priority Flow Control on Cisco 8808 and Cisco 8812 Modular Chassis Line Cards               | Release 7.5.3       | <p>Priority Flow Control is now supported on the following line card in the buffer-internal mode:</p> <ul style="list-style-type: none"> <li>• 88-LC0-34H14FH</li> </ul> <p>The feature is supported in the buffer-internal and buffer-extended modes on:</p> <ul style="list-style-type: none"> <li>• 88-LC0-36FH</li> </ul> <p>Apart from the buffer-external mode, support for this feature now extends to the buffer-internal mode on the following line cards:</p> <ul style="list-style-type: none"> <li>• 88-LC0-36FH-M</li> <li>• 8800-LC-48H</li> </ul>                                                                                   |
| Shortlink Priority Flow Control                                                             | Release 7.3.3       | This feature and the <b>hw-module profile priority-flow-control</b> command are supported on 88-LC0-36FH line card.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Priority Flow Control Support on Cisco 8800 36x400 GbE QSFP56-DD Line Cards (88-LC0-36FH-M) | Release 7.3.15      | <p>This feature and the <b>hw-module profile priority-flow-control</b> command are supported on 88-LC0-36FH-M and 8800-LC-48H line cards.</p> <p>All previous functionalities and benefits of this feature are available on these line cards. However, the buffer-internal mode is not supported.</p> <p>In addition, to use the buffer-extended mode on these line cards, you are required to configure the performance capacity or headroom values. This configuration requirement ensures that you can better provision and balance workloads to achieve lossless behavior, which in turn ensures efficient use of bandwidth and resources.</p> |

| Feature Name          | Release Information | Feature Description                                                                                     |
|-----------------------|---------------------|---------------------------------------------------------------------------------------------------------|
| Priority Flow Control | Release 7.3.1       | This feature and the <a href="#">hw-module profile priority-flow-control</a> command are not supported. |

### PFC analogy

The PFC mechanism is like a multi-lane highway where only the congested lane is closed temporarily while others keep moving.

## Benefits of Priority Flow Control

These benefits make Priority Flow Control (PFC) critical in converged networks carrying both lossy and lossless traffic.

- Provides lossless transport for traffic classes that require it, for example, storage or Remote Direct Memory Access (RDMA) traffic.
- Prevents head-of-line blocking by pausing only specific traffic classes.
- Enhances coexistence of loss-sensitive and best-effort applications on the same link.
- Reduces packet drops, retransmissions, and latency spikes for critical traffic.

## How Priority Flow Control works

### Summary

This process describes how Priority Flow Control (PFC) operates when congestion occurs on a link.

When a traffic class queue exceeds a configured threshold, the switch generates a PFC pause frame to upstream devices, temporarily stopping transmission for that specific class.

The process ensures that lossless traffic continues without packet drops while non-congested traffic flows normally.

### Workflow

These stages describe how PFC manages congestion events on a link.

1. Congestion detected: The switch/line card detects that a queue for a specific CoS is approaching a configured threshold.
2. Pause frame generated: A PFC frame is sent upstream to the peer, with the CoS bit set and pause quanta specified.
3. Traffic paused: The upstream device stops transmitting for that CoS, while all other traffic classes continue.
4. Congestion clears: Once the downstream buffer drains, the switch stops sending pause frames or sends a pause frame with a timer value of zero.
5. Traffic resumes: Normal transmission continues without packet loss for the protected class.

## Priority Flow Control modes

A Priority Flow Control (PFC) operating mode is a configuration profile that

- determines whether buffer thresholds and pause behaviors are set at the line-card or interface level
- manages how Explicit Congestion Notification (ECN) and queue limits are applied and,
- controls how memory resources are shared across lossless and lossy traffic.

### Details about Priority Flow Control modes

- Before you begin, verify whether each PFC operating mode is supported on your specific hardware and line card. Mode support varies across platforms. (See Hardware Support for Priority Flow Control.)
- Only one mode can be active per line card at any time.
- A selected mode applies to all ports on a line card.
- Choose the appropriate mode based on network requirements such as link distance, latency sensitivity, and traffic isolation needs.

**Table 45: Priority Flow Control modes and their details**

| buffer-internal                                                                                 | buffer-extended                                                                                                            | buffer-extended hybrid                                                                                             |
|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Configures pause-threshold, headroom, and ECN thresholds entirely inside the line-card profile. | Configures pause-threshold in the line-card profile, but moves ECN and queue limits into interface-level queuing policies. | Splits High-Bandwidth Memory (HBM) into two separate pools—one for lossless traffic and another for lossy traffic. |
| Effective queue limit = pause-threshold + headroom.                                             | Enables flexible per-interface tuning.                                                                                     | Each pool can be individually sized (such as up to 8 GB each on high-capacity line cards).                         |
| Recommended for short-haul (less than 1 km), low-latency environments                           | Recommended for long-haul networks or networks with mixed traffic.                                                         | Recommended for deployments at high scale that require isolation between traffic types.                            |

## Hardware support for Priority Flow Control modes

The table lists hardware that supports Priority Flow Control (PFC) by release, and indicates the PFC mode available for each.

**Table 46: PFC hardware support matrix**

| Release        | Supported Hardware                                                                 | PFC Mode                                                     |
|----------------|------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Release 25.3.1 | <ul style="list-style-type: none"> <li>• 8101-32FH</li> <li>• 8201-32FH</li> </ul> | buffer-extended, buffer-internal, and buffer-extended hybrid |

| Release        | Supported Hardware                                                                                            | PFC Mode                            |
|----------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Release 24.1.1 | <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> </ul> | buffer-extended and buffer-internal |
| Release 7.5.3  | 88-LC0-36FH                                                                                                   | buffer-extended and buffer-internal |
|                | 88-LC0-36FH-M                                                                                                 | buffer-extended and buffer-internal |
|                | 8800-LC-48H                                                                                                   | buffer-extended and buffer-internal |
|                | 88-LC0-34H14FH                                                                                                | buffer-internal                     |
| Release 7.3.15 | <ul style="list-style-type: none"> <li>• 88-LC0-36FH-M</li> <li>• 88-LC0-36FH</li> </ul>                      | buffer-extended                     |
|                |                                                                                                               |                                     |
| Release 7.0.11 | 8800-LC-48H                                                                                                   | buffer-internal                     |

## Best practices for Priority Flow Control

### Restrictions for Priority Flow Control

These restrictions are hardware or configuration conditions that can prevent Priority Flow Control (PFC) from functioning as intended.

- Unsupported hardware: PFC is not supported on fixed chassis systems.
- Unsupported interface types: Not supported on bundle interfaces, sub-interfaces, or when breakout is configured.
- Unsupported modes: Not supported in 4XVOQ mode or when VOQ counter sharing is enabled.
- Scope limitations: Applies only to 40 GbE, 100 GbE, and 400 GbE physical ports.
- Consistency requirement: On modular chassis (such as 8808 and 8812), configure PFC consistently on all line cards.

### Guidelines for configuring Priority Flow Control

Follow these guidelines to achieve efficient, interoperable PFC behavior.

- Always configure PFC on both ends of a link for the same traffic classes (CoS symmetry).
- Before enabling PFC, determine whether buffer-internal, buffer-extended, or buffer-extended hybrid mode is required for your deployment.
- Validate hardware capacity and firmware prerequisites.
- For multiline card systems, maintain consistent threshold and Explicit Congestion Notification (ECN) configurations across cards.
- Use telemetry (pause-duration, ECN marks, and queue-occupancy counters) to tune thresholds over time.

- Periodically verify **show controllers npu priority-flow-control** after configuration changes.

## Accessing Priority Flow Control operational state programmatically

You can retrieve and monitor Priority Flow Control (PFC) configuration and statistics programmatically using a Cisco IOS XR data model.

- Data model: Cisco-IOS-XR-ofa-npu-pfc-oper.yang
- Access: via NETCONF or RESTCONF interfaces
- Information available: PFC configuration state, pause statistics, and ECN counters
- Reference guide: *Programmability Configuration Guide for Cisco 8000 Series Routers*

# buffer-internal mode

A buffer-internal mode is a Priority Flow Control (PFC) operating mode that

- configures pause-threshold, headroom, and ECN thresholds in the line card profile
- calculates the effective queue limit as the sum of pause-threshold and headroom, and
- applies uniformly to all ports on a line card.

## Supporting details for buffer-internal mode

- Explicit Congestion Notification (ECN) and queue-limit values in interface queuing policies are ignored when buffer-internal mode is active.
- Recommended for short-haul PFC peers (less than 1 km apart) because thresholds are enforced close to the hardware buffers.
- Provides predictable, lossless behavior for designated traffic classes.

## Lane traffic analogy for buffer-internal mode

Each lane has a small local holding bay. As the bay fills, a yellow warning line (ECN threshold) signals drivers to ease off. If it continues filling to the red stop line (pause threshold), a stop sign is raised for that lane. Extra reserved space beyond the stop line (headroom) catches cars already en route while the stop takes effect.

# Best practices for configuring buffer-internal mode

## buffer-internal mode threshold ranges

Use this information to confirm valid threshold ranges and relationships for buffer-internal mode.

- Pause threshold values: 307,200 to 422,400 bytes.
- Headroom threshold values: 345,600 to 537,600 bytes.
- Explicit Congestion Notification (ECN) threshold values: 153,600 to 403,200 bytes.

## Guidelines to configure buffer-internal mode

Follow these guidelines to ensure a stable Priority Flow Control (PFC) configuration in the buffer-internal mode.

- For Cisco 8808 and Cisco 8812 chassis, configure Priority Flow Control (PFC) thresholds on all line cards in the chassis.
- Reload the line card whenever you add or remove a traffic class. You must also reload the line card when you configure the buffer-internal threshold values for the first time on a new traffic class.
- Add or remove ECN configuration using the **hw-module profile priority-flow-control** command. Then, reload the line card to apply the ECN changes.

## Restrictions while configuring buffer-internal mode

Enforce these restrictions while configuring the buffer-internal mode thresholds.

- Ensure the ECN threshold value is less than the pause threshold value.
- Ensure that the combined configuration values for pause threshold and headroom do not exceed 844,800 bytes; otherwise, the configuration is rejected.
- Configure only one of buffer-internal, buffer-extended, or buffer-extended hybrid mode on each line card.

# Configure buffer-internal thresholds

Use this task to configure pause-threshold, headroom, and Explicit Congestion Notification (ECN) thresholds for traffic classes in buffer-internal mode.

- Applies uniformly to all interfaces on the line card.
- Use only when buffer-extended or buffer-extended hybrid mode is not configured on the same line card.

### Before you begin

Ensure that peer devices are also configured for Priority Flow Control on the same class of service (CoS).

Follow these steps to configure buffer-internal thresholds.

### Procedure

**Step 1** Enter the line card profile configuration.

#### Example:

```
Router(config)#hw-module profile priority-flow-control mode buffer-internal
```

This example selects **buffer-internal** mode for PFC on the line card.

**Step 2** Configure thresholds per traffic class.

#### Example:

```
Router(config-pfc-profile)#traffic-class 3
Router(config-pfc-profile-tc)#pause-threshold 307200
```



```
Router(config-pfc-profile-tc)#headroom 345600
Router(config-pfc-profile-tc)#ecn-threshold 153600
```

Each value in this example is described here.

- **traffic-class 3**: Selects traffic class 3 out of 0–7. This TC is tied to a Layer 2 CoS (802.1p) or an internal mapping, so your QoS policy should map the desired CoS or DSCP to traffic class 3 if you want this class to be lossless.
- **pause-threshold 307200**: When instantaneous buffer occupancy for TC 3 reaches 307,200 bytes, the device starts sending PFC pause frames for priority 3. This prevents loss by pausing upstream senders before the queue overflows.
- **headroom 345600**: This is the reserved buffer for traffic class 3 to absorb traffic after a pause is asserted. It's the guaranteed space the queue can grow into while the pause takes effect.
- **ecn-threshold 153600**: Explicit Congestion Notification (ECN) marking begins once occupancy exceeds 153,600 bytes. This value should be lower than the pause threshold so congestion is signaled (via ECN) before resorting to PFC pauses.

**Step 3** (Optional) Configure ECN maximum threshold and probability percentage (Pmax).

**Example:**

```
Router(config-pfc-profile-tc)#ecn-max-threshold 403200
Router(config-pfc-profile-tc)#probability-percentage 5
Router(config-pfc-profile-tc)#exit
Router(config-pfc-profile)#exit
Router(config)#commit
```

Here's what each value indicates in this example.

- **ecn-max-threshold 403200**: Upper bound of the ECN marking ramp. Between **ecn-threshold** and **ecn-max-threshold**, marking probability increases linearly. At or above this level, the marking probability reaches the configured maximum percentage (Pmax).
- **probability-percentage 5**: Indicates 5% as the maximum ECN marking probability (Pmax) at the **ecn-max-threshold** value. In effect, this means that there will be 0% marking at 153,600 bytes, linearly increasing to 5% at 403,200 bytes and above.

**Step 4** View the configured thresholds and runtime counters for the specific line card.

**Example:**

```
Router#show controllers npu priority-flow-control location 0/0/CPU0
TC  Pause-threshold  Headroom  ECN-threshold  ECN-max  Pmax  TxPause
3   307200           345600    153600        403200   5%    24
```

Each value in this example is described here.

- **TC**: Traffic Class (3)
- **Pause-threshold**: 307,200 bytes (pause onset).
- **Headroom**: 345,600 bytes (reserved buffer after pause).
- **ECN-threshold**: 153,600 bytes (start of ECN marking).
- **ECN-max**: 403,200 bytes (top of ECN ramp).
- **Pmax**: 5% (maximum ECN marking probability).

- **TxPause:** 24 (number of PFC pause frames transmitted for this TC).

---

## Configurable ECN threshold and marking probability values

Configurable ECN threshold and marking probability values are Priority Flow Control (PFC) congestion optimization mechanisms that

- activate Explicit Congestion Notification (ECN) thresholds before PFC pause thresholds
- provide fine-grained control over when ECN marking begins and how marking probability increases, and
- help prevent aggressive throttling of source traffic by signaling congestion early and gradually.

### Priority Flow Control modes and support

- This functionality is supported only when PFC is configured in buffer-internal mode.
- In buffer-extended mode, ECN thresholds are derived from QoS policy maps and are documented under the Congestion Avoidance feature.

## How Explicit Congestion Notification works within Priority Flow Control buffer-internal mode

In buffer-internal mode, Explicit Congestion Notification (ECN) acts as an early congestion notifier within the PFC control loop. It gradually signals congestion to senders through ECN markings before resorting to link-level pause or packet drops.

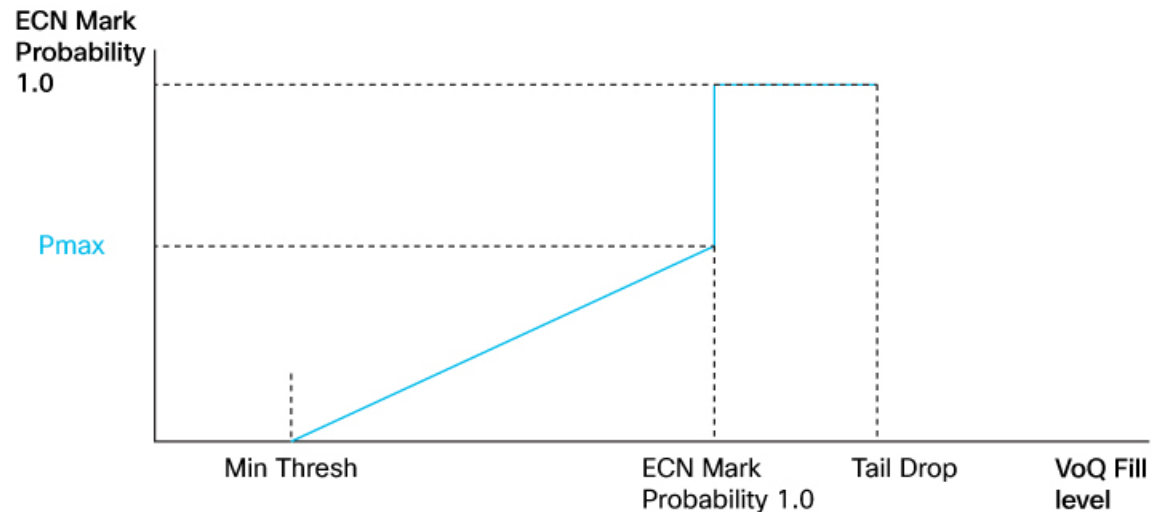
### Summary

The figure, ECN Mark Probability vs. Queue Length (VOQ Fill Levels) illustrates the progressive relationship between queue depth and router response:

- ECN Min threshold: Start of the probabilistic marking region
- ECN Max threshold : End of the linear region; ECN marking probability reaches Pmax
- Pause threshold: Queue depth where PFC pause frames are triggered
- Tail drop threshold: Final limit beyond which packets are dropped deterministically.

## Workflow

Figure 8: ECN Mark Probability vs. Queue Length (VOQ Fill Levels)



522803

The stages show how ECN, PFC, and tail drop operate sequentially as queue depth increases.

1. Queue occupancy increases: The router monitors each traffic class queue within the Shared Memory System (SMS). ECN marking does not occur while depth is below the ECN Min Threshold.
2. ECN marking begins (early warning): When queue depth exceeds the ECN Min Threshold, the router marks packets with ECN bits at a low probability. This action provides early congestion feedback to upstream devices.
3. Marking probability rises linearly: As the queue grows from the ECN Min to Max threshold, marking probability increases linearly—from 0 percent to the configured Pmax value, for example, 5 percent.
4. Full ECN marking region reached: After the ECN Max Threshold is crossed, marking probability becomes 1 (100%). Every packet is ECN-marked, but all traffic continues to be forwarded with no packet drops.
5. Pause threshold reached (PFC activation): If congestion continues and queue depth reaches the configured pause threshold, the router sends PFC PAUSE frames to upstream devices for the affected traffic class. ECN marking stays active at 100%, while PFC halts incoming traffic temporarily to allow queues to drain.
6. Tail drop protection: If the queue continues to grow and crosses the tail drop threshold, the router starts dropping packets deterministically. This action protects against buffer exhaustion when ECN and PFC feedback have not restored stability.
7. Congestion clears: When queue depth falls below the ECN Min Threshold, ECN marking ceases, pause frames stop, and traffic resumes normal operation.

| When...                             | Then...                                       | And...                             |
|-------------------------------------|-----------------------------------------------|------------------------------------|
| Queue depth < ECN min threshold     | Router does not mark packets                  | Marking probability = 0%           |
| Queue depth ≥ ECN min threshold     | Router begins ECN marking                     | Probability increases linearly     |
| Queue depth between ECN min and max | ECN marking probability rises from 0% to Pmax | Provides early congestion feedback |

| When...                                                      | Then...                                | And...                           |
|--------------------------------------------------------------|----------------------------------------|----------------------------------|
| Queue depth $\geq$ ECN max threshold but $<$ pause threshold | All packets are ECN-marked             | No packet drops yet              |
| Queue depth $\geq$ pause threshold                           | Router sends PFC pause frames          | ECN marking continues at 100%    |
| Queue depth $\geq$ Tail-Drop Threshold                       | Router drops packets deterministically | Protects against buffer overflow |
| Queue depth $<$ ECN Min Threshold (after drain)              | ECN marking and pause actions cease    | Normal operations resume         |

## PFC buffer internal mode configuration options and behaviors

You configure the max-threshold and probability-percentage options for this feature within the **hw-module profile priority-flow-control** command.

This feature gives you the flexibility to choose one of the following configuration options:

- Default **max-threshold** and **probability-percentage** values
- User-defined (configurable) **max-threshold** and **probability-percentage** values
- Priority Flow Control (PFC) in buffer-internal mode without these new options, as supported in releases prior to Release 7.5.4.

**Table 47: Useful Tips**

| If you...                                                                                                                                                                   | ...you must...                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Want to switch from the default configuration mode to the custom configuration mode<br>OR<br>Want to switch from the custom configuration mode to the default configuration | <ol style="list-style-type: none"> <li>1. Use the <b>no</b> form of the <b>hw-module profile priority-flow-control</b> command to remove the existing configuration.</li> <li>2. Configure the new mode and settings using the <b>hw-module profile priority-flow-control</b> command.</li> <li>3. Reload the line card</li> </ol> |
| Configured PFC in buffer-internal mode without configuring the <b>max-threshold</b> and <b>probability-percentage</b> parameters, but now want to configure them            | <ol style="list-style-type: none"> <li>1. Configure the <b>max-threshold</b> and <b>probability-percentage</b> parameters using the <b>hw-module profile priority-flow-control</b> command.</li> <li>2. Reload the line card.</li> </ol>                                                                                           |
| Want to change the buffer-internal parameters in the custom mode                                                                                                            | Configure the buffer-internal parameters using the <b>hw-module profile priority-flow-control</b> command. You do not need to reload the line card.                                                                                                                                                                                |

| If you...                                                                                                   | ...you must...                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Want to continue configuring PFC in buffer-internal mode the way you did in releases prior to Release 7.5.4 | <p>Configure the buffer-internal parameters using the <b>hw-module profile priority-flow-control</b> command, but ensure that you only configure values for <b>pause-threshold</b>, <b>headroom</b>, and <b>ecn</b>. If you don't configure values for <b>max-threshold</b>, the router takes the <b>ecn</b> value as the ECN maximum threshold value.</p> <p>For more details, refer to Option 3 in the section titled Configure ECN threshold and maximum marking probability values.</p> |

## Best practices for configuring ECN threshold and marking probability

Follow these principles to ensure that configuration is consistent and operation is predictable for Explicit Congestion Notification (ECN) thresholds and marking probabilities within Priority Flow Control (PFC).

- Mode dependency: This functionality is supported only when PFC is configured in buffer-internal mode.
- ECN value derivation: If you configure PFC values in the buffer-internal mode, the ECN value for the line card is derived from the buffer-internal configuration, unlike in the buffer-extended mode where the ECN value is derived from the policy map.
- Hardware Support:
 

Supported line cards include:

  - 88-LC0-36FH
  - 88-LC0-36FH-M
- Supported interface types include:
  - Physical interfaces
  - Bundle interfaces
  - Subinterfaces
  - Bundle subinterfaces
- Interface speeds: This functionality is supported across all interface speeds.
- Policy map applicability:
 

If your policy map enables maximum ECN marking probability for one or more classes, you can:

  - apply the map to any of the supported interface types
  - remove the map from any of the supported interface types
  - modify the map while you're attaching it to multiple interfaces.
- Class configuration dependency: The probability percentage option is valid only when **random-detect ecn** is configured in the same class. Otherwise, the policy is rejected when applied.

- Device level consistency: Ensure that the configured probability percentage value is identical for all traffic classes, because this setting is enforced at the device level.

## Configure ECN threshold and maximum marking probability values

Use this task to configure the Explicit Congestion Notification (ECN) thresholds and marking probability values for Priority Flow Control (PFC) operating in buffer-internal mode.

Depending on your network requirements, you can choose the default options, custom options, or use the existing configuration without the new options

In buffer-internal mode, you can configure PFC with adjustable ECN thresholds to optimize congestion control. You can manage queue behavior by defining the minimum and maximum thresholds and the marking probability.

The task supports three configuration paths:

- Option 1: Default configuration mode (uses predefined ECN and marking values)
- Option 2: Custom configuration mode (you define every parameter, including **max-threshold** and **probability-percentage**)
- Option 3: Configuration without the new options, as used before Release 7.5.4.

### Before you begin

- Ensure that PFC is enabled on the router.
- Confirm that the buffer-internal mode is enabled.

Follow these steps to configure ECN threshold and maximum marking probability values.

### Procedure

**Step 1** Enable PFC on the interface.

#### Example:

```
Router(config)#interface FourHundredGigE0/6/0/1
Router(config-if)#priority-flow-control mode on
```

**Step 2** Choose one configuration option that meets your requirements.

- Option 1: Default configuration mode

Configure PFC in buffer-internal mode using predefined buffer values and default ECN thresholds.

```
Router(config)#hw-module profile priority-flow-control location 0/6/0/1
Router(config-pfc-loc)#buffer-internal traffic-class 3
Router(config-pfc-loc)#buffer-internal traffic-class 4
Router(config-pfc-loc)#commit
```

This mode applies the system's default **max-threshold** and **probability-percentage** values for ECN.

- Option 2: Custom configuration modeConfigure PFC in buffer-internal mode with custom thresholds for all parameters, including **max-threshold** and **probability-percentage**.

```

Router(config)#hw-module profile priority-flow-control location 0/6/0/1
Router(config-pfc-loc)#buffer-internal traffic-class 3 pause-threshold 1574400 bytes headroom
1651200 bytes ecn 629760 bytes max-threshold 1416960 bytes probability-percentage 50
Router(config-pfc-loc)#buffer-internal traffic-class 4 pause-threshold 1574400 bytes headroom
1651200 bytes ecn 629760 bytes max-threshold 1416960 bytes probability-percentage 50
Router(config-pfc-loc)#commit

```

You define all thresholds and probabilities. This option provides the highest control over ECN marking behavior.

- Option 3: Configuration mode without ECN enhancements

Configure PFC in buffer-internal mode without **max-threshold** and **probability-percentage** parameters.

```

Router(config)#hw-module profile priority-flow-control location 0/6/0/1
Router(config-pfc-loc)#buffer-internal traffic-class 3 pause-threshold 1574400 bytes headroom
1651200 bytes ecn 629760 bytes
Router(config-pfc-loc)#buffer-internal traffic-class 4 pause-threshold 1574400 bytes headroom
1651200 bytes ecn 629760 bytes
Router(config-pfc-loc)#commit

```

This option uses configurations prior to Release 7.5.4.

If you do not configuring **max-threshold**, the router uses the ECN threshold value as the ECN maximum threshold.

### Step 3 Verify the configuration.

#### Example:

```
Router#show controllers npu priority-flow-control location all
```

- Option 1: Default configuration

```

Location: 0/6/CPU0
PFC: Enabled
PFC Mode: buffer-internal
TC  Pause-threshold  Headroom  ECN      ECN-MAX  Prob-per
-----
3   1574400 bytes    1651200  629760   1416960  5
4   1574400 bytes    1651200  629760   1416960  5

```

Both traffic classes (3 and 4) use the system defaults for ECN maximum threshold and marking probability (5 percent).

- Option 2: Custom configuration modeConfigure PFC in buffer-internal mode with custom thresholds for all parameters, including **max-threshold** and **probability-percentage**.

```

Location: 0/6/CPU0
PFC: Enabled
PFC Mode: buffer-internal
TC  Pause-threshold  Headroom  ECN      ECN-MAX  Prob-per
-----
3   1574400 bytes    1651200  629760   1416960  50
4   1574400 bytes    1651200  629760   1416960  50

```

The ECN max threshold (1416960 bytes) and marking probability (50 percent) reflect custom values.

- Option 3: Configuration mode without **max-threshold** and **probability-percentage** parameters.

```

Location: 0/6/CPU0
PFC: Enabled
PFC Mode: buffer-internal
TC  Pause-threshold  Headroom  ECN      ECN-MAX  Prob-per
-----

```

```

3  1574400 bytes    1651200    629760    not-configured not-configured
4  1574400 bytes    1651200    629760    not-configured not-configured

```

Since **max-threshold** and **probability-percentage** are not configured, these fields display **not-configured**, and the ECN threshold functions as the effective maximum threshold.

---

You have successfully configured ECN thresholds and marking probabilities for buffer-internal mode. The router applies ECN marking and congestion feedback based on your chosen configuration option.

## buffer-extended mode

A buffer-extended mode is a Priority Flow Control (PFC) operating mode that

- allows pause-thresholds to be configured in the line card profile
- requires Explicit Notification Congestion (ECN) and queue limit configuration in the interface's egress queuing policies, and
- applies consistently to all ports on the line card.

### Supporting details for buffer-extended mode

- Provides more flexibility than buffer-internal mode, since ECN and queue limits can differ per interface.
- Recommended for long-haul PFC peers, such as data center interconnects over 1 km.
- Queue-limit and ECN configurations in policies take effect, unlike buffer-internal mode where they are ignored.

### Lane traffic analogy for buffer-extended mode

Each lane still has its own small holding bay, but the city traffic authority sets one common stop line for everyone. This is the **pause-threshold** defined in the line-card profile.

Each lane's local signal controller (the interface-level QoS policy) decides when to flash its yellow warning light, which is the ECN threshold that each interface can tune separately.

So, while all lanes share the same stop rule, each lane can warn drivers earlier or later depending on how busy it gets.

## Best practices for configuring buffer-extended mode

### buffer-extended mode threshold ranges

Set the pause-threshold value range for buffer-extended mode between 2 milliseconds (ms) and 25 ms or from 2000 microseconds (μs) and 25000 μs.

### Guidelines to configure buffer-extended mode

Follow these guidelines to ensure a stable Priority Flow Control (PFC) configuration in the buffer-extended mode.



- For Cisco 8808 and Cisco 8812 chassis, configure Priority Flow Control (PFC) thresholds on all line cards in the chassis.
- Reload the line card whenever you add or remove a traffic class.
- Add or remove ECN configuration using the **hw-module profile priority-flow-control** command. Then, reload the line card to apply the ECN changes.
- For line card 88-LC0-36FH-M, use kilobytes (KB) or megabytes (MB) for both pause and headroom thresholds. The headroom value range is 4 to 75,000.
- For line card 8800-LC-48H, use milliseconds (ms) or microseconds (µs) for pause thresholds. Do not configure headroom values. Always use ms or µs even if KB or MB are shown in the CLI.

## Restrictions while configuring buffer-extended mode

Enforce these restrictions while configuring the buffer-extended mode thresholds.

- Ensure the ECN threshold value is less than the pause threshold value.
- Ensure that the combined configuration values for pause threshold and headroom do not exceed 844,800 bytes; otherwise, the configuration is rejected.
- Configure only one of buffer-internal, buffer-extended, or buffer-extended hybrid mode on each line card.

## Configure buffer-extended thresholds

Use this task to configure Priority Flow Control (PFC) pause-thresholds in the line-card profile and define Explicit Congestion Notification (ECN) and queue-limit parameters in interface-level QoS policies.

buffer-extended mode separates global pause configuration from per interface congestion control:

- Pause thresholds are configured once in the line-card profile.
- ECN and queue limits are set per interface in QoS policies.

This enables fine-grained traffic management across multiple interfaces on the same line card.

### Before you begin

- The router must already be configured for buffer-extended mode.
- PFC must be globally enabled (**priority-flow-control mode on**).

Follow these steps to configure buffer-extended thresholds.

### Procedure

**Step 1** Enter the line card PFC profile and enable buffer-extended mode.

#### Example:

```
Router(config)#hw-module profile priority-flow-control mode buffer-extended
```

**Step 2** Specify the **pause-threshold** value for the desired traffic class.

**Example:**

```
Router(config-pfc-profile)#traffic-class 3
Router(config-pfc-profile-tc)#pause-threshold 2000 us
```

- All **pause-threshold** values configured under this profile will apply uniformly to every port on that line card.
- This step defines the point at which PFC sends a pause frame upstream for traffic class 3 when congestion is detected.
- The unit ( $\mu$ s, ms, KB, or MB) depends on your specific Cisco 8000 router, but the function remains the same: this value triggers pausing traffic on the specified class of service (CoS).

**Step 3** (Optional) Configure additional buffer parameters if supported by your line card.

**Example:**

```
Router(config-pfc-profile-tc)#headroom 600 KB
```

- On some Cisco 8000 routers, you can explicitly reserve additional buffer space beyond the pause threshold (called headroom) to absorb packets already in flight when a pause frame is issued.
- On hardware where headroom is not configurable, this step is omitted.

**Step 4** Create or edit the interface-level QoS policy to define ECN thresholds.

**Example:**

```
Router(config)#policy-map qos-policy
Router(config-pmap)#class COS_3
Router(config-pmap-c)#random-detect min-threshold 200 KB max-threshold 400 KB probability 5
Router(config-pmap-c)#random-detect ecn
```

- This step creates or edits the interface-level QoS policy that defines ECN thresholds and probabilities.
- The **random-detect** command enables Weighted Random Early Detection (WRED), the mechanism that provides early congestion signaling before a PFC pause frame is triggered.
- The **min-threshold** (200 KB) marks the queue depth where ECN marking begins.
- The **max-threshold** (400 KB) is the queue depth where ECN marking probability increases to the configured probability (5 %).
- Between these two points, the system automatically increases the marking probability in a linear fashion.
- If ECN is enabled, packets are marked with ECN bits instead of being dropped.
- If ECN is disabled, packet drops occur early to prevent queue buildup.

This configuration allows the router to signal congestion early, reducing the likelihood of abrupt PFC pauses and maintaining smoother traffic flow.

**Step 5** Apply the QoS policy to the required interface.

**Example:**

```
Router(config)#interface HundredGigE0/0/0/2
Router(config-if)#service-policy output qos-policy
Router(config)#exit
Router(config)#commit
```

- This step creates or edits the interface-level QoS policy that defines ECN thresholds and probabilities.

- The **random-detect** command enables Weighted Random Early Detection (WRED), the mechanism that provides early congestion signaling before a PFC pause frame is triggered.
- The **min-threshold** (200 KB) marks the queue depth where ECN marking begins.
- The **max-threshold** (400 KB) is the queue depth where ECN marking probability increases to the configured probability (5%).
- Between these two points, the system automatically increases the marking probability in a linear fashion.
- If ECN is enabled, packets are marked with ECN bits instead of being dropped.
- If ECN is disabled, packet drops occur early to prevent queue buildup.

This configuration allows the router to signal congestion early, reducing the likelihood of abrupt PFC pauses and maintaining smoother traffic flow.

**Step 6** View pause thresholds applied on the line card.

**Example:**

```
Router#show controllers npu priority-flow-control location 0/1/CPU0
Traffic Class  Pause-Threshold  ECN Source  Status
3              2000 us          Policy-Map  Enabled
```

- The configured traffic class (3) now uses a 2000  $\mu$ s pause threshold.
- The ECN source is listed as **Policy-Map**, confirming that ECN thresholds are coming from interface-level configuration.
- The **min-threshold** (200 KB) marks the queue depth where ECN marking begins.
- The **max-threshold** (400 KB) is the queue depth where ECN marking probability increases to the configured probability (5 %).
- Between these two points, the system automatically increases the marking probability in a linear fashion.
- If ECN is enabled, packets are marked with ECN bits instead of being dropped.
- If ECN is disabled, packet drops occur early to prevent queue buildup.

This configuration allows the router to signal congestion early, reducing the likelihood of abrupt PFC pauses and maintaining smoother traffic flow.

**Step 7** View ECN thresholds and queue limits on the interface.

**Example:**

```
Router#show policy-map interface HundredGigE0/0/0/2
Class-map: COS_3
  Random Detect (WRED):
    Min Threshold : 200 KB
    Max Threshold : 400 KB
    Probability   : 5 %
    Mark Type     : ECN
```

- The configured ECN minimum and maximum thresholds are applied to the interface.
- **Mark Type:** ECN confirms that early congestion notification is active instead of packet drops.
- The **min-threshold** (200 KB) marks the queue depth where ECN marking begins.

- The **max-threshold** (400 KB) is the queue depth where ECN marking probability increases to the configured probability (5 %).
- Between these two points, the system automatically increases the marking probability in a linear fashion.
- If ECN is enabled, packets are marked with ECN bits instead of being dropped.
- If ECN is disabled, packet drops occur early to prevent queue buildup.

This configuration allows the router to signal congestion early, reducing the likelihood of abrupt PFC pauses and maintaining smoother traffic flow.

## Configure PFC on interfaces

Use this task to enable Priority Flow Control (PFC) on supported line cards and interfaces, and to apply the classification and queuing policies that allow selected traffic classes to operate losslessly.

- PFC must be enabled globally and per line card before it can be applied to traffic classes.
- The operating mode (buffer-internal, buffer-extended, or buffer-extended hybrid) must be chosen at the line card profile level.

### Before you begin

- Install the supported line card.
- Configure PFC on matching CoS values.
- Ensure there are no breakout, bundle, or sub-interface configurations on PFC-enabled line cards.

Follow these steps to configure PFC on an interface.

### Procedure

**Step 1** Enable PFC globally.

#### Example:

```
Router(config)#priority-flow-control mode on
```

**Step 2** Configure the line card profile for the required PFC mode.

#### Example:

```
Router(config)#priority-flow-control mode on
```

- Option A: buffer-internal mode

```
Router(config)#hw-module profile priority-flow-control mode buffer-internal
Router(config-pfc-profile)#traffic-class 3
Router(config-pfc-profile-tc)#pause-threshold 307200
Router(config-pfc-profile-tc)#headroom 345600
Router(config-pfc-profile-tc)#ecn-threshold 153600
```

This option defines all threshold values directly in the LC profile so each traffic class uses fixed on-chip buffers.

- Option B: buffer-extended mode

```
Router(config)#hw-module profile priority-flow-control mode buffer-extended
Router(config-pfc-profile)#traffic-class 3
Router(config-pfc-profile-tc)#pause-threshold 2000 us
```

- This option configures only the global **pause-threshold** in the line card profile while leaving Explicit Congestion Notification (ECN) and queue thresholds to interface policies.
- This option centralizes pause control across the line card but allows per-interface tuning through QoS policies.

- Option C: buffer-extended hybrid

```
Router(config)#hw-module profile priority-flow-control hybrid-mode enable
Router(config-pfc-profile)#hbm-buffers-percentage lossless 60 lossy 40
Router(config-pfc-profile)#max-non-pfc-voqs 2500
```

This option divides High Bandwidth Memory (HBM) between lossless and lossy traffic:

- **hbm-buffers-percentage** allocates 60 percent of HBM to the lossless pool and 40 percent to lossy traffic.
- **max-non-pfc-voqs** caps the number of lossy queues that share HBM resources.
- The configuration takes effect after a router restart.

### Step 3 Configure and attach classification and queuing policies.

#### Example:

```
Router(config)#policy-map pfc-ingress
Router(config-pmap)#class COS_3
Router(config-pmap-c)#priority-flow-control enable
Router(config)#policy-map qos-policy
Router(config-pmap)#class COS_3
Router(config-pmap-c)#random-detect min-threshold 200 KB max-threshold 400 KB probability 5
Router(config-pmap-c)#random-detect ecn
Router(config-pmap-c)#exit
Router(config-pmap)#exit
Router(config)#interface HundredGigE0/0/0/1
Router(config-if)#service-policy input pfc-ingress
Router(config-if)#service-policy output qos-policy
Router(config)#commit
```

This example:

- creates a policy map named **pfc-ingress** to handle incoming traffic related to PFC.
- defines a class named **COS\_3** inside the **pfc-ingress** policy map.
- enables PFC processing on incoming traffic for **COS\_3**.
- creates another policy map named **qos-policy** to apply QoS mechanisms to outgoing traffic.
- defines a class named **COS\_3** within the **qos-policy** policy map.
- configures Random Early Detection (RED), **min-threshold**, **max-threshold**, and the probability of dropping packets for **COS\_3** traffic.
- enables Explicit Congestion Notification (ECN) in conjunction with RED.

- applies the **pfc-ingress** policy map to the input direction of the HundredGigE0/0/0/1 interface. This means that any traffic arriving on this interface will be evaluated against the **pfc-ingress** policy, and the router will honor incoming PFC pause frames for **COS\_3** traffic.
- applies the **qos-policy** policy map to the output direction of the HundredGigE0/0/0/1 interface. This means that any traffic being transmitted out of this interface will be evaluated against the **qos-policy**, and the RED or ECN mechanisms will be applied to **COS\_3** traffic to manage congestion.

**Step 4** Verify the hardware-level PFC profile configuration on a specific line card.

**Example:**

```
Router#show controllers npu priority-flow-control location 0/0/CPU0
Location: 0/0/CPU0
PFC Mode: buffer-internal
```

| Traffic Class | Pause-threshold | Headroom | ECN-threshold | Tx Pause Frames |
|---------------|-----------------|----------|---------------|-----------------|
| 3             | 307200          | 345600   | 153600        | 24              |

A non-zero and increasing **Tx Pause Frames** count indicates that the router's NPU is actively experiencing congestion for this traffic class and is sending pause frames to its neighbors. This is a key indicator that PFC is working as intended.

**Step 5** View the PFC statistics for a specific interface.

**Example:**

```
Router#show controllers HundredGigE0/0/0/1 priority-flow-control
Priority flow control information for interface HundredGigE0/0/0/1:
Priority Flow Control:
Total Rx PFC Frames : 95
Total Tx PFC Frames : 102
Rx Data Frames Dropped: 0
CoS Status Rx Frames
---
0 on 0
1 on 0
2 on 0
3 on 95
4 on 0
5 on 0
6 on 0
7 on 0
```

- **Total Rx PFC Frames : 95** indicates that this interface has received 95 PFC pause frames in total.
- **Total Tx PFC Frames : 102** indicates that this interface has transmitted 102 PFC pause frames in total.
- **Rx Data Frames Dropped: 0** indicates that the PFC mechanism is effectively preventing data frame drops on ingress due to congestion on this interface.

**Step 6** To view the consolidated statistics for all interfaces on the router, see [View global statistics for Priority Flow Control and Priority Flow Control watchdog, on page 177](#).

# High Bandwidth Memory congestion detection

High Bandwidth Memory (HBM) congestion detection is a Priority Flow Control (PFC) congestion monitoring functionality that

- helps monitor buffer occupancy across the Shared Memory System (SMS) and High-Bandwidth Memory (HBM)
- detects when Virtual Output Queue (VOQ) spillover into HBM indicates congestion, and
- records time-stamped congestion events and history for post-event analysis and troubleshooting.

**Table 48: Feature History Table**

| Feature Name                            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detect High Bandwidth Memory Congestion | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)</p> <p>This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                                                                              |
| Detect High Bandwidth Memory Congestion | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name                            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detect High Bandwidth Memory Congestion | Release 7.5.3       | <p>We provide detailed insights into congestion on the High Bandwidth Memory (HBM), such as the devices on which congestion has occurred, the time stamps, and when the device returned to its normal state. With such details, you can investigate the cause of congestion and identify the source ports causing congestion for future preventive actions.</p> <p>You must configure PFC in the buffer-extended mode for this option.</p> <p>The feature introduces the following to enable the option to detect HBM congestion:</p> <ul style="list-style-type: none"> <li>• YANG data model (at <a href="#">Github</a> under the 753 folder):<br/>Cisco-IOS-XR-um-8000-hw-module-profile-cfg</li> <li>• CLI: <b>hw-module profile npu memory buffer-extended location bandwidth-congestion-detection enable</b></li> </ul> <p>It also introduces the following to view the congestion and memory usage details:</p> <ul style="list-style-type: none"> <li>• YANG data model (at <a href="#">Github</a> under the 753 folder):<br/>Cisco-IOS-XR-8000-platforms-npu-memory-oper</li> <li>• CLI: <b>show controllers npu packet-memory</b></li> </ul> |

## How HBM congestion detection works

This process explains how queue congestion moves from the Shared Memory System (SMS) to the High-Bandwidth Memory (HBM) on your router. It also describes how the HBM congestion detection feature captures buffer utilization details for post-congestion analysis.

### Summary

The key components involved in High Bandwidth Memory congestion detection are:

- Shared Memory System (SMS): the on-chip buffer where queues are normally stored and monitored for occupancy thresholds.
- High Bandwidth Memory (HBM): the external, off-chip buffer that temporarily stores excess packets evicted from SMS during congestion.
- Virtual Output Queues (VOQs): logical queues that carry per-class traffic from SMS to HBM.
- Detection logic within the NPU: tracks queue depth, HBM use, and packet age to identify congestion.



### Workflow

These stages describe how HBM congestion detection works.

1. Normal queue operation: Under normal conditions, packets egressing from an interface are enqueued into the Shared Memory System (SMS), the router's on-chip buffer memory.
2. Onset of congestion: When congestion begins, the SMS continues to buffer packets until queue occupancy exceeds per queue usage criteria.

These criteria are based on two parameters.

- Buffer space threshold per VOQ, and
  - Packet age, measured in milliseconds.
3. Eviction to HBM: Once the usage threshold is crossed, the router evicts packets from the SMS to the HBM.
  4. Accumulation of HBM load: A sustained HBM load marks the onset of HBM congestion conditions.
  5. HBM congestion metrics: By configuring the command `hw-module profile npu memory buffer-extended bandwidth-congestion-detection enable`, you can view information such as devices on which congestion has occurred and the timestamps, and the current as well as highest buffer memory usage watermark since the last reading.

### Result

Use this information for post-event analysis and reporting.

## Best practices for HBM congestion detection

Use these practices to ensure accurate monitoring when enabling High Bandwidth Memory (HBM) congestion detection on your router.

- Priority Flow Control mode: You must configure Priority Flow Control (PFC) in buffer-extended mode to enable this functionality.
- Line card reload: No line card reload is required after running the **hw-module profile npu memory buffer-extended bandwidth-congestion-detection enable** command.
- Supported hardware: This functionality is supported on:
  - Cisco Silicon One Q200-based routers and line cards
  - 8201-32FH routers
  - 88-LC0-48TH-MO, 88-LC0-36FH, and 88-LC0-36FH-M line cards

## Configure High Bandwidth Memory congestion detection

Capture congestion events and memory usage data for High Bandwidth Memory (HBM) and Shared Memory System (SMS).

### Before you begin

Enable Priority Flow Control (PFC) buffer-extended mode.

Follow these steps to enable HBM congestion detection and view the details.

## Procedure

**Step 1** Enable congestion detection.

### Example:

```
Router#configure
Router(config)#hw-module profile npu buffer-extended location 0/6/CPU0
bandwidth-congestion-detection enable
Router(config)#commit
Router(config)#exit
```

Line card reload is not required.

**Step 2** View the buffer usage and HBM congestion details.

| If ...                                    | Then...                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| You want to view buffer details           | <p>Run</p> <ul style="list-style-type: none"> <li>• <b>show controllers npu packet-memory usage instance all location all</b></li> <li>• <b>show controllers npu packet-memory usage verbose instance all location all</b></li> </ul> <p><b>Note</b></p> <ul style="list-style-type: none"> <li>• These commands provide data for the current use and the highest watermark reached since the last reading for both SMS and HBM.</li> <li>• The refresh interval for the information is 30 seconds.</li> </ul>                                                                                                                                                                             |
| You want to view HBM bandwidth congestion | <p>Run</p> <ul style="list-style-type: none"> <li>• <b>show controllers npu packet-memory congestion instance all location all</b></li> <li>• <b>show controllers npu packet-memory congestion detail instance all location all</b></li> <li>• <b>show controllers npu packet-memory congestion verbose instance all location all</b></li> </ul> <p><b>Note</b></p> <ul style="list-style-type: none"> <li>• These commands provide data for when the HBM congestion occurred or is about to happen.</li> <li>• The output maintains a history of the last 120 events regardless of the elapsed time.</li> <li>• The refresh interval for new events to be added is 30 seconds.</li> </ul> |

- View the buffer details.

```
Router#show controllers npu packet-memory usage instance all location all
HW memory Information For Location: 0/6/CPU0
```

| Timestamp(msec)                 | Device | Buff-int<br>Usage | Buff-int<br>Max WM | Buff-ext<br>Usage | Buff-ext<br>Max WM |
|---------------------------------|--------|-------------------|--------------------|-------------------|--------------------|
| Wed 2022-09-21 01:54:11.154 UTC | 0      | 7                 | 8                  | 0                 | 0                  |
| Wed 2022-09-21 01:54:12.154 UTC | 0      | 7                 | 8                  | 0                 | 0                  |
| Wed 2022-09-21 01:54:24.023 UTC | 1      | 22                | 22                 | 0                 | 0                  |
| Wed 2022-09-21 01:54:34.088 UTC | 2      | 11                | 12                 | 0                 | 0                  |
| Wed 2022-09-21 01:54:35.088 UTC | 2      | 11                | 12                 | 0                 | 0                  |
| Wed 2022-09-21 01:54:36.088 UTC | 2      | 11                | 12                 | 0                 | 0                  |
| Wed 2022-09-21 01:54:37.088 UTC | 2      | 11                | 12                 | 0                 | 0                  |
| Wed 2022-09-21 01:54:38.089 UTC | 2      | 11                | 12                 | 0                 | 0                  |
| Wed 2022-09-21 01:54:39.089 UTC | 2      | 11                | 12                 | 0                 | 0                  |
| Wed 2022-09-21 01:54:40.089 UTC | 2      | 11                | 12                 | 0                 | 0                  |

This example displays:

- The timestamp at which data is sampled.
  - The network processor name (**Device**).
  - The packet memory usage for that timestamp for SMS (**Buff-int Usage** in units of buffers) and HBM (**Buff-ext Usage** in units of 8 KB blocks).
  - The highest maximum watermark reached for SMS (**Buff-int Max WM**) and HBM (**Buff-ext Max WM**) since the last reading.
- View the timestamp in milliseconds and the buffer details.

```
Router#show controllers npu packet-memory usage verbose instance all location all
```

HW memory Information For Location: 0/RP0/CPU0

\* Option 'verbose' formatted data is for internal consumption.

| Timestamp(msec) | Device | Buff-int<br>Usage | Buff-int<br>Max WM | Buff-ext<br>Usage | Buff-ext<br>Max WM |
|-----------------|--------|-------------------|--------------------|-------------------|--------------------|
| 1663958881006   | 0      | 2455              | 2676               | 637               | 640                |
| 1663958882007   | 0      | 2461              | 2703               | 635               | 640                |
| 1663958883007   | 0      | 2364              | 2690               | 635               | 640                |
| 1663958884007   | 0      | 71603             | 75325              | 3183              | 18336              |
| 1663958885008   | 0      | 2458              | 2852               | 1275              | 1279               |
| 1663958886008   | 0      | 2484              | 2827               | 1275              | 1279               |

- Check if HBM congestion has occurred and view the timestamp of the congestion state.

```
Router#show controllers npu packet-memory congestion instance all location all
HW memory Information For Location: 0/6/CPU0
```

| Timestamp(msec)                 | Device | Buff-ext<br>Event Type |
|---------------------------------|--------|------------------------|
| Wed 2022-09-21 02:14:41.709 UTC | 1      | Congest                |
| Wed 2022-09-21 02:14:41.959 UTC | 1      | Congest                |
| Wed 2022-09-21 02:14:42.960 UTC | 1      | Congest                |
| Wed 2022-09-21 02:14:43.960 UTC | 1      | Congest                |
| Wed 2022-09-21 02:14:45.210 UTC | 1      | Congest                |

```

Wed 2022-09-21 02:14:45.710 UTC      Congest      1
Wed 2022-09-21 02:14:47.711 UTC      Normal       1

```

The system displays the last 120 events and adds new events every 30 seconds. To view updated data, run the command again.

After 120 events, the system replaces the oldest event with the newest. You cannot remove events from the list.

- View additional details about HBM congestion.

```

Router#show controllers npu packet-memory congestion detail instance all location all
Fri Sep 23 18:49:50.640 UTC
HW memory Information For Location: 0/RP0/CPU0

```

\* Option 'detail' formatted data is for internal consumption.

| Timestamp(msec)                 | Evicted-buff | Buff-int | Buff-int | Buff-ext   | Buff-ext | Device | Slice | VOQ | VOQ-buff |
|---------------------------------|--------------|----------|----------|------------|----------|--------|-------|-----|----------|
|                                 |              |          |          | Event Type |          |        |       |     |          |
|                                 | int-WM       | UC-WM    | Usage    | Max WM     | Usage    | Max WM |       |     |          |
| Fri 2022-09-23 18:42:30.349 UTC | 63969        | 65451    | 70410    | 70410      | Congest  | 34405  | 34405 | 5   | 534      |
| Fri 2022-09-23 18:42:31.101 UTC | 0            | 900      | 2440     | 2440       | Normal   | 0      | 0     | 5   | 534      |
| Fri 2022-09-23 18:42:37.354 UTC | 63984        | 65493    | 70573    | 70573      | Congest  | 34408  | 34408 | 5   | 534      |
| Fri 2022-09-23 18:42:38.354 UTC | 0            | 915      | 2455     | 2455       | Normal   | 0      | 0     | 5   | 534      |
| Fri 2022-09-23 18:42:44.606 UTC | 64002        | 65520    | 70081    | 70081      | Congest  | 34532  | 34532 | 5   | 534      |
| ...                             | ...          | ...      | ...      | ...        | ...      | ...    | ...   | ... | ...      |

This example displays:

- The network processor name (Device ) and the slice number (Slice ) for that device. Every network processor has a fixed number of slices, and each slice, in turn, has a set number of ports.
- Single VOQ buffer and aggregated SMS VOQ buffers.
- The packet memory usage for that timestamp for SMS (**Buff-int Usage** in units of buffers) and HBM (**Buff-ext Usage** in units of 8 KB blocks).
- The highest maximum watermark reached for SMS (**Buff-int Max WM** ) and HBM (**Buff-ext Max WM**) since the last reading.

- View additional HBM congestion details.

```

Router#show controllers npu packet-memory congestion verbose instance all location all
HW memory Information For Location: 0/RP0/CPU0
* Option 'verbose' formatted data is for internal consumption.

```

| Timestamp(msec) | Event    | Device   | Slice    | VOQ  | VOQ-buff | Evicted-buff | Buff-int |
|-----------------|----------|----------|----------|------|----------|--------------|----------|
| Buff-int        | Buff-int | Buff-ext | Buff-ext | Type |          |              |          |
|                 | Max WM   | Usage    | Max WM   |      | int-WM   | int-WM       | UC-WM    |
|                 |          |          |          |      |          |              | Usage    |
| 1663958550349   | 0        | 0        | 5        | 534  | 16011    | 63969        | 65451    |
| 70410           | 70410    | 34405    | 34405    |      |          |              |          |
| 1663958551101   | 1        | 0        | 5        | 534  | 0        | 0            | 900      |
| 2440            | 2440     | 0        | 0        |      |          |              |          |
| 1663958557354   | 0        | 0        | 5        | 534  | 16011    | 63984        | 65493    |
| 70573           | 70573    | 34408    | 34408    |      |          |              |          |
| 1663958558354   | 1        | 0        | 5        | 534  | 0        | 0            | 915      |

```

2455      2455      0      0
1663958564606      0      0      5      534      16011      64002      65520
70081      70081      34532      34532
1663958565356      1      0      5      534      0      0      915
2417      2417      0      0

```

This example also displays:

- Event type, where **0** is single VOQ-based congestion and **1** is single VOQ-based congestion backoff (**VOQ-buff int-WM**), **2** is congestion in aggregated SMS buffers for VOQ and **3** is congestion backoff in aggregated SMS buffers for VOQ (**Evicted-buff int-WM**).
- The buffer internal for unicast (**Buff-int UC-WM**), which is for information only.

## Check available Shared Memory System and High Bandwidth Memory buffers

By enabling instantaneous display of available or free buffers for Shared Memory System (SMS) and High Bandwidth Memory (HBM), you can analyze the congestion affecting buffer occupancy accurately, especially during rapid traffic fluctuations.

### Before you begin

To detect HBM and SMS congestion, use the **hw-module profile npu memory buffer-extended bandwidth-congestion-detection enable** command.

### Procedure

**Step 1** Run the command to view buffer usage statistics.

#### Example:

```
Router#show controller npu packet-memory usage instance all location all
HW memory Information For Location: 0/6/CPU0
```

| Timestamp(msec)                 | Device | Buff-int | Buff-int | Buff-ext | Buff-ext | Buff-int-free |
|---------------------------------|--------|----------|----------|----------|----------|---------------|
| Buff-ext-free                   |        | Usage    | Max WM   | Usage    | Max WM   | Min WM        |
| Min WM                          |        |          |          |          |          |               |
| Wed 2023-08-30 23:47:40.918 UTC | 0      | 1518     | 6668     | 17154    | 17656    | 293394        |
| 982846                          |        |          |          |          |          |               |
| Wed 2023-08-30 23:47:41.918 UTC | 0      | 1227     | 5631     | 16010    | 16427    | 293685        |
| 983990                          |        |          |          |          |          |               |
| Wed 2023-08-30 23:47:42.919 UTC | 0      | 1398     | 8295     | 15041    | 15734    | 293514        |
| 984959                          |        |          |          |          |          |               |
| Wed 2023-08-30 23:47:43.919 UTC | 0      | 1765     | 8892     | 14744    | 15678    | 293147        |
| 985256                          |        |          |          |          |          |               |
| Wed 2023-08-30 23:47:41.011 UTC | 1      | 10380    | 12419    | 37532    | 38165    | 284532        |
| 962468                          |        |          |          |          |          |               |
| Wed 2023-08-30 23:47:42.011 UTC | 1      | 10463    | 11977    | 37315    | 38326    | 284449        |
| 962685                          |        |          |          |          |          |               |
| Wed 2023-08-30 23:47:43.013 UTC | 1      | 9145     | 12604    | 37714    | 38242    | 285767        |
| 962286                          |        |          |          |          |          |               |

Wed 2023-08-30 23:47:44.013 UTC 1 10996 13272 37429 38051 283916  
962571

**Step 2** Review the output. Focus on:

- **Buff-int-free Min WM** (available SMS buffer)
- **Buff-ext-free Min WM** (available HBM buffer)

The current availability of SMS and HBM buffers is displayed, allowing you to analyze congestion and buffer occupancy in real-time.

## Available buffer in Shared Memory System and High Bandwidth Memory

This functionality provides instantaneous visibility of available, or free, buffer space in both Shared Memory System (SMS) and High Bandwidth Memory (HBM), complementing existing watermark data.

These counters for available buffers are accounted for in this way:

- Available SMS buffer (**Buff-int-free Min WM**) at a given instant= Maximum SMS buffer - highest maximum watermark reached for SMS (**Buff-int Max WM**)
- Available HBM buffer (**Buff-ext-free Min WM**) at a given instant = Maximum HBM buffer - highest maximum watermark reached for HBM (**Buff-ext Max WM**)

**Table 49: Feature History Table**

| Feature Name                                                     | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Available Shared Memory System and High Bandwidth Memory Buffers | Release 25.1.1      | Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)<br><br>This feature is supported on: <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                                                                              |
| Available Shared Memory System and High Bandwidth Memory Buffers | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name                                                     | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Available Shared Memory System and High Bandwidth Memory Buffers | Release 24.1.1      | <p>You can now view buffer availability for Shared Memory System (SMS) and High Bandwidth Memory (HBM) with higher accuracy without any lag between the minimum and maximum watermark readings, especially when the packet buffers are used and released rapidly. This is possible because we've enabled the instantaneous display of available or free SMS and HBM.</p> <p>Previously, you could view details only for the highest watermark readings for SMS and HBM.</p> <p>You must configure PFC in the buffer-extended mode for this option, and this functionality is available only for Cisco Silicon One Q200-based routers and line cards.</p> <p>This functionality modifies the following:</p> <ul style="list-style-type: none"> <li>• CLI: <b>show controllers npu packet-memory</b></li> <li>• YANG Data Model:<br/>Cisco-IOS-XR-8000-platforms-npu-memory-oper</li> </ul> |

## Global pause frames for High Bandwidth Memory congestion

A global pause frame (X-Off) for High Bandwidth Memory (HBM) congestion is a pause-protection functionality that

- prevents packet drops on Priority Flow Control (PFC)–enabled queues when HBM bandwidth is saturated
- ensures simultaneous pausing of all active lossless queues to maintain traffic integrity, and
- preserves bandwidth guarantees during extreme congestion without impacting uncongested queues.

When HBM congestion occurs (see High Bandwidth Memory congestion detection), global pause frames (X-Off) are triggered for all PFC-enabled queues, regardless of whether those queues caused the congestion.

Idle queues that are not receiving traffic do not transmit X-Off signals.

This selective behavior ensures that congestion protection does not activate prematurely or unnecessarily, avoiding performance degradation in unaffected queues.

This functionality operates only when PFC is configured in buffer-extended mode and HBM congestion detection is enabled.

Table 50: Feature History Table

| Feature Name                                             | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Global Pause Frames for High Bandwidth Memory Congestion | Release 7.5.4       | <p>We ensure no packet drops on PFC-enabled queues due to High Bandwidth Memory (HBM) congestion. Such prevention of drops is possible because we have enabled the triggering of global pause frames (X-Off) whenever there's HBM congestion.</p> <p>This functionality is disabled by default. You have the following options to enable it:</p> <ul style="list-style-type: none"> <li>• CLI: <b>hw-module profile npu memory buffer-extended bandwidth-congestion-protect enable</b></li> <li>• YANG Data Model:<br/><b>Cisco-IOS-XR-um-8000-hw-module-profile-cfg</b> (see <a href="#">GitHub</a>, <a href="#">YANG Data Models Navigator</a>)</li> </ul> <p>This feature introduces the <b>show hw-module bandwidth-congestion-protect</b> command to view the status of the global X-Off configuration.</p> |

## Recommendations for configuring global pause frames for High Bandwidth Memory congestion

Follow these recommendations when enabling and configuring global pause frames (X-Off) for High Bandwidth Memory (HBM) congestion to ensure consistent lossless behavior and stable Priority Flow Control (PFC) operation.

- **Distance Limitation:** This functionality is not supported when devices operating in buffer-extended mode are more than 0.5 km apart.
- **Headroom Limit:** Configuring the command **hw-module profile npu memory buffer-extended bandwidth-congestion-protect enable** on line cards with headroom values greater than 6,144,000 bytes can cause a commit failure or prevent the feature from activating.
- **Reload Requirement:** You must reload the line card after enabling the command **hw-module profile npu memory buffer-extended bandwidth-congestion-protect enable** for the configuration to take effect.
- **Supported Hardware:** This functionality is supported only on 88-LC0-36FH and 88-LC0-36FH-M line cards.

## Configure global pause frames for High Bandwidth Memory congestion

Use this procedure to enable and verify global pause frames (X-off) for High Bandwidth Memory (HBM) congestion on line cards configured with the buffer-extended mode.



This task ensures lossless behavior by pausing all Priority Flow Control enabled queues when HBM utilization crosses the congestion threshold.

This feature applies to buffer-extended mode. It prevents packet drops during extreme congestion by triggering a global X-off condition. You must reload the line card for the configuration to take effect.

## Procedure

**Step 1** Enable global pause (X-off) protection for HBM congestion.

### Example:

```
Router#config
Router(config)#hw-module profile npu buffer-extended location 0/1/CPU0 bandwidth-congestion-protect
enable
Router(config)#commit
```

**Step 2** Verify global X-off configuration status.

### Example:

```
Router#show hw-module bandwidth-congestion-protect location 0/1/CPU0
Location    Configured    Applied      Action
-----
0/1/CPU0    Yes          No          Reload
```

The table lists possible outputs for different command and commit actions.

**Table 51: Command Output Scenarios**

| If you configured...                                                                                                                               | Configured field displays... | Applied field displays..                                                                                                                                                                 | Then Action field displays... |
|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Configure the <b>hw-module profile npu memory buffer-extended</b> command                                                                          | Yes                          | No                                                                                                                                                                                       | Reload                        |
| Use the <b>no</b> form of the <b>hw-module profile npu memory buffer-extended</b> command after configuring it, but before reloading the line card | No                           | No                                                                                                                                                                                       | N/A                           |
| Configure the <b>hw-module profile npu memory buffer-extended</b> command for a supported variant and reload the line card                         | Yes                          | Yes, Active<br><b>Note</b><br>Yes indicates that the configuration is programmed to the hardware, <b>Active</b> indicates that the global X-off functionality is active on the hardware. | N/A                           |

| If you configured...                                                                                                                            | Configured field displays...                                                                                | Applied field displays..                                                                                    | Then Action field displays... |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------|
| Use the no form of the <b>hw-module profile npu memory buffer-extended</b> command when it is active and commit without reloading the line card | <b>No</b><br><b>Note</b><br>At this stage, the output displays the user action and not the hardware status. | <b>No</b><br><b>Note</b><br>At this stage, the output displays the user action and not the hardware status. | <b>Reload</b>                 |
| Reload the line card after committing the no form of the <b>hw-module profile npu memory buffer-extended</b> command                            | <b>No</b><br><b>Note</b><br>At this stage, the output displays the hardware status.                         | <b>No</b><br><b>Note</b><br>At this stage, the output displays the hardware status.                         | <b>N/A</b>                    |

## buffer-extended hybrid mode

A buffer-extended hybrid mode is a Priority Flow Control (PFC) operating mode that

- divides High Bandwidth Memory (HBM) into two independent pools—one for lossless traffic and another for lossy traffic,
- enables larger HBM allocation (up to 8 GB per pool) for higher-capacity line cards, and
- prevents lossy traffic from monopolizing buffer resources.

### Supporting details for buffer-extended hybrid mode

In the absence of the buffer-extended hybrid mode, with both lossy and lossless traffic, lossy traffic classes experienced tail drops because the combined HBM usage exceeded the 4 GB limit.

The buffer-extended hybrid mode feature addresses this limitation.

- Lossless traffic: Mapped to HBM pool 0 (operates in buffer-extended mode).
- Lossy traffic: Mapped to HBM pool 1 (operates in buffer-internal mode).
- Configuration knobs:
  - **hbm-buffers-percentage** to split the pools.
  - **max-non-pfc-voqs** to cap the number of lossy queues that can share HBM.
- Telemetry: Pool statistics are visible in **show controllers npu priority-flow-control** outputs.

### Lane traffic analogy for buffer-extended hybrid mode

The city adds an expressway divided into two distinct zones. One zone is reserved for priority vehicles (lossless traffic mapped to HBM pool 0) and another for regular traffic (lossy traffic mapped to HBM pool 1).

Both zones still follow the same central stop rules (**pause-thresholds** defined in the line-card profile), but they now draw from separate holding areas (dedicated HBM pools) so busy regular lanes do not block the priority lanes.

Within each zone, the local signal controllers (interface-level QoS policies) still decide when to flash their yellow warning lights (ECN thresholds), but the size of each zone's holding area is set by the planners (HBM pool allocation using the **hbm-buffers-percentage** command).

## Best practices for configuring buffer-internal hybrid mode

Follow these best practices to ensure correct and stable configuration of buffer-internal hybrid mode for PFC on supported line cards.

- Lossless pool allocation: Allocate 50 to 80 percent of High Bandwidth Memory (HBM) to the lossless pool using the **hbm-buffers-percentage** lossless setting
- Lossy pool allocation: Assign the remaining percentage to the lossy pool, ensuring the total allocation equals 100 percent.
- VOQ maximum setting: Set **max-non-pfc-voqs** to a value between 1 and 3,800, according to system needs.
- Buffer pool sizing: Ensure each buffer pool does not exceed 8 GB, depending on the line-card hardware.
- Profile configuration order: Globally configure the buffer-extended hybrid mode profile before applying PFC or QoS policies.
- HBM pool division: Specify the HBM percentage to divide the pool between lossless and lossy traffic, as required by your application.
- Recommended lossless percentage for special traffic: Allocate a larger percentage (such as 60 to 70 percent) to the lossless pool when handling Remote Direct Memory Access (RDMA), storage, or latency-sensitive traffic.
- Lossy queue cap: Adjust **max-non-pfc-voqs** to limit the number of lossy queues sharing HBM resources.
- Router restart requirement: Restart the router after enabling buffer-extended hybrid mode for the configuration to take effect.

## Configure buffer-extended hybrid mode with default settings

Use this procedure to configure buffer-extended hybrid mode with default settings.

### Procedure

**Step 1** Enter the configuration mode.

**Example:**

```
Router# configure
```

**Step 2** Enter the configuration mode for PFC profiling on an interface.

**Example:**

```
Router(config)# hw-module profile priority-flow-control location 0/0/CPU0
```

- Step 3** Enable buffer-extended PFC for traffic classes 3 and 4, indicating that these traffic classes are lossless and use dedicated buffer resources.

**Example:**

```
Router(config-pfc-profile)# buffer-extended traffic-class 3
Router(config-pfc-profile)# buffer-extended traffic-class 4
```

- Step 4** Configure the non-PFC or lossy traffic classes to use the percentage of the available HBM buffers. In this example, the HBM percentage specified is 60.

**Example:**

```
Router(config-pfc-profile)# buffer-extended non-pfc-tcs hbm-buffers percentage 60
Router(config-pfc-profile)# !
Router(config-pfc-profile)# end
```

- Step 5** Save the configuration.

**Example:**

```
Router(config)# commit
```

- Step 6** Verify the configuration.

**Example:**

```
Router#show controllers npu priority-flow-control location 0/0/CPU0
Fri Mar 14 17:38:43.948 UTC
Location:                0/0/CPU0
PFC:                      Enabled
PFC Mode:                 buffer-extended
PFC max evict Lossy:      24(Default)
TC      Pause-threshold   Headroom
-----
3      62914560 bytes      76800000 bytes
4      62914560 bytes      76800000 bytes
-----
TC      HbmPoolNum        TcGroup
-----
0              1              1
1              1              2
2              1              3
3              0              0
4              0              0
5              1              1
6              1              2
7              1              3
-----
Total hbm buffers          :    984576
Lossy Max hbm buffers per  :    60 (590745 buffers)
Lossless Max hbm buffers per:   40 (393830 buffers)
-----
```

## Configure buffer-extended hybrid mode with user-specified settings

Use this procedure to configure buffer-extended hybrid mode with user-specified settings.

## Procedure

**Step 1** Enter the configuration mode.

**Example:**

```
Router# configure
```

**Step 2** Enter the configuration mode for PFC profiling on an interface.

**Example:**

```
Router(config)# hw-module profile priority-flow-control location 0/0/CPU0
```

**Step 3** Enable buffer-extended PFC for traffic class 3 and class 4, indicating that this traffic class is lossless and uses dedicated buffer resources.

**Example:**

```
Router(config-pfc-profile)# buffer-extended traffic-class 3
```

```
Router(config-pfc-profile)# buffer-extended traffic-class 4
```

**Step 4** Configure the non-PFC or lossy traffic classes. It specifies that a maximum of 32 non-PFC VOQs can use 60% of the available HBM buffers.

**Example:**

```
Router(config-pfc-profile)# buffer-extended non-pfc-tcs max-non-pfc-voqs 32 hbm-buffers-percentage 60
```

```
Router(config-pfc-profile)# !
```

```
Router(config-pfc-profile)# end
```

**Step 5** Save the configuration.

**Example:**

```
Router(config)# commit
```

**Step 6** Verify the configuration.

**Example:**

```
Router#show running-config | include "hw-module|buffer"
```

```
Fri Aug 29 15:31:50.548 UTC
```

```
hw-module profile priority-flow-control location 0/0/CPU0
```

```
buffer-extended traffic-class 3
```

```
buffer-extended traffic-class 4
```

```
buffer-extended non-pfc-tcs max-non-pfc-voqs 32 hbm-buffers-percentage 60
```

```
Router#
```

```
Router#show controllers npu priority-flow-control location 0/0/CPU0
```

```
Fri Aug 29 15:31:55.165 UTC
```

```
Location: 0/0/CPU0
```

```
PFC: Enabled
```

```
PFC Mode: buffer-extended
```

```
PFC max evict Lossy: 32
```

```
TC Pause-threshold Headroom
```

```
-----
3 62914560 bytes 76800000 bytes
4 62914560 bytes 76800000 bytes
-----
```

```
TC HbmPoolNum TcGroup
```

```

-----
0          1          1
1          1          2
2          1          3
3          0          0
4          0          0
5          1          1
6          1          2
7          1          3
-----
Total hbm buffers      :    984576
Lossy Max hbm buffers per :    60 (590745 buffers)
Lossless Max hbm buffers per:   40 (393830 buffers)
-----
Router#

```

## Configure buffer-extended hybrid mode

You can configure buffer-extended hybrid mode by using these configuration settings:

| If..                                                                                                    | In..                                               | Then..                                                                                            | Example                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| You configure <b>hbm-buffers-percentage</b>                                                             | the default <b>max-non-pfc-voqs</b> setting        | the router uses its pre-defined, default number of non-PFC VOQs.                                  | You can assign X percent of HBM to regular traffic. The router then automatically decides how many of your regular traffic queues can share that X percent of HBM. In this setup, the router still works to separate lossless traffic and lossy traffic. |
| You specify a particular value for <b>max-non-pfc-voqs</b> along with the <b>hbm-buffers-percentage</b> | the user-specified <b>max-non-pfc-voqs</b> setting | you can explicitly limit the number of non-PFC VOQs that can use the specified percentage of HBM. | You can assign Y number of the regular traffic queues to use X percent of HBM. This gives you more direct control over how many lossy queues can access the allocated HBM space.                                                                         |



## CHAPTER 11

# Priority Flow Control Watchdog and Monitoring Metrics

- [Stages of Priority Flow Control Watchdog and monitoring, on page 171](#)
- [Priority Flow Control watchdog, on page 172](#)
- [Global statistics counters for Priority Flow Control and Priority Flow Control watchdog, on page 176](#)
- [Traffic class queue pause duration, on page 179](#)

## Stages of Priority Flow Control Watchdog and monitoring

After configuring Priority Flow Control (PFC), use this reference to understand the key stages of monitoring and optimizing PFC. These include watchdog operation, pause-duration analysis, and global-counter review.

**Table 52: Stages of Priority Flow Control Watchdog and Monitoring**

| Stage                                   | Description                                                                                                                                                     | See Section                                                                                                          |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Understand watchdog monitoring          | Learn how the PFC watchdog monitors queues, detects stuck conditions, and prevents pause storms by transitioning through monitor, shutdown, and restore states. | <a href="#">Priority Flow Control watchdog, on page 172</a>                                                          |
| Configure and verify watchdog operation | Enable or disable the watchdog globally or per interface, adjust interval multipliers, and verify its operation.                                                | <a href="#">Configure a Priority Flow Control watchdog interval, on page 173</a>                                     |
| Review global counters                  | Display aggregated PFC and watchdog counters to assess network-wide pause and congestion activity.                                                              | <a href="#">Global statistics counters for Priority Flow Control and Priority Flow Control watchdog, on page 176</a> |

| Stage                        | Description                                                                                                                    | See Section                                                     |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Analyze queue pause duration | View transmit (Tx) and receive (Rx) pause percentages and durations to identify whether congestion is temporary or persistent. | <a href="#">Traffic class queue pause duration, on page 179</a> |

## Priority Flow Control watchdog

The Priority Flow Control (PFC) watchdog is a PFC fault-detection and recovery mechanism that

- identifies queue-stuck conditions (PFC storms) caused by sustained pause frames
- prevents pause-frame propagation and looping across the network, and
- restores normal traffic flow automatically after congestion clears.

### PFC watchdog parameters and their role in queue monitoring

The PFC watchdog reduces persistent pause conditions by enforcing time-based monitoring of queues and managing state transitions:

- Watchdog interval: The polling interval (in milliseconds) to check the status of PFC queues to determine if they are stalled due to excessive PFC pause frames.
- Shutdown multiplier: The multiple of the watchdog interval after which the queue is shut down if congestion persists.
- Auto-restore multiplier: Wait period, in multiples of the watchdog interval, before the watchdog restores the queue after congestion clears.

## How the Priority Flow Control watchdog works

The Priority Flow Control (PFC) watchdog monitors queues receiving pause frames. It enforces queue transitions between monitored, waiting, shutdown, and restored states to maintain network stability.

### Summary

The key components involved in the PFC watchdog process are:

- Watchdog module: detects and reacts to prolonged pause frames.
- Line card hardware: monitors queue drain states and notifies the Watchdog module.
- System timers: govern queue shutdown, monitoring, and restoration intervals.

### Workflow

These stages describe how the PFC Watchdog process works.



1. Monitoring of pause activity: the watchdog module observes PFC-enabled queues for excessive pause frames during the configured watchdog interval.
2. Detection of sustained congestion: when hardware detects that packets are not draining due to persistent pause frames, it notifies the watchdog module.
3. Queue transition to wait-to-shutdown: The watchdog starts the shutdown timer. If the queue remains blocked, the module transitions it to a wait-to-shutdown state.
4. Queue shutdown and packet drop: If the queue doesn't drain before the shutdown timer expires, the watchdog changes its state to "drop." All outgoing packets on that queue are dropped, halting pause propagation.
5. Queue monitoring during shutdown: the watchdog continues to poll the queue. If pause frames persist, the queue remains in the drop state.
6. Auto-restore evaluation: when pause frames stop, the auto-restore timer starts. If pause frames reappear during the timer interval, it resets.

If no pause frames are detected, the queue transitions to the "restored" state and resumes traffic.

## Best practices for Priority Flow Control watchdog configuration

Use these best practices to configure and operate the Priority Flow Control watchdog effectively.

- Configure PFC before enabling the watchdog: Configure Priority Flow Control (PFC) and its associated policies before enabling the PFC Watchdog. This approach ensures that pause frame behavior and queue management policies are already defined before the watchdog begins monitoring.
- Understand global watchdog behavior: When the global watchdog mode is disabled, it overrides all interface-level configurations. This means that even if the watchdog is configured on individual interfaces, it remains inactive until the global mode is enabled.
- Leverage interface-level overrides: When the global watchdog mode is enabled, interface-level attributes such as interval, shutdown multiplier, and auto-restore multiplier override global settings. This flexibility allows finer control for high-priority interfaces with unique traffic profiles.
- Tune watchdog parameters per interface: Use appropriate values for interval, shutdown multiplier, and auto-restore multiplier for each interface. This configuration helps avoid premature shutdown or delayed recovery of queues. Incorrectly set timers can result in unnecessary packet drops or slow queue restoration.

## Configure a Priority Flow Control watchdog interval

Use this task to configure and verify Priority Flow Control (PFC) watchdog parameters at either the global or interface level. The watchdog detects queue-stuck conditions caused by sustained PFC pause frames and automatically restores normal queue operation after congestion clears.

The PFC watchdog monitors queues for pause storms and takes corrective actions when queues fail to drain within a specified interval. You can configure watchdog timers globally for all interfaces. Alternatively, override timers individually for specific interfaces.

### Before you begin

Ensure that PFC and its policies are configured and active.

Follow these steps to configure and verify the PFC watchdog interval.

## Procedure

**Step 1** Enable the watchdog globally.

### Example:

```
Router(config)#priority-flow-control watchdog mode on
```

This step activates the watchdog feature for all PFC-enabled interfaces.

**Step 2** Configure global watchdog parameters.

### Example:

```
Router(config)#priority-flow-control watchdog interval 100
Router(config)#priority-flow-control watchdog shutdown-multiplier 2
Router(config)#priority-flow-control watchdog auto-restore-multiplier 10
Router(config)#commit
```

This step sets watchdog timing behavior:

- checks queues every 100 milliseconds
- declares queues stuck after 200 milliseconds (100 milliseconds x 2), and
- restores queues after 1000 milliseconds (100 x 10)

**Step 3** (Optional) Configure interface-level overrides.

### Example:

```
Router(config)#interface HundredGigE0/2/0/0
Router(config-if)#priority-flow-control watchdog mode on
Router(config-if)#commit
```

This step enables the Watchdog for a specific interface. Interface-level settings override global values if defined.

**Step 4** Verify watchdog configuration.

- Verify global watchdog configuration.

```
Router#show run priority-flow-control watchdog
priority-flow-control watchdog interval 100
priority-flow-control watchdog shutdown-multiplier 2
priority-flow-control watchdog auto-restore-multiplier 10
priority-flow-control watchdog mode on
```

- Verify active watchdog configuration on an interface.

```
Router#show controllers HundredGigE0/2/0/0 priority-flow-control

Priority flow control information for interface HundredGigE0/2/0/0:

Priority flow control watchdog configuration:
(D) : Default value
U : Unconfigured
-----
Configuration Item          Global  Interface  Effective
-----
PFC watchdog state :       Enabled   U          Enabled
```

```

Poll interval :          100 (D)   U          100 (D)
Shutdown multiplier :    2 (D)     U          2 (D)
Auto-restore multiplier : 10 (D)    U          10 (D)

```

- **Global** column shows configured values.
- **Interface** column shows interface-specific overrides (if any).
- **Effective** column shows which values are actively applied to the interface.
- Verify watchdog operational statistics.

```

Router#show controllers HundredGigE0/2/0/0 priority-flow-control watchdog-stats
Priority flow control information for interface HundredGigE0/2/0/0:

```

```

Priority flow control watchdog statistics:
SAR: Auto restore and shutdown

```

| Traffic Class            | : | 0 | 1 | 2 | 3         | 4         | 5 | 6 |
|--------------------------|---|---|---|---|-----------|-----------|---|---|
| 7                        |   |   |   |   |           |           |   |   |
| Watchdog Events          | : | 0 | 0 | 0 | 3         | 3         | 0 | 0 |
| 0                        |   |   |   |   |           |           |   |   |
| Shutdown Events          | : | 0 | 0 | 0 | 3         | 3         | 0 | 0 |
| 0                        |   |   |   |   |           |           |   |   |
| Auto Restore Events      | : | 0 | 0 | 0 | 3         | 3         | 0 | 0 |
| 0                        |   |   |   |   |           |           |   |   |
| SAR Events               | : | 0 | 0 | 0 | 3510      | 3510      | 0 | 0 |
| 0                        |   |   |   |   |           |           |   |   |
| SAR Instantaneous Events | : | 0 | 0 | 0 | 1172      | 1172      | 0 | 0 |
| 0                        |   |   |   |   |           |           |   |   |
| Total Dropped Packets    | : | 0 | 0 | 0 | 941505767 | 941488166 | 0 | 0 |
| 0                        |   |   |   |   |           |           |   |   |
| Dropped Packets          | : | 0 | 0 | 0 | 314855466 | 314887161 | 0 | 0 |
| 0                        |   |   |   |   |           |           |   |   |

- **Watchdog Events** displays the number of times the watchdog detected a stalled queue.
- **Shutdown Events** displays the number of times the line card was shut down due to stuck queues.
- **Auto Restore Events** displays the number of times the watchdog restored the queue after congestion cleared.
- **Total Dropped Packets** displays the cumulative virtual output queue (VOQ) and output queue drops across all watchdog events since the last clear.
- **Dropped Packets** displays the cumulative VOQ and output queue drops in the most recent watchdog shutdown event.
- Ignore the **SAR Events** and **SAR Instantaneous Events** entries. These numbers are not relevant to your operations.

# Global statistics counters for Priority Flow Control and Priority Flow Control watchdog

A global statistics counter for Priority Flow Control (PFC) and PFC watchdog is an observability mechanism that

- consolidates per-interface counters into a unified, system-wide view
- collects statistics from all line cards and aggregates them in the local statistics infrastructure, and
- provides fast, tabular summaries of PFC and PFC watchdog behavior across all interfaces.

This feature enables you to quickly monitor overall PFC and watchdog activity without running per-interface commands. It improves troubleshooting speed and provides a holistic view of congestion control across the chassis.

**Table 53: Feature History Table**

| Feature Name                                                                            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Global Statistics Counters for Priority Flow Control and Priority Flow Control Watchdog | Release 25.1.1      | Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*)<br><br>This feature is supported on: <ul style="list-style-type: none"> <li>• 8712-MOD-M</li> <li>• 8011-4G24Y4H-I</li> </ul>                                                                                                                                              |
| Global Statistics Counters for Priority Flow Control and Priority Flow Control Watchdog | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul> |

| Feature Name                                                                            | Release Information              | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Global Statistics Counters for Priority Flow Control and Priority Flow Control Watchdog | Release 7.5.5<br>Release 24.2.11 | <p>You can now view statistics for Priority Flow Control (PFC) and PFC Watchdog for all interfaces in a consolidated, compact, tabular, and easy-to-read format.</p> <p>We've also made the display of these global statistics faster by ensuring data is collected from all line cards for their interfaces and cumulatively sent to the local statistics infrastructure from where the show commands collect the data.</p> <p>Previously, you could view statistics only per interface for PFC and PFC Watchdog, wherein the show commands get the data from each interface.</p> <p>This feature modifies the following command:</p> <ul style="list-style-type: none"> <li>• <a href="#">show controllers</a></li> </ul> |

## View global statistics for Priority Flow Control and Priority Flow Control watchdog

Use this task to view the consolidated statistics for all interfaces on the router, enabling faster validation of Priority Flow Control (PFC) or PFC watchdog behavior.

### Procedure

**Step 1** View global PFC statistics.

#### Example:

```
Router#show controllers all priority-flow-control statistics location all
Interface          TC    RxPFC  TxPFC  RxDropped
FourHundredGigE0/0/0/0  0      0       0       NA
FourHundredGigE0/0/0/0  7      0       0       NA
FourHundredGigE0/0/0/0 all  0      0       0       0
```

This example displays traffic-class-based PFC statistics for all interfaces where:

- **Rx PFC** displays the number of received PFC frames
- **Tx PFC** displays the number of transmitted PFC frames
- **Rx Dropped** displays the number of PFC frames dropped. **0** indicates that no frames were dropped, while **NA** indicates the information is not available because the line card has not sent the data.

**Step 2** View global PFC watchdog statistics.

#### Example:

```
Router#show controllers all priority-flow-control watchdog-stats location all
```

| Interface              | TC     | Watchdog | Shutdown | Auto Restore | SAR    | SAR Instantaneous | Total Dropped |
|------------------------|--------|----------|----------|--------------|--------|-------------------|---------------|
| Dropped                | Events | Events   | Events   | Events       | Events | Events            |               |
| FourHundredGigE0/0/0/0 | 0      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/0 | 7      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/1 | 0      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/1 | 7      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/2 | 0      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/2 | 7      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/3 | 0      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/3 | 7      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |
| FourHundredGigE0/0/0/4 | 0      | 0        | 0        | 0            | 0      | 0                 | 0             |
| 0                      |        |          |          |              |        |                   |               |

This example displays statistics across all interfaces for watchdog activity for every traffic class, where:

- **Watchdog Events** indicates how many times the watchdog module receives notifications that excess PFC frames have been received
- **Shutdown events** displays the number of times the PFC watchdog moves the queue to the shutdown state
- **Auto restore Events** displays the number of times the PFC watchdog sets the auto-restore timer
- **Total Dropped** displays the cumulative virtual output queue (VOQ) drops and the output drops in all the watchdog events from the time you ran the **clear controllers priority-flow-control watchdog statistics** command to clear the statistics counter
- **Dropped** indicates the cumulative VOQ drops and the output queue drops in the most recent watchdog shutdown event.

Disregard the **SAR Events** and **SAR Instantaneous Events** counters because they have no operational impact.

You can verify that PFC frames, pause activity, and watchdog operations are visible across all interfaces. This visibility helps confirm overall congestion health.

## Supported commands for global statistics for Priority Flow Control and Priority Flow Control watchdog

Use these commands to display or clear global statistics for Priority Flow Control (PFC) and PFC watchdog.

| Operation                  | Command                                                                   | Description                                                                                                                |
|----------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| View global PFC statistics | <b>show controllers all priority-flow-control statistics location all</b> | Displays consolidated PFC statistics, including pause frame counts and flow control events across all interfaces and NPUs. |

| Operation                           | Command                                                                       | Description                                                                                                                                                                                       |
|-------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| View global PFC watchdog statistics | <b>show controllers all priority-flow-control watchdog-stats location all</b> | Displays cumulative PFC watchdog statistics, including watchdog events, shutdowns, and auto-restore counts for each traffic class.                                                                |
| Clear PFC statistics                | <b>clear controller priority-flow-control statistics</b>                      | Clears all PFC global counters and resets the corresponding per-interface counters that the <b>show controllers all priority-flow-control statistics location all</b> command displays.           |
| Clear PFC watchdog statistics       | <b>clear controller priority-flow-control watchdog-stats</b>                  | Clears all PFC watchdog global counters and resets the per-interface watchdog statistics that the <b>show controllers all priority-flow-control watchdog-stats location all</b> command displays. |

After running a clear command, all counter values reset to zero and begin incrementing again as new pause or watchdog events occur.

## Traffic class queue pause duration

A traffic class queue pause duration is a Priority Flow Control (PFC) congestion monitoring functionality that

- reports how long per-traffic-class queues remain paused, presented as the percent of time and the absolute duration (in microseconds), for both transmitting (Tx) and receiving (Rx) paths
- correlates pause activity across adjacent devices (Tx on the receiver versus Rx on the transmitter) to distinguish short-lived bursts from sustained congestion, and
- supports multi-window sampling, such as one minute and five minutes, and offers per-interface granularity to facilitate troubleshooting and buffer PFC threshold tuning.

Table 54: Feature History Table

| Feature Name                            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enhanced PFC pause duration monitoring  | Release 25.2.1      | <p>Introduced in this release on: Modular Systems (8800 [LC ASIC: Q200])</p> <p>This feature improves network congestion analysis by providing granular visibility into Priority Flow Control (PFC) pause durations.</p> <p>It introduces the ability to monitor both transmit (Tx) and receive (Rx) pause durations in microseconds, allowing for precise identification of congestion patterns.</p> <p>Previously, only pause percentages were available, but this enhancement provides a more detailed and accurate understanding of PFC pause events.</p> <p>The feature introduces these changes:</p> <p>CLI:</p> <ul style="list-style-type: none"> <li>• <a href="#">show controllers npu packet-memory interface [interface] tx-pause-duration location</a></li> <li>• <a href="#">show controllers npu packet-memory interface [interface] rx-pause-duration location</a></li> </ul> <p>YANG data models</p> <ul style="list-style-type: none"> <li>• <a href="#">Cisco-IOS-XR-platforms-ofa-oper</a> (see <a href="#">GitHub</a>, <a href="#">YANG Data Models Navigator</a>)</li> </ul> |
| View Traffic Class Queue Pause Duration | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8010 [ASIC: A100], 8700 [ASIC:K100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8011-4G24Y4H-I</li> <li>• 8712-MOD-M</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| View Traffic Class Queue Pause Duration | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8711-32FH-M</li> <li>• 8212-48FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-12TH24FH-E</li> <li>• 88-LC1-52Y8H-EM</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |



| Feature Name                            | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| View Traffic Class Queue Pause Duration | Release 24.2.11     | <p>Introduced in this release on Cisco 8000 Series Routers with Cisco Silicon One Q200 network processors that support the PFC buffer-extended mode function.</p> <p>For traffic flows between routers, you can view the pause duration of output and input queues in the transmitting and receiving routers, respectively.</p> <p>The pause duration values of the impacted traffic class queues are displayed for regular intervals within a specified time duration.</p> <p>With the information, you can view the extent of congestion on PFC-enabled interfaces over a period of time and identify whether traffic congestion is due to small bursts of traffic or other causes.</p> <p>The feature introduces these changes:</p> <p>CLI:</p> <ul style="list-style-type: none"> <li>• <a href="#">show controllers npu packet-memory interface</a></li> </ul> <p>YANG data models:</p> <ul style="list-style-type: none"> <li>• <a href="#">Cisco-IOS-XR-platforms-oft-ops</a> (see <a href="#">GitHub</a>, YANG Data Models Navigator)</li> </ul> |

## How traffic class queue pause duration works

In PFC-enabled networks, routers exchange PAUSE and NO-PAUSE (X-on) frames to manage congestion without packet drops. When queue occupancy crosses the configured pause-threshold, the receiver issues PAUSE frames upstream. When congestion clears, it sends X-on frames.

The router records how long each queue remains paused during each sampling window and reports this as a pause percentage or pause duration.

### Summary

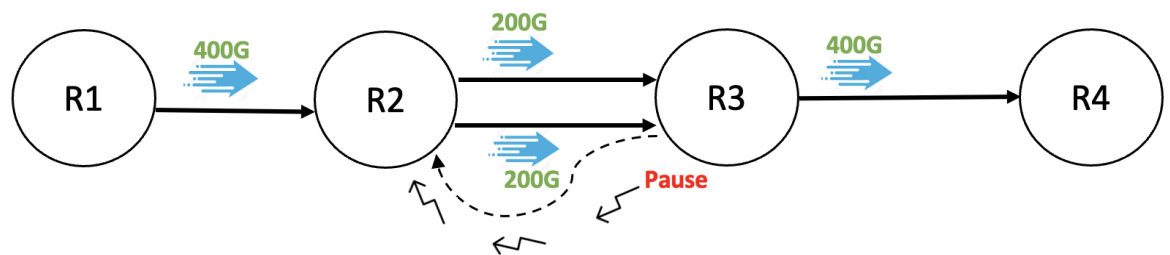
This summary refers to the figure to clarify the workflow.

- Transmitting router (R2): Sends traffic and pauses output queues when PAUSE frames are received.
- Receiving router (R3): Detects congestion, issues PAUSE or X-on frames, and measures pause duration.
- Upstream router (R1): Feeds traffic into R2 and can receive propagated PAUSE frames.
- Downstream router (R4): Receives traffic forwarded from R3.
- 

These components work together to regulate traffic flow and record how long queues remain paused.

## Workflow

Figure 9: Traffic class queue pause duration scenario



These stages describe how traffic class queue pause duration works.

1. R1 transmits: R1 sends packets to R2 at a line rate of 400 Gbps.
2. R2 transmits normally: R2 forwards 400 G of traffic toward R3 across two 200 G PFC-enabled ECMP links
3. R3 queues begin to fill: R3 forwards 399 G to R4 and buffers the remaining 1 G in input queues. Mild congestion starts but remains below the pause-threshold.
4. R3 crosses the pause-threshold: R3 detects the threshold breach and sends PFC PAUSE frames to R2. R2 halts transmission and buffers packets in High Bandwidth Memory (HBM).
5. Pause propagates upstream: R2 sends PAUSE frames further upstream to R1. Traffic slows from R1 to R2 to R3 and queues stabilize.
6. R3 drains queues and resumes: As congestion clears, R3 sends X-on frames to R2. Transmission resumes at line rate.
7. R2 releases backlog: R2 transmits its buffered packets at full 400 G, causing a brief traffic burst. R3's queues refill temporarily, triggering another PAUSE.
8. Pause cycles repeat: R2 and R3 alternate between PAUSE and X-on states. For example, both routers record approximately 50 percent pause-percentage, which indicates short, periodic congestion.
9. Administrator monitors pause duration: Run `show controllers npu packet-memory interface <interface>` to display pause statistics for each traffic class and interval. High pause percentages (greater than or equal to 70 percent) suggest sustained congestion; lower values indicate normal operation.

## Result

This process quantifies queue pause behavior in terms of time percentages or durations. This helps you distinguish transient bursts from persistent congestion and tune thresholds.

# Best practices for using traffic class queue pause duration

Use these guidelines to interpret and validate traffic class queue pause-duration data accurately.

- Select sampling intervals appropriately:

Select a short observation window (30 seconds to 1 minute) to detect micro-bursts or transient queue spikes.

Select a longer observation window (5 minutes) to capture congestion trends and recurring pause patterns over time.

- Avoid over-sampling in **detail** option: Avoid using the **detail** option in automation scripts or telemetry integrations, as it generates a large volume of records
- Verify Priority Flow Control (PFC) configuration: Ensure that PFC buffer-extended mode is enabled on the transmitting and receiving router nodes.
- PFC mode support for viewing pause duration in microseconds: Use buffer-internal or buffer-extended mode to view pause duration in microseconds.

## View traffic class queue pause duration

Use the **show controllers npu packet-memory interface** command to view pause statistics per interface and traffic class to determine how long queues were paused, as a percentage or in microseconds, for selected intervals.

### Before you begin

Enable PFC buffer-extended mode on the transmitting and receiving router nodes.

Follow these steps to view queue pause duration on a PFC-enabled router.

### Procedure

- 
- Step 1** Select Tx or Rx view.
- **rx-pause-percent**—Specifies the input queues on the receiving router.
  - **tx-pause-percent**—Specifies the output queues on the transmitting router.
- Step 2** (Optional) Choose observation window or detail level.
- **one-minute**—Specifies the average pause duration of the queues in percentage for the last minute.
  - **five-minute**—Specifies the average pause duration of the queues in percentage for the last five minutes.
  - **detail**—Displays a maximum of 120 records at a frequency of one record per 250-millisecond interval.
  - **Default**—30 records are displayed for each traffic class at a frequency of one record per second.
  - **verbose**—View the time stamp in a raw mode.
- Step 3** Run the show command.
- ```
Router#show controllers npu packet-memory interface <intf> tx-pause-percent <observation window in
time or detail> location <node>
```
- OR

## View traffic class queue pause duration

```
Router#show controllers npu packet-memory interface <intf> rx-pause-percent <observation window in
time or detail> location <node>
```

- **one-minute**—Specifies the average pause duration of the queues in percentage for the last minute.
- **five-minute**—Specifies the average pause duration of the queues in percentage for the last five minutes.
- **detail**—Displays a maximum of 120 records at a frequency of one record per 250-millisecond interval.
- **Default**—30 records are displayed for each traffic class at a frequency of one record per second.
- **verbose**—View the time stamp in a raw mode.
- View the average pause duration for input queues for a specific observation window.

```
Router#show controllers npu packet-memory interface all tx-pause-percent one-minute location
0/6/CPU0
```

```
-----
Node ID: 0/6/CPU0
Source Queue Pause Percentage Info for interface(s) all
Intf          TC          Pause-Percentage
name
-----
FH0/6/0/10    2          0.00000
FH0/6/0/11    2          0.00000
FH0/6/0/13    2          0.00000
FH0/6/0/14    2          0.00000
FH0/6/0/15    2          0.00000
FH0/6/0/16    2          0.00000
FH0/6/0/18    2          0.00000
FH0/6/0/21    2          53.01604
FH0/6/0/22    2          0.00000
FH0/6/0/23    2          53.13991
```

- View the average pause duration of output queues for a specific observation window.

```
show controllers npu packet-memory interface all rx-pause-percent one-minute location 0/6/CPU0
```

```
-----
Node ID: 0/6/CPU0
Out Queue Pause Percentage Info for interface(s) all
Intf          TC          Pause-Percentage
name
-----
FH0/6/0/10    2          0.00000
FH0/6/0/11    2          0.00000
FH0/6/0/13    2          0.00000
FH0/6/0/14    2          0.00000
FH0/6/0/15    2          0.00000
FH0/6/0/16    2          0.00000
FH0/6/0/18    2          0.00000
FH0/6/0/21    2          53.01604
FH0/6/0/22    2          0.00000
FH0/6/0/23    2          53.13991
```

- View pause duration for Tx pause frames in microseconds.

```
Router#show controllers npu packet-memory interface all tx-pause-duration location 0/2/CPU0
```

```
-----
Node ID: 0/2/CPU0
Out Queue Pause Duration Info for interface(s) all

Intf          TC          Pause-Duration
name
```

```

-----
FH0/2/0/0      3      0
FH0/2/0/0      4      0
FH0/2/0/35     3      0
FH0/2/0/35     4      0
FH0/2/0/34     3      0
FH0/2/0/34     4      0
FH0/2/0/33     3      0
FH0/2/0/33     4      0
FH0/2/0/31     3      33282645110
FH0/2/0/31     4      31821853078
FH0/2/0/30     3      33304272074
FH0/2/0/30     4      31835301237
FH0/2/0/29     3      0
FH0/2/0/29     4      0
FH0/2/0/28     3      33227075106
FH0/2/0/28     4      31802440395
FH0/2/0/27     3      33241333365
FH0/2/0/27     4      31740514268
FH0/2/0/26     3      33256748403
FH0/2/0/26     4      32711889077
FH0/2/0/25     3      33344738029
FH0/2/0/25     4      31827152759
FH0/2/0/24     3      33407963260
FH0/2/0/24     4      32542402762
FH0/2/0/1      3      0
FH0/2/0/1      4      0
FH0/2/0/2      3      0
FH0/2/0/2      4      0
FH0/2/0/3      3      0
FH0/2/0/3      4      0
FH0/2/0/4      3      0
FH0/2/0/4      4      0
FH0/2/0/5      3      0
FH0/2/0/5      4      0
FH0/2/0/6      3      0
FH0/2/0/6      4      0
FH0/2/0/7      3      0
FH0/2/0/7      4      0
FH0/2/0/8      3      0
FH0/2/0/8      4      0
FH0/2/0/9      3      0
FH0/2/0/9      4      0
FH0/2/0/10     3      0
FH0/2/0/10     4      0
FH0/2/0/11     3      0
FH0/2/0/11     4      0

```

- View pause duration for Rx pause frames in microseconds.

```

Router#show controllers npu packet-memory interface all rx-pause-duration location 0/2/CPU0
Fri Feb  7 05:02:28.551 UTC

```

```

-----
Node ID: 0/2/CPU0
Source Queue Pause Duration Info for interface(s) all

```

```

Intf          TC          Pause-Duration
name
-----
FH0/2/0/0      3          66113289
FH0/2/0/0      4          20373867793
FH0/2/0/35     3          0
FH0/2/0/35     4          0
FH0/2/0/34     3          0

```

## View traffic class queue pause duration

|            |   |             |
|------------|---|-------------|
| FH0/2/0/34 | 4 | 0           |
| FH0/2/0/33 | 3 | 0           |
| FH0/2/0/33 | 4 | 0           |
| FH0/2/0/31 | 3 | 0           |
| FH0/2/0/31 | 4 | 0           |
| FH0/2/0/30 | 3 | 0           |
| FH0/2/0/30 | 4 | 0           |
| FH0/2/0/29 | 3 | 0           |
| FH0/2/0/29 | 4 | 0           |
| FH0/2/0/28 | 3 | 0           |
| FH0/2/0/28 | 4 | 0           |
| FH0/2/0/27 | 3 | 0           |
| FH0/2/0/27 | 4 | 0           |
| FH0/2/0/26 | 3 | 0           |
| FH0/2/0/26 | 4 | 0           |
| FH0/2/0/25 | 3 | 0           |
| FH0/2/0/25 | 4 | 0           |
| FH0/2/0/24 | 3 | 0           |
| FH0/2/0/24 | 4 | 0           |
| FH0/2/0/1  | 3 | 34701848    |
| FH0/2/0/1  | 4 | 19824055492 |
| FH0/2/0/2  | 3 | 5594691     |
| FH0/2/0/2  | 4 | 19013361290 |
| FH0/2/0/3  | 3 | 19015180    |
| FH0/2/0/3  | 4 | 19924428067 |
| FH0/2/0/4  | 3 | 27727815    |
| FH0/2/0/4  | 4 | 19841945429 |
| FH0/2/0/5  | 3 | 0           |
| FH0/2/0/5  | 4 | 0           |
| FH0/2/0/6  | 3 | 29833476    |
| FH0/2/0/6  | 4 | 19835847699 |
| FH0/2/0/7  | 3 | 29578038    |
| FH0/2/0/7  | 4 | 19837595706 |
| FH0/2/0/8  | 3 | 0           |
| FH0/2/0/8  | 4 | 0           |
| FH0/2/0/9  | 3 | 0           |
| FH0/2/0/9  | 4 | 0           |
| FH0/2/0/10 | 3 | 0           |
| FH0/2/0/10 | 4 | 0           |
| FH0/2/0/11 | 3 | 0           |
| FH0/2/0/11 | 4 | 0           |

---



## CHAPTER 12

# QoS Deployment across Network Scenarios

- [Information flow, on page 187](#)
- [QoS support for VPWS traffic, on page 188](#)
- [QoS Policy Propagation via BGP, on page 193](#)
- [QoS on Pseudowire Headend, on page 200](#)
- [Bridged Virtual Interface two-pass QoS, on page 235](#)

## Information flow

This content explains how QoS can be deployed in various network scenarios. It focuses on how classification and queuing decisions are influenced by traffic type and tunneling methods. The content also describes configurations that extend QoS capabilities beyond standard interfaces, such as Border Gateway Protocol (BGP) policy propagation, pseudowire headends and Bridged Virtual Interface (BVI) two-pass QoS.

**Table 55: Information flow**

| Deployment scenario                                         | What it covers                                                                                                                                                                                                                                                                                                                           | What it helps you achieve                                                                                                                                                                                           |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">QoS support for VPWS traffic</a>                | Describes how QoS classification and remarking are applied to Virtual Private Wire Service (VPWS) traffic on Layer 2 attachment circuits. Explains ingress classification based on IEEE 802.1p Priority Code Point (PCP) and Drop Eligible Indicator (DEI) fields in the VLAN header, and the use of QoS policies on VPWS traffic flows. | Enables consistent and differentiated service treatment for point-to-point Layer 2 services. Ensures predictable QoS behavior for VPWS traffic by preserving and acting on Layer 2 priority information at ingress. |
| <a href="#">QoS Policy Propagation via BGP, on page 193</a> | Describes how QoS classification and marking can be derived from BGP route attributes instead of ACLs. Explains the process of mapping prefixes to QoS groups and applying policies through routing table integration.                                                                                                                   | Enables scalable, destination-based QoS classification and consistent marking across networks where BGP is the control plane. Reduces manual ACL management and improves classification accuracy.                   |

| Deployment scenario                                                 | What it covers                                                                                                                                                                                                                                                                          | What it helps you achieve                                                                                                                                                                           |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">QoS on Pseudowire Headend, on page 200</a>              | Explains how QoS policies apply to traffic bridging between Layer 2 and Layer 3 domains through BVIs. Covers marking, policing, and shaping behavior for bridged and routed traffic.                                                                                                    | Ensures differentiated service treatment for traffic traversing bridge domains, maintaining consistent QoS handling between L2 and L3 segments.                                                     |
| <a href="#">Bridged Virtual Interface two-pass QoS, on page 235</a> | Describes how the router performs two independent QoS inspections: one at the Layer 2 bridge-domain ingress and a second at the Layer 3 BVI ingress. Covers supported classification and marking actions and how traffic transitions from bridging to routing using the two-pass model. | Provides consistent QoS treatment for traffic moving from Layer 2 to Layer 3, improves accuracy of classification and marking, and enables more flexible policy design for bridged-to-routed flows. |

## QoS support for VPWS traffic

QoS support for Virtual Private Wire Service (VPWS) is a QoS feature that:

- enables packet classification and remarking for VPWS traffic by using Layer 2 header fields
- supports ingress classification on Layer 2 attachment circuits (ACs)
- applies IEEE 802.1p Priority Code Point (PCP) and Drop Eligible Indicator (DEI) fields to differentiate traffic and allows QoS policies to be applied to VPWS traffic flows.

EVPN-VPWS is a BGP-based control-plane solution for point-to-point services that establishes an EVPN instance between a pair of provider edge (PE) routers and forwards traffic without requiring MAC address lookup.

Without QoS, VPWS traffic is handled using a best-effort service model, where packets are forwarded without guarantees for delay, throughput, or reliability.

**Table 56: Feature History Table**

| Feature Name         | Release Information | Feature Description                                                                                                                                                |
|----------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS Support for VPWS | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is now supported on the Cisco 8011-4G24Y4H-I routers. |



| Feature Name         | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS Support for VPWS | Release 24.4.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>*This feature is now supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 8712-MOD-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-52Y8H-EM</li> <li>• 88-LC1-12TH24FH-E</li> </ul> |
| QoS Support for VPWS | Release 7.7.1       | <p>Now, you can apply QoS packet classification on Layer 2 subinterfaces, and for VPWS traffic. This feature provides ingress classification support for L2 attachment circuit (AC) traffic based on the 802.1p field priority value.</p> <p>With this feature, you can use QoS policies on VPWS traffic in your network on Cisco 8000 Series routers.</p>                                                                   |

### VLAN tag–based classification for VPWS traffic

VLAN tag–based classification is a Layer 2 traffic classification mechanism that uses fields in the IEEE 802.1Q or 802.1ad VLAN header to differentiate traffic classes.

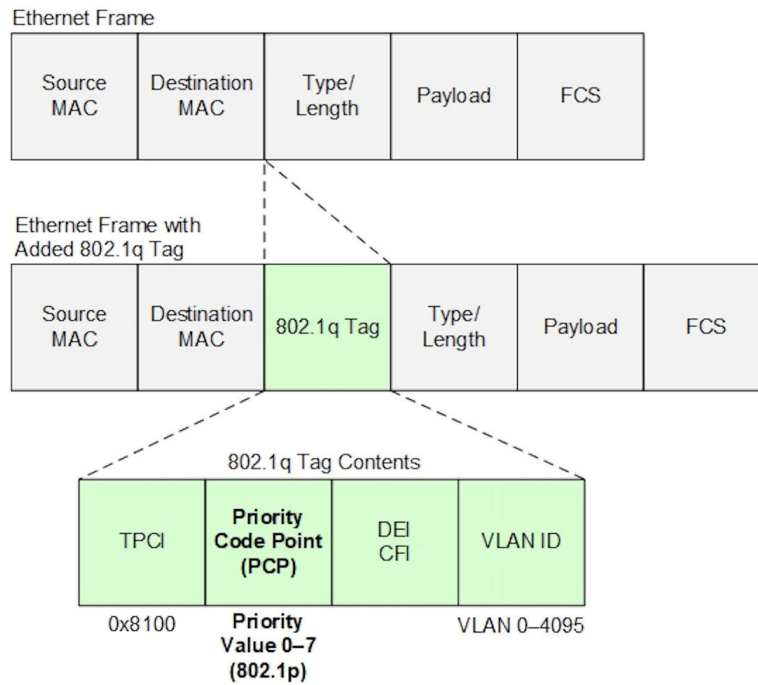
The VLAN tag includes:

- A 3-bit Priority Code Point (PCP) field that identifies traffic priority values from 0 to 7.
- A 1-bit Drop Eligible Indicator (DEI) field that marks frames as eligible for drop during congestion.

Modular QoS Configuration (MQC) supports class-map matching based on PCP and DEI values. This classification is supported on 802.1Q and 802.1ad interfaces and applies to:

- Attachment circuit (AC) to AC traffic flows
- Attachment circuit (AC) to pseudowire (PWE) traffic flows

Figure 10: IEEE 802.1Q VLAN Tag and 802.1p Priority Fields



## Configure ingress QoS classification for VPWS

Use this task to configure ingress QoS classification and remarking for Virtual Private Wire Service (VPWS) traffic based on VLAN PCP and DEI fields.

### Before you begin

VPWS must be configured on the router. See the *L2VPN Configuration Guide for Cisco 8000 Series Routers*.

Follow these steps to configure ingress QoS classification for VPWS traffic.

### Procedure

**Step 1** Define class maps that match PCP and DEI values in the VLAN header.

#### Example:

```
Router# configure terminal
Router(config)#class-map match-all CONTROL_PLANE
Router(config-cmap)#match cos 7
Router(config-cmap)#end-class-map

Router(config)#class-map match-all VOIP
Router(config-cmap)#match cos 6
Router(config-cmap)#end-class-map

Router(config)#class-map match-all VIDEO_STREAM
```

```

Router(config-cmap)#match cos 5
Router(config-cmap)#end-class-map

Router(config)#class-map match-all TRANSACTIONAL_DATA
Router(config-cmap)#match cos 4
Router(config-cmap)#end-class-map

Router(config)#class-map match-all DB_SYNC
Router(config-cmap)#match cos 3
Router(config-cmap)#match dei 1
Router(config-cmap)#end-class-map

Router(config)#class-map match-all BULK_DATA
Router(config-cmap)#match cos 2
Router(config-cmap)#match dei 1
Router(config-cmap)#end-class-map

Router(config)#class-map match-all SCAVENGER
Router(config-cmap)#match cos 1
Router(config-cmap)#match dei 1
Router(config-cmap)#end-class-map
Router(config)#commit

```

**Step 2** Associate the defined class maps with a policy map and set traffic-class values.

**Example:**

```

Router# configure terminal
Router(config)# policy-map INGRESS_L2_AC
Router(config-pmap)# class CONTROL_PLANE
Router(config-pmap-c)# set traffic-class 7
Router(config-pmap-c)# exit

Router(config-pmap)# class VOIP
Router(config-pmap-c)# set traffic-class 6
Router(config-pmap-c)# exit

Router(config-pmap)# class VIDEO_STREAM
Router(config-pmap-c)# set traffic-class 5
Router(config-pmap-c)# exit

Router(config-pmap)# class TRANSACTIONAL_DATA
Router(config-pmap-c)# set traffic-class 4
Router(config-pmap-c)# exit

Router(config-pmap)# class DB_SYNC
Router(config-pmap-c)# set traffic-class 3
Router(config-pmap-c)# exit

Router(config-pmap)# class BULK_DATA
Router(config-pmap-c)# set traffic-class 2
Router(config-pmap-c)# exit

Router(config-pmap)# class SCAVENGER
Router(config-pmap-c)# set traffic-class 1
Router(config-pmap-c)# exit

Router(config-pmap)# class class-default
Router(config-pmap-c)# exit
Router(config-pmap)# end-policy-map
Router(config)# commit

```

**Step 3** Configure ingress traffic remarking by setting CoS and DEI values for traffic classes as required.

**Example:**

```
Router# configure terminal
Router(config)# policy-map INGRESS_L2_AC
Router(config-pmap)# class CONTROL_PLANE
Router(config-pmap-c)# set traffic-class 7
Router(config-pmap-c)# exit

Router(config-pmap)# class VOIP
Router(config-pmap-c)# set traffic-class 6
Router(config-pmap-c)# set cos 7
Router(config-pmap-c)# exit

Router(config-pmap)# class VIDEO_STREAM
Router(config-pmap-c)# set traffic-class 5
Router(config-pmap-c)# set cos 5
Router(config-pmap-c)# exit

Router(config-pmap)# class TRANSACTIONAL_DATA
Router(config-pmap-c)# set traffic-class 4
Router(config-pmap-c)# set cos 5
Router(config-pmap-c)# exit

Router(config-pmap)# class DB_SYNC
Router(config-pmap-c)# set traffic-class 3
Router(config-pmap-c)# set dei 0
Router(config-pmap-c)# exit

Router(config-pmap)# class BULK_DATA
Router(config-pmap-c)# set traffic-class 2
Router(config-pmap-c)# set cos 3
Router(config-pmap-c)# set dei 0
Router(config-pmap-c)# exit

Router(config-pmap)# class SCAVENGER
Router(config-pmap-c)# set traffic-class 1
Router(config-pmap-c)# exit

Router(config-pmap)# class class-default
Router(config-pmap-c)# set dei 1
Router(config-pmap-c)# end-policy-map
Router(config)# commit
```

**Step 4** Attach the ingress service policy to the Layer 2 subinterface.

**Example:**

```
Router(config)# interface hundredGigE 0/11/0/31.102
Router(config-subif)# service-policy input INGRESS_L2_AC
Router(config-subif)# commit
```

**Step 5** Verify application of ingress QoS policies and correct classification of traffic.

**Example:**

```
Router# show policy-map interface Hu0/11/0/31.102 input

HundredGigE0/11/0/31.102 input: INGRESS_L2_AC

Class CONTROL_PLANE
```

```

Classification statistics      (packets/bytes)      (rate - kbps)
Matched      :      9813350/13738690000      936847
  Transmitted      :      9813350/13738690000      936847
  Total Dropped      :      0/0      0
Class VOIP
Classification statistics      (packets/bytes)      (rate - kbps)
Matched      :      117760245/164864343000      11242157
  Transmitted      :      117760245/164864343000      11242157
  Total Dropped      :      0/0      0
Class VIDEO_STREAM
Classification statistics      (packets/bytes)      (rate - kbps)
Matched      :      49066792/68693508800      4684233
  Transmitted      :      49066792/68693508800      4684233
  Total Dropped      :      0/0      0
Class TRANSACTIONAL_DATA
Classification statistics      (packets/bytes)      (rate - kbps)
Matched      :      225707344/315990281600      21547467
  Transmitted      :      225707344/315990281600      21547467
  Total Dropped      :      0/0      0
Class DB_SYNC
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched      :      0/0      0
  Transmitted      :      0/0      0
  Total Dropped      :      0/0      0
Class BULK_DATA
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched      :      0/0      0
  Transmitted      :      0/0      0
  Total Dropped      :      0/0      0
Class SCAVENGER
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched      :      0/0      0
  Transmitted      :      0/0      0
  Total Dropped      :      0/0      0
Class class-default
Classification statistics      (packets/bytes)      (rate - kbps)
Matched      :      500482413/700675378200      47779164
  Transmitted      :      500482413/700675378200      47779164
  Total Dropped      :      0/0      0
Policy Bag Stats time: 1657528790084 [Local Time: 07/11/22 08:39:50.084]

```

Ingress QoS classification and remarking are enabled for VPWS traffic on the specified subinterface.

## QoS Policy Propagation via BGP

A QoS Policy Propagation via BGP (QPPB) mechanism is a Quality of Service (QoS) classification technique that

- enables QoS classification by leveraging BGP attributes distributed through the BGP control plane
- facilitates destination-based (or source-based) traffic classification by associating BGP routes with QoS groups, and
- simplifies packet marking and prioritization without relying solely on access control lists (ACLs).

A QoS group is a numeric value associated with a route in the forwarding information base (FIB). The router uses this value to determine the traffic class or priority for packets that match that route.

Table 57: Feature History Table

| Feature Name                   | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS Policy Propagation via BGP | Release 25.1.1      | Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*)<br><br>*This feature is now supported on the Cisco 8011-4G24Y4H-I routers.                                                                                                                                                                                                                                                    |
| QoS Policy Propagation via BGP | Release 24.4.1      | Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)<br><br>*This feature is now supported on: <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 8712-MOD-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-52Y8H-EM</li> <li>• 88-LC1-12TH24FH-E</li> </ul> |
| QoS Policy Propagation via BGP | Release 7.5.2       | You now have the ability to install a BGP route in the routing table with a QoS Group so that IP packets that match the route receive the QoS policies associated with the QoS group.<br><br>This functionality enables convenient classification and marking when BGP is deployed, overcoming the administrative challenges of classifying based on ACLs.                                                            |

### QPPB integration of BGP and QoS groups

QPPB integrates the BGP control plane with the IP data forwarding plane. It uses routing policies and table maps to match specific prefixes learned through BGP neighbors and assigns a QoS group value. The FIB retains this value so the data plane can apply QoS actions such as remarking, shaping, or policing.

### QPPB for Prioritizing Business-Critical Traffic

In an enterprise network, an administrator may want to prioritize mission-critical traffic, such as web-based business applications, over general data traffic. Using QPPB, the administrator can tag destination prefixes representing these business applications within the BGP route policy. When packets arrive at the ingress interface, the router automatically classifies and marks them according to their associated QoS group, ensuring consistent prioritization across the network.

## Benefits of QoS Policy Propagation via BGP

QPPB provides these benefits:

- IP prefix–based classification: enables IP prefix-based QoS classification and marking to differentiate traffic based on destination networks.
- Traffic prioritization: prioritizes traffic to specific destination prefixes over other traffic classes for predictable service delivery.
- Control plane integration: leverages the BGP control plane to tag prefixes using standard route-map techniques, ensuring consistent policy propagation.
- Flexible remarking and prioritization: uses Modular QoS CLI (MQC) mechanisms for flexible remarking, queuing, and prioritization actions.
- Control and forwarding plane alignment: establishes a direct connection between the BGP control plane routing decisions and the IP data forwarding plane for QoS enforcement.

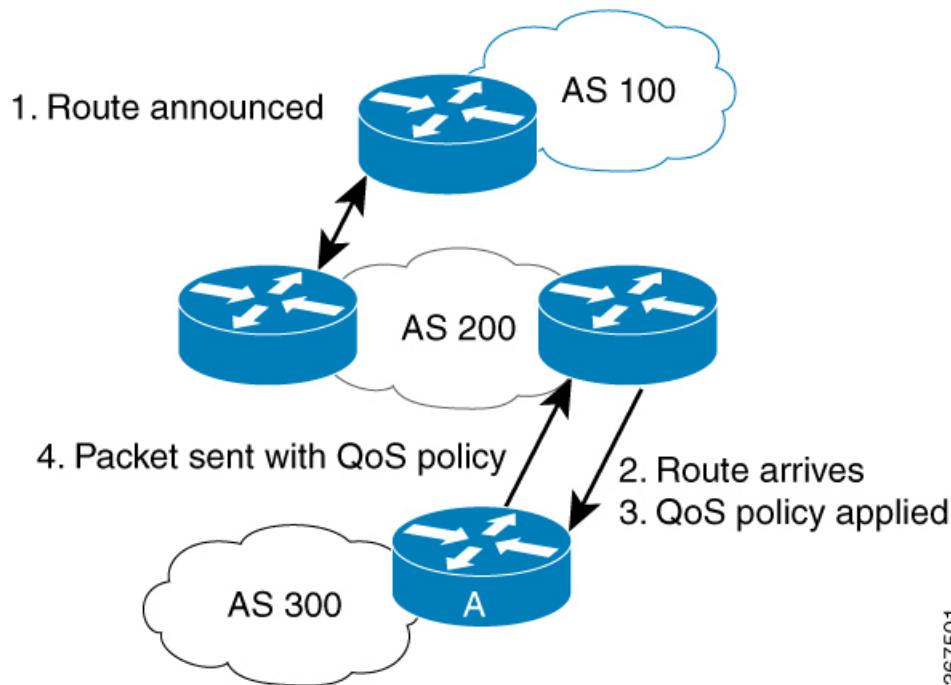
## How QoS Policy Propagation via BGP works

When a router receives routes from BGP peers, it can classify traffic based on route attributes rather than source information. QoS Policy Propagation via BGP (QPPB) works within the BGP process to assign QoS group values to specific destination prefixes. These values are then propagated to the forwarding plane, where packet-level actions are applied based on your policy.

### Summary

The key components that are involved in the QPPB process are:

- BGP control plane: learns routes and applies the table map or route policy to assign QoS group values.
- Routing policy: defines which prefixes receive which QoS group values.
- Forwarding plane (FIB): stores the assigned QoS group for each route and uses it to classify incoming packets.
- QoS policy: uses the assigned QoS group values for marking, queuing, and shaping actions.

**Workflow****Figure 11: QPPB process example**

These stages describe how QPPB works:

1. Router A in the figure receives routes from BGP peers (for example, AS 100 and AS 200) and evaluates each route using the configured routing policy.
2. The routing policy (table map) matches the destination prefix against defined criteria such as community lists, prefix sets, or autonomous system (AS) paths and assigns a QoS group value.
3. The router installs the route, along with its assigned QoS group, into the routing information base (RIB) and then into the forwarding information base (FIB).
4. When an incoming packet's destination matches one of these prefixes, the router retrieves the corresponding QoS group from the FIB and classifies the packet accordingly.
5. The ingress QoS policy uses this QoS group to apply marking, queuing, or other actions.

**Result**

The QPPB process results in a scalable, BGP-driven QoS model that reduces administrative complexity and ensures consistent policy enforcement for traffic toward specific destinations.

## Best practices for configuring QoS Policy Propagation via BGP

### Configuration best practices

Configure QPPB with supported QoS group values and destination-prefix policies to ensure valid, consistent, destination-based classification.

- Define route policies using the **set qos-group** command to assign QoS group values between 0 and 7.
- Use QoS group-based classification. IP precedence-based QPPB is not supported.



- Use IP destination prefix mode on input policies. This is the supported mode for QPPB.

## Interface and hardware recommendations

Apply QPPB to supported interfaces and platforms. This approach prevents configuration failures and ensures correct QoS group propagation.

- Apply QPPB on physical interfaces, bundle interfaces, or their subinterfaces based on your topology.
- Do not configure QPPB on Q100-based routers or line cards, which do not support this feature.

## Interactions with other features

Coordinate QPPB with ACL, FlowSpec, Peering QoS, and policing behavior to maintain predictable precedence.

- Plan your QoS and ACL configurations considering ascending order feature precedence.
  - For QoS group-based policy, the order is: Security ACL (0), BGP FlowSpec (1), QPPB (2), Peering QoS (3)
  - For ACL-based policy, the order is: Security ACL (0), ACL-based QoS (1), BGP FlowSpec (2), QPPB (3)
- Do not configure QPPB on Q100-based routers or line cards, which do not support this feature.
- QPPB overwrites QoS group values set by security ACLs.
- Peering QoS overwrites QoS group values set by QPPB.
- Peering QoS and QPPB overwrite BGP FlowSpec actions, except for drop and policer.
- QPPB does not support egress policing.

## Compatibility with SPAN and remarking features

Verify Switched Port Analyzer (SPAN) and remarking compatibility before enabling QPPB to prevent interface conflicts and preserve expected QoS behavior.

- You cannot enable ACL-based SPAN on interfaces where QPPB is configured. Disable ACL-based SPAN before applying QPPB on interfaces.
- QPPB supports the same remarking and policing behavior as the current QoS features.

# Configure QoS Policy Propagation via BGP

Use this task to configure QoS Policy Propagation via BGP (QPPB) so that traffic classification and marking occur automatically based on BGP-learned destination prefixes.

QPPB integrates BGP routing information with QoS classification. You define route policies that assign QoS group values to specific prefixes and apply them to BGP through a table-policy. Ingress and egress QoS policies then use those values to apply differentiated treatment to packets.

### Before you begin

- Ensure BGP is configured and routes are being exchanged with peers.
- Verify that the target interfaces support QoS configuration.
- Confirm that no ACL-based SPAN sessions are active on the same interfaces.

Follow these steps to configure QPPB.

## Procedure

**Step 1** Define a routing policy. Use the Routing Policy Language (RPL) to set up destination prefixes and assign QoS groups.

### Example:

```
Router#configure
Router(config)#prefix-set prefix-list-v4
Router(config-prefix-set)#70.1.1.1
Router(config-prefix-set)#70.2.1.0/24
Router(config-prefix-set)#70.2.2.0/24 ge 28
Router(config-prefix-set)#70.2.3.0/24 le 28
Router(config-prefix-set)#end-set
Router(config)#prefix-set prefix-list-v6
Router(config-prefix-set)#2001:300::2
Router(config-prefix-set)#2003:200::3
Router(config-prefix-set)#end-set
Router(config)#route-policy qppb1
Router(config-route-policy)#if destination in (60.60.0.2/24) then
Router(config-route-policy)#set qos-group 5
Router(config-route-policy)#elseif destination in prefix-list-v4 then
Router(config-route-policy)#set qos-group 4
Router(config-route-policy)#else
Router(config-route-policy)#set qos-group 1
Router(config-route-policy)#pass
Router(config-route-policy)#endif
Router(config-route-policy)#end-policy
Router(config)#commit
```

This step uses prefix sets and route policies to classify BGP learned routes and tag them with QoS groups. For example:

- **prefix set prefix-list-v4** includes 70.1.1.1, 70.2.1.0/24, and ranges like 70.2.2.0/24 ge 28.
- Route policy assigns **qos-group 5** to destination 60.60.0.2/24, **qos-group 4** to prefixes in **prefix-list-v4**, and **qos-group 1** otherwise.

This step enables granular classification of traffic based on destination prefixes, essential for scalable QoS enforcement.

**Step 2** Attach the routing policy at the table-policy point under BGP to apply QoS group values.

### Example:

```
Router#configure
Router(config)#router bgp 900
Router(config-bgp)#bgp router-id 22.22.22.22
Router(config-bgp)#address-family ipv4 unicast
Router(config-bgp-af)#table-policy qppb1
Router(config-bgp-af)#exit
Router(config-bgp)#address-family ipv6 unicast
Router(config-bgp-af)#table-policy qppb2
Router(config-bgp-af)#exit
Router(config-bgp)#neighbor 30.2.2.1
Router(config-bgp-nbr)#remote-as 500
Router(config-bgp-nbr)#address-family ipv4 unicast
Router(config-bgp-nbr-af)#route-policy pass in
Router(config-bgp-nbr-af)#route-policy pass out
Router(config-bgp-nbr-af)#exit
Router(config-bgp-nbr)#address-family ipv6 unicast
```

```

Router(config-bgp-nbr-af) #route-policy pass in
Router(config-bgp-nbr-af) #route-policy pass out
Router(config-bgp-nbr-af) #exit
Router(config-bgp-nbr) #exit
Router(config-bgp) #exit
Router(config) #commit

```

This step attaches the route policy to BGP address families so routes carry QoS group values when installed in the RIB and FIB. For example, for **router bgp 900**, it applies **table-policy qppb1** for IPv4 unicast and **table-policy qppb2** for IPv6 unicast address families.

This step ensures QoS group values propagate with routing information, aligning control and forwarding planes for QoS.

**Step 3** Configure ingress QoS and enable IPv4 or IPv6 BGP propagation.

**Example:**

```

Router#configure
Router(config) #class-map match-any qos-group5
Router(config-cmap) #match qos-group 5
Router(config-cmap) #exit
Router(config) #class-map match-any qos-group4
Router(config-cmap) #match qos-group 4
Router(config-cmap) #exit
Router(config) #policy-map ingress-marker-po1
Router(config-pmap) #class qos-group5
Router(config-pmap-c) #set precedence 0
Router(config-pmap-c) #set discard-class 0
Router(config-pmap-c) #set traffic-class 1
Router(config-pmap-c) #exit
Router(config-pmap) #class qos-group4
Router(config-pmap-c) #set precedence 1
Router(config-pmap-c) #set discard-class 1
Router(config-pmap-c) #set traffic-class 2
Router(config-pmap-c) #exit
Router(config-pmap) #class class-default
Router(config-pmap) #exit
Router(config-pmap) #exit
Router(config) #interface HundredGigE0/0/0/0
Router(config-if) #ipv4 address 25.1.1.1/24
Router(config-if) #ipv6 address 2001:db8:a0b:12f0::1/64
Router(config-if) #ipv4 bgp policy propagation input qos-group destination
Router(config-if) #ipv6 bgp policy propagation input qos-group destination
Router(config-if) #service-policy input ingress-marker-po1
Router(config-if) #exit
Router(config) #commit

```

This step defines class-maps matching QoS groups, creates a policy-map to set packet markings, and applies this policy on ingress interfaces. It also enables BGP policy propagation for IPv4 and IPv6. Specifically:

- **class-map qos-group5** matches **qos-group 5**, **class-map qos-group4** matches **qos-group 4**.
- **policy-map ingress-marker-po1** sets **precedence 0** for **qos-group5** and **precedence 1** for **qos-group4**.

This step automates packet marking based on destination prefixes, eliminating manual ACLs and simplifying QoS enforcement.

**Step 4** Configure the egress interface policy.

**Example:**

```

Router#configure
Router(config) #class-map match-any level1

```

```

Router(config-cmap)#match traffic-class 1
Router(config-cmap)#exit
Router(config)#class-map match-any level2
Router(config-cmap)#match traffic-class 2
Router(config-cmap)#exit
Router(config)#policy-map output-pol
Router(config-pmap)#class level1
Router(config-pmap-c)#bandwidth percent 50
Router(config-pmap-c)#exit
Router(config-pmap)#class level2
Router(config-pmap-c)#bandwidth percent 20
Router(config-pmap-c)#queue-limit 50 ms
Router(config-pmap-c)#exit
Router(config-pmap)#exit
Router(config)#interface HundredGigE0/5/0/0
Router(config-if)#ipv4 address 30.1.1.1/24
Router(config-if)#ipv6 address 2001:da8:b0a:12f0::1/64
Router(config-if)#service-policy output output-pol
Router(config-if)#exit
Router(config)#commit

```

The egress interface policy uses the traffic class assigned at ingress as an internal match criterion. The router applies QoS treatments such as bandwidth allocation and queue limits based on this class. Because traffic class is internal and not part of the packet header, the router enforces QoS consistently within the device without modifying packets. This approach leads to scalable and effective traffic management on egress interfaces.

This step enforces QoS treatment such as bandwidth guarantees and queue management, ensuring consistent traffic handling downstream.

---

Your router automatically classifies and marks packets based on BGP-learned destination prefixes. This enables scalable, policy-driven QoS control without manual ACL-based classification.

## QoS on Pseudowire Headend

Quality of Service (QoS) on Pseudowire Headend (PWHE) is a network traffic management feature that

- applies QoS classification, marking, policing, and queuing on pseudowire headend interfaces
- ensures prioritized forwarding of customer traffic that enters a pseudowire, and
- enhances the performance and reliability of Layer 2 and Layer 3 VPN services on edge routers.

**Pseudowire (PW)**—A pseudowire is a virtual, point-to-point connection that emulates the properties of a physical wire. It allows the transport of various Layer 2 services, such as Ethernet, Frame Relay, or Asynchronous Transmission Mode (ATM), over a Layer 3 packet-switched network like IP or MPLS.

**Pseudowire Headend (PWHE)**—PWHE refers to the interface and configuration at the start of a pseudowire connection on an edge router. It provides the logical boundary where traffic from a customer or access network is encapsulated into a pseudowire for transport across the core.

### QoS on PWHE and the importance of headend

PWHE allows Layer 2 services to be extended over a Layer 3 network. By applying QoS at the headend, traffic entering the pseudowire can be policed, shaped, and marked to maintain consistent QoS treatment

across the MPLS or IP backbone. This ensures that latency-sensitive traffic such as voice or video is forwarded with the appropriate priority.

**Table 58: Feature History Table**

| Feature Name | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QoS on PWHE  | Release 25.1.1      | <p>Introduced in this release on: Fixed Systems (8700 [ASIC:K100])(select variants only*)</p> <p>*This feature is now supported on Cisco 8712-MOD-M routers.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| QoS on PWHE  | Release 24.3.1      | <p>Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*)</p> <p>You can now prioritize critical services like voice, video, and real-time applications for network traffic entering a Pseudowire Headend (PWHE) interface, which then forwards the prioritized traffic into the pseudowire to be transported over a packet-switched network, such as an IP or MPLS network.</p> <p>The feature introduces the <b>show qos-ea default-queue</b> command.</p> <p>The feature modifies the <code>Cisco-IOS-XR-qos-ma-oper.yang</code> (see <a href="#">GitHub</a>, <a href="#">YANG Data Models Navigator</a>) data model.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"> <li>• 8212-48FH-M</li> <li>• 8711-32FH-M</li> <li>• 88-LC1-36EH</li> <li>• 88-LC1-52Y8H-EM</li> <li>• 88-LC1-12TH24FH-E</li> </ul> |

### QoS on PWHE in practice

For example, a service provider can configure QoS on a PWHE interface so that voice and video services are prioritized when they enter the pseudowire. These packets are then transported across the MPLS core with markings that preserve their service class, ensuring consistent performance end-to-end.

## How QoS on Pseudowire Headend works

Pseudowire Headend (PWHE) extends QoS control to pseudowire interfaces. This enables per-class differentiation and bandwidth management alongside core interfaces.

### Summary

The key components in the QoS process on PWHE interfaces are:

- **Classification and marking:** The router examines Layer 2 and Layer 3 header fields on ingress and egress to identify traffic classes and apply appropriate QoS group and traffic class markings.
- **Policing and queuing:** The router applies policing on ingress to control traffic rates and uses queuing and shaping on egress to manage congestion and enforce bandwidth guarantees.
- **Bandwidth distribution:** The system allocates excess bandwidth between PWHE and non-PWHE traffic that share the same interface.
- **Aggregate shaper:** The line card supports an aggregate shaping policy that coexists with subinterface policies.
- **Traffic pin-down:** Each PWHE interface maps its traffic to a single physical interface, known as a pin-down member.
- **QoS accounting:** The router collects QoS statistics based on the customer IP packet, Layer 2 header, and any configured overhead.

### Workflow

These stages describe how QoS on PWHE operates.

1. **Ingress classification and policing:** The router receives incoming traffic on the PWHE interface and classifies it based on Layer 2 and Layer 3 header fields. This stage ensures that packets entering the pseudowire are marked and rate-limited in alignment with service policies.
2. **Pin-down selection:** The router assigns each PWHE interface to a single physical interface called a pin-down member, selected from a generic interface list. This mapping remains constant for all traffic belonging to that PWHE interface. The pin-down mechanism ensures that traffic flows consistently through the same physical path, maintaining predictable forwarding and shaping behavior.
3. **Port shaping interaction:** The router enforces port-level shaping policies configured on the pin-down member. These shaping parameters limit the maximum bandwidth that all PWHE traffic mapped to that member can use. As a result, even if the PWHE interface allows a higher percentage of bandwidth, its effective throughput is constrained by the shaping policy on the pin-down member.

4. Egress queuing and shaping: When packets are forwarded toward the pseudowire egress, the router derives the egress queues and shaping actions from the QoS policies applied to the PWHE main or subinterfaces. These egress queues are replicated on each member or bundle interface as applicable, ensuring consistent queuing behavior across all physical members.
5. Bandwidth remaining distribution: After high-priority traffic receives its guaranteed bandwidth, the router distributes the remaining bandwidth among PWHE and non-PWHE traffic on the same pin-down member. It uses the bandwidth remaining ratios configured in the parent class-default of the PWHE policies to control how the excess bandwidth is shared across different classes and interfaces.
6. QoS accounting and verification: The router collects QoS accounting statistics that include the customer IP packet, customer Layer 2 header, and any configured additional overhead. It excludes outer transport and encapsulation headers. You can use these statistics to verify policy behavior, evaluate bandwidth distribution, and fine-tune configurations for optimal performance.

### Result

QoS on PWHE ensures that traffic is consistently prioritized, shaped, and accounted for, maintaining predictable performance for pseudowire-based services and fair resource allocation across shared physical interfaces.

## Functional details and behaviors of QoS on PWHE

QoS on PWHE provides the framework for applying consistent service differentiation to pseudowire-based traffic.

Each topic in this section explains a specific operational aspect, such as bandwidth distribution, aggregate shaping, or policing behavior, that contributes to the overall performance of QoS on PWHE interfaces.

Before configuring QoS on PWHE interfaces, it's essential to understand the features and functionalities this technology provides and the restrictions that accompany them.

**Table 59: Reference Information about Functional Details and Behaviors**

| Information                                                                                        | What It Covers                                                                                                         |
|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <a href="#">General features and support information for QoS on PWHE, on page 204</a>              | Lists address-family support, policy directions, and inheritance rules for main and subinterfaces.                     |
| <a href="#">Traffic mapping for Pseudowire Headend interfaces to pin-down members, on page 205</a> | Explains how PWHE traffic maps to a single physical interface (pin-down member) for predictable flow paths.            |
| <a href="#">Port shaping effect on pin-down members, on page 206</a>                               | Describes how port-level shaping on a pin-down member constrains the total bandwidth available to mapped PWHE traffic. |
| <a href="#">Bandwidth distribution for QoS on Pseudowire Headend, on page 206</a>                  | Details how remaining bandwidth is distributed fairly among PWHE and non-PWHE traffic on shared members.               |
| <a href="#">QoS accounting for Pseudowire Headend traffic, on page 206</a>                         | Defines how the router measures and reports PWHE traffic for policing, shaping, and statistics.                        |

| Information                                                                             | What It Covers                                                                                                      |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <a href="#">Marking for QoS on Pseudowire Headend, on page 207</a>                      | Summarizes supported marking types and restrictions for ingress and egress directions.                              |
| <a href="#">Policing and queuing for QoS on Pseudowire Headend, on page 207</a>         | Explains ingress policing and egress queuing behavior with and without policy maps, including bundle replication    |
| <a href="#">Pseudowire Headend aggregate shaper, on page 208</a>                        | Describes aggregate shaping policies that coexist with subinterface policies and the supported action combinations. |
| <a href="#">Pseudowire Ethernet subinterface policy, on page 209</a>                    | Provides information on applying QoS policies at the subinterface level and setting bandwidth limits.               |
| <a href="#">Scale information for QoS on Pseudowire Headend interfaces, on page 209</a> | Lists the supported number of PWHE interfaces per line card and scale considerations.                               |
| <a href="#">Pseudowire Headend without QoS policy, on page 210</a>                      | Explains default traffic behavior when no QoS policies are applied to a PWHE interface.                             |

## General features and support information for QoS on PWHE

Read this topic to understand the foundational support details for QoS on PWHE, including address-family compatibility, policy directions, and subinterface inheritance behavior. Knowing these details helps you apply policies correctly on main and subinterfaces. This ensures that configurations remain supported and behave as expected across IPv4 and IPv6 deployments.

The feature supports:

- IPv4 address families
- IPv6 address families

QoS policies supported:

- Ingress: support for policing and marking.
- Egress: support for marking and queuing.

Subinterface policies:

- You can apply policy maps on PW-Ether subinterfaces.
- PW-Ether subinterfaces inherit policies from their main PW-Ether interface.

This table summarizes the policies you can configure on PWHE main and subinterfaces for egress.



**Table 60: Egress Policy Matrix (Main versus Subinterface)**

| On the egress PWHE main interface                                                                                                                                                                     | On the egress PWHE subinterface                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| when you configure one of the following: <ul style="list-style-type: none"> <li>• two-level H-QoS queuing policy or flat QoS queuing policy,</li> <li>• marking policy, or</li> <li>• both</li> </ul> | you cannot configure any policies.                                                                                                                                                                   |
| if you do not configure any policies                                                                                                                                                                  | you can configure one of the following: <ul style="list-style-type: none"> <li>• two-level H-QoS queuing policy or flat QoS queuing policy,</li> <li>• marking policy, or</li> <li>• both</li> </ul> |
| if you configure aggregate shaper, you cannot configure a second policy                                                                                                                               | you can configure a marking policy, but you cannot set the traffic class. You can set the bandwidth allocation in the queuing policy.                                                                |

Simultaneous policies on port and PWHE interfaces:

- You can apply policies at the port for transit traffic while also applying policies for PWHE interfaces.
- This application enables concurrent control of core and pseudowire traffic on the same device.

## Traffic mapping for Pseudowire Headend interfaces to pin-down members

By pinning down a member, you keep specific network resources in use. This method gives you stability, predictability, and control over operations. Use it for complex network environments, where dynamic changes can cause issues.

A Pseudowire Headend (PWHE) interface sends its traffic to a specific physical (or pin-down) member. The mapping determines how flows are assigned and maintained for predictable forwarding and shaping.

- Consider a case where the generic interface list has 10G and 100G interfaces, and 2 PWHE interfaces named PWHE-1 and PWHE-2 use this generic list.
  - If the traffic for PWHE-1 is hashed (or assigned) to the 10G interface, all the traffic for PWHE-1 will exclusively use the pinned-down 10G interface.
  - If the traffic for PWHE-2 is hashed to the 100G interface, all the traffic for PWHE-2 will exclusively use the pinned-down 100G interface.
- Each PWHE interface directs its traffic through a single physical interface from the generic interface list, so all data flows for that PWHE interface use the same physical path.

## Port shaping effect on pin-down members

Port shaping determines how much bandwidth a pin-down member can provide to pseudowire headend (PWHE) traffic.

Understanding this behavior helps you predict effective throughput when PWHE interfaces share the same physical interface.

Port shaping defines how bandwidth limits configured on a physical member influence the total rate available to PWHE traffic pinned to that member.

- **Rate limitation:** A shaping policy on a member interface restricts the maximum amount of traffic that can pass through that port. This constraint affects all PWHE interfaces that are hashed to the same physical member.
- **Example scenario:** If a pin-down member has a shaping policy set to 10 G on its default traffic class (TC-0), the maximum bandwidth allowed on that interface is 10 G. When a PWHE interface (for example, PWHE-1) is configured to use 50% of available bandwidth and is pinned to that member, the effective maximum bandwidth for PWHE-1 is 5 Gbps.
- **Operational summary:** The shaping configuration on the physical port directly determines the throughput available to PWHE traffic. Even if a PWHE interface has its own shaping policy, its actual transmission rate is constrained by the shaping policy of the pin-down member it uses.

## Bandwidth distribution for QoS on Pseudowire Headend

Bandwidth distribution controls how excess capacity is shared across classes once high-priority traffic has received its guaranteed bandwidth. Understanding this mechanism helps maintain fairness and efficient utilization of shared physical resources.

Bandwidth distribution determines how remaining capacity is shared between Pseudowire Headend (PWHE) and non-PWHE traffic after guaranteed service levels are met.

- **Shared resource control:** PWHE and non-PWHE traffic on the same pin-down member share scheduling resources dynamically based on configured bandwidth ratios.
- **Policy tuning:** Configure **bandwidth remaining** ratio values in the parent class-default of PWHE policies to control excess-bandwidth allocation.
  - between PWHE and non-PWHE traffic and
  - between various PWHE interfaces and physical interfaces.
- **Operational guidance:** Adjust these ratios to maintain fairness and prevent oversubscription between pseudowire traffic and core traffic.

## QoS accounting for Pseudowire Headend traffic

QoS accounting measures how pseudowire traffic consumes bandwidth and applies QoS functions, including policing, shaping, and collecting statistics. Accurate accounting enables you to verify performance, troubleshoot issues, and optimize configurations.

QoS accounting describes how the router measures and reports traffic that is handled by PWHE interfaces. This helps validate policy effectiveness and ensures accurate performance analysis.

- Measurement basis: Packet length calculations when performing QoS functions (such as policing, shaping, and gathering statistics) include the customer IP packet, the customer Layer 2 header, and any configured overhead.
- Exclusion scope: Outer MPLS headers (such as VC and transport labels) and outer Layer 2 headers (Layer 2 encapsulation of the underlying physical interface) of the physical interface are not counted.
- Operational use: These statistics provide insight into actual customer traffic and enable validation of QoS policy outcomes.

## Marking for QoS on Pseudowire Headend

Marking identifies packet classes and preserves differentiated treatment as traffic enters and exits the pseudowire. Understanding supported marking options ensures consistent end-to-end QoS behavior.

Marking defines which header fields the router can set on Pseudowire Headend (PWHE) interfaces and how those markings apply in the ingress and egress directions.

- Supported fields: Marking of the customer IP header (DSCP or precedence), **qos-group**, and **discard-class** is supported.
- Transport CoS limitation: Marking of CoS bits in the transport Layer 2 header is not supported.
- Unconditional marking (ingress and egress): You can set DSCP or precedence, **discard-class**, and **qos-group**.
- Conditional policer marking (ingress): You can set at most two of these—DSCP or precedence, **qos-group**, **discard-class**.
- Unconditional policer marking (egress): You can set DSCP or precedence and **discard-class**.

**Table 61: Marking Options and Supported Traffic Direction**

| Type of marking       | Traffic direction  | Supported markings                                                      |
|-----------------------|--------------------|-------------------------------------------------------------------------|
| Unconditional         | Ingress and egress | DSCP or precedence, <b>discard-class</b> , <b>qos-group</b>             |
| Conditional policer   | Ingress            | Any two of: DSCP or precedence, <b>qos-group</b> , <b>discard-class</b> |
| Unconditional policer | Egress             | DSCP or precedence, <b>discard-class</b>                                |
| Transport L2 CoS      | Ingress or egress  | Not supported                                                           |

## Policing and queuing for QoS on Pseudowire Headend

Policing regulates ingress rates while queuing and shaping control egress scheduling. Understanding these behaviors ensures predictable bandwidth enforcement and priority handling for Pseudowire Headend (PWHE) traffic.

Policing and queuing govern how PWHE traffic complies with configured rates and how packets are scheduled on egress:

- Ingress policing: All policing features supported on normal Layer 3 interfaces are supported on the PWHE main interface and subinterfaces.
- Bundles: If a PWHE member is a bundle, policy maps are replicated on bundle members.
- Statistics: Show commands provide egress statistics per pin-down member and per bundle member. Aggregate shaper statistics summarize subinterface queues per pin-down member.

If you attach a policy map to the PWHE interface, queuing behavior changes for ingress and egress traffic.

**Table 62: PWHE Queuing and Policing Behavior**

| Policy map on PWHE | Egress queues                                                                                                                                                                                                        | Ingress policers                                                                                                                                       |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| None               | Each PWHE member has default queues for each port. Egress traffic uses the member's port default queue. Queuing is supported in the egress direction only when a service policy is attached in the output direction. | Not applicable                                                                                                                                         |
| Present            | Egress queues in the policy maps are replicated on each PWHE member.                                                                                                                                                 | Policer bandwidth is calculated based on active members per slice. These are the members attached to the generic interface list on the PWHE interface. |

## Pseudowire Headend aggregate shaper

The aggregate shaper limits the total egress rate for Pseudowire Headend (PWHE) traffic while allowing subinterfaces to enforce class-level policies.

The PWHE aggregate shaper operates on **class-default** and can coexist with subinterface policies using supported action combinations.

- Action scope: The aggregate shaper policy contains **class-default** with **shape** and **bandwidth remaining** actions.
- Supported combinations for the aggregate shaper policy:
  - Only shape action
  - Only bandwidth remaining action, or
  - both shape and bandwidth remaining actions (recommended)
- Coexistence rule: Coexistence rule: The aggregate shaper can coexist with subinterface policies. If a shaper is configured on the main interface, you cannot configure another shaping policy.

**Table 63: Aggregate Shaper Action Variants**

| Policy variant | Actions in class-default |
|----------------|--------------------------|
| Only shape     | shape average <rate>     |

| Policy variant                                           | Actions in class-default                                                 |
|----------------------------------------------------------|--------------------------------------------------------------------------|
| Only bandwidth remaining                                 | <b>bandwidth remaining ratio</b> <value>                                 |
| Both shape and bandwidth remaining actions (recommended) | <b>shape average</b> <rate> and <b>bandwidth remaining ratio</b> <value> |

## Pseudowire Ethernet subinterface policy

Subinterface policies enable fine-grained QoS management at lower levels of the interface hierarchy. You can specify how bandwidth and marking are applied to specific classes of traffic on each pseudowire subinterface.

Apply PW-Ethernet subinterface policies for granular QoS management when no policy exists on the parent PW-Ether interface. Key features include:

- **Applicability:** You can apply QoS policies on PW-Ether subinterfaces only when no policy exists on the main PW-Ether interface.
- **Granular bandwidth control:** You can configure bandwidth limits as percentage values for each class within the subinterface policy hierarchy. For example, if you have a policy that manages traffic on a PW-Ether subinterface, you can specify that a particular class of traffic should use, say, 20 percent of the available bandwidth.
- **Differentiated treatment:** These subinterface-level configurations let you assign distinct service levels for different customers or applications.
- **Hierarchy consistency:** Subinterface policies align with parent aggregate and shaping policies to maintain overall QoS consistency.

## Scale information for QoS on Pseudowire Headend interfaces

Scale information helps you plan configurations that remain within hardware resource limits and maintain consistent performance.

Router-specific scale values define how many Pseudowire Headend (PWHE) interfaces can simultaneously support ingress or egress QoS policies without depleting queueing resources.

- **Line Card Capacity:**
  - Each line card supports up to 256 PWHE interfaces with ingress or egress QoS policies.
  - This scale is valid only if queueing resources are not exhausted.
  - The actual scale also depends on available queueing and shaping resources and the complexity of the policies applied.
  - Exceeding these limits may cause policy application failures or degraded performance.
- **Resource Dependency:** The actual scale depends on available queueing and shaping resources. Higher policy complexity can reduce effective scale.

## Pseudowire Headend without QoS policy

When a PWHE interface does not have an explicit QoS policy, the router still applies basic classification by copying IP header markings into MPLS labels. Knowing this default behavior helps you predict traffic handling before applying explicit policies.

Default QoS on PWHE interfaces exhibits these behaviors:

- PWHE ingress to core-facing egress (access to core): DSCP or precedence values from the customer IP packet are copied to the EXP bits of all imposed labels (VPN and transport).
- PWHE egress (core to access): DSCP or precedence values from the customer IP packet are copied to the EXP field in the access facing direction.
- Operational impact: These inherited markings preserve basic service differentiation. Bandwidth and queueing controls are not enforced until explicit policies are applied.

## Limitations for QoS on Pseudowire Headend

These constraints apply to QoS on pseudowire headend interfaces:

- Interface support: Configure QoS only on PW-Ether main or subinterfaces operating in Layer 3 mode.
- QoS accounting scope: QoS accounting does not include pseudowire (PW) headers when measuring and recording packet length.

Configuration rules

- When defining QoS policies, include at least one match criterion per class.
- Use supported classification methods for matching access groups in class maps in IPv4 or IPv6. Packet-length or Time to Live (TTL)-based classification is not supported.
- Classification and marking inheritance
  - Do not use Layer 2 header-based classification and marking on Layer 3 PWHE interfaces.
  - Subinterfaces inherit classification and marking behavior from the PW-Ether main interface if no explicit policy is configured.
- Egress QoS-group classification: Use classification methods other than QoS group for BUM traffic in access-to-pseudowire flows.

## Configuration options for QoS on Pseudowire Headend

Use this reference to identify which QoS configuration best suits your Pseudowire Headend deployment.

The table helps you compare scenarios and select the configuration that matches your traffic type and shaping requirements.

Table 64: Selecting the Appropriate QoS Configuration on PWHE Interfaces

| Configuration option                                                                   | When to use                                                             | Network scale   | Recommended for                                                                        |
|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------|----------------------------------------------------------------------------------------|
| <a href="#">Configure Flat QoS on Pseudowire Headend, on page 211</a>                  | Simple traffic management and basic prioritization; no hierarchy needed | Small to medium | Uniform traffic in which marking and queuing apply directly at the interface           |
| <a href="#">Configure Hierarchical QoS on Pseudowire Headend, on page 215</a>          | Multilevel control with parent and child policies                       | Large           | Differentiated voice, video, and data with layered shaping and bandwidth               |
| <a href="#">Configure QoS with aggregate shaper on Pseudowire Headend, on page 221</a> | Control total bandwidth and smooth bursts while preserving class QoS    | Large           | Multiple PW-Ether subinterfaces share physical bandwidth and require port level limits |

## Configure Flat QoS on Pseudowire Headend

Configure a single-level policy on a PWHE interface to prioritize traffic classes.

Flat QoS provides a single policy level on the PW-Ether interface. You manage ingress marking and policing, as well as egress queuing and shaping directly.

### Before you begin

Identify classes and priority. Ensure there is no conflicting aggregate shaper on the main interface.

Follow these steps to configure flat QoS on a PWHE interface.

### Procedure

**Step 1** Apply an input service policy to the physical interface

#### Example:

```
Router(config)#interface HundredGigE0/0/0/59
Router(config-if)#service-policy input MAIN_IN_PMAP5
Router(config-if)#commit
```

This example applies an input service policy named **MAIN\_IN\_PMAP5** to the **HundredGigE0/0/0/59** physical interface.

**Step 2** Apply output service policies to the PWHE interface.

#### Example:

```
Router(config)#interface PW-Ether1
Router(config-if)#service-policy output shape_flat_bwrr
Router(config-if)#service-policy output MAIN_OUT_PMAP5_Q_TC_MARK
Router(config-if)#commit
```

This example applies two service policies named **shape\_flat\_bwrr** and **MAIN\_OUT\_PMAP5\_Q\_TC\_MARK** to the PWHE interface named **PW-Ether**.

**Step 3** Define class maps and policy maps for traffic classes.**Example:**

```

Router(config)#policy-map type qos shape_flat_bwrr
Router(config-pmap)#class type qos MAIN_OUT_CMAP_1
Router(config-pmap-c)#bandwidth remaining ratio 20
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_2
Router(config-pmap-c)#bandwidth remaining ratio 30
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_3
Router(config-pmap-c)#bandwidth remaining ratio 40
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_4
Router(config-pmap-c)#bandwidth remaining ratio 50
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_5
Router(config-pmap-c)#bandwidth remaining ratio 60
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_6
Router(config-pmap-c)#shape average percent 30
Router(config-pmap-c)#priority level 2
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_7
Router(config-pmap-c)#shape average percent 40
Router(config-pmap-c)#priority level 1
Router(config-pmap-c)#exit
Router(config-pmap)#end-policy-map

```

This step configures bandwidth remaining ratio in the **shape\_flat\_bwrr** service policy on the **PW-Ether1** interface for different traffic classes. It defines how bandwidth is distributed and prioritized among classes on the PWHE interface.

**Step 4** Set traffic classes on the PWHE interface.**Example:**

```

Router(config)#policy-map type qos MAIN_OUT_PMAP5_Q_TC_MARK
Router(config-pmap)#class type qos MAIN_OUT_MARK_CMAP_1
Router(config-pmap-c)#set traffic-class 1
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_CMAP_2
Router(config-pmap-c)#set traffic-class 2

Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_CMAP_3
Router(config-pmap-c)#set traffic-class 3
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_CMAP_4
Router(config-pmap-c)#set traffic-class 4
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_CMAP_5
Router(config-pmap-c)#set traffic-class 5
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_CMAP_6
Router(config-pmap-c)#set traffic-class 6
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_CMAP_7
Router(config-pmap-c)#set traffic-class 7
Router(config-pmap-c)#exit
Router(config-pmap)#end-policy-map

```

This example sets traffic classes in the **MAIN\_OUT\_PMAP5\_Q\_TC\_MARK** policy on the **PW-Ether1** interface. This enables consistent QoS treatment across the network.



**Step 5** Set traffic classes and QoS groups on the physical interface.**Example:**

```

Router(config)#policy-map type qos MAIN_IN_PMAP5
Router(config-pmap)#class type qos MAIN_IN_CMAP8_1
Router(config-pmap-c)#set traffic-class 1
Router(config-pmap-c)#set qos-group 1
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_IN_CMAP8_2
Router(config-pmap-c)#set traffic-class 2
Router(config-pmap-c)#set qos-group 2
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_IN_CMAP8_3
Router(config-pmap-c)#set traffic-class 3
Router(config-pmap-c)#set qos-group 3
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_IN_CMAP8_4
Router(config-pmap-c)#set traffic-class 4
Router(config-pmap-c)#set qos-group 4
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_IN_CMAP8_5
Router(config-pmap-c)#set traffic-class 5
Router(config-pmap-c)#set qos-group 5
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_IN_CMAP8_6
Router(config-pmap-c)#set traffic-class 6
Router(config-pmap-c)#set qos-group 6
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_IN_CMAP8_7
Router(config-pmap-c)#set traffic-class 7
Router(config-pmap-c)#set qos-group 7
Router(config-pmap-c)#exit
Router(config-pmap)#end-policy-map
Router(config-pmap)#commit

```

This example sets traffic classes and QoS groups in the **MAIN\_IN\_PMAP5** policy on the **HundredGigE0/0/0/59** physical interface. It ensures ingress traffic is properly classified and marked for QoS enforcement.

**Step 6** Verify the flat QoS configuration for the PWHE interface.**Example:**

```
Router#show policy-map interface PW-Ether1 output
```

```
PW-Ether1 output: shape_flat_bwrr
```

```

Class MAIN_OUT_CMAP_1
  Classification statistics          (packets/bytes)      (rate - kbps)
    Matched                        : 3713933519/885747042770    20297753
    Transmitted                    : 819154447/194810747182    4464282
    Total Dropped                  : 2894779072/690936295588      15833471
  Queueing statistics
    Queue ID                      : None (Pseudo-Wire)
    Taildropped(packets/bytes)    : 2894779072/690936295588
Class MAIN_OUT_CMAP_2
  Classification statistics          (packets/bytes)      (rate - kbps)
    Matched                        : 3713933859/885747126352    20297723
    Transmitted                    : 1226431948/292261487200    6697427
    Total Dropped                  : 2487501911/593485639152    13600296
  Queueing statistics
    Queue ID                      : None (Pseudo-Wire)
    Taildropped(packets/bytes)    : 2487501911/593485639152

```

## Configure Flat QoS on Pseudowire Headend

```

Class MAIN_OUT_CMAP_3
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3713934320/885747236808      20297726
    Transmitted   :      1635701991/389672415000      8929683
    Total Dropped :      2078232329/496074821808      11368043
  Queueing statistics
    Queue ID      : None (Pseudo-Wire)
    Taildropped(packets/bytes) : 2078232329/496074821808
Class MAIN_OUT_CMAP_4
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3713934620/885747307472      20297725
    Transmitted   :      1962135193/467620808304      10715952
    Total Dropped :      1751799427/418126499168      9581773
  Queueing statistics
    Queue ID      : None (Pseudo-Wire)
    Taildropped(packets/bytes) : 1751799427/418126499168
Class MAIN_OUT_CMAP_5
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3713934983/885747394920      20297724
    Transmitted   :      2453607357/584506995776      13394493
    Total Dropped :      1260327626/301240399144      6903231
  Queueing statistics
    Queue ID      : None (Pseudo-Wire)
    Taildropped(packets/bytes) : 1260327626/301240399144
Class MAIN_OUT_CMAP_6
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3713935277/885747464816      20297721
    Transmitted   :      3713522895/885652168176      20295533
    Total Dropped :      412382/95296640      2188
  Queueing statistics
    Queue ID      : None (Pseudo-Wire)
    Taildropped(packets/bytes) : 412382/95296640
Class MAIN_OUT_CMAP_7
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3713935660/885747555696      20297721
    Transmitted   :      3713935660/885747555696      20297721
    Total Dropped :      0/0      0
  Queueing statistics
    Queue ID      : None (Pseudo-Wire)
    Taildropped(packets/bytes) : 0/0
Class class-default
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3713933237/885746977848      20297727
    Transmitted   :      40921134/9742495832      223233
    Total Dropped :      3673012103/876004482016      20074494
Policy Bag Stats time: 1722242150998 [Local Time: 07/29/24 08:35:50.998]

```

PW-Ether1 output: MAIN\_OUT\_PMAP5\_Q\_TC\_MARK

```

Class MAIN_OUT_MARK_CMAP_1
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714522947/885887620056      20317550
    Transmitted   :      3714522947/885887620056      20317550
    Total Dropped :      0/0      0
Class MAIN_OUT_MARK_CMAP_2
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714523360/885887719136      20317549
    Transmitted   :      3714523360/885887719136      20317549
    Total Dropped :      0/0      0
Class MAIN_OUT_MARK_CMAP_3
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714523737/885887808376      20317548
    Transmitted   :      3714523737/885887808376      20317548
    Total Dropped :      0/0      0

```

```

Class MAIN_OUT_MARK_CMAP_4
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714524115/885887898616      20317547
    Transmitted   :      3714524115/885887898616      20317547
    Total Dropped :      0/0      0
Class MAIN_OUT_MARK_CMAP_5
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714524499/885887990496      20317545
    Transmitted   :      3714524499/885887990496      20317545
    Total Dropped :      0/0      0
Class MAIN_OUT_MARK_CMAP_6
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714524873/885888079352      20317545
    Transmitted   :      3714524873/885888079352      20317545
    Total Dropped :      0/0      0
Class MAIN_OUT_MARK_CMAP_7
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714525265/885888173128      20317546
    Transmitted   :      3714525265/885888173128      20317546
    Total Dropped :      0/0      0
Class class-default
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :      3714525658/885888266160      20317545
    Transmitted   :      3714525658/885888266160      20317545
    Total Dropped :      0/0      0
Policy Bag Stats time: 1722242151064 [Local Time: 07/29/24 08:35:51.064]

```

This example shows:

- large numbers of matched and transmitted packets per class indicating active classification and forwarding.
- significant drops in some classes (for example, **MAIN\_OUT\_CMAP\_1** to **MAIN\_OUT\_CMAP\_5**), indicating traffic shaping or congestion control is active and working.
- zero drops in marking policy classes (**MAIN\_OUT\_MARK\_CMAP\_1** to **MAIN\_OUT\_MARK\_CMAP\_7**), confirming marking is applied without loss.

---

Flat QoS enforces simple per-class bandwidth allocation and marking on PWHE interfaces without hierarchical nesting.

## Configure Hierarchical QoS on Pseudowire Headend

Configure hierarchical parent and child policies on a Pseudowire Headend (PWHE) interface to achieve granular control over traffic scheduling and shaping.

In a hierarchical QoS (H-QoS) setup, the parent policy defines the overall shaping rate for the interface, while the child policy determines how that bandwidth is distributed among individual traffic classes. This configuration enables precise control of traffic priorities and ensures that critical services continue to receive resources even when congestion occurs.

### Before you begin

Before configuring H-QoS:

- Determine which traffic types in your network are critical and need prioritization with finer control.

- See [General features and support information for QoS on PWHE, on page 204](#) for important information about the policies you can configure on PWHE main and subinterfaces for egress.
- See [Pseudowire Headend aggregate shaper, on page 208](#) for details about aggregate shaper policies.

Follow these steps to configure H-QoS on a PWHE interface:

## Procedure

**Step 1** Apply the parent output policy to the PW-Ether interface.

### Example:

```
Router(config)#interface PW-Ether1
Router(config-if)#service-policy output port_shaper_combination_hqos_bwrr
Router(config-if)#commit
```

This example attaches the parent QoS policy **map parent\_shape\_hqos\_bwrr** to the egress (output) of the PW-Ether1 interface. This ensures that all outbound traffic on this interface is controlled by the shaping and bandwidth rules defined in the referenced policy map.

**Step 2** Create the parent QoS policy map with shaping rules.

### Example:

```
Router(config)#policy-map type qos parent_shape_hqos_bwrr
Router(config-pmap)#class type qos class-default
Router(config-pmap-c)#shape average percent 70
Router(config-pmap-c)#end-policy-map
```

This example creates a parent QoS policy named **parent\_shape\_hqos\_bwrr**. Within this policy, the **class-default** traffic class is shaped to use up to 70 percent of the interface's bandwidth.

**Step 3** Reference a child policy map within the parent policy map.

### Example:

```
Router(config)#policy-map type qos parent_shape_hqos_bwrr
Router(config-pmap)#class type qos class-default
Router(config-pmap-c)#service-policy type qos shape_hqos_bwrr
```

This step links the child policy map **shape\_hqos\_bwrr** to the **class-default** class of the parent policy. This allows for further breakdown and detailed QoS handling of the shaped traffic, enabling more granular bandwidth management and queuing within the already shaped group.

**Step 4** Define the child policy map.

### Example:

```
Router(config)#policy-map type qos shape_hqos_bwrr
Router(config-pmap)#class type qos MAIN_OUT_CMAP_1
Router(config-pmap-c)#bandwidth remaining ratio 20
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_2
Router(config-pmap-c)#bandwidth remaining ratio 30
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_3
Router(config-pmap-c)#bandwidth remaining ratio 40
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_4
Router(config-pmap-c)#bandwidth remaining ratio 50
```

```

Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_5
Router(config-pmap-c)#bandwidth remaining ratio 60
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_6
Router(config-pmap-c)#shape average percent 10
Router(config-pmap-c)#priority level 2
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_7
Router(config-pmap-c)#shape average percent 20
Router(config-pmap-c)#priority level 1
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos class-default
Router(config-pmap-c)#bandwidth remaining ratio 10
Router(config-pmap-c)#end-policy-map
Router(config)#commit

```

This example defines the child policy map **shape\_hqos\_bwrr**. Each **MAIN\_OUT\_CMAP\_X** class is assigned a specific bandwidth share (bandwidth remaining ratio) or shaping parameter (shape average percent and priority level).

This allows for further breakdown and detailed QoS handling of the shaped traffic, enabling more granular bandwidth management and queuing within the already shaped group. Specifically:

- Classes 1 to 5 are allocated bandwidth in increasing ratios.
- Classes 6 and 7 are shaped and prioritized, ensuring critical traffic gets faster service.
- The **class-default** ensures any unclassified traffic gets a minimal share.
- Traffic is separated into classes, each with explicit bandwidth allocation or priority, providing precise traffic management.

## Step 5 Verify the H-QoS operation.

### Example:

```
Router#show policy-map interface PW-Ether1 output
```

```
PW-Ether1 output: MAIN_OUT_PMAP5_Q_TC_MARK
Class MAIN_OUT_MARK_CMAP_1
```

| Classification statistics | (packets/bytes)        | (rate - kbps) |
|---------------------------|------------------------|---------------|
| Matched :                 | 567459747/135334786816 | 12797449      |
| Transmitted :             | 567459747/135334786816 | 12797449      |
| Total Dropped :           | 0/0                    | 0             |

```
Class MAIN_OUT_MARK_CMAP_2
```

| Classification statistics | (packets/bytes)        | (rate - kbps) |
|---------------------------|------------------------|---------------|
| Matched :                 | 567460159/135334884896 | 12797447      |
| Transmitted :             | 567460159/135334884896 | 12797447      |
| Total Dropped :           | 0/0                    | 0             |

```
Class MAIN_OUT_MARK_CMAP_3
```

| Classification statistics | (packets/bytes) | (rate - kbps) |
|---------------------------|-----------------|---------------|
|---------------------------|-----------------|---------------|

## Configure Hierarchical QoS on Pseudowire Headend

```

    Matched      :          567460559/135334979696      12797443
    Transmitted   :          567460559/135334979696      12797443
    Total Dropped :          0/0                        0

Class MAIN_OUT_MARK_CMAP_4
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :          567460923/135335066400      12797441
    Transmitted   :          567460923/135335066400      12797441
    Total Dropped :          0/0                        0

Class MAIN_OUT_MARK_CMAP_5
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :          567461313/135335159920      12797441
    Transmitted   :          567461313/135335159920      12797441
    Total Dropped :          0/0                        0

Class MAIN_OUT_MARK_CMAP_6
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :          567461672/135335245112      12797434
    Transmitted   :          567461672/135335245112      12797434
    Total Dropped :          0/0                        0

Class MAIN_OUT_MARK_CMAP_7
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :          567462071/135335339784      12797433
    Transmitted   :          567462071/135335339784      12797433
    Total Dropped :          0/0                        0

Class class-default
  Classification statistics      (packets/bytes)      (rate - kbps)
    Matched      :          567462460/135335433176      12797433
    Transmitted   :          567462460/135335433176      12797433
    Total Dropped :          0/0                        0

Policy Bag Stats time: 1722242661147

PW-Ether1 output: parent_shape_hqos_bwrr

Class class-default
  Classification statistics      (packets/bytes)      (rate - kbps)

```

```

Matched          :          4534918248/1081543126494      103380648
Transmitted      :          1754661178/418305616732      40446522
Total Dropped    :          2780257070/663237509762      62934126

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_1

Classification statistics          (packets/bytes)      (rate - kbps)

Matched          :          566863835/135192664022      13582722
Transmitted      :          116771460/27808308708      3393098
Total Dropped    :          450092375/107384355314      10189624

Queueing statistics

Queue ID          : None (Pseudo-Wire)

Taildropped(packets/bytes)        : 450092375/107384355314

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_2

Classification statistics          (packets/bytes)      (rate - kbps)

Matched          :          566864236/135192760288      12828274
Transmitted      :          175109685/41713170712      3958057
Total Dropped    :          391754551/93479589576      8870217

Queueing statistics

Queue ID          : None (Pseudo-Wire)

Taildropped(packets/bytes)        : 391754551/93479589576

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_3

Classification statistics          (packets/bytes)      (rate - kbps)

Matched          :          566864574/135192841920      12828272
Transmitted      :          233516796/55616629856      5277334
Total Dropped    :          333347778/79576212064      7550938

Queueing statistics

Queue ID          : None (Pseudo-Wire)

Taildropped(packets/bytes)        : 333347778/79576212064

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_4

Classification statistics          (packets/bytes)      (rate - kbps)

Matched          :          566864964/135192934568      12828276
Transmitted      :          280180776/66740681128      6332876
Total Dropped    :          286684188/68452253440      6495400

```

## Queueing statistics

Queue ID : None (Pseudo-Wire)  
 Taildropped(packets/bytes) : 286684188/68452253440

## Policy shape\_hqos\_bwrr Class MAIN\_OUT\_CMAP\_5

| Classification statistics |   | (packets/bytes)        | (rate - kbps) |
|---------------------------|---|------------------------|---------------|
| Matched                   | : | 566865352/135193027832 | 12828278      |
| Transmitted               | : | 350225162/83425674520  | 7916096       |
| Total Dropped             | : | 216640190/51767353312  | 4912182       |

## Queueing statistics

Queue ID : None (Pseudo-Wire)  
 Taildropped(packets/bytes) : 216640190/51767353312

## Policy shape\_hqos\_bwrr Class MAIN\_OUT\_CMAP\_6

| Classification statistics |   | (packets/bytes)        | (rate - kbps) |
|---------------------------|---|------------------------|---------------|
| Matched                   | : | 566865746/135193120992 | 12828275      |
| Transmitted               | : | 180140646/43035061400  | 4083496       |
| Total Dropped             | : | 386725100/92158059592  | 8744779       |

## Queueing statistics

Queue ID : None (Pseudo-Wire)  
 Taildropped(packets/bytes) : 386725100/92158059592

## Policy shape\_hqos\_bwrr Class MAIN\_OUT\_CMAP\_7

| Classification statistics |   | (packets/bytes)        | (rate - kbps) |
|---------------------------|---|------------------------|---------------|
| Matched                   | : | 566866035/135193189376 | 12828277      |
| Transmitted               | : | 360261283/86063059512  | 8166361       |
| Total Dropped             | : | 206604752/49130129864  | 4661916       |

## Queueing statistics

Queue ID : None (Pseudo-Wire)  
 Taildropped(packets/bytes) : 206604752/49130129864

## Policy shape\_hqos\_bwrr Class class-default

| Classification statistics |   | (packets/bytes)        | (rate - kbps) |
|---------------------------|---|------------------------|---------------|
| Matched                   | : | 566863506/135192587496 | 12828274      |
| Transmitted               | : | 58455370/13903030896   | 1319204       |
| Total Dropped             | : | 508408136/121289556600 | 11509070      |



```

Queueing statistics

Queue ID                               : None (Pseudo-Wire)

Taildropped(packets/bytes)             : 508408136/121289556600

Policy Bag Stats time: 1722242661011

```

Use this example to confirm that:

- all expected classes appear with growing counters.
- critical classes have minimal or zero drops.
- traffic rates match your policy allocations.

---

After configuration, the H-QoS policy maintains fairness across all classes by providing fine-grained control of pseudowire queues. During congestion, high-priority traffic retains required bandwidth and performance, while overall bandwidth is shaped efficiently.

## Configure QoS with aggregate shaper on Pseudowire Headend

Configure an aggregate shaper on the Pseudowire Ethernet (PW-Ether) main interface to control the total egress bandwidth for all pseudowire subinterfaces. This setup ensures that overall traffic leaving the interface stays within a defined limit, while each subinterface continues to manage its own class-based marking and queuing.

An aggregate shaper defines the maximum rate for all PWHE subinterfaces combined. It operates on the **class-default** of the policy map and enforces a collective bandwidth cap across all associated traffic.

You can configure an aggregate shaper to coexist with subinterface policies if these configurations meet the rules for shaping and bandwidth-remaining actions. This approach allows you to balance total throughput with per-subinterface service differentiation.

### Before you begin

Before you configure the aggregate shaper:

- Determine which types of traffic are critical and need prioritization with finer control.
- See [General Features and Support Information for QoS on PWHE](#) for important information about the policies you can configure on PWHE main and subinterfaces for egress.
- See [PWHE Aggregate Shaper](#) for details about aggregate shaper policies.

Follow these steps to configure an aggregate shaper on a PWHE interface:

### Procedure

---

**Step 1** Define the aggregate shaper parent policy map.

**Example:**

```

Router(config)#policy-map parent_shape_hqos_bwrr
Router(config-pmap)#class class-default
Router(config-pmap-c)#shape average 100000000
Router(config-pmap-c)#service-policy type qos shape_hqos_bwrr
Router(config-pmap-c)#exit
Router(config-pmap)#end-policy-map
Router(config)#commit

```

The policy map (**parent\_shape\_hqos\_bwrr**) defines how all output traffic on the **PW-Ether1** main interface is managed:

- The **class-default** command matches all traffic on the interface that isn't explicitly matched by another class (and since only **class-default** is present, this means all egress traffic).
- The **shape average** command enforces an aggregate shaping limit of 100 Mbps. The total egress bandwidth for all traffic combined cannot exceed this value.
- The **service-policy type qos shape\_hqos\_bwrr** command attaches a child policy. After applying the aggregate shaping limit, the traffic is further divided and managed into classes as defined in the child policy.

**Step 2** Define the child policy map for class-based shaping and bandwidth allocation.

**Example:**

```

Router(config)#policy-map type qos shape_hqos_bwrr
Router(config-pmap)#class type qos MAIN_OUT_CMAP_1
Router(config-pmap-c)#bandwidth remaining ratio 20
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_2
Router(config-pmap-c)#bandwidth remaining ratio 30
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_3
Router(config-pmap-c)#bandwidth remaining ratio 40
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_4
Router(config-pmap-c)#bandwidth remaining ratio 50
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_5
Router(config-pmap-c)#bandwidth remaining ratio 60
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_6
Router(config-pmap-c)#shape average percent 10
Router(config-pmap-c)#priority level 2
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_CMAP_7
Router(config-pmap-c)#shape average percent 20
Router(config-pmap-c)#priority level 1
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos class-default
Router(config-pmap-c)#bandwidth remaining ratio 10
Router(config-pmap-c)#end-policy-map
Router(config)#commit

```

The child policy (**shape\_hqos\_bwrr**) breaks down the aggregate bandwidth among several classes:

- the bandwidth remaining ratio assigns relative bandwidth shares to each class.
- the shape average percent and the priority level give special handling to specific classes.

**Step 3** Apply the aggregate shaper parent policy to the main PWHE interface.

**Example:**

```
Router(config)#interface PW-Ether1
Router(config-if)#service-policy output parent_shape_hqos_bwrr
Router(config-if)#commit
```

This command applies the aggregate shaper and all class-based bandwidth and priority controls to all egress traffic on the main interface (**PW-Ether1**).

**Step 4** Apply a DSCP marking policy to the PWHE subinterface.

**Example:**

```
Router(config)#interface PW-Ether1.1
Router(config-if)#service-policy output MAIN_OUT_DSCP_MARK
Router(config-if)#commit
```

This example ensures traffic leaving the subinterface **PW-Ether1.1** is marked with the correct DSCP values for downstream QoS treatment.

**Step 5** Define the DSCP marking policy map for the subinterface.

**Example:**

```
Router(config)#policy-map type qos MAIN_OUT_DSCP_MARK
Router(config-pmap)#class type qos MAIN_OUT_MARK_DSCP_CMAP_1
Router(config-pmap-c)#set dscp af11
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_DSCP_CMAP_2
Router(config-pmap-c)#set dscp af12
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_DSCP_CMAP_3
Router(config-pmap-c)#set dscp af13
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_DSCP_CMAP_4
Router(config-pmap-c)#set dscp af41
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_DSCP_CMAP_5
Router(config-pmap-c)#set dscp af42
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_DSCP_CMAP_6
Router(config-pmap-c)#set dscp af43
Router(config-pmap-c)#exit
Router(config-pmap)#class type qos MAIN_OUT_MARK_DSCP_CMAP_7
Router(config-pmap-c)#set dscp ef
Router(config-pmap-c)#end-policy-map
Router(config)#commit
```

The policy map marks each class of traffic with a specific DSCP value as it leaves the subinterface:

- Assured Forwarding (**af**) values (for example, **af11** and **af41**) indicate differentiated levels of forwarding assurance and drop precedence for business-critical or prioritized services.
- Expedited Forwarding (**ef**) is typically used for low-latency, high-priority traffic such as voice or real-time applications.

**Step 6** Confirm shaping and class statistics on main interface.

**Example:**

```
Router#show qos-ea default-queue PW-Ether 1 member interface HundredGigE0/7/0/2 output

Interface Name      =      PE1
Interface Handle    =      1c000180
Location            =      0/7/CPU0
Asic Instance       =      0
VOQ Base           =      53312
```

## Configure QoS with aggregate shaper on Pseudowire Headend

```

Port Speed(kbps)      = 100000000
Local Port            = etm_local
VOQ Mode              = 8
-----
ReceivedPkts    ReceivedBytes    DroppedPkts    DroppedBytes
-----
TC_0 = 150295879    150596322940    762151909    763676346056
TC_1 = 0             0             0             0
TC_2 = 0             0             0             0
TC_3 = 0             0             0             0
TC_4 = 0             0             0             0
TC_5 = 0             0             0             0
TC_6 = 0             0             0             0
TC_7 = 0             0             0             0

```

This example shows queue statistics for each traffic class (TC\_0 to TC\_7) on the main interface **PW-Ether1**. TC\_0 (class-default) reflects packet shaping and drops.

**Step 7** View detailed QoS statistics and configuration information for output traffic from the main interface.

### Example:

```

Router#show qos interface PW-Ether1 output member HundredGigE0/7/0/2
NOTE:- Configured values are displayed within parentheses
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id: 0
Total number of classes: 9
Interface Bandwidth: 100000000 kbps
Policy Name: parent_shape_hqos_bwrr
Accounting Type: Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class = class-default
Level2 Class = MAIN_OUT_CMAP_1
Level2 Class = MAIN_OUT_CMAP_2
Level2 Class = MAIN_OUT_CMAP_3
Level2 Class = MAIN_OUT_CMAP_4
Level2 Class = MAIN_OUT_CMAP_5
Level2 Class = MAIN_OUT_CMAP_6
Level2 Class = MAIN_OUT_CMAP_7
Level2 Class = class-default
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id: 0
Total number of classes: 8
Interface Bandwidth: 100000000 kbps
Policy Name: MAIN_OUT_PMAP5_Q_TC_MARK
Accounting Type: Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class = MAIN_OUT_MARK_CMAP_1
New traffic class = 1
Level1 Class = MAIN_OUT_MARK_CMAP_2
New traffic class = 2
Level1 Class = MAIN_OUT_MARK_CMAP_3
New traffic class = 3
Level1 Class = MAIN_OUT_MARK_CMAP_4

```

```

New traffic class                                = 4

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_5
New traffic class                                = 5

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_6
New traffic class                                = 6

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_7
New traffic class                                = 7

Level1 Class                                    =  class-default
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                         3
Total number of classes:                       9
Interface Bandwidth:                           100000000 kbps
Policy Name:                                   parent_shape_hqos_bwrr
Accounting Type:                               Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                    =  class-default

    Level2 Class                                =  MAIN_OUT_CMAP_1

    Level2 Class                                =  MAIN_OUT_CMAP_2

    Level2 Class                                =  MAIN_OUT_CMAP_3

    Level2 Class                                =  MAIN_OUT_CMAP_4

    Level2 Class                                =  MAIN_OUT_CMAP_5

    Level2 Class                                =  MAIN_OUT_CMAP_6

    Level2 Class                                =  MAIN_OUT_CMAP_7

    Level2 Class                                =  class-default
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                         3
Total number of classes:                       8
Interface Bandwidth:                           100000000 kbps
Policy Name:                                   MAIN_OUT_PMAP5_Q_TC_MARK
Accounting Type:                               Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                    =  MAIN_OUT_MARK_CMAP_1
New traffic class                                = 1
Level1 Class                                    =  MAIN_OUT_MARK_CMAP_2
New traffic class                                = 2

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_3
New traffic class                                = 3

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_4
New traffic class                                = 4

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_5
New traffic class                                = 5

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_6
New traffic class                                = 6

Level1 Class                                    =  MAIN_OUT_MARK_CMAP_7
New traffic class                                = 7

Level1 Class                                    =  class-default

```

## Configure QoS with aggregate shaper on Pseudowire Headend

```

Interface HundredGigE0/7/0/2 Ifh 0x1c000180 (PinDown Member) -- output policy
NPU Id:                                0
Total number of classes:                9
Interface Bandwidth:                   100000000 kbps
Policy Name:                           parent_shape_hqos_bwrr
VOQ Base:                              53312
Accounting Type:                       Layer1 (Include Layer 1 encapsulation and above)
VOQ Mode:                              8
Shared Counter Mode:                   1
-----
Level1 Class                           = class-default
Queue Max. BW.                         = 70000000 kbps (70 %)
Inverse Weight / Weight                 = 255 / (BWR not configured)

Level2 Class                           = MAIN_OUT_CMAP_1
Egressq Queue ID                       = 53313 (LP queue)
Queue Max. BW.                         = no max (default)
Inverse Weight / Weight                 = 12 / (20)
Guaranteed service rate                 = 4711538 kbps
TailDrop Threshold                     = 614400 bytes / 1 ms (614400 bytes)
WRED not configured for this class

Level2 Class                           = MAIN_OUT_CMAP_2
Egressq Queue ID                       = 53314 (LP queue)
Queue Max. BW.                         = no max (default)
Inverse Weight / Weight                 = 8 / (30)
Guaranteed service rate                 = 7067307 kbps
TailDrop Threshold                     = 614400 bytes / 702 us (614400 bytes)
WRED not configured for this class

Level2 Class                           = MAIN_OUT_CMAP_3
Egressq Queue ID                       = 53315 (LP queue)
Queue Max. BW.                         = no max (default)
Inverse Weight / Weight                 = 6 / (40)
Guaranteed service rate                 = 9423076 kbps
TailDrop Threshold                     = 614400 bytes / 527 us (614400 bytes)
WRED not configured for this class

Level2 Class                           = MAIN_OUT_CMAP_4
Egressq Queue ID                       = 53316 (LP queue)
Queue Max. BW.                         = no max (default)
Inverse Weight / Weight                 = 5 / (50)
Guaranteed service rate                 = 11307692 kbps
TailDrop Threshold                     = 614400 bytes / 421 us (614400 bytes)
WRED not configured for this class

Level2 Class                           = MAIN_OUT_CMAP_5
Egressq Queue ID                       = 53317 (LP queue)
Queue Max. BW.                         = no max (default)
Inverse Weight / Weight                 = 4 / (60)
Guaranteed service rate                 = 14134615 kbps
TailDrop Threshold                     = 614400 bytes / 351 us (614400 bytes)
WRED not configured for this class

Level2 Class (HP2)                     = MAIN_OUT_CMAP_6
Egressq Queue ID                       = 53318 (HP2 queue)
Queue Max. BW.                         = 7000000 kbps (10 %)
Guaranteed service rate                 = 7000000 kbps
TailDrop Threshold                     = 614400 bytes / 702 us (614400 bytes)
WRED not configured for this class

Level2 Class (HP1)                     = MAIN_OUT_CMAP_7
Egressq Queue ID                       = 53319 (HP1 queue)
Queue Max. BW.                         = 14000000 kbps (20 %)

```

```

    Guaranteed service rate           = 14000000 kbps
    TailDrop Threshold                = 614400 bytes / 351 us (614400 bytes)
    WRED not configured for this class

    Level2 Class                      = class-default
    Egressq Queue ID                  = 53312 (Default LP queue)
    Queue Max. BW.                    = no max (default)
    Inverse Weight / Weight            = 24 / (10)
    Guaranteed service rate            = 2355769 kbps
    TailDrop Threshold                = 614400 bytes / 2 ms (614400 bytes)
    WRED not configured for this class

Interface HundredGigE0/7/0/2 Ifh 0x1c000180 (PinDown Member) -- output policy
NPU Id:                             0
Total number of classes:              8
Interface Bandwidth:                  100000000 kbps
Policy Name:                          MAIN_OUT_PMAP5_Q_TC_MARK
VOQ Base:                            0
Accounting Type:                      Layer1 (Include Layer 1 encapsulation and above)
VOQ Mode:                             8
Shared Counter Mode:                  1
-----
Level1 Class                          = MAIN_OUT_MARK_CMAP_1
New traffic class                     = 1

Level1 Class                          = MAIN_OUT_MARK_CMAP_2
New traffic class                     = 2

Level1 Class                          = MAIN_OUT_MARK_CMAP_3
New traffic class                     = 3

Level1 Class                          = MAIN_OUT_MARK_CMAP_4
New traffic class                     = 4

Level1 Class                          = MAIN_OUT_MARK_CMAP_5
New traffic class                     = 5

Level1 Class                          = MAIN_OUT_MARK_CMAP_6
New traffic class                     = 6

Level1 Class                          = MAIN_OUT_MARK_CMAP_7
New traffic class                     = 7

Level1 Class                          = class-default

Node 0/0/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                             0
Total number of classes:              9
Interface Bandwidth:                  25000000 kbps
Policy Name:                          parent_shape_hqos_bwrr
Accounting Type:                      Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                          = class-default

    Level2 Class                      = MAIN_OUT_CMAP_1

    Level2 Class                      = MAIN_OUT_CMAP_2

    Level2 Class                      = MAIN_OUT_CMAP_3

    Level2 Class                      = MAIN_OUT_CMAP_4

    Level2 Class                      = MAIN_OUT_CMAP_5

```

```

Level2 Class                                =    MAIN_OUT_CMAP_6

Level2 Class                                =    MAIN_OUT_CMAP_7

Level2 Class                                =    class-default
Node 0/0/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                     0
Total number of classes:                    8
Interface Bandwidth:                        250000000 kbps
Policy Name:                               MAIN_OUT_PMAP5_Q_TC_MARK
Accounting Type:                           Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                =    MAIN_OUT_MARK_CMAP_1
New traffic class                           =    1

Level1 Class                                =    MAIN_OUT_MARK_CMAP_2
New traffic class                           =    2

Level1 Class                                =    MAIN_OUT_MARK_CMAP_3
New traffic class                           =    3

Level1 Class                                =    MAIN_OUT_MARK_CMAP_4
New traffic class                           =    4

Level1 Class                                =    MAIN_OUT_MARK_CMAP_5
New traffic class                           =    5

Level1 Class                                =    MAIN_OUT_MARK_CMAP_6
New traffic class                           =    6

Level1 Class                                =    MAIN_OUT_MARK_CMAP_7
New traffic class                           =    7

Level1 Class                                =    class-default

```

This example displays applied policy name (**parent\_shape\_hqos\_bwrr**), class structure, bandwidth, queue, and drop thresholds. Ensure that the class and queue structure match the configuration.

**Step 8** View detailed QoS statistics and configuration information for output traffic from the main interface.

#### Example:

```

Router#show qos interface PW-Ether1 output member HundredGigE0/7/0/2
NOTE:- Configured values are displayed within parentheses
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                     0
Total number of classes:                    9
Interface Bandwidth:                        1000000000 kbps
Policy Name:                               parent_shape_hqos_bwrr
Accounting Type:                           Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                =    class-default

Level2 Class                                =    MAIN_OUT_CMAP_1

Level2 Class                                =    MAIN_OUT_CMAP_2

Level2 Class                                =    MAIN_OUT_CMAP_3

Level2 Class                                =    MAIN_OUT_CMAP_4

Level2 Class                                =    MAIN_OUT_CMAP_5

Level2 Class                                =    MAIN_OUT_CMAP_6

```



```

Level2 Class                                =    MAIN_OUT_CMAP_7

Level2 Class                                =    class-default
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                     0
Total number of classes:                   8
Interface Bandwidth:                       100000000 kbps
Policy Name:                               MAIN_OUT_PMAP5_Q_TC_MARK
Accounting Type:                           Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                =    MAIN_OUT_MARK_CMAP_1
New traffic class                          =    1

Level1 Class                                =    MAIN_OUT_MARK_CMAP_2
New traffic class                          =    2

Level1 Class                                =    MAIN_OUT_MARK_CMAP_3
New traffic class                          =    3

Level1 Class                                =    MAIN_OUT_MARK_CMAP_4
New traffic class                          =    4

Level1 Class                                =    MAIN_OUT_MARK_CMAP_5
New traffic class                          =    5

Level1 Class                                =    MAIN_OUT_MARK_CMAP_6
New traffic class                          =    6

Level1 Class                                =    MAIN_OUT_MARK_CMAP_7
New traffic class                          =    7

Level1 Class                                =    class-default
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                     3
Total number of classes:                   9
Interface Bandwidth:                       100000000 kbps
Policy Name:                               parent_shape_hqos_bwrr
Accounting Type:                           Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                =    class-default

Level2 Class                                =    MAIN_OUT_CMAP_1

Level2 Class                                =    MAIN_OUT_CMAP_2

Level2 Class                                =    MAIN_OUT_CMAP_3

Level2 Class                                =    MAIN_OUT_CMAP_4

Level2 Class                                =    MAIN_OUT_CMAP_5

Level2 Class                                =    MAIN_OUT_CMAP_6

Level2 Class                                =    MAIN_OUT_CMAP_7

Level2 Class                                =    class-default
Node 0/7/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                     3
Total number of classes:                   8
Interface Bandwidth:                       100000000 kbps
Policy Name:                               MAIN_OUT_PMAP5_Q_TC_MARK
Accounting Type:                           Layer1 (Include Layer 1 encapsulation and above)
-----

```

## Configure QoS with aggregate shaper on Pseudowire Headend

```

Level1 Class                                =   MAIN_OUT_MARK_CMAP_1
New traffic class                          =   1
Level1 Class                                =   MAIN_OUT_MARK_CMAP_2
New traffic class                          =   2

Level1 Class                                =   MAIN_OUT_MARK_CMAP_3
New traffic class                          =   3

Level1 Class                                =   MAIN_OUT_MARK_CMAP_4
New traffic class                          =   4

Level1 Class                                =   MAIN_OUT_MARK_CMAP_5
New traffic class                          =   5

Level1 Class                                =   MAIN_OUT_MARK_CMAP_6
New traffic class                          =   6

Level1 Class                                =   MAIN_OUT_MARK_CMAP_7
New traffic class                          =   7

Level1 Class                                =   class-default
Interface HundredGigE0/7/0/2 Ifh 0x1c000180 (PinDown Member) -- output policy
NPU Id:                                    0
Total number of classes:                   9
Interface Bandwidth:                       100000000 kbps
Policy Name:                              parent_shape_hqos_bwrr
VOQ Base:                                  53312
Accounting Type:                           Layer1 (Include Layer 1 encapsulation and above)
VOQ Mode:                                  8
Shared Counter Mode:                       1
-----
Level1 Class                                =   class-default
Queue Max. BW.                             =   70000000 kbps (70 %)
Inverse Weight / Weight                    =   255 / (BWR not configured)

Level2 Class                                =   MAIN_OUT_CMAP_1
Egressq Queue ID                          =   53313 (LP queue)
Queue Max. BW.                             =   no max (default)
Inverse Weight / Weight                    =   12 / (20)
Guaranteed service rate                   =   4711538 kbps
TailDrop Threshold                        =   614400 bytes / 1 ms (614400 bytes)
WRED not configured for this class

Level2 Class                                =   MAIN_OUT_CMAP_2
Egressq Queue ID                          =   53314 (LP queue)
Queue Max. BW.                             =   no max (default)
Inverse Weight / Weight                    =   8 / (30)
Guaranteed service rate                   =   7067307 kbps
TailDrop Threshold                        =   614400 bytes / 702 us (614400 bytes)
WRED not configured for this class

Level2 Class                                =   MAIN_OUT_CMAP_3
Egressq Queue ID                          =   53315 (LP queue)
Queue Max. BW.                             =   no max (default)
Inverse Weight / Weight                    =   6 / (40)
Guaranteed service rate                   =   9423076 kbps
TailDrop Threshold                        =   614400 bytes / 527 us (614400 bytes)
WRED not configured for this class

Level2 Class                                =   MAIN_OUT_CMAP_4
Egressq Queue ID                          =   53316 (LP queue)
Queue Max. BW.                             =   no max (default)
Inverse Weight / Weight                    =   5 / (50)
Guaranteed service rate                   =   11307692 kbps

```

```

TailDrop Threshold                = 614400 bytes / 421 us (614400 bytes)
WRED not configured for this class

Level2 Class                      = MAIN_OUT_CMAP_5
Egressq Queue ID                 = 53317 (LP queue)
Queue Max. BW.                  = no max (default)
Inverse Weight / Weight          = 4 / (60)
Guaranteed service rate          = 14134615 kbps
TailDrop Threshold               = 614400 bytes / 351 us (614400 bytes)
WRED not configured for this class

Level2 Class (HP2)               = MAIN_OUT_CMAP_6
Egressq Queue ID                 = 53318 (HP2 queue)
Queue Max. BW.                  = 7000000 kbps (10 %)
Guaranteed service rate          = 7000000 kbps
TailDrop Threshold               = 614400 bytes / 702 us (614400 bytes)
WRED not configured for this class

Level2 Class (HP1)               = MAIN_OUT_CMAP_7
Egressq Queue ID                 = 53319 (HP1 queue)
Queue Max. BW.                  = 14000000 kbps (20 %)
Guaranteed service rate          = 14000000 kbps
TailDrop Threshold               = 614400 bytes / 351 us (614400 bytes)
WRED not configured for this class

Level2 Class                      = class-default
Egressq Queue ID                 = 53312 (Default LP queue)
Queue Max. BW.                  = no max (default)
Inverse Weight / Weight          = 24 / (10)
Guaranteed service rate          = 2355769 kbps
TailDrop Threshold               = 614400 bytes / 2 ms (614400 bytes)
WRED not configured for this class

Interface HundredGigE0/7/0/2 Ifh 0x1c000180 (PinDown Member) -- output policy
NPU Id:                          0
Total number of classes:          8
Interface Bandwidth:              100000000 kbps
Policy Name:                      MAIN_OUT_PMAP5_Q_TC_MARK
VOQ Base:                        0
Accounting Type:                  Layer1 (Include Layer 1 encapsulation and above)
VOQ Mode:                        8
Shared Counter Mode:              1
-----
Level1 Class                      = MAIN_OUT_MARK_CMAP_1
New traffic class                 = 1

Level1 Class                      = MAIN_OUT_MARK_CMAP_2
New traffic class                 = 2

Level1 Class                      = MAIN_OUT_MARK_CMAP_3
New traffic class                 = 3

Level1 Class                      = MAIN_OUT_MARK_CMAP_4
New traffic class                 = 4

Level1 Class                      = MAIN_OUT_MARK_CMAP_5
New traffic class                 = 5

Level1 Class                      = MAIN_OUT_MARK_CMAP_6
New traffic class                 = 6

Level1 Class                      = MAIN_OUT_MARK_CMAP_7
New traffic class                 = 7

```

## Configure QoS with aggregate shaper on Pseudowire Headend

```

Level1 Class                                =    class-default

Node 0/0/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                     0
Total number of classes:                    9
Interface Bandwidth:                        25000000 kbps
Policy Name:                               parent_shape_hqos_bwrr
Accounting Type:                            Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                =    class-default

    Level2 Class                            =    MAIN_OUT_CMAP_1

    Level2 Class                            =    MAIN_OUT_CMAP_2

    Level2 Class                            =    MAIN_OUT_CMAP_3

    Level2 Class                            =    MAIN_OUT_CMAP_4

    Level2 Class                            =    MAIN_OUT_CMAP_5

    Level2 Class                            =    MAIN_OUT_CMAP_6

    Level2 Class                            =    MAIN_OUT_CMAP_7

    Level2 Class                            =    class-default
Node 0/0/CPU0, Interface PW-Ether1 Ifh 0x7800104c (PW-Ether) -- output policy
NPU Id:                                     0
Total number of classes:                    8
Interface Bandwidth:                        25000000 kbps
Policy Name:                               MAIN_OUT_PMAP5_Q_TC_MARK
Accounting Type:                            Layer1 (Include Layer 1 encapsulation and above)
-----
Level1 Class                                =    MAIN_OUT_MARK_CMAP_1
New traffic class                          =    1

Level1 Class                                =    MAIN_OUT_MARK_CMAP_2
New traffic class                          =    2

Level1 Class                                =    MAIN_OUT_MARK_CMAP_3
New traffic class                          =    3

Level1 Class                                =    MAIN_OUT_MARK_CMAP_4
New traffic class                          =    4

Level1 Class                                =    MAIN_OUT_MARK_CMAP_5
New traffic class                          =    5

Level1 Class                                =    MAIN_OUT_MARK_CMAP_6
New traffic class                          =    6

Level1 Class                                =    MAIN_OUT_MARK_CMAP_7
New traffic class                          =    7

Level1 Class                                =    class-default

```

This example displays applied policy name (**parent\_shape\_hqos\_bwrr**), class structure, bandwidth, queue, and drop thresholds. Look for class and queue structure matching configuration.

**Step 9** View per-class statistics and queuing details for a specific member interface on the main interface.

**Example:**

```
Router#show policy-map interface PW-Ether1 output member HundredGigE0/7/0/2
Interface:PW-Ether1 Member:HundredGigE0/7/0/2 output: parent_shape_hqos_bwrr
```

```
Class class-default
  Classification statistics          (packets/bytes)      (rate - kbps)
  Matched                          :      1048830183/250133071040      0
  Transmitted                      :      405970112/96745121320      0
  Total Dropped                    :      642860071/153387949720      0

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_1
  Classification statistics          (packets/bytes)      (rate - kbps)
  Matched                          :      131102864/31266417264      0
  Transmitted                      :      27020068/6431611720      0
  Total Dropped                    :      104082796/24834805544      0
  Queueing statistics
    Queue ID                       :      53313
    Taildropped(packets/bytes)      :      104082796/24834805544

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_2
  Classification statistics          (packets/bytes)      (rate - kbps)
  Matched                          :      131103199/31266497640      0
  Transmitted                      :      40521792/9647311792      0
  Total Dropped                    :      90581407/21619185848      0
  Queueing statistics
    Queue ID                       :      53314
    Taildropped(packets/bytes)      :      90581407/21619185848

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_3
  Classification statistics          (packets/bytes)      (rate - kbps)
  Matched                          :      131103636/31266600664      0
  Transmitted                      :      54004138/12863485816      0
  Total Dropped                    :      77099498/18403114848      0
  Queueing statistics
    Queue ID                       :      53315
    Taildropped(packets/bytes)      :      77099498/18403114848

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_4
  Classification statistics          (packets/bytes)      (rate - kbps)
  Matched                          :      131103920/31266669280      0
  Transmitted                      :      64802769/15436026336      0
  Total Dropped                    :      66301151/15830642944      0
  Queueing statistics
    Queue ID                       :      53316
    Taildropped(packets/bytes)      :      66301151/15830642944

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_5
  Classification statistics          (packets/bytes)      (rate - kbps)
  Matched                          :      131104311/31266762056      0
  Transmitted                      :      80969302/19295430376      0
  Total Dropped                    :      50135009/11971331680      0
  Queueing statistics
    Queue ID                       :      53317
  Taildropped(packets/bytes)      :      50135009/11971331680

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_6
  Classification statistics          (packets/bytes)      (rate - kbps)
  Matched                          :      131104735/31266863416      0
  Transmitted                      :      41698712/9952700928      0
  Total Dropped                    :      89406023/21314162488      0
  Queueing statistics
    Queue ID                       :      53318
    Taildropped(packets/bytes)      :      89406023/21314162488

Policy shape_hqos_bwrr Class MAIN_OUT_CMAP_7
```

## Configure QoS with aggregate shaper on Pseudowire Headend

```

Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      131105067/31266942536      0
  Transmitted                 :      83425746/19902839360      0
  Total Dropped               :      47679321/11364103176      0
Queueing statistics
  Queue ID                   :      53319
  Taildropped(packets/bytes) :      47679321/11364103176

Policy shape_hqos_bwrr Class class-default
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      131102451/31266318184      0
  Transmitted                 :      13527585/3215714992      0
  Total Dropped               :      117574866/28050603192      0
Queueing statistics
  Queue ID                   :      53312
  Taildropped(packets/bytes) :      117574866/28050603192
Policy Bag Stats time: 1722238640916 [Local Time: 07/29/24 07:37:20.916]

Interface:PW-Ether1 Member:HundredGigE0/7/0/2 output: MAIN_OUT_PMAP5_Q_TC_MARK

Class MAIN_OUT_MARK_CMAP_1
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Class MAIN_OUT_MARK_CMAP_2
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Class MAIN_OUT_MARK_CMAP_3
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Class MAIN_OUT_MARK_CMAP_4
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Class MAIN_OUT_MARK_CMAP_5
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Class MAIN_OUT_MARK_CMAP_6
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Class MAIN_OUT_MARK_CMAP_7
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Class class-default
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      0/0      0
  Transmitted                 :      0/0      0
  Total Dropped               :      0/0      0
Policy Bag Stats time: 1722238674599 [Local Time: 07/29/24 07:37:54.599]

```

This example shows matched, transmitted, and dropped counts per class, queueing info, and policy name on the member interface **HundredGigE0/7/0/2**.

This end-to-end configuration directly implements your design. It enforces a bandwidth cap of 100 Mbps across all subinterfaces on class-default. You can also apply distinct QoS policies, such as marking, at the subinterface level for service differentiation.

## Bridged Virtual Interface two-pass QoS

Bridged Virtual Interface (BVI) two-pass QoS is a QoS processing mechanism that

- performs a Layer 2 QoS inspection on packets within the bridge domain
- applies a second, independent Layer 3 QoS classification and treatment as packets transition through the BVI, and
- ensures consistent QoS behavior for traffic that moves from bridging to routing without relying solely on Layer 3 attributes.

**Table 65: Feature History Table**

| Feature Name                                | Release Information | Feature Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BVI two-pass QoS ingress TC and EXP marking | Release 25.4.1      | <p>Introduced in this release on: Fixed Systems (8200 Series [ASIC: P100], 8700 Series [ASIC: P100]) (select variants only*)</p> <p>You can now apply consistent QoS treatment for traffic that moves from a bridge domain to a routed BVI. Your router performs two independent ingress QoS processing passes: it first classifies and marks traffic using Layer 2 attributes inside the bridge domain, and then applies a second Layer 3-based classification at the BVI ingress. At the BVI ingress, only traffic class (TC) or MPLS EXP marking actions are supported.</p> <p>This helps with accurate class assignment and predictable queuing for bridged-to-routed flows.</p> <p>*This feature is supported on:</p> <ul style="list-style-type: none"><li>• 8212-48FH-M</li><li>• 8711-32FH-M</li></ul> |

## Benefits of BVI two-pass QoS

The Bridge Virtual Interface (BVI) two-pass QoS feature provides several key benefits for networks that bridge traffic into routed BVIs.

Independent Layer 2 and Layer 3 QoS processing:

You can apply separate ingress QoS processing at the Layer 2 bridge domain and at the Layer 3 BVI boundary. This ensures that packets are evaluated using the appropriate header fields at each stage of the forwarding path.

Consistent traffic treatment across Layer 2 and Layer 3 transitions:

The router performs classification twice: once using Layer 2 attributes, and then using Layer 3 attributes. This preserves accurate class assignment for bridged-to-routed flows and allows Layer 3 QoS policies to work predictably, even when traffic originates in a bridge domain.

Greater control over traffic classification and marking:

At the BVI, which functions at Layer 3, you can classify packets on ingress using precedence or DSCP and Access Control Lists (ACL). Marking actions supported on BVI ingress are the MPLS Experimental (EXP) value and traffic class. This Layer 3 ingress classification and marking enable more precise control over how traffic is prioritized and forwarded.

Improved policy design flexibility:

The feature lets you design QoS policies that use Layer 2 attributes, Layer 3 attributes, or both. You are not constrained by a single-pass QoS model. This flexibility is especially useful in multi-tenant or service-edge deployments.

## How BVI two-pass QoS works

When traffic arrives on a Layer 2 interface that belongs to a bridge domain and is routed through a BVI (Bridge Virtual Interface), the router processes the packet twice before forwarding it out of the device. This two-pass model ensures that packets receive the correct QoS treatment as they transition from Layer 2 bridging to Layer 3 routing, even when routed forwarding begins only at the BVI. The BVI supports QoS in the ingress direction only, limited to classification and marking.

### Summary

The BVI two-pass QoS model uses these components:

- L2 interface: Receives packets into the bridge domain; performs the first QoS inspection.
- Bridge domain: Forwards packets at Layer 2 to the BVI.
- BVI: Acts as the Layer 3 boundary and applies the second QoS pass using ingress QoS classification and marking.
- Egress physical or bundle port: Forwards the routed packet out of the router.

### Workflow

These stages describe how the router processes packets through the BVI two-pass QoS model as they transition from Layer 2 bridging to Layer 3 routing.

1. Packet arrives on the Layer 2 interface (first pass, Layer 2 QoS):



- A packet enters a physical or bundle interface associated with a bridge domain.
  - The router performs Layer 2 forwarding and QoS inspection. This may involve encapsulation, such as VLAN tags, or classification rules on the Layer 2 ingress interface.
2. Packet reaches the BVI (second pass, Layer 3 QoS):
    - The packet transitions from bridging to routing when it arrives at the BVI.
    - The router performs a Layer 3 forwarding lookup on the packet.
    - If a QoS ingress policy map is configured on the BVI, the router applies:
      - Ingress classification (DSCP, Prec, or ACL match criteria)
      - Marking at ingress (EXP or traffic class)

The BVI does not perform shaping or queuing in this direction.

3. Packet is routed toward the egress physical or bundle port:
  - After Layer 3 processing, the router selects the appropriate egress physical or bundle interface.
  - The packet is forwarded across the internal fabric to the egress line card.
4. Packet is transmitted out the egress physical or bundle interface:
  - The packet is sent out the final interface.
  - Any egress QoS configured on that physical or bundle port is applied separately and is not part of this two-pass ingress model.

## Best practices for BVI two-pass QoS

### Review supported QoS classification and marking actions for the second pass

Review the supported QoS classification and marking actions applicable during the second pass (BVI ingress) in the two-pass forwarding model. These supported actions are:

- Classification by precedence or DSCP
- Classification by Access Control List
- Setting MPLS Experimental value
- Setting traffic class

### Understand scale limitations before applying BVI two-pass QoS in large or complex deployments

You can use BVI two-pass QoS in this release, but you should not rely on it for very large or complex deployments where you might need to apply many QoS policies or expect high scalability. For this release,

there are no guaranteed performance or capacity numbers. You might encounter limitations or issues if you try to scale the feature beyond typical or simple use cases.

## Restrictions while configuring BVI two-pass QoS for Release 25.4.1

### Ingress marking capabilities are limited

Release 25.4.1 supports only a subset of ingress QoS marking features on Bridged Virtual Interfaces (BVIs) in two-pass mode:

- Matching actions supported on the BVI Layer 3 attachment circuit (L3AC) ingress are ACL, DSCP, and Prec. Marking actions supported on the BVI L3AC ingress include setting traffic class and setting the EXP value.

No other ingress or egress QoS marking actions are supported on BVIs in Release 25.4.1.

- Supported streams include traffic transitioning from Layer 2 to Layer 3 and IP-to-IP or IP-to-MPLS forwarding paths.

### Policing is not supported

Ingress or egress QoS policers are not supported on BVIs in this release.

## Configure BVI two-pass QoS for ingress traffic

Apply QoS policies to Bridged Virtual Interface (BVIs) using two-pass forwarding. This allows flexible classification and marking of ingress traffic.

This process allows you to classify, mark, and manage ingress traffic on BVI interfaces. You apply a policy map that references class maps and sets traffic classes and MPLS experimental values.

### Procedure

**Step 1** Define class maps to match DSCP values.

#### Example:

```
Router#configure
Router(config)#class-map match-all DSCPAF23_TABLE
Router(config-cmap)#match dscp af23
Router(config-cmap)#end-class-map
Router(config)#class-map match-all DSCPAF41_TABLE
Router(config-cmap)#match dscp af41
Router(config-cmap)#end-class-map
Router(config)#class-map match-all DSCPAF42_TABLE
Router(config-cmap)#match dscp af42
Router(config-cmap)#end-class-map
```

This example defines class maps for your QoS policy. Specifically:

- Each **class-map match-all DSCPAFxx\_TABLE** command creates a logical grouping (class map) that matches packets with a certain DSCP value.
- The line **match dscp af23** (and similar for other values) tells the router to identify (or classify) packets whose DSCP field is set to **af23** (or **af41** and **af42**).

**Step 2** Create a policy map and assign actions.**Example:**

```

Router(config)#policy-map INGRESS
Router(config-pmap)#class DSCPAF23_TABLE
Router(config-pmap-c)#set traffic-class 2
Router(config-pmap-c)#set mpls experimental imposition 2
Router(config-pmap-c)#exit
Router(config-pmap)#class DSCPAF41_TABLE
Router(config-pmap-c)#set traffic-class 5
Router(config-pmap-c)#set mpls experimental imposition 5
Router(config-pmap-c)#exit
Router(config-pmap)#class DSCPAF42_TABLE
Router(config-pmap-c)#set traffic-class 6
Router(config-pmap-c)#set mpls experimental imposition 6
Router(config-pmap-c)#exit
Router(config-pmap)#class class-default
Router(config-pmap-c)#set traffic-class 0
Router(config-pmap-c)#exit
Router(config-pmap)#end-policy-map

```

This example creates a QoS policy map (named **INGRESS**) that defines actions for traffic on the BVI interface. Each class map (such as **DSCPAF23\_TABLE**) is associated with specific actions.

For each class, the policy matches packets classified earlier by their DSCP value. It then sets the internal traffic class or MPLS experimental (EXP) bits corresponding to the matched class—meaning the value will be 2, 5, or 6 depending on which class the packet matches.

Additionally, this example:

- Adds a **class-default** entry for any packets not matching a specific class map, setting their traffic class to **0**.
- Prepares the policy for attachment to the BVI in the ingress (input) direction.
- Enables granular and independent QoS marking as part of the BVI two-pass architecture. The **INGRESS** policy is applied on the BVI interface (Layer 3).

**Step 3** Attach the policy map to the BVI interface.**Example:**

```

Router(config)#interface bvi 1
Router(config-if)#service-policy input INGRESS
Router(config-if)#commit

```

**Step 4** Display the QoS policy applied to the BVI interface.**Example:**

```

Router#show qos interface bvi 1 input location 0/RP0/CPU0

```

```

Interface BVI1 ifh 0x78000014  -- input policy
NPU Id:                        0
Total number of classes:       4
Interface Bandwidth:           1048576000 kbps
Policy Name:                   INGRESS

```

```

-----
Level1 Class                    =  DSCPAF23_TABLE
New traffic class                =  2
New imposition exp               =  2
Policer not configured for this class

```

## Configure BVI two-pass QoS for ingress traffic

```

Level1 Class                = DSCPAF41_TABLE
New traffic class           = 5
New imposition exp          = 5
Policer not configured for this class

Level1 Class                = DSCPAF42_TABLE
New traffic class           = 6
New imposition exp          = 6
Policer not configured for this class

Level1 Class                = class-default
New traffic class           = 0
Policer not configured for this class

```

In this example:

- **Policy Name:** Shows that the **INGRESS** policy is attached in the ingress (input) direction on **BVI1**. In BVI two-pass mode, this is Layer 3 where user QoS policy is applied.

**Level1 Class:** Each line represents a class defined in your QoS policy. This classification phase happens during the Layer 3 (second pass) processing.

- **New traffic class** and **New imposition exp:** These reflect the marking actions taken on each packet that matches the class.

**New traffic class** sets an internal marking for traffic categorization.

**New imposition exp** sets the MPLS Experimental bits for MPLS domain.

- **Policer not configured for this class:** Indicates that there is no rate-limiting (policing) on the class, which is normal unless you have explicitly configured a policer.

On ingress, the router processes the packet through the Layer 2 pipeline, where it is classified and marked according to your policy on the Layer 2 interface.

After Layer 2 features are applied, the packet is passed to the BVI, where additional (independent) Layer 3 QoS features can be applied in the second pass.

### Step 5 Check statistics for each class in the policy.

#### Example:

```
Router#show policy-map interface bvi 1 input
```

```

BVI1 input: INGRESS
Class DSCPAF23_TABLE
  Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                      : 1200000/720000000      12000
  Transmitted                   : 1200000/720000000      12000
  Total Dropped                 : 0/0                      0

Class DSCPAF41_TABLE
  Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                      : 1100000/660000000      11000
  Transmitted                   : 1100000/660000000      11000
  Total Dropped                 : 0/0                      0

Class DSCPAF42_TABLE
  Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                      : 1050000/630000000      10500
  Transmitted                   : 1050000/630000000      10500
  Total Dropped                 : 0/0                      0

```

```
Class class-default
Classification statistics      (packets/bytes)      (rate - kbps)
  Matched                    :      880000/528000000      8800
  Transmitted                 :      880000/528000000      8800
  Total Dropped               :           0/0              0
```

In this example:

- **Matched:** Number of packets or bytes that matched each class-map. This shows that the classification step in the Layer 3 pipeline is working and how much traffic falls under each rule.
- **Transmitted:** Number of packets or bytes forwarded after policy action. This should match **Matched** if no drops are configured. This confirms that after Layer 3 ingress policy actions at the BVI interface, the packet is forwarded further.
- **Total Dropped:** Indicates how many packets were dropped at this stage. This value should be **0** unless you configure drop actions or policing.

Two-pass context: These statistics show how Layer 3 (second-pass) QoS classification and actions are being enforced on ingress to the BVI.

---

